Rockchip RKStudio_Tool Algorithmic Control Description

ID:RK-XX-XX-nnn

Release Version:V1.2.0

Release Date:2025-01-22

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY.

THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document is a reference document for RKStudio_Tool algorithmic control, describing the location, functions, parameter settings, and practical demonstration of the algorithmic control in the tool.

Product Vision

Chipset	Kernel Version
RK2118/RK2116/RK2108/RK3308/RK3506/RK356X	This document is unrelated to the kernel and serves solely as a reference document for the `RKStudio_Tool algorithm control`

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Huaping Liao	2024-02-22	Initial version
V1.1.0	Huaping Liao	2024-08-15	Update Algorithm and illustration
V1.2.0	Di Jiang	2025-01-20	Update Algorithm and illustration
V1.3.0	Huaping Liao	2025-10-27	Update Algorithm and illustration
V1.4.0	Huaping Liao	2026-02-02	Update Algorithm and illustration

Contents

- Rockchip RKStudio_Tool Algorithmic Control Description
  - 1 RK_Algorithms
    - 1.1 Loudness
      - 1.1.1 Loudness (Low & High)
      - 1.1.2 Loudness (Low End)
      - 1.1.3 Loudness (High End)
      - 1.1.4 Loudness (Low & High) External Ctrl
      - 1.1.5 Loudness (Low & High) Dynamic
    - 1.2 Shift
      - 1.2.1 Phat Shift(FIR)
      - 1.2.2 Phat Shift (IIR)
      - 1.2.3 Frequency Shift （FIR)
      - 1.2.4 Frequency Shift (IIR)
    - 1.3 Surround Sound
    - 1.4 Pitch Shifting
      - 1.4.1 Phase Vocoder
      - 1.4.2 Phase Vocoder (Formant)
      - 1.4.3 WSOLA
      - 1.4.4 WSOLA (Formant)
    - 1.5 Phat Stereo
    - 1.6 DTS
      - 1.6.1 DTS:X
      - 1.6.2 DTS Virtual:X
    - 1.7 Dolby
    - 1.8 BF_ECNR
    - 1.9 AVAS
      - 1.9.1 AVAS
      - 1.9.2 AVAS External Source
      - 1.9.3 AVAS Standard Compliance and Configuration Specification
    - 1.10 Voice Separators
      - 1.10.1 Voice Separator (MS)
      - 1.10.2 Voice Separator (LRM)
    - 1.11 CrossTalk Suppressor
  - 2 Basic DSP
    - 2.1 Arithmetic Operations
      - 2.1.1 Absolute Value
      - 2.1.2 Inverse Square Root
      - 2.1.3 Log2
      - 2.1.4 Log10

- 2.1.5 Exp2
- 2.1.6 Square Root
- 2.1.7 Signal Add
- 2.1.8 Signal Subtract
- 2.1.9 Divide
- 2.1.10 Multiply
- 2.1.11 Signal Invert
- 2.1.12 Multi Signal Invert

▼ 2.2 Adjustable Gain
- 2.2.1 Gain
- 2.2.2 Gain (Slew)
- 2.2.3 Gain (Slew Double)
- 2.2.4 Gain External Mute (Slew Double)

▼ 2.3 DSP Functions
- 2.3.1 Delay
- 2.3.2 Multi-Delay
- 2.3.3 Multi Delay (Index Selectable)
- 2.3.4 Multi Delay Slew (Index Selectable)
- 2.3.5 DSP Feedback

▼ 2.4 Logic
- 2.4.1 AbsMax
- 2.4.2 Max
- 2.4.3 Min
- 2.4.4 Logic AB
- 2.4.5 Logic ABCD
- 2.4.6 Logic Calc
- 2.4.7 Zero Comparator
- 2.4.8 Value Hold
- 2.4.9 AbsMax Value Hold
- 2.4.10 Max Value Hold
- 2.4.11 Min Value Hold

▼ 3 Dynamics Processors

▼ 3.1 Envelope
- 3.1.1 Envelope RMS
- 3.1.2 Envelope Peak

▼ 3.2 Compressors
- 3.2.1 Compressor RMS
- 3.2.2 Compressor RMS External Input
- 3.2.3 Compressor Peak
- 3.2.4 Compressor Peak External Input

▼ 3.3 Limiter
- 3.3.1 Limiter RMS
- 3.3.2 Limiter Peak

- 3.3.3 Limiter Ratio
- 3.3.4 Limiter Clip
- 3.4 Noise Gate
  - 3.4.1 Noise Gate
  - 3.4.2 Noise Gate (Ext)
- 3.5 Agc

4 Filters
- 4.1 FIR
- 4.2 DC-Blocking
- 4.3 Emphasis
  - 4.3.1 Pre-Emphasis
  - 4.3.2 De-Emphasis
- 4.4 IIR - 1st Order
- 4.5 IIR - 2nd Order
  - 4.5.1 IIR - 2nd Order
  - 4.5.2 IIR - 2nd order (Index Selectable)
  - 4.5.3 IIR - 2nd order (Double Precision)
- 4.6 IIR Nth Order (Single Precision)
- 4.7 IIR Nth Order (Double Precision)
- 4.8 IIR - Coeff
- 4.9 Graphical EQ
  - 4.9.1 Graphical EQ
  - 4.9.2 Graphic EQ (High Precision)
- 4.10 Graphical EQ Group
  - 4.10.1 Graphical EQ Group
  - 4.10.2 Graphic EQ Group (Index Selectable)
- 4.11 Crossover (Single Precision)
  - 4.11.1 2-Way Crossover (Single Precision)
  - 4.11.2 3-Way Crossover (Single Precision)
  - 4.11.3 4-Way Crossover(Single Precision)
- 4.12 Crossover (Double Precision)
- 4.13 Dynamic EQ
- 4.14 Text-In EQ
- 4.15 Resample
  - 4.15.1 Up-sampler
  - 4.15.2 Down-sampler
- 4.16 Crossover EQ
- 4.17 Allpass Delay

5 IO
- 5.1 Audio Input
- 5.2 Audio Output
- 5.3 Trigger Signal IO
- 5.4 Debug IO

- 6 Mixers/Splitters
  - 6.1 Mixers
    - 6.1.1 Signal Merge
    - 6.1.2 Single Control Mixer
    - 6.1.3 Single Control Mixer (Slew)
    - 6.1.4 Multiple Control Mixer
    - 6.1.5 Multiple Control Mixer (Slew)
    - 6.1.6 Cross Mixer
    - 6.1.7 Cross Mixer Fast
    - 6.1.8 Cross Mixer (Slew)
    - 6.1.9 Cross Mixer Fast (Slew)
    - 6.1.10 Cross Mixer Slew (Index)
  - 6.2 Splitters
    - 6.2.1 Single Control Splitter
    - 6.2.2 Single Control Splitter (Slew)
    - 6.2.3 Multiple Control Splitter
    - 6.2.4 Multiple Control Splitter (Slew)
  - 6.3 Fader Balance
  - 6.4 Matrix Mixer
  - 6.5 Matrix Mixer Custom Plug-in
- 7 Muxes/DeMuxes
  - 7.1 Multiplexers Switch
    - 7.1.1 Mono Switch Mux
    - 7.1.2 Stereo Switch Mux
    - 7.1.3 Multi Switch Nx4
    - 7.1.4 Multi Switch Nx6
    - 7.1.5 Multi Switch Nx8
    - 7.1.6 Multi Switch Nx12
    - 7.1.7 Multi Switch NxM
  - 7.1_ Multiplexers Switch (Slew)
    - 7.1.8 Mono Switch Mux (Slew)
    - 7.1.9 Stereo Switch Mux (Slew)
    - 7.1.10 Multi Switch Nx4 (Slew)
    - 7.1.11 Multi Switch Nx6 (Slew)
    - 7.1.12 Multi Switch Nx8 (Slew)
    - 7.1.13 Multi Switch Nx12 (Slew)
    - 7.1.14 NxM Switch Mux (Slew)
  - 7.2 Multiplexers Index Selectable
    - 7.2.1 Mono Index Selectable Mux
    - 7.2.2 Stereo Index Selectable Mux
    - 7.2.3 Multi Index Selectable Nx4
    - 7.2.4 Multi Index Selectable Nx8
  - 7.2_ Multiplexers Index Selectable (Slew)

- 7.2.5 Mono Index Selectable Mux (Slew)
- 7.2.6 Stereo Index Selectable Mux (Slew)
- 7.2.7 Multi Index Selectable Nx4 (Slew)
- 7.2.8 Multi Index Selectable Nx8 (Slew)

▼ 7.3 DeMultiplexers Switch

- 7.3.1 Mono Switch Demux
- 7.3.2 Stereo Switch Demux
- 7.3.3 NxM Switch Demux
- 7.3.4 Mono Switch Demux (Slew)
- 7.3.5 Stereo Switch Demux (Slew)
- 7.3.6 NxM Switch Demux (Slew)

▼ 7.4 DeMultiplexers Index Selectable

- 7.4.1 Mono Index Selectable Demux
- 7.4.2 Stereo Index Selectable Demux
- 7.4.3 Mono Index Selectable Demux (Slew)
- 7.4.4 Stereo Index Selectable Demux (Slew)

▼ 8 Source

- 8.1 DC
- 8.2 Beeq
- 8.3 Sawtooth
- 8.4 Sine
- 8.5 Square
- 8.6 Triangle
- 8.7 Chime
- 8.8 Switch(0,1)
- 8.9 Voltage Controlled
- 8.10 White Noise
- 8.11 Pink Noise
- 8.12 Wav Player
- 8.13 LFO
- 8.14 Ctrl Rate Noise

▼ 9 Volume Controls

- 9.1 Generic Gain Envelope

▼ 9.2 Mute

- 9.2.1 Mute
- 9.2.2 Mute (Slew)
- 9.2.3 Multiple Mute
- 9.2.4 Multiple Mute (Slew)

▼ 9.3 Volume

- 9.3.1 Single Volume
- 9.3.2 Single Volume (Slew)
- 9.3.3 Multiple Volume
- 9.3.4 Multiple Volume (Slew)

- 9.3.5 Single Volume(Slew Linear)
- 9.3.6 Multiple Volume(Slew Linear)
- 9.3.7 Single Volume(Slew Ext)

▼ 10 Detectors
- 10.1 Multiple Band Level Detector
- 10.2 Single Band Level Detector
- 10.3 1D Index Lookup Table
- 10.4 2D Index Lookup Table

▼ 11 Audio Effects
- ▼ 11.1 Wah-Wah
  - 11.1.1 Auto Wah
- 11.2 Flanger
- ▼ 11.3 Echo
  - 11.3.1 Echo
  - 11.3.2 Ping-Pong Echo (Mono)
- 11.4 Virtual Bass
- ▼ 11.5 Reverbs
  - 11.5.1 Simple Reverb
  - 11.5.2 Dattorr Reverb
  - 11.5.3 Advance Reverb
  - 11.5.4 Early Reflection
  - 11.5.5 Room Reverb
- 11.6 Chorus
- 11.7 Transient Shaper
- 11.8 Vibrato
- 11.9 Robot Voice

▼ 12 Cores
- 12.1 Core_ Input
- 12.2 Core_ Output

▼ 13 Custom Algorithms
- 13.1 CustomHowlingSupporess
- 13.2 CustomVocalSeparation
- 13.3 CustomAlgo_V1
- 13.4 CustomAlgo_V2

▼ 14 Counters
- 14.1 Counter
- 14.2 Counter_Ext

▼ 15 Non Linear Process
- 15.1 Hard Clip
- 15.2 Soft Clip
- 15.3 Soft Clip Bilateral
- 15.4 Non Linear

▼ Appendix 1 Schematic Design

- Appendix 1.1 T Connection
- Appendix 1.2 User Comment
- Appendix 1.3 Hierarchy Board
- Appendix 1.4 Hierarchy Input
- Appendix 1.5 Hierarchy Output
- Appendix 1.6 Alias

▼ Appendix 2 Conversion Formulas

- Appendix 2.1 Parameter Conversion Description
- Appendix 2.2 IIR1st Calculation Formulas and Description
- Appendix 2.3 IIR2nd Calculation Formulas and Description
- Appendix 2.4 IIR_Nnd Calculation Formula and Explanation
- Appendix 2.5 Graphical EQ Filter Coefficient Calculation Formula and Explanation
- Appendix 2.6 Crossover Control Calculation Formulas and Explanations
- Appendix 2.7 Using Python Library Functions to Calculate FIR Filter Coefficients
- Appendix 2.8 Ratio Gain Conversion to Linear Gain
- Appendix 2.9 SlewRate Parameter Conversion to Smoothing Coefficient
- Appendix 2.10 Conversion for Delay- and Control-Time-Related Parameters
- Appendix 2.11 Conversion for Smoothing-Time-Constant Parameters
- Appendix 2.12 Conversion for Gain-Change Rate Parameters
- Appendix 2.13 Conversion for Gain-Related Parameters

▼ Appendix 3 Glossary

- Appendix 3.1 biquads:
- Appendix 3.2 attack_time:
- Appendix 3.3 release_time:
- Appendix 3.4 Quality Factor Q:
- Appendix 3.5 Boost (Filter Gain) and Gain (Overall Output Gain):
- Appendix 3.6 Dry and Wet Signals:
- Appendix 3.7 Loudness and Sound Intensity:

---

1 RK_Algorithms

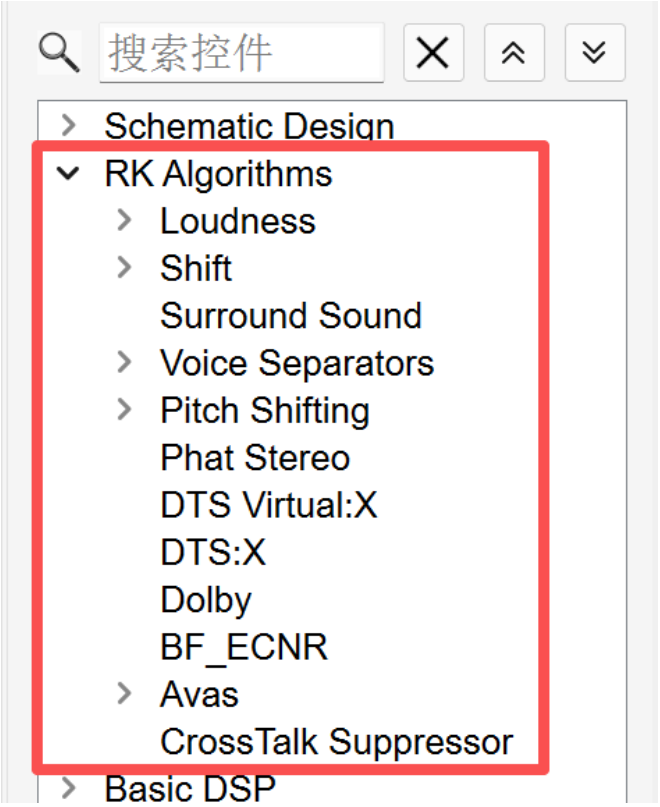


Figure 1-1 The position of RK_Algorithms in the algorithm list

1.1 Loudness

1.1.1 Loudness (Low & High)

Function: Loudness control, using a combination of low-pass and high-pass filters to apply gain to low-frequency and high-frequency signals respectively. (Loudness* refers to the subjective perception of sound level by humans, while sound intensity is the objective physical measure of sound level. This control adjusts the gain of different frequency sounds to create different loudness effects.)

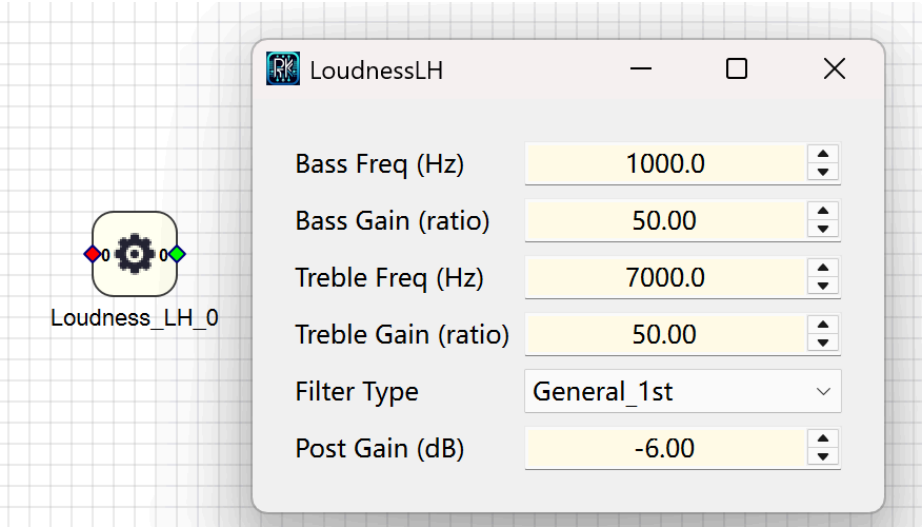


Figure 1.1.1-1 Loudness (Low & High) control and settings interface

Number of Parameters: 6

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
BassFreq (HZ)	0	float	Low-pass cutoff frequency	Hz	<1000	1000
BassGain (ratio)*	1	float	Percentage applied to low frequencies (higher values increase low-frequency gain)	%	0 ~ 100	50
TrebleFreq (HZ)	2	float	High-pass cutoff frequency	Hz	>7000	7000
TrebleGain (ratio)*	3	float	Percentage applied to high frequencies (higher values increase high-frequency gain)	%	0 ~ 100	50
FilterType	4	int	Filter type		0/1	0
PostGain (dB)	5	float	Overall gain reduction	dB	0 ~ -60	-6

Note: For the parameter FilterType, the value 0 corresponds to General 1st (first-order general filter), and the value 1 corresponds to Butterworth filter.

Demonstration of actual effect:
Input white noise source to obtain output, and list typical operating conditions:

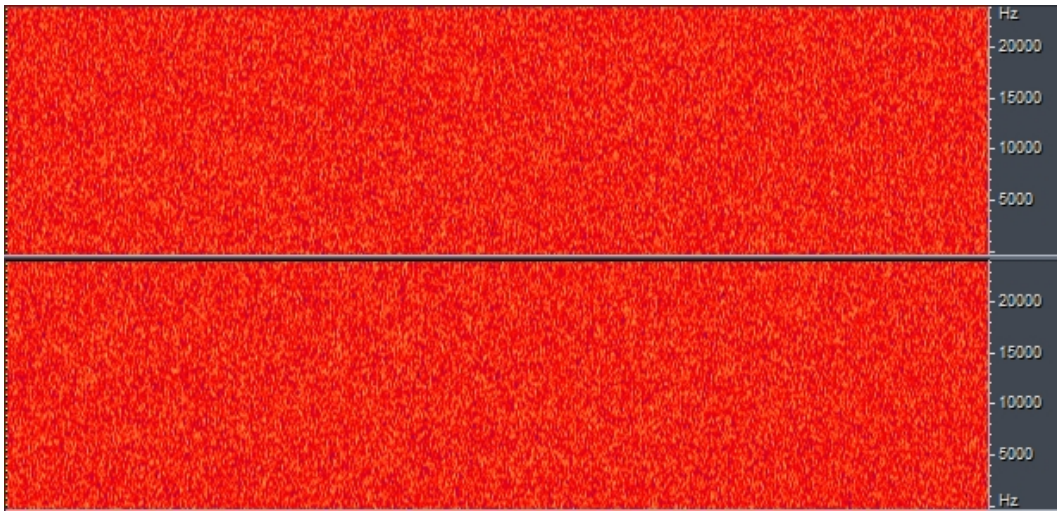


Figure 1.1.1-2 Input white noise spectrum

Channel 1: Parameters are configured in sequence as 1000, 50, 7000, 50, 0, -6； i.e., LP cutoff frequency at 1000 Hz, LP enabled at 50%, HP cutoff frequency at 7000 Hz, HP enabled at 50%, using a first-order general filter, and post-gain at -6 dB.

Channel 2: Parameters are configured in sequence as 1000, 100, 7000, 100, 0, -6； i.e., LP cutoff frequency at 1000 Hz, LP enabled at 100%, HP cutoff frequency at 7000 Hz, HP enabled at 100%, using a first-order general filter, and post-gain at -6 dB.

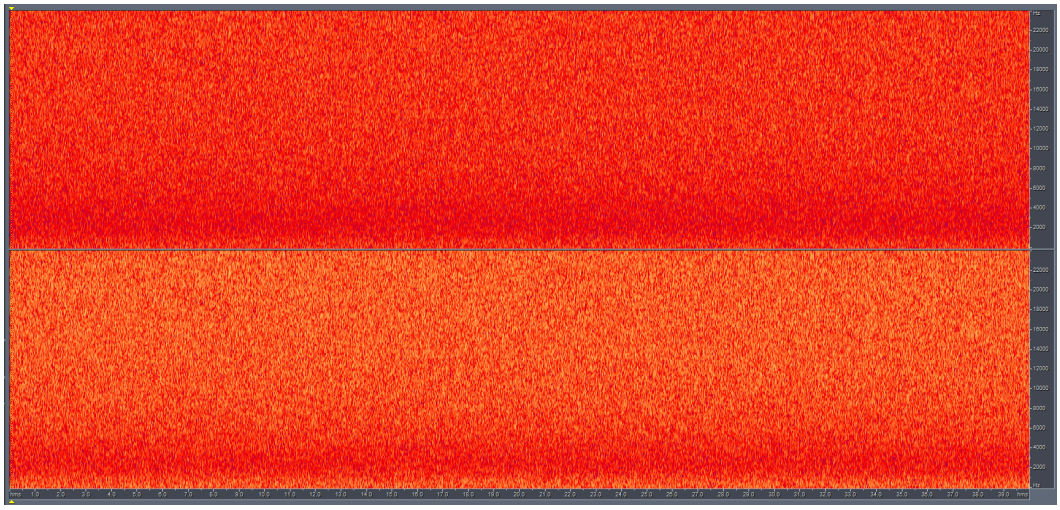


Figure 1.1.1-3 RKStudio output spectrum (above the horizontal line in the image is Channel 1 output, below is Channel 2 output)

1.1.2 Loudness (Low End)

Function: Loudness control, using a low-pass filter to apply gain to low-frequency signals.

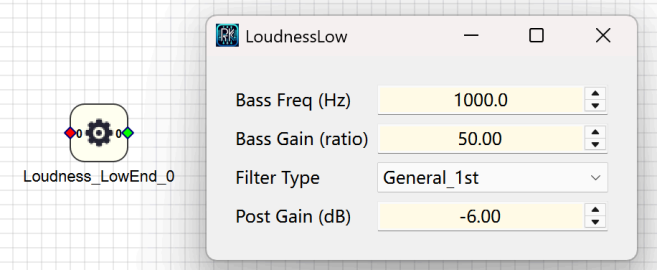


Figure 1.1.2-1 Loudness (Low End) control and settings interface

Number of Parameters: 4

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
BassFreq (HZ)	0	float	Low-pass cutoff frequency	HZ	<1000	1000
BassGain (ratio)*	1	float	Percentage applied to low frequencies (higher values increase low-frequency gain)	%	0~100	50
FilterType	2	int	Filter type		0/1	0
PostGain (dB)	3	float	Gain	dB	0 ~ -60	-6

Note: For the parameter FilterType, the value 0 corresponds to General 1st (first-order general filter), and the value 1 corresponds to Butterworth filter.

1.1.3 Loudness (High End)

Function: Loudness control, using a high-pass filter to apply gain to high-frequency signals.

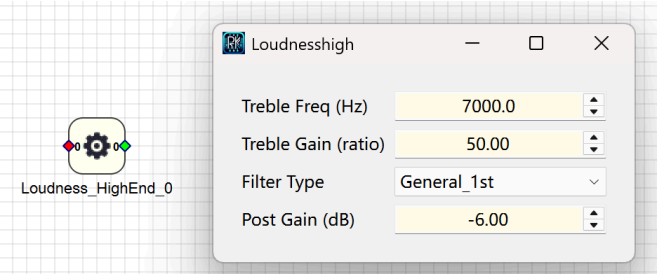


Figure 1.1.3-1 Loudness (High End) control and settings interface

Number of Parameters: 4

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
TrebleFreq (HZ)	0	float	High-pass cutoff frequency	HZ	>7000	7000
TrebleGain (ratio)*	1	float	Percentage applied to high frequencies (higher values increase high-frequency gain)	%	0~100	50
FilterType	2	int	Filter type		0/1	0
PostGain (dB)	3	float	Gain	dB	0 ~ -60	-6

Note: For the parameter FilterType, the value 0 corresponds to General 1st (first-order general filter), and the value 1 corresponds to Butterworth filter.

1.1.4 Loudness (Low & High) External Ctrl

Function: Loudness control, using a combination of low-pass and high-pass filters to apply gain to low-frequency and high-frequency signals respectively. The EXT module externally inputs Post Gain for overall gain reduction, with a range of 0-1, where 0.5 corresponds to -3dB.

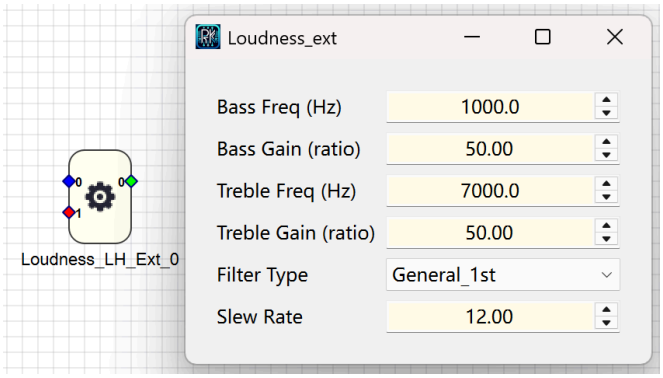


Figure 1.1.4-1 Loudness (Low & High) External Ctrl control and settings interface

Number of Parameters: 6

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
BassFreq (HZ)	0	float	Low-pass cutoff frequency	Hz	<1000	1000
BassGain (ratio)*	1	float	Percentage applied to low frequencies (higher values	%	0 ~ 100	50

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
			increase low-frequency gain)			
TrebleFreq (HZ)	2	float	High-pass cutoff frequency	Hz	>7000	7000
TrebleGain (ratio)*	3	float	Percentage applied to high frequencies (higher values increase high-frequency gain)	%	0 ~ 100	50
FilterType	4	int	Filter type		0/1	0
SlewRate*	5	float	Smoothing rate (higher slew values result in slower changes)		1-23	12

Note: For the parameter FilterType, the value 0 corresponds to General 1st (first-order general filter), and the value 1 corresponds to Butterworth.

This module is similar to the above modules, with the difference being that post_gain is input externally via EXT, so no additional effect description is provided.

1.1.5 Loudness (Low & High) Dynamic

Function:Dynamic loudness control. When the volume is low, it boosts low and high frequencies; when the audio volume is high, it attenuates mid-frequencies, maintaining the overall loudness near the "Keep Level".

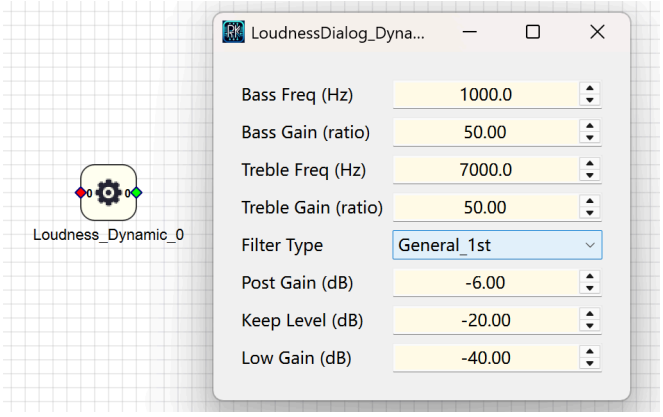


Figure 1.1.5-1 Loudness (Low & High) Dynamic control and settings interface

Number of Parameters:8

Parameter Name	Param Index	Type	Function	Unit	Recommended Value	Default Value
BassFreq (HZ)	0	float	Low-pass cutoff frequency	Hz	<1000	1000
BassGain (ratio)*	1	float	Low-frequency application percentage; larger values indicate greater low-frequency gain	%	0 ~ 100	50
TrebleFreq (HZ)	2	float	High-pass cutoff frequency	Hz	> 7000	7000
TrebleGain (ratio)*	3	float	High-frequency application percentage; larger values indicate greater high-frequency gain	%	0 ~ 100	50
FilterType	4	float	Filter type		0/1	0
PostGain (dB)	5	float	High-level calculation ceiling. Signal levels above this threshold will maintain the current calculated energy	dB	0 ~ -60	-6
KeepLevel (dB)	6	float	Maintenance Level (Activation Threshold). Loudness is attenuated above this level and boosted below this level	dB	-60 ~ 0	-20
Low Gain (dB)	7	float	Low-level calculation floor. Signal levels below this threshold will maintain the current calculated energy	dB	-100 ~ -20	-40

Note: For the FilterType parameter, the value 0 corresponds to a General 1st (first-order general filter), and the value 1 corresponds to a Butterworth filter.

Practical Effect Demonstration (Sine sweep signal time-domain waveform comparison):

Case 1: Low-level sweep input test

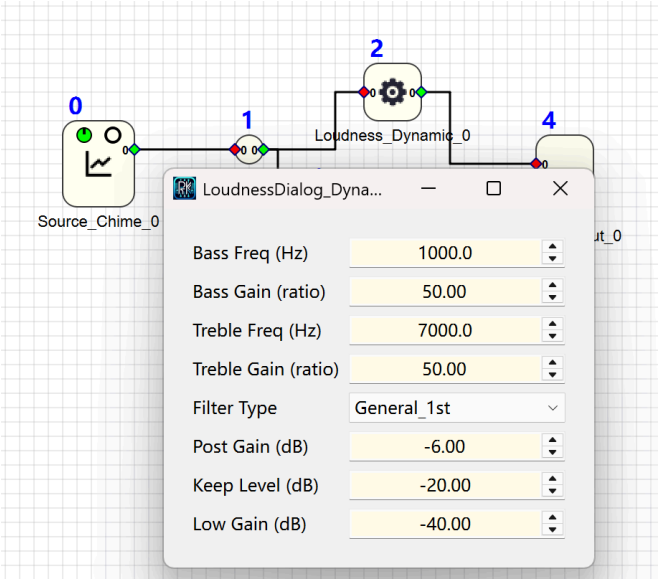


Figure 1.1.5-2 Case 1 Control Parameters

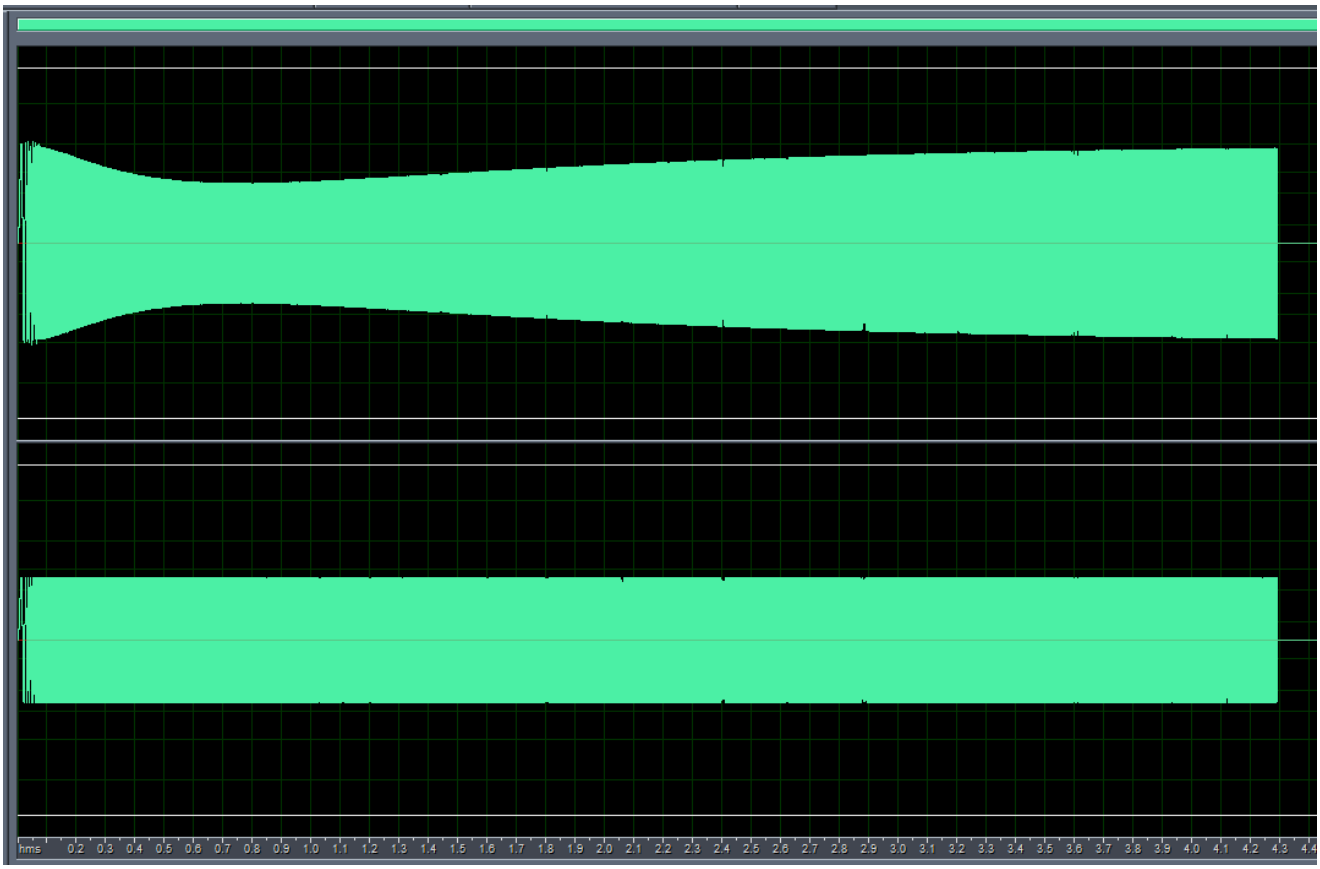


Figure 1.1.5-3 Case 1 Waveform Comparison

Result Analysis: Since the input electric level is lower than the set maintenance threshold (Keep Level), the algorithm activated loudness compensation. As seen in the waveform, the output signal (top trace) shows a full-band gain boost compared to the input signal (bottom trace). Specifically, the regions below 1000Hz and above 7000Hz exhibit a larger magnitude of dynamic enhancement, which aligns with the preset compensation characteristics.

Case 2: High-level sweep input test

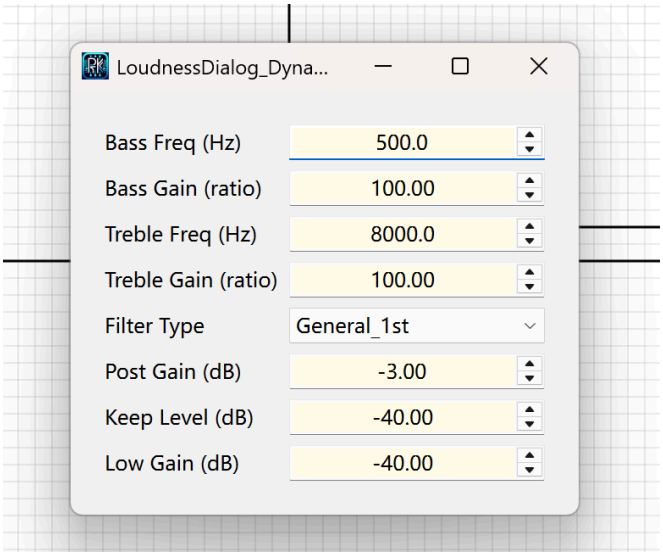


Figure 1.1.5-4 Case 2 Control Parameters

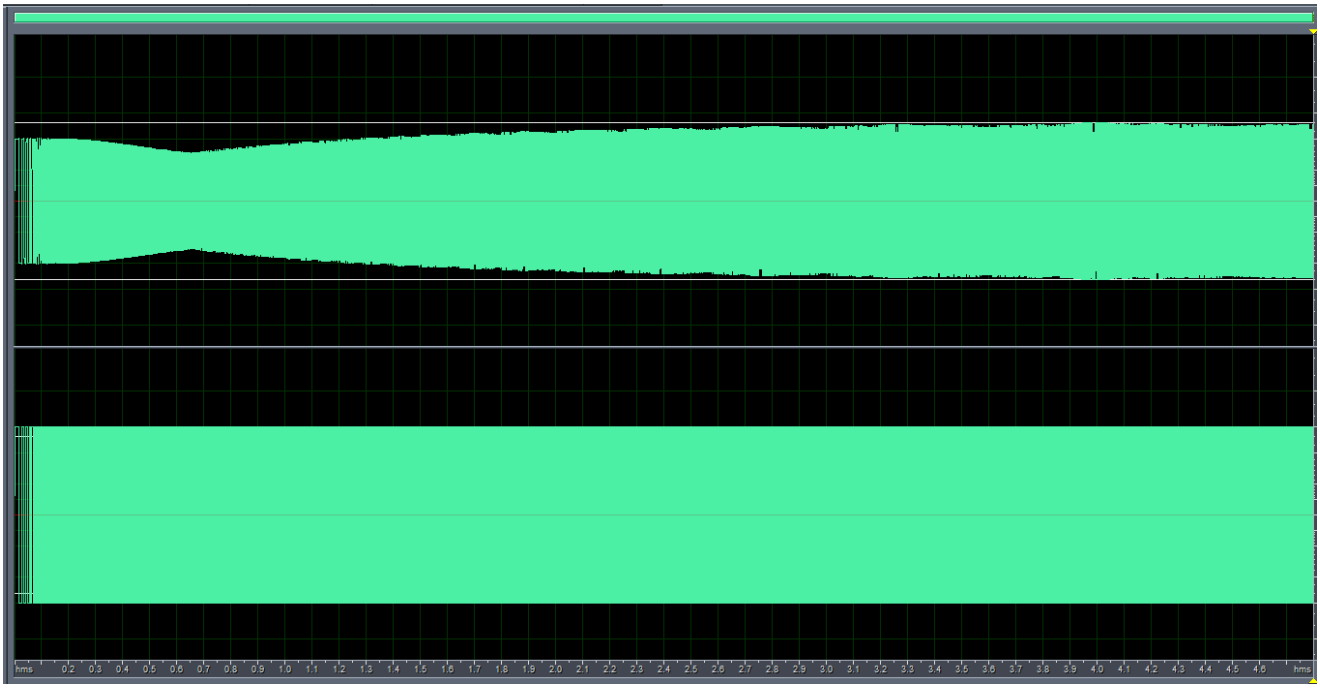


Figure 1.1.5-5 Case 2 Waveform Comparison

Result Analysis: Since the input electric level is higher than the set maintenance threshold , the algorithm determines that the current loudness is sufficient and triggers the "mid-range attenuation" mechanism for high-level conditions. As seen in the waveform, the output signal (top trace) exhibits a significant envelope contraction in the middle frequency range of the sweep compared to the flat input signal (bottom trace), forming a characteristic narrow-waist dynamic envelope. This visually demonstrates that under high dynamic input, the algorithm actively suppresses energy in the ear-sensitive mid-range frequencies to optimize the listening sensation and prevent harshness. This perfectly validates the preset processing logic of "attenuating when above Keep Level."

1.2 Shift

1.2.1 Phat Shift(FIR)

Function: Performs phase shifting in the time domain. This module uses FIR filters to achieve phase shifting. When the filter order is low, performance on low-frequency signals is relatively poor and may generate noise. When the filter order is high, it increases latency and computational load. In this case it is recommended to use Phat Shift (IIR)*.

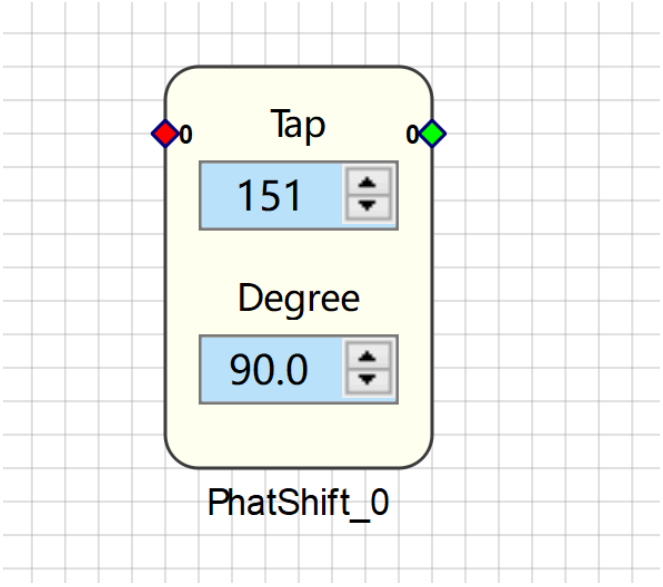


Figure 1.2.1-1 Phat Shift control and settings interface

Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Tap	0	int	Phase shift filter length	None	151	151
Degree	1	float	Phase shift angle	Degrees	0 ~ 359	90

Demonstration of actual effect:
Input a 1kHz sine wave source to obtain the output. With parameter settings of 151, 0 and 151, 90, the output will have a phase difference of 90 degrees:

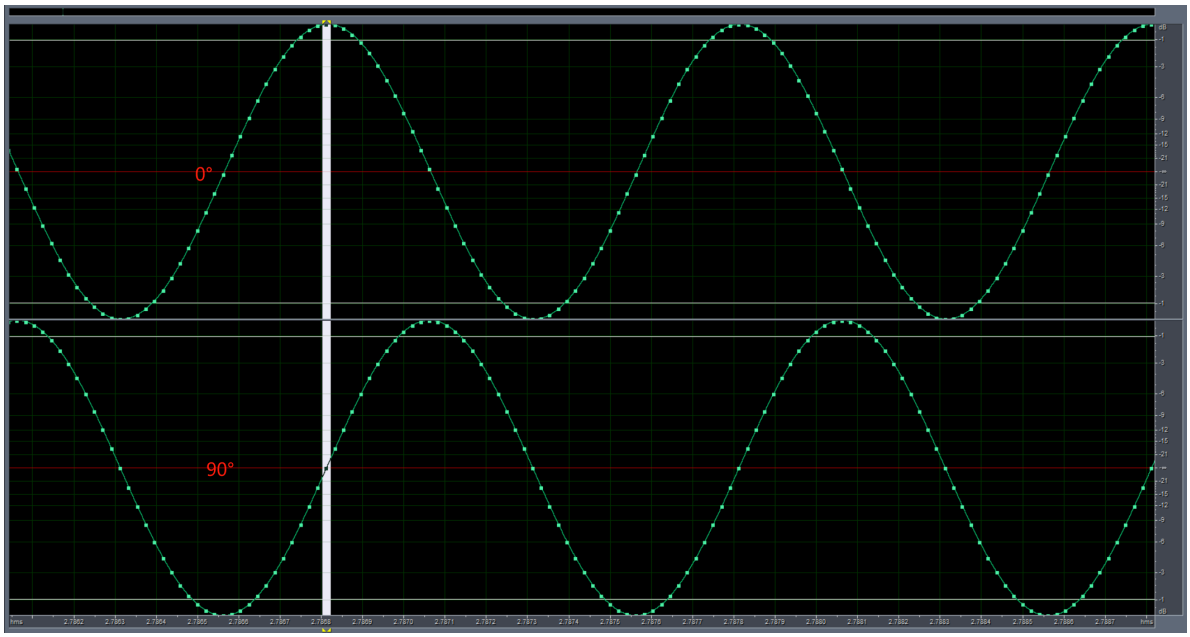


Figure 1.2.1 Sine wave time-domain phase shift of 0° (Channel 1) and 90° (Channel 2) output

1.2.2 Phat Shift (IIR)

Function: Performs phase shifting using an FIR filter.

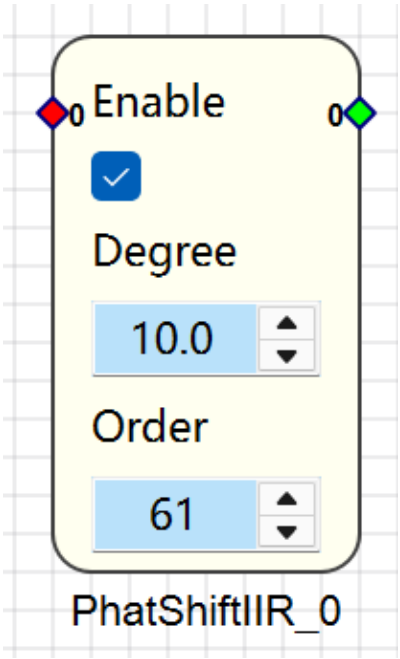


Figure 1.2.2-1 Phat Shift (IIR) control and settings interface

Number of Parameters:2

Parameter Name	Param Index	Type	Function	Unit	Recommended Value	Default Value
Degree	0	float	Phase shift angle	Degree	0 ~ 359	90

Parameter Name	Param Index	Type	Function	Unit	Recommended Value	Default Value
Order	1	float	Phase shift filter order	None	5 ~ 61	61

1.2.3 Frequency Shift（FIR)

Function: Performs frequency shifting in the frequency domain. This module uses FIR filters to achieve phase shifting. When the filter order is low, performance on low-frequency signals is relatively poor and may generate noise. When the filter order is high, it increases latency and computational load. In this case it is recommended to use Frequency Shift (IIR)*.

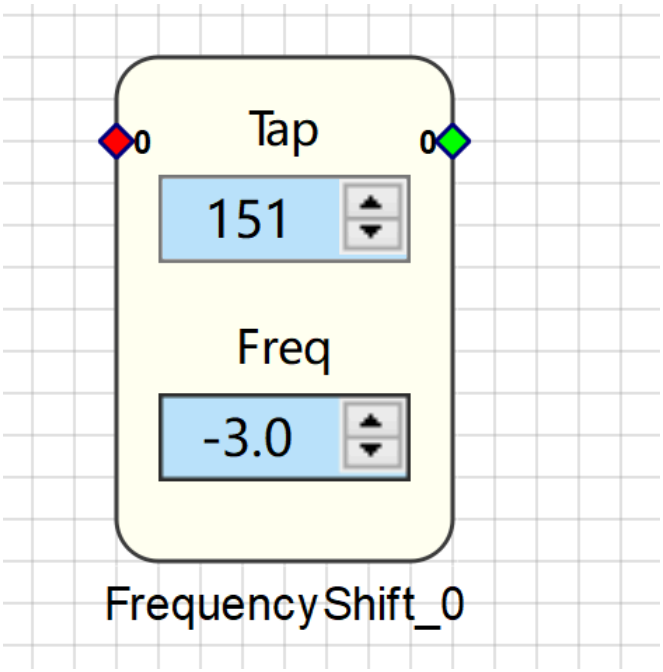


Figure 1.2.3-1 Frequency Shift control and settings interface

Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Tap	0	float	Frequency shift filter length	None	151	151
Freq	1	float	Frequency shift	Hz	-10 ~ 10	-3

Demonstration of actual effect:

Input a 1kHz sine wave source to obtain the output. With parameter settings of 151, 0 and 151, 10, the output will have a frequency difference of 10 Hz:

The time-domain results:

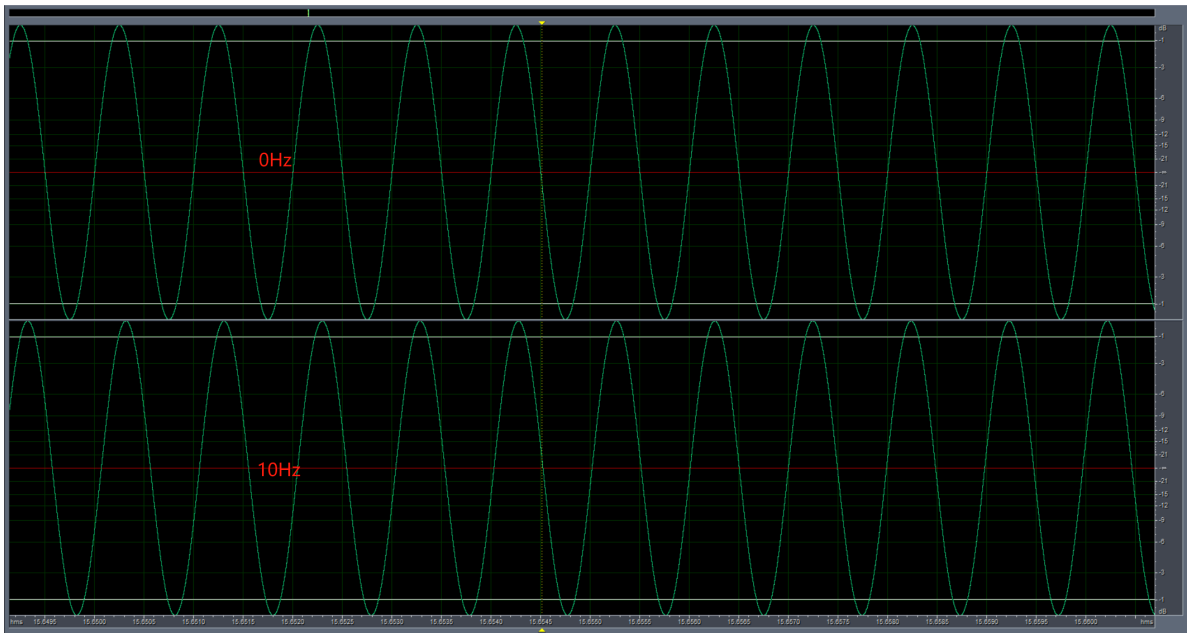


Figure 1.2.3-2 Sine wave frequency-domain frequency shift of 0Hz (Channel 1) and 10Hz (Channel 2) output

The corresponding frequency analysis results are shown in the figure below:

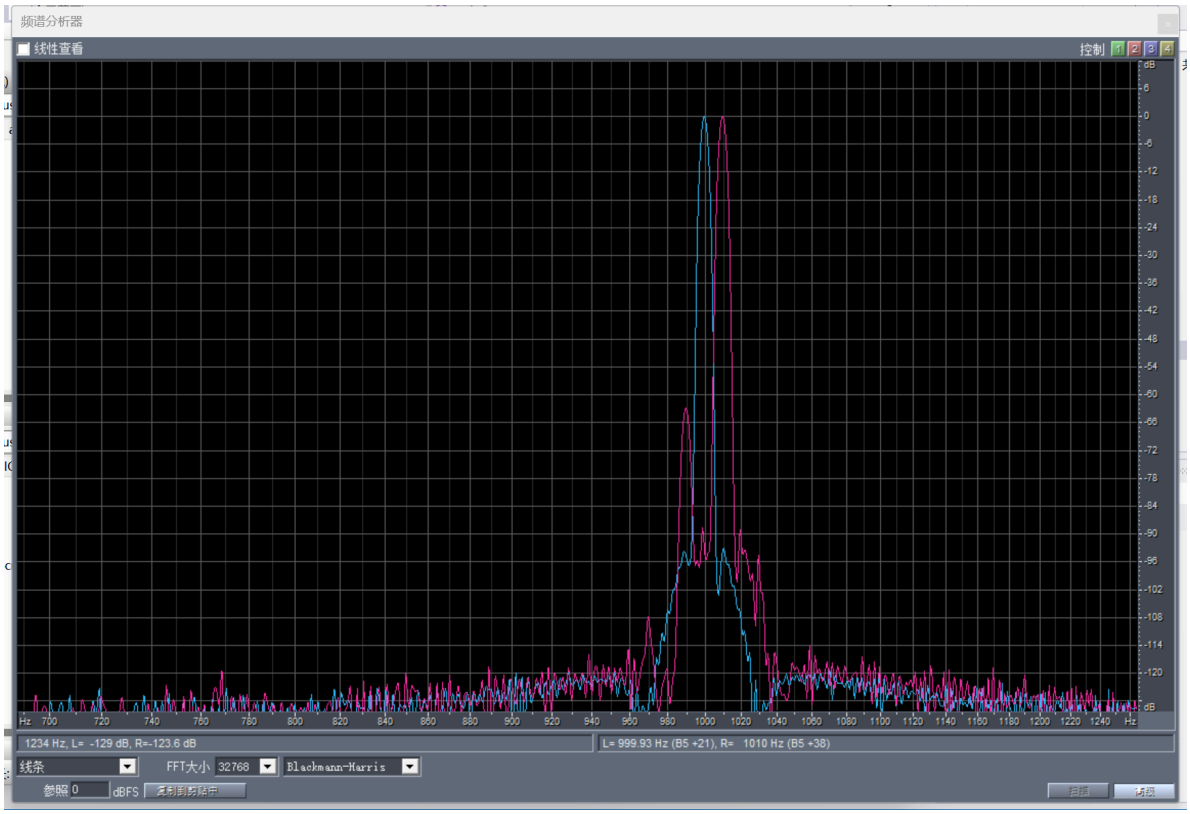


Figure 1.2.3-3 Sine wave frequency-domain frequency shift of 0Hz (blue line) and 10Hz (purple line) frequency spectrum analysis

1.2.4 Frequency Shift (IIR)

Function：Performs frequency shifting using IIR filters.

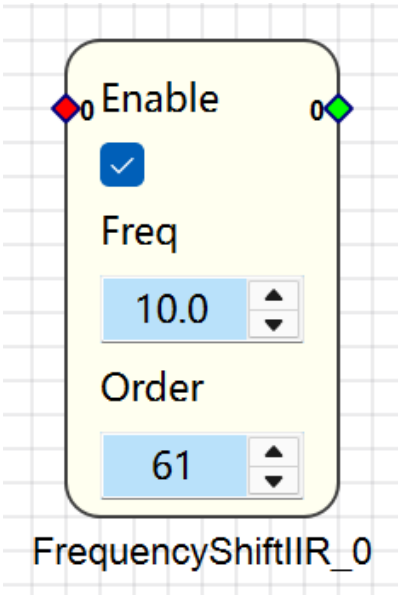


Figure 1.2.4-1 Frequency Shift (IIR) control and settings interface

Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Freq	0	float	Frequency to shift	Hz	-10 ~ 10	-3
Tap	1	float	Frequency shifting filter order	None	5~61	61

1.3 Surround Sound

Function: Traditional surround sound module, supports upmixing stereo to 5 channels. The 5 output channels are FL (front left), FR (front right), RL (rear left surround), RR (rear right surround), and C (center subwoofer).

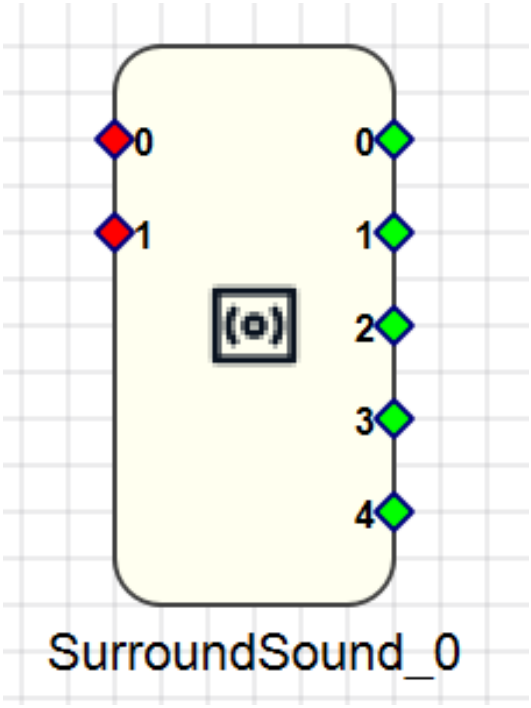


Figure 1.3-1 Surround Sound control

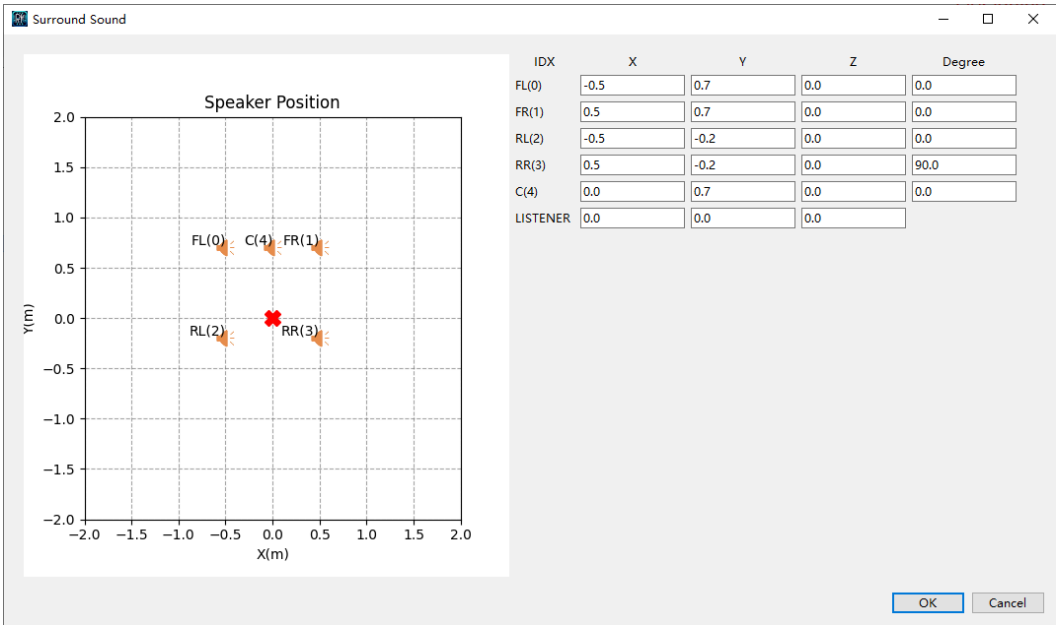


Figure 1.3-2 Surround Sound parameter interface

Number of Parameters: 3 + 4*5 = 23

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
X(listener)	0	float	X-axis coordinate of the listener position	meters	None	0

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Y(listener)	1	float	Y-axis coordinate of the listener position	meters	None	0
Z(listener)	2	float	Z-axis coordinate of the listener position	meters	None	0
X(speaker)	1*n+2	float	X-axis coordinate of the speaker layout	meters	None	As shown in the figure
Y(speaker)	2*n+2	float	Y-axis coordinate of the speaker layout	meters	None	As shown in the figure
Z(speaker)	3*n+2	float	Z-axis coordinate of the speaker layout	meters	None	As shown in the figure
Degree(speaker)	4*n+2	float	Output phase shift angle	Degrees	None	0

1.4 Pitch Shifting

1.4.1 Phase Vocoder

Function: Implements pitch shifting based on a phase vocoder. This module can be used for pitch shifting of both human voice and music. It has relatively high requirements for frame length, preferably above 384.

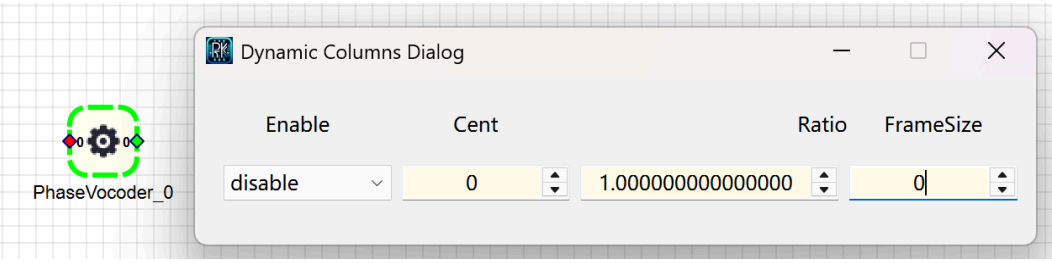


Figure 1.4-1 Phase Vocoder control and settings interface

Number of parameters: 4

Parameter Name	Param Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	float	Enable/Disable	None	0/1	0
Cent	1	float	Pitch shift amount (Unit: cents)	Cent	-1200 ~ 1200	0
Ratio	2	float	Pitch shift coefficient: <1 lowers pitch, >1 raises pitch		0.5 ~ 2	1
FrameSize	3	int	Internal processing frame size. Set to 0 to follow system size; set > 0 to enable internal buffering.	Samples	512 / 1024	0

Note 1: Cent and Ratio are essentially the same parameter represented in different units. They satisfy the relationship: Ratio = 2^(Cent/1200).

Note 2 on FrameSize (Internal Buffering Strategy):
The Phase Vocoder algorithm requires high frequency resolution to perform accurate pitch shifting. It typically yields optimal audio quality with larger frame sizes (recommended 384 samples or above).

- FrameSize = 0 (Default / Pass-through Mode):
  The module processes audio using the system's Global Frame Length.
- FrameSize > 0 (Buffering Mode):
  The module enables an internal buffering mechanism. Regardless of the system's input block size, the module accumulates incoming samples until the buffer reaches the specified FrameSize (e.g., 512 samples) before performing the algorithm.
  - Advantage: Allows for high-quality, distortion-free processing even when the host system is running at a very low frame length (e.g., 64 samples).
  - Trade-off: Introduces additional processing latency.(Latency Calculation: Total Latency = FrameSize - SystemFrameLength).

1.4.2 Phase Vocoder (Formant)

Function: Based on the Phase Vocoder, this adds formant preservation/protection functionality.

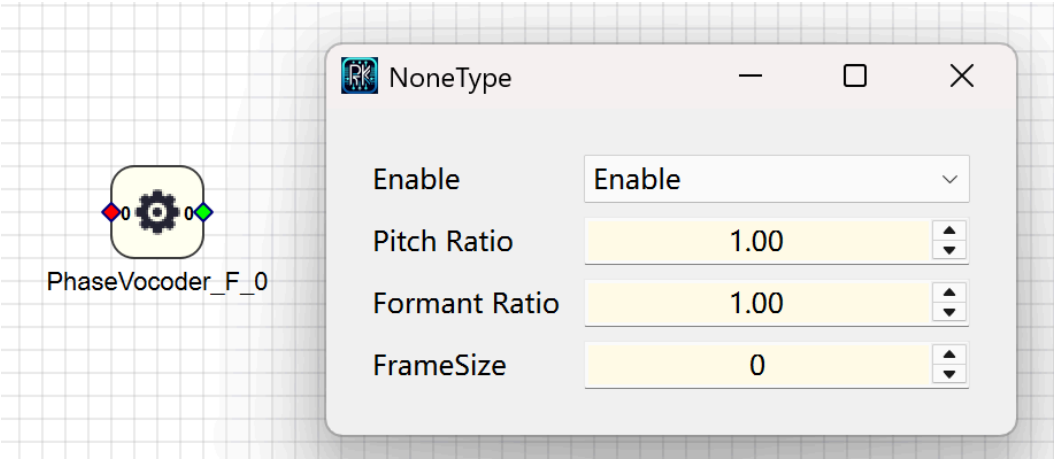


Figure 1.4-2 Phase Vocoder (Formant) control and settings interface

Number of Parameters: 4

Parameter Name	Param Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	float	Enable/Disable	None	0/1	0
Pitch Ratio	1	float	Pitch shift coefficient: <1 lowers pitch, >1 raises pitch	None	0.5 ~ 2	1
Formant Ratio	2	float	Formant envelope modification coefficient	None	0.5 ~ 2	1
FrameSize	3	int	Internal processing frame size. Set to 0 to follow system size; set > 0 to enable internal buffering.	Samples	512 / 1024	0

Note: In pure Phase Vocoder pitch shifting, the spectral envelope (i.e., formant frequencies) is shifted proportionally with the pitch ratio, leading to a "chipmunk" (high pitch) or "monster" (low pitch) vocal sound. The Formant Ratio parameter allows independent control of the spectral envelope's scaling/displacement, decoupling "pitch" from "timbre" (formant frequencies).

Formant Ratio = 1.0 (Recommended Default): Preserves the original formant frequencies as much as possible, thus protecting the timbre; it only changes the fundamental frequency/harmonics, achieving "natural" pitch shifting.

Formant Ratio ≈ Pitch Ratio: The formants shift synchronously with the pitch, equivalent to not preserving the formants. The resulting sound is similar to traditional Phase Vocoder.

Formant Ratio > 1: Shifts formants upward (simulating smaller vocal tract / brighter tone).

Formant Ratio < 1: Shifts formants downward (simulating larger vocal tract / darker tone).

1.4.3 WSOLA

Function: Implements pitch shifting using WSOLA . This module is suitable for voice pitch shifting and requires a slightly lower frame length, preferably above 192.

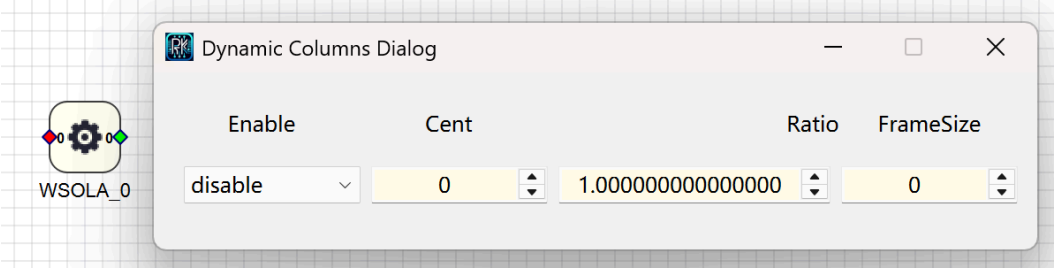


Figure 1.4-3 WSOLA control and settings interface

Number of Parameters：4

Parameter Name	Param Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	float	Enable/Disable	None	0/1	0
Cent	1	float	Pitch shift amount (Unit: cents)	Cent	-1200 ~ 1200	0
Ratio	2	float	Pitch shift coefficient: <1 lowers pitch, >1 raises pitch	None	0.5 ~ 2	1
FrameSize	3	int	Internal processing frame size. Set to 0 to follow system size; set > 0 to enable internal buffering.	Samples	512 / 1024	0

Note 1: Cent and Ratio are essentially the same parameter represented in different units. They satisfy the relationship: Ratio = 2^(Cent/1200).

Note 2 on FrameSize (Internal Buffering Strategy):
The Phase Vocoder algorithm requires high frequency resolution to perform accurate pitch shifting. It typically yields optimal audio quality with larger frame sizes (recommended 384 samples or above).

- FrameSize = 0 (Default / Pass-through Mode):
  The module processes audio using the system's Global Frame Length.
- FrameSize > 0 (Buffering Mode):
  The module enables an internal buffering mechanism. Regardless of the system's input block size, the module accumulates incoming samples until the buffer reaches the specified FrameSize (e.g., 512 samples) before performing the algorithm.
  - Advantage: Allows for high-quality, distortion-free processing even when the host system is running at a very low frame length (e.g., 64 samples).

- Trade-off: Introduces additional processing latency.(Latency Calculation: Total Latency = FrameSize - SystemFrameLength).

1.4.4 WSOLA (Formant)

Function: Adds formant preservation to WSOLA.

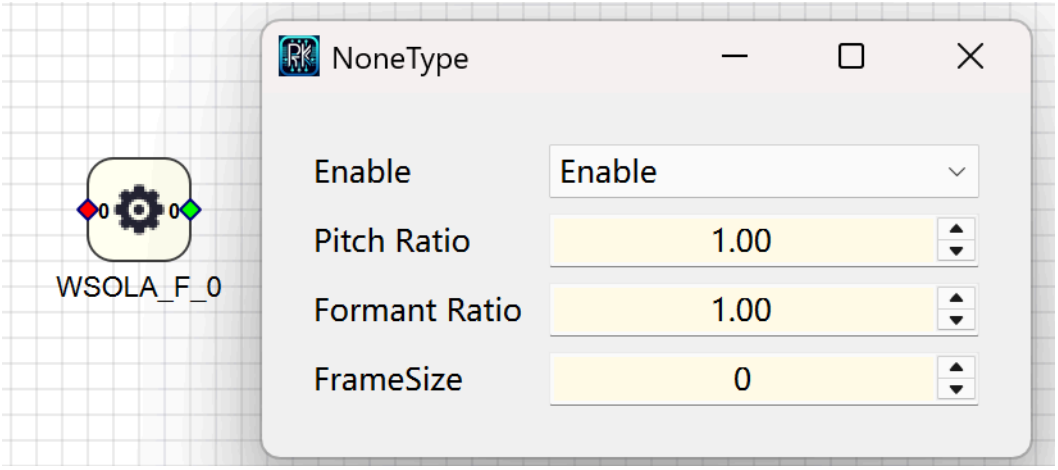


Figure 1.4-4 WSOLA (Formant) control and settings interface

Number of Parameters：4

Parameter Name	Param Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	float	Enable/Disable	None	0/1	0
Pitch Ratio	1	float	Pitch shift coefficient: <1 lowers pitch, >1 raises pitch	None	0.5 ~ 2	1
Formant Ratio	2	float	Formant envelope modification coefficient	None	0.5 ~ 2	1
FrameSize	3	int	Internal processing frame size. Set to 0 to follow system size; set > 0 to enable internal buffering.	Samples	512 / 1024	0

1.5 Phat Stereo

Function: Enhances stereo effect, making the audio sound wider, fuller, and more spacious.

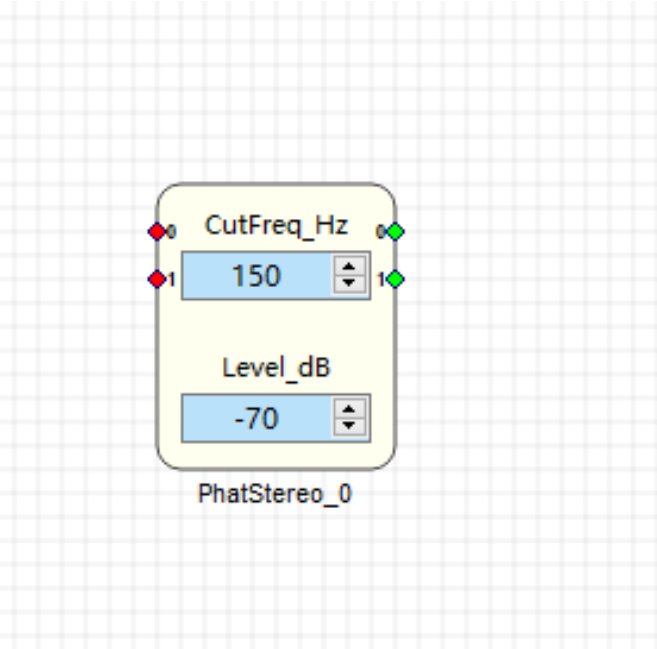


Figure 1.5-1 Phat Stereo control

Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
CutFreq_Hz	0	int	Cutoff frequency of the low-pass filter	Hz	20 ~ 20000	150
Level_dB	1	int	Control the output level of the audio	dB	-96 ~ 6	-70

1.6 DTS

1.6.1 DTS:X

DTS:X is primarily used for DTS certification.

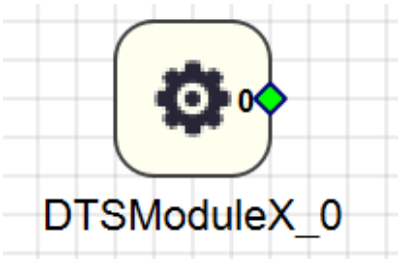


Figure 1.6.1-1 DTS:X control

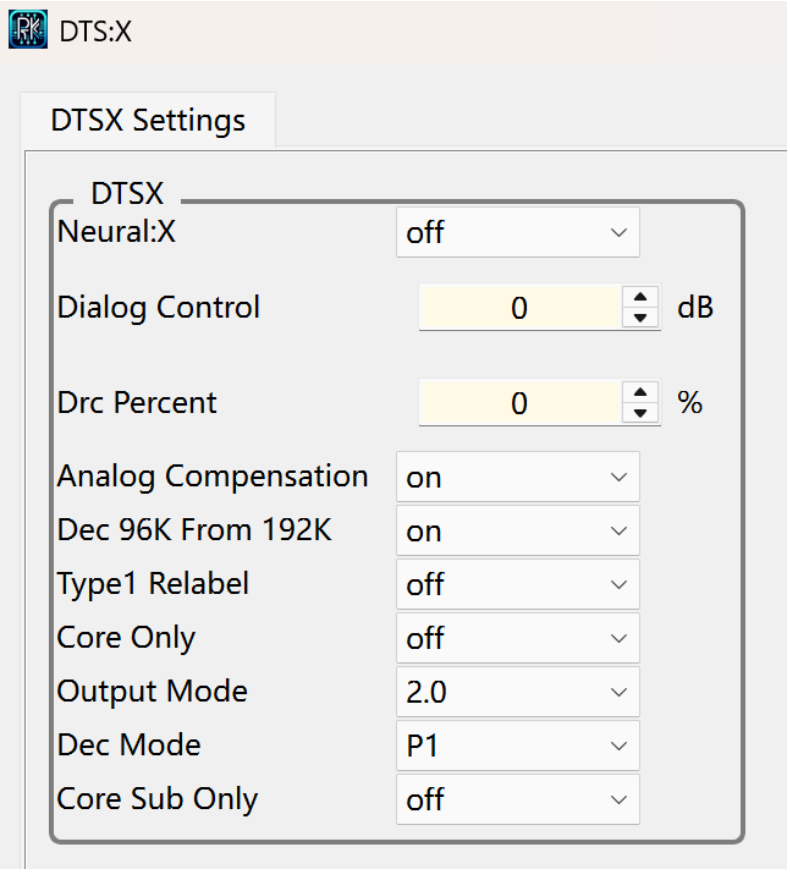


Figure 1.6.1-2 DTS:X parameter interface

This control involves copyright licensing. Please contact business/development personnel for details.

1.6.2 DTS Virtual:X

DTS Virtual:X is primarily used for DTS post-processing.

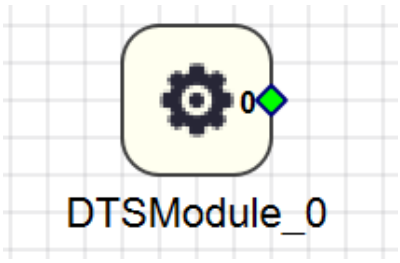


Figure 1.6.2-1 DTS Virtual:X control

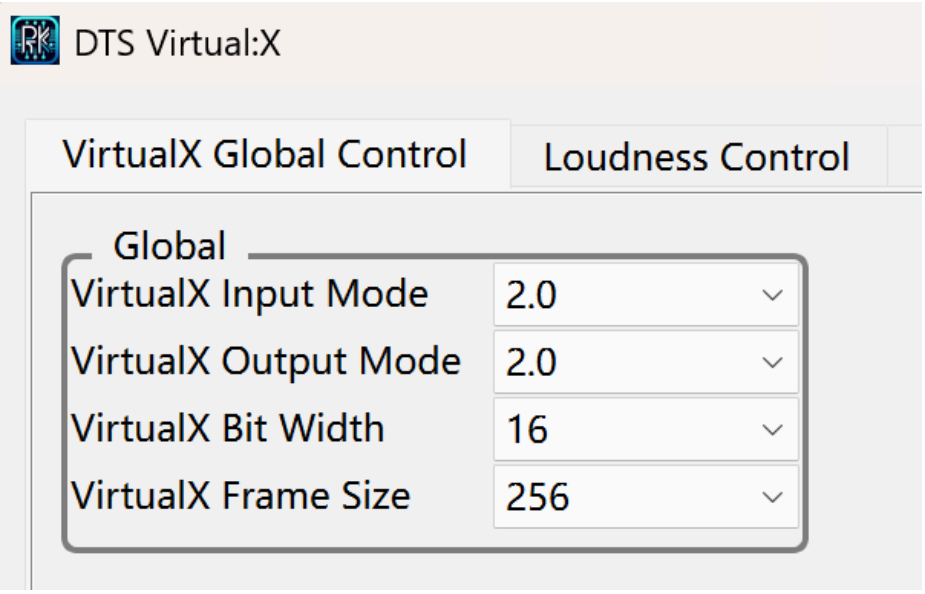


Figure 1.6.2-2 DTS Virtual:X parameter interface

This control involves copyright licensing. Please contact business/development personnel for details.

1.7 Dolby

The Dolby plugin is primarily used for Dolby certification.

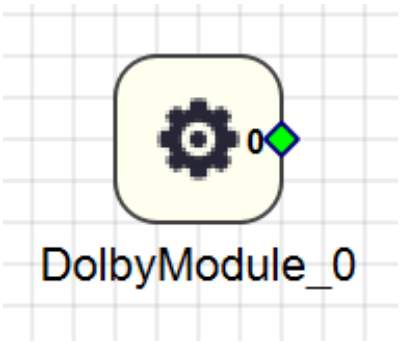


Figure 1.7-1 Dolby control

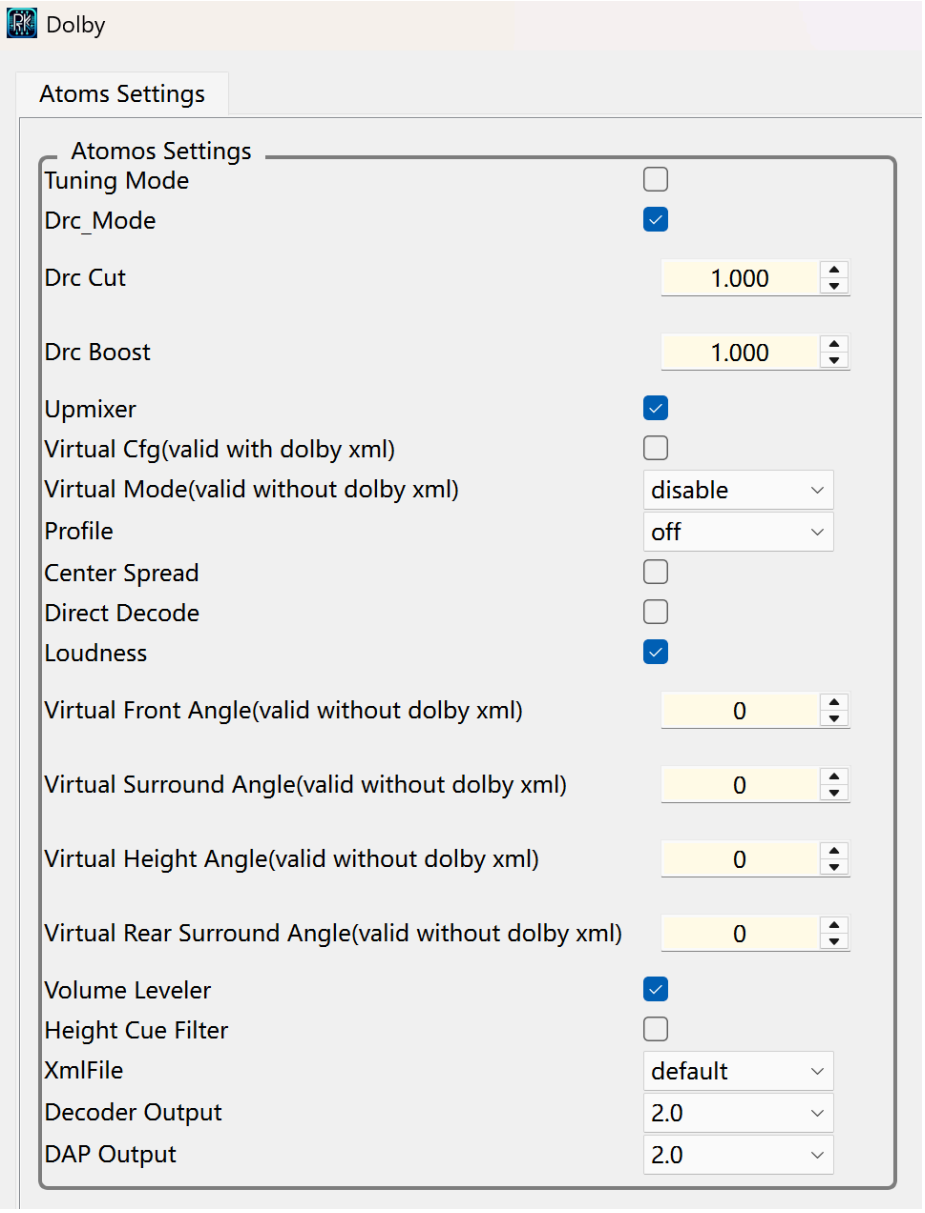


Figure 1.7-2 Dolby parameter interface

This control involves copyright licensing. Please contact business/development personnel for details.

1.8 BF_ECNR

BF_ECNR (Beamforming, Echo Cancellation, and Noise Reduction) is used for voice noise reduction and echo cancellation. It requires manual configuration of the number of channels, sampling rate, and frame length.

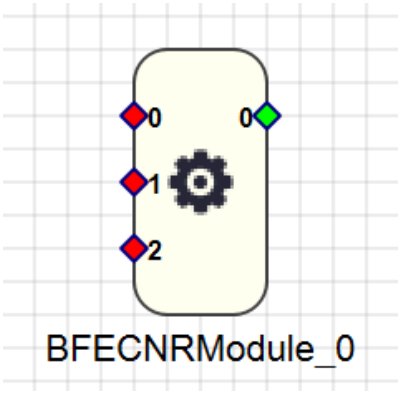


Figure 1.8-1 BF_ECNR control

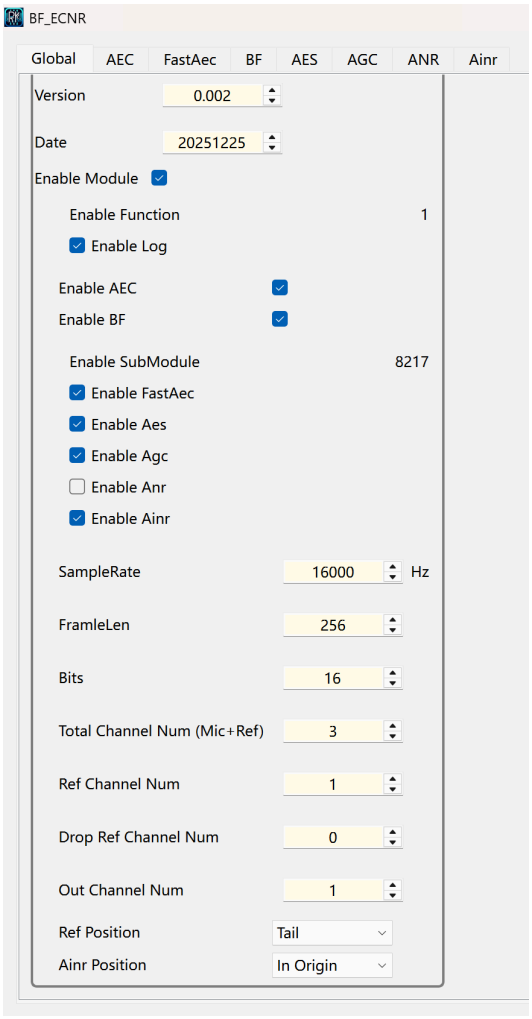


Figure 1.8-2 BF_ECNR parameter interface

> **Total Input Channel Num = Mic Num + Ref Num**
> **Total Channel Num = Mic Num + Ref Num**

(1) Total Channel Num (Mic+Ref) = 3

- Description: Indicates that this algorithm module requires a total of 3 input signal channels.
- These 3 channels consist of "microphone recording data" and "reference echo data". The specific distribution depends on the parameters below.

(2) Ref Channel Num = 1

- Description: Among the 3 total inputs above, 1 channel is the reference signal. Since the total is 3 and the reference is 1, the microphone count (Mic Num) = 3 - 1 = 2.
- Conclusion: This is a configuration of Dual Microphones (2 Mic) + Single Reference (1 Ref).

(3) Out Channel Num = 1

- Description: After algorithm processing (Noise Reduction, Echo Cancellation, Beamforming), it finally outputs 1 channel of clean audio.

This control is detailed in another document:
Rockchip_Developer_Guide_Audio_Algorithm_VQE_Introduction_EN.pdf

1.9 AVAS

1.9.1 AVAS

Function: Acoustic Vehicle Alerting System (low-speed driving warning sound). The signal source uses files from the wav partition as input.

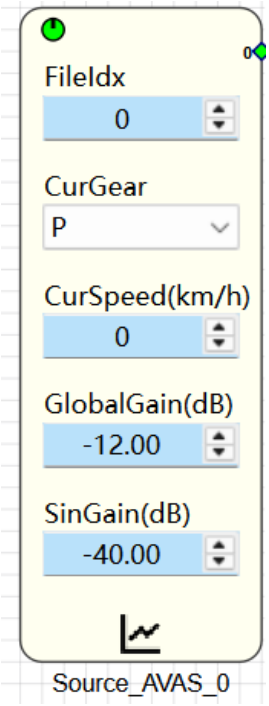
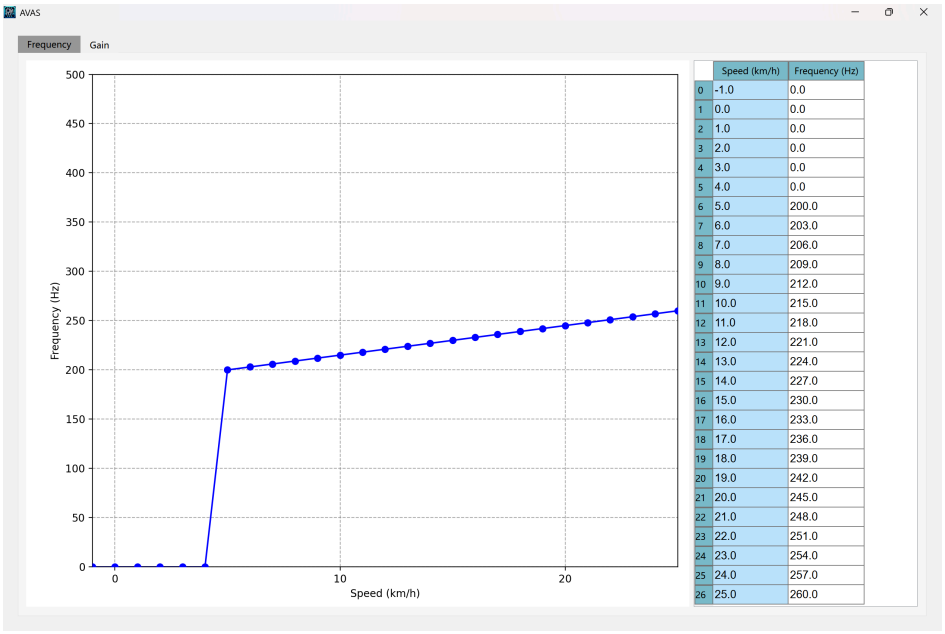
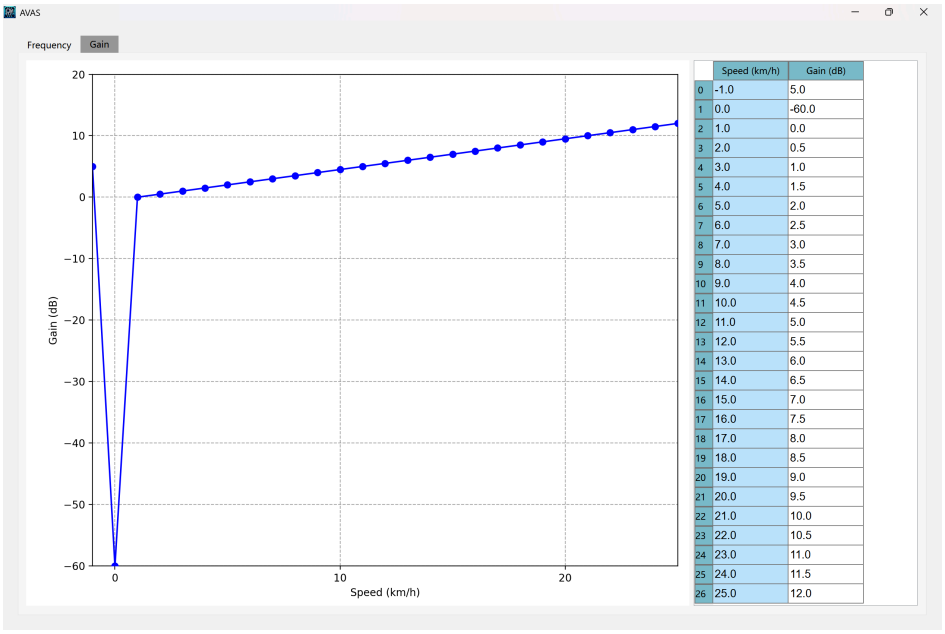


Figure 1.9.1-1 AVAS control



	Speed (km/h)	Frequency (Hz)
0	-1.0	0.0
1	0.0	0.0
2	1.0	0.0
3	2.0	0.0
4	3.0	0.0
5	4.0	0.0
6	5.0	200.0
7	6.0	203.0
8	7.0	206.0
9	8.0	209.0
10	9.0	212.0
11	10.0	215.0
12	11.0	218.0
13	12.0	221.0
14	13.0	224.0
15	14.0	227.0
16	15.0	230.0
17	16.0	233.0
18	17.0	236.0
19	18.0	239.0
20	19.0	242.0
21	20.0	245.0
22	21.0	248.0
23	22.0	251.0
24	23.0	254.0
25	24.0	257.0
26	25.0	260.0

Figure 1.9.1-2 AVAS Frequency Settings Interface



	Speed (km/h)	Gain (dB)
0	-1.0	5.0
1	0.0	-60.0
2	1.0	0.0
3	2.0	0.5
4	3.0	1.0
5	4.0	1.5
6	5.0	2.0
7	6.0	2.5
8	7.0	3.0
9	8.0	3.5
10	9.0	4.0
11	10.0	4.5
12	11.0	5.0
13	12.0	5.5
14	13.0	6.0
15	14.0	6.5
16	15.0	7.0
17	16.0	7.5
18	17.0	8.0
19	18.0	8.5
20	19.0	9.0
21	20.0	9.5
22	21.0	10.0
23	22.0	10.5
24	23.0	11.0
25	24.0	11.5
26	25.0	12.0

Figure 1.9.1-3 AVAS Gain Settings Interface

Number of Parameters：5

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
FileIdx	0	float	Audio file ID	None	0 ~ 127	0
CurGear	1	float	Current gear selection	None	0 ~ 3	0
CurSpeed	2	float	Current vehicle	km/h	0 ~ 200	0

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
			speed			
GlobalGain	3	float	Global gain	dB	-100 ~ 100	-12
SinGain	4	float	Signal gain	dB	-100 ~ 100	-40

1.9.2 AVAS External Source

Function: Acoustic Vehicle Alerting System (Low-speed vehicle warning sound). The function is the same as AVAS*, but the input source uses an input signal.

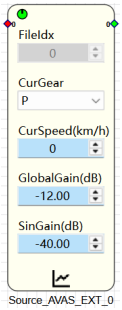


Figure 1.9.2-1 AVAS_External Source Control

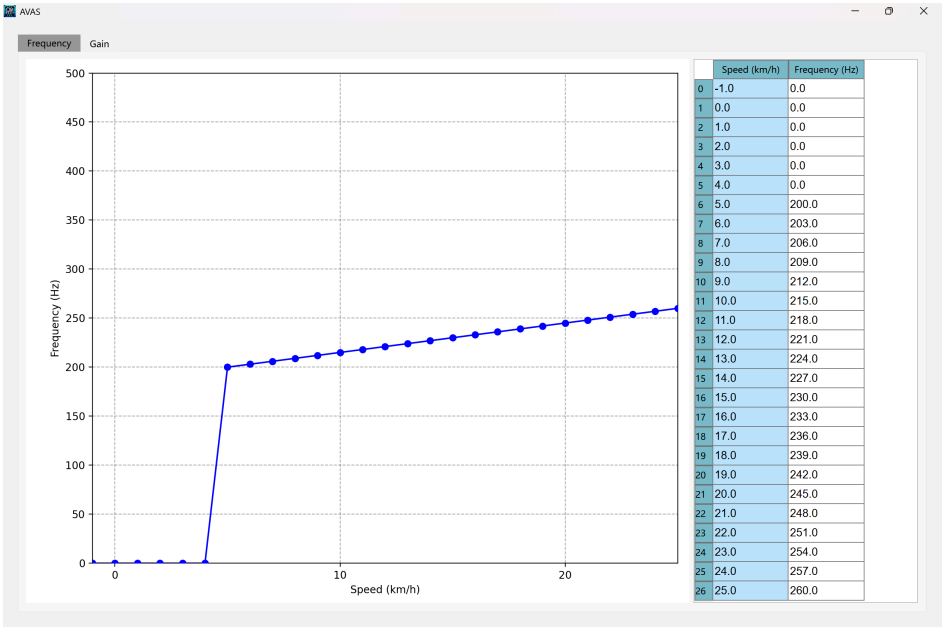
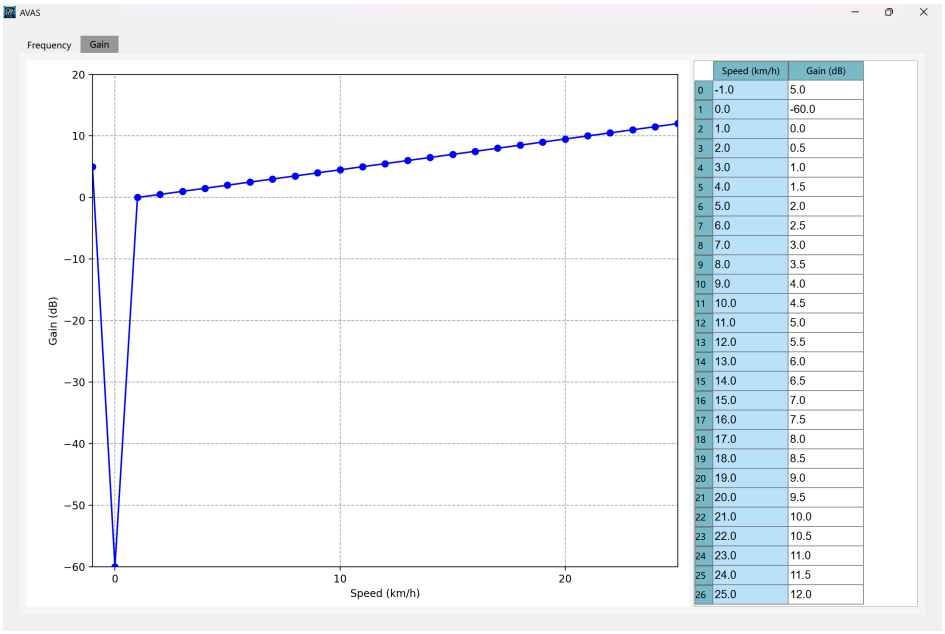


Figure 1.9.2-2 AVAS_External Source Settings Interface



	Speed (km/h)	Gain (dB)
0	-1.0	5.0
1	0.0	-60.0
2	1.0	0.0
3	2.0	0.5
4	3.0	1.0
5	4.0	1.5
6	5.0	2.0
7	6.0	2.5
8	7.0	3.0
9	8.0	3.5
10	9.0	4.0
11	10.0	4.5
12	11.0	5.0
13	12.0	5.5
14	13.0	6.0
15	14.0	6.5
16	15.0	7.0
17	16.0	7.5
18	17.0	8.0
19	18.0	8.5
20	19.0	9.0
21	20.0	9.5
22	21.0	10.0
23	22.0	10.5
24	23.0	11.0
25	24.0	11.5
26	25.0	12.0

Figure 1.9.2-3 AVAS_External Source Settings Interface

1.9.3 AVAS Standard Compliance and Configuration Specification

The parameter configuration of this module aims to satisfy the **GB/T 37153-2018 "Acoustic Vehicle Alerting System of Electric Vehicles Running at Low Speed"** mandatory standard. The specific configuration logic is as follows:

1、Frequency Response (Frequency Table)
The frequency must be configured to increase linearly with vehicle speed (0-20 km/h) to meet the standard's defined "frequency shift" requirement (frequency shift rate ≥ 0.8% /(km/h)), thereby simulating vehicle acceleration characteristics.

2、Gain Control (Gain Table)
It is necessary to ensure that the output sound pressure level in the low-speed range (e.g., 10 km/h, 20 km/h) reaches the national standard minimum limit (usually ≥ 50 dB), and attenuates after exceeding the cutoff speed.

3、Gear Logic (CurGear)
When the system identifies the R gear, regardless of the vehicle speed, it must continuously output the reverse warning sound. This corresponds to the parameter configuration for speed = -1.

1.10 Voice Separators

Uses traditional coherence methods to separate human voice from background sound.

1.10.1 Voice Separator (MS)

Function: A voice separation module based on frequency-domain cross-correlation. It separates a stereo signal into a Mid channel (M) and a Side channel (S).

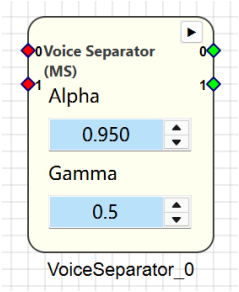


Figure 1.10.1-1 Voice Separator (MS) control and settings interface

Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Alpha	0	float	Smoothing coefficient	None	0.5-0.999	0.95
Gamma	1	float	Curve mapping coefficient	None	0.5 ~ 2	1

- Note 1: Alpha affects the smoothing degree of the internally calculated frequency-domain autocorrelation coefficients between different frames. A larger value results in a more noticeable smoothing effect but a slower response to sudden sounds. This parameter should be modified primarily based on the frame length; the longer the frame length, the smaller the recommended Alpha value.

- Note 2: Gamma affects the non-linear mapping curve between the frequency-domain autocorrelation coefficient and the retention level of the M channel. A smaller value means that even small autocorrelation coefficients will result in greater retention, making the M channel sound more complete. A smaller value also makes the M channel sound cleaner, but may result in the loss of some sound information. This parameter should be adjusted based on actual listening perception.

1.10.2 Voice Separator (LRM)

Function: A voice separation module based on frequency-domain cross-correlation. It separates a stereo signal into a Mid channel (M), a Left channel (L), and a Right channel (R).

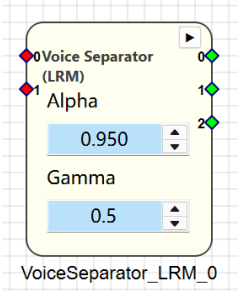


Figure 1.10.2-1 Voice Separator (LRM) control and settings interface

Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Alpha	0	float	Smoothing coefficient	None	0.5-0.999	0.95
Gamma	1	float	Curve mapping coefficient	None	0.5 ~ 2	1

- Note 1: Alpha affects the smoothing degree of the internally calculated frequency-domain autocorrelation coefficients between different frames. A larger value results in a more noticeable smoothing effect but a slower response to sudden sounds. This parameter should be modified primarily based on the frame length; the longer the frame length, the smaller the recommended Alpha value.

- Note 2: Gamma affects the non-linear mapping curve between the frequency-domain autocorrelation coefficient and the retention level of the M channel. A smaller value means that even small autocorrelation coefficients will result in greater retention, making the M channel sound more complete. A smaller value also makes the M channel sound cleaner, but may result in the loss of some sound information. This parameter should be adjusted based on actual listening perception.

1.11 CrossTalk Suppressor

Function: Used to eliminate the interference of dual-channel audio on both ears (crosstalk), thereby expanding the sound field.

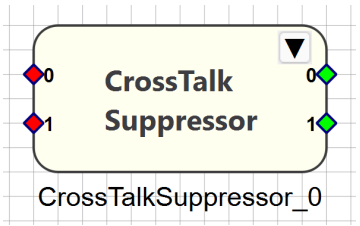


Figure 1.11-1 CrossTalk Suppressor Control

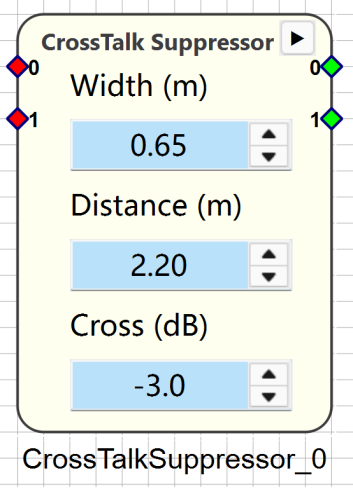


Figure 1.11-2 CrossTalk Suppressor Parameter Interface

2 Basic DSP

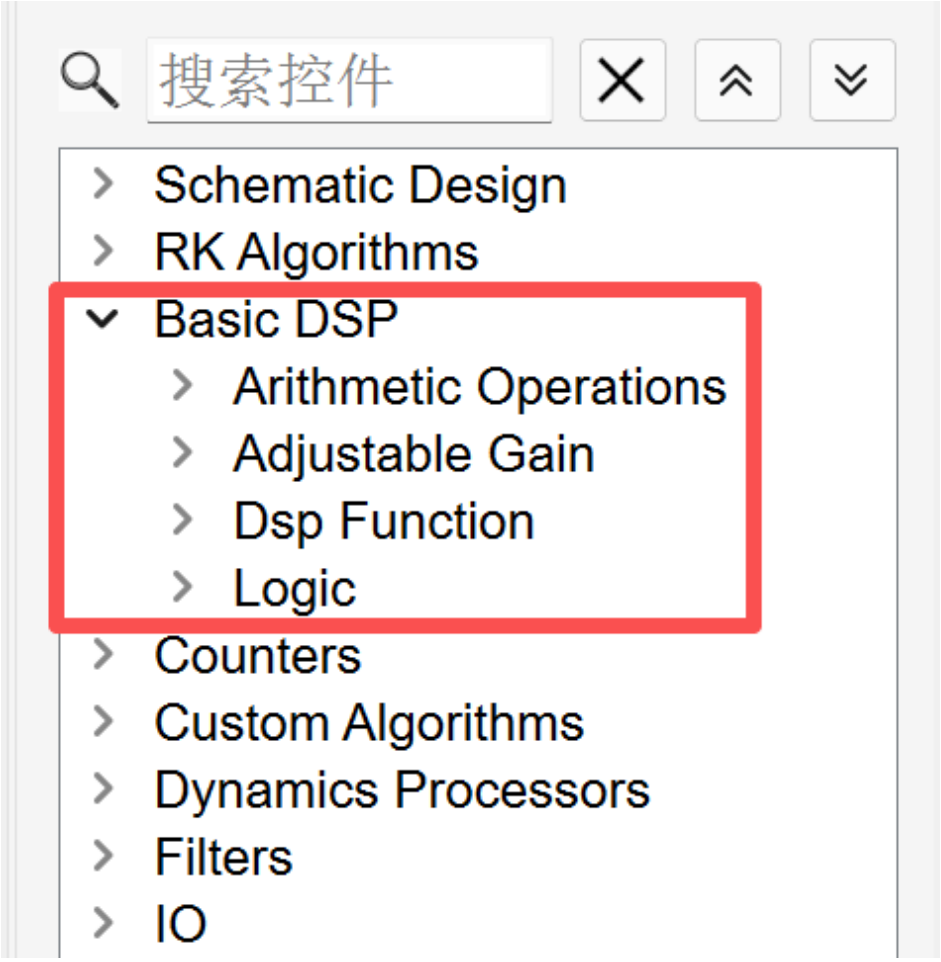


Figure 2-1 Basic DSP location in the tool

2.1 Arithmetic Operations

2.1.1 Absolute Value

Function Description: Calculate the absolute value of the data: $output = abs(input)$.

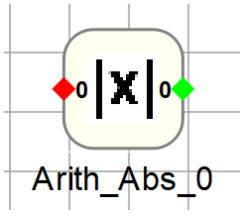


Figure 2.1.1-1 Absolute Value control

Parameters: None

2.1.2 Inverse Square Root

Function Description: Calculate the inverse of the square root of the data: $output = 1/\sqrt{input}$.

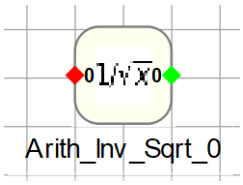


Figure 2.1.2-1 Inverse Square Root control

Parameters: None

2.1.3 Log2

Function Description: Calculate the log2 value of the data: $output = log_2(input)$.

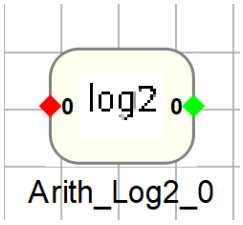


Figure 2.1.3-1 Log2 control

Parameters: None

2.1.4 Log10

Function Description: Calculate the log10 value of the data: $output = log_{10}(input)$.

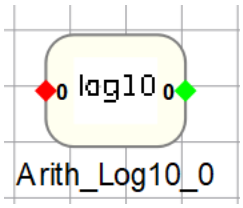


Figure 2.1.4-1 Log10 control

Parameters: None

2.1.5 Exp2

Function Description: Calculate the value of 2 raised to the power of the input data: $output = 2^{input}$.

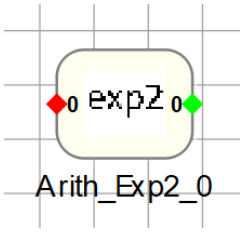


Figure 2.1.5-1 Exp2 control

Parameters: None

2.1.6 Square Root

Function Description: Calculate the square root of the data: $output = \sqrt{input}$.

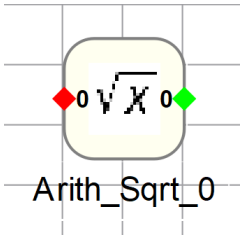


Figure 2.1.6-1 Square Root control

Parameters: None

2.1.7 Signal Add

Function Description: Sum all the data: $output = input0 + input1 + ...$

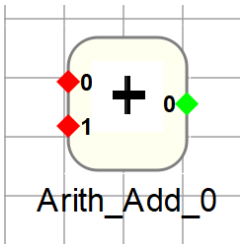


Figure 2.1.7-1 Signal Add control

2.1.8 Signal Subtract

Function Description: Subtract all the data: $output = input0 - input1 - ....$

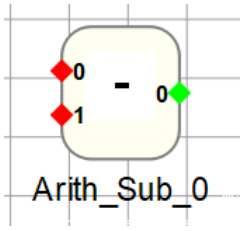


Figure 2.1.8-1 Signal Subtract control

Parameters: None

2.1.9 Divide

Function Description: Divide the data of each pair of channels, with the first channel divided by the second channel and the result output to the first channel, the third channel divided by the fourth channel and the result output to the second channel, and so on. The formula is as follows:

$output0 = input0/input1$

$output1 = input2/input3$

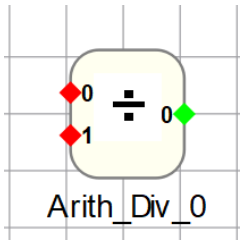


Figure 2.1.9-1 Divide control

Parameters: None

Actual effect display:

Typical condition: Input is DC 0.5 and DC 1.0.

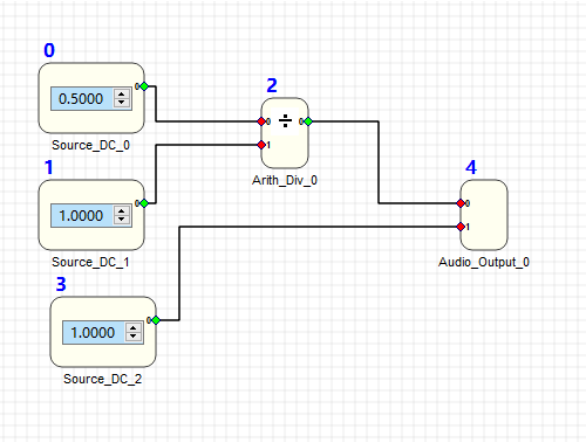


Figure 2.1.9-2 Divide test example

Figure 2.1.9-3 Output waveforms

Channel 0 (Left) at 50% and Channel 1 (Right) at 100%.

2.1.10 Multiply

Function Description: Multiply the data of each pair of channels, with the first channel multiplied by the second channel and the result output to the first channel, the third channel multiplied by the fourth channel and the result output to the second channel, and so on. The formula is as follows:

$output0 = input0 * input1$

$output1 = input2 * input3$

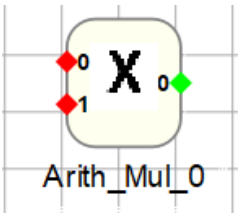


Figure 2.1.10-1 Multiply control

Parameters: None

2.1.11 Signal Invert

Function Description: Invert the polarity of the signal. If the checkbox is selected, the signal is inverted, with the formula: $output = -input$.

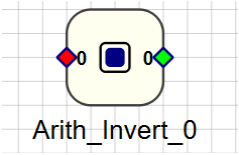


Figure 2.1.11-1 Signal Invert control

Parameters: None

2.1.12 Multi Signal Invert

Function Description: Invert the polarity of multiple signals, with individual control for each channel, with the formula:

$output0 = -input0$

$output1 = -input1.$

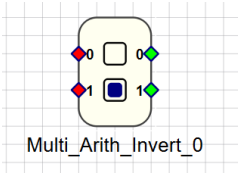


Figure 2.1.12-1 Multi Signal Invert control

Parameters: None

2.2 Adjustable Gain

2.2.1 Gain

Function： Multiplies the input signal by the gain and outputs the result. The input box is for the gain value in dB, which will be converted to linear gain* and passed to the algorithm for use.

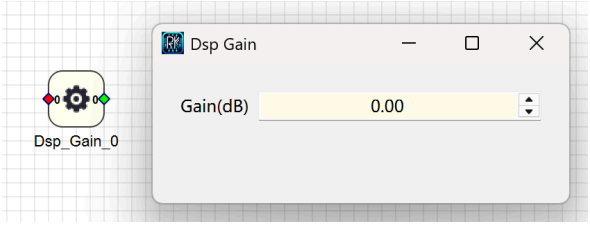


Figure 2.2.1-1 Gain Module and Settings Interface

Number of Parameters: 1

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Gain	0	float	Gain value	dB	-138 ~ 100	0

Refer to Gain (Slew) for specific effects; this module lacks the smoothing function.

Note: The formula for converting dB to linear gain is:

```
def db_to_linear(db, en_rms=False):
    if en_rms:
        return 10**(db/10)
    else:
        return 10**(db/20)
```

All controls in Section 2.2 use the else branch calculation method.

2.2.2 Gain (Slew)

Function: Multiplies the input signal by the gain and outputs the result. The input box is for the gain value in dB, which will be converted to linear gain* and passed to the algorithm for use. It performs smoothing operations during gain changes based on the set slew rate.

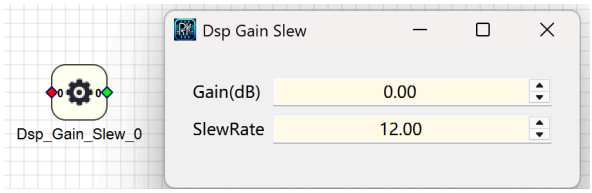


Figure 2.2.2-1 Gain (Slew) Module and Settings Interface

Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Gain	0	float	Gain value	dB	-138 ~ 100	0
SlewRate	1	float	Smoothing speed for gain changes; higher slew rate means slower changes	None	1 ~ 23	12

Demonstration of Actual Effects:

Input data is set to a DC signal with a value of 1. There are two typical cases:

- Case 1: Parameters are set in order to -12, 8 ； i.e., gain is set to -12 dB, and slew rate is 8.
- Case 2: Parameters are set in order to -6, 8 ； i.e., gain is set to -6 dB, and slew rate is 8.

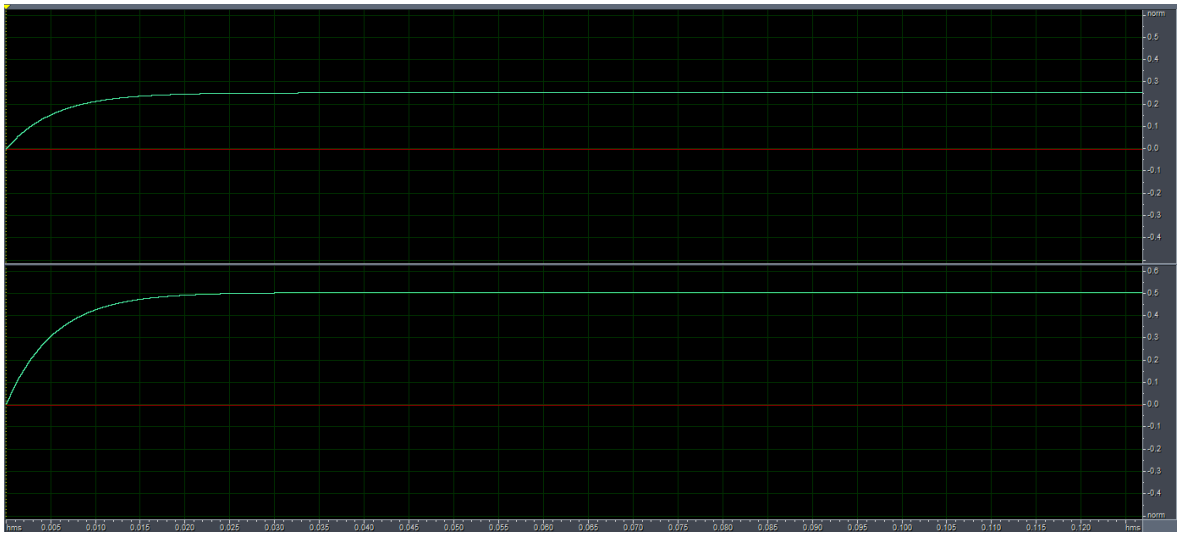


Figure 2.2.2-2 RKStudio Output Time Domain; Case 1 is the upper figure, and Case 2 is the lower figure.

2.2.3 Gain (Slew Double)

Function: Multiplies the input signal by the gain and outputs the result. The input box is for the gain value in dB, which will be converted to linear gain* and passed to the algorithm for use. It performs smoothing operations during gain changes based on the set RampUP and RampDown. RampUP is the smoothing coefficient for gain increase, and RampDown is the smoothing coefficient for gain decrease.

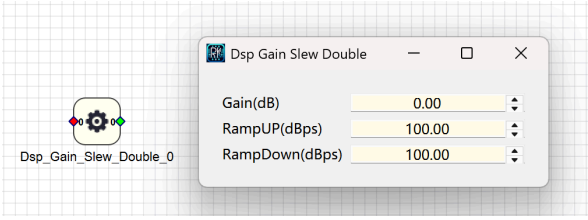


Figure 2.2.3-1 Gain (Slew Double) Module and Settings Interface

Number of Parameters: 3

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Gain(dB)	0	float	Gain value	dB	-138 ~ 100	0
RampUP(dBps)	1	float	Gain increase rate	dB/s	1 ~ 1000	200
RampDown(dBps)	2	float	Gain decrease rate	dB/s	1 ~ 1000	200

Demonstration of Actual Effects:

The input data is set to a DC signal with a value of 0.5. There are two typical cases below, which demonstrate the effect of the smoothing coefficient:

- Case 1: The parameters are set in order to: 6, 200, 200； i.e., the gain is set to 6 dB, with a RampUP rate of 200 dB/s and a RampDown rate of 200 dB/s.
- Case 2: The parameters are set in order to: 6, 400, 400 ； i.e., the gain is set to 6 dB, with a RampUP rate of 400 dB/s and a RampDown rate of 400 dB/s.

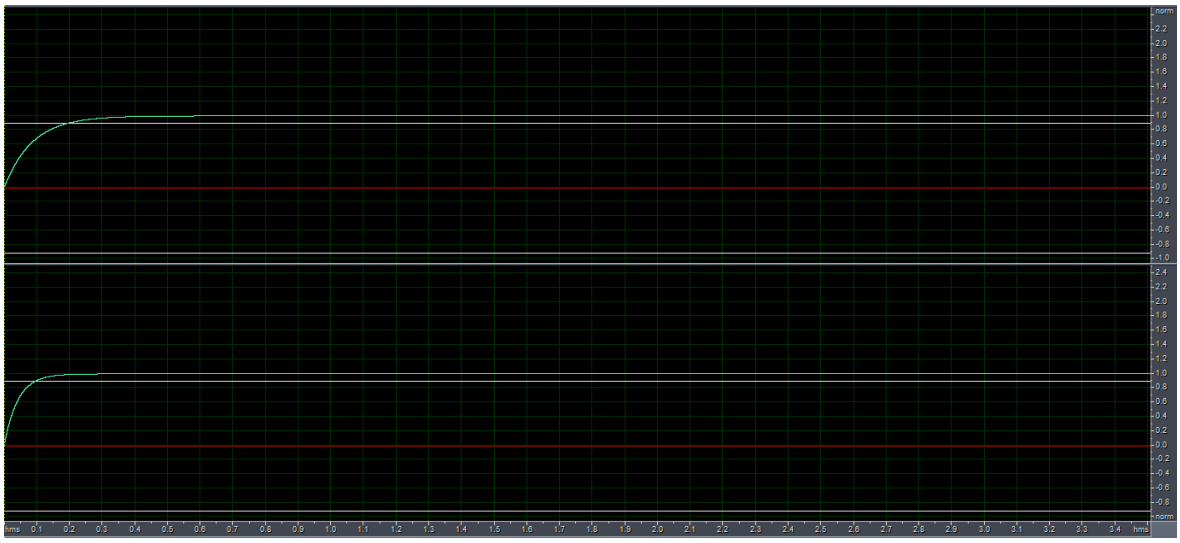


Figure 2.2.3-2 RKStudio Output Time Domain; Case 1 is the upper figure, and Case 2 is the lower figure.

2.2.4 Gain External Mute (Slew Double)

Function: The parameter settings and functionality are the same as in Section 2.2.3 Gain (Slew Double), with dB gain also converted to linear gain* for algorithm calculation. The main difference is that the first channel is an external control signal used to mute the output channel. If the control signal is not zero, the output will be muted; otherwise, normal gain calculation will be performed.

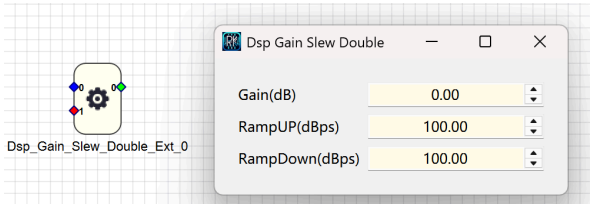


Figure 2.2.4-1 Gain_Slew_EXT Module and Settings Interface

2.3 DSP Functions

2.3.1 Delay

Function Description: Delays the input signal by the set delay value in samples or delay_ms time before outputting.

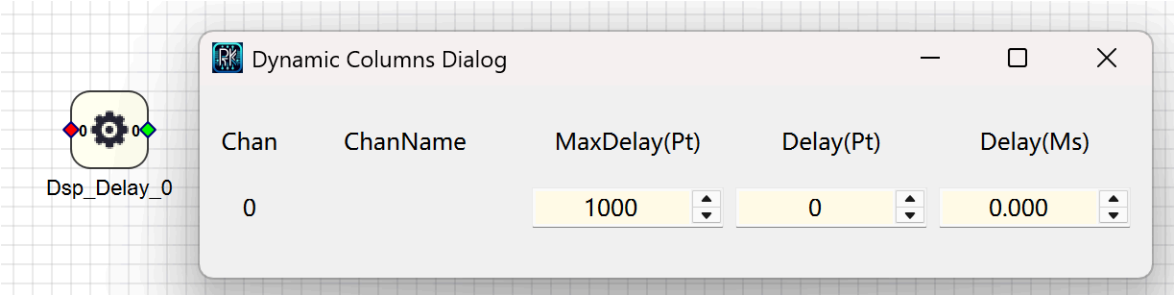


Figure 2.3.1-1 Delay Module and Settings Interface

Number of Parameters: 3

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
MaxDelay	0	int	Maximum delay	sample (number of samples)	1 ~ 20000	1500
Delay	1	int	Delay value	sample (number of samples)	0 ~ MaxDelay	0
Delay_ms	2	float	Delay value	ms	0 ~ MaxDelay/Fs	0

Note: The second parameter Delay (in samples) and the third parameter delay_ms (in ms) are related. Setting the Delay (in samples) will automatically calculate the delay time Delay_ms, and vice versa, setting delay_ms will also automatically calculate the delay samples.

Demonstration of Actual Effects:

Input data is set to a DC signal with a value of 1. There are two typical cases:

- Case 1: Parameters are set in order to 1500, 500,-- ； i.e., maximum delay is set to 1500, and current delay is set to 500, with the delay unit being samples.
- Case 2: Parameters are set in order to 1500, --, 30 ； i.e., maximum delay is set to 1500, and current delay is set to 30, with the delay unit being ms.

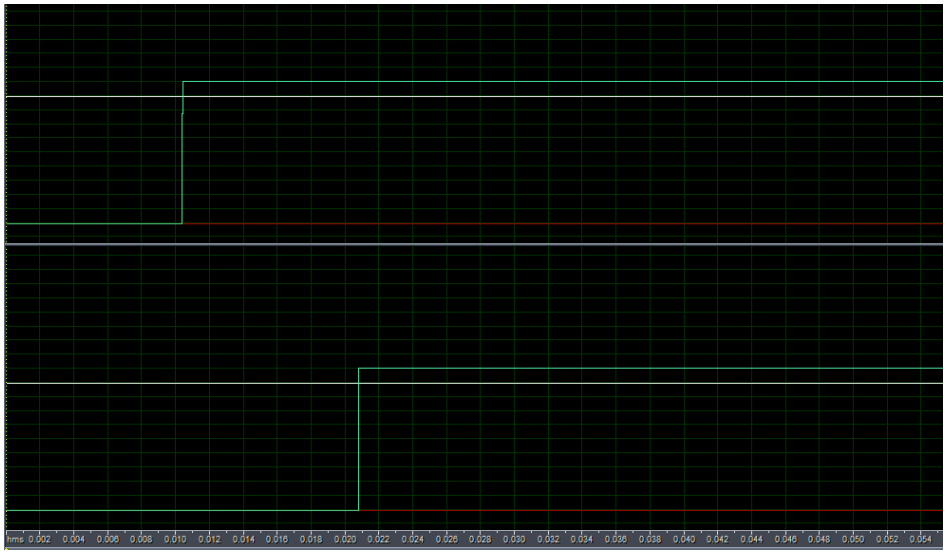


Figure 2.3.1-2 RKStudio Output Time Domain; Case 1 is the upper figure, and Case 2 is the lower figure. It can be seen that under a sampling rate of 48 kHz, the delays are 10 ms and 30 ms, respectively, as expected.

2.3.2 Multi-Delay

Function Description: Delays the input signal by the set delay value in samples or delay_ms time before outputting. Each channel's delay can be controlled separately.

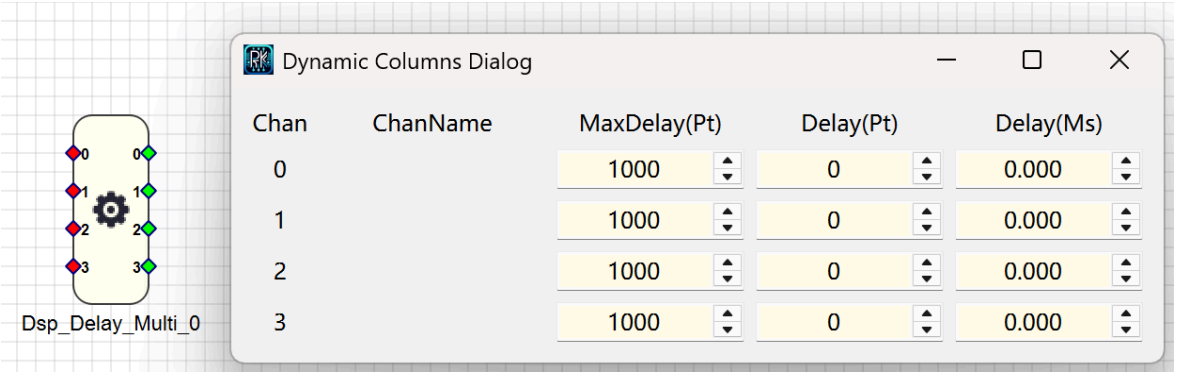


Figure 2.3.2-1 Multi-Delay Module and Settings Interface

Number of Parameters: Number of input channels * 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
MaxDelay	0	int	Maximum delay	sample (number of samples)	1 ~ 20000	1500
Delay	1	int	Delay value	sample (number of samples)	0 ~ MaxDelay	0

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Delay_ms	2	float	Delay value	ms	0 ~ MaxDelay/Fs	0
...						

Right-click the control to add channels.

Note: The second parameter Delay (in samples) and the third parameter delay_ms (in ms) are related. Setting the Delay (in samples) will automatically calculate the delay time Delay_ms, and vice versa, setting delay_ms will also automatically calculate the delay samples.

Refer to the Delay module for the operating effect.

2.3.3 Multi Delay (Index Selectable)

Function: Indexed multi-channel delay. Unlike the standard Multi Delay, this module allows for pre-configuring multiple sets of delay parameters. By changing the Index parameter value, you can select the delay configuration to be used from the preset list.

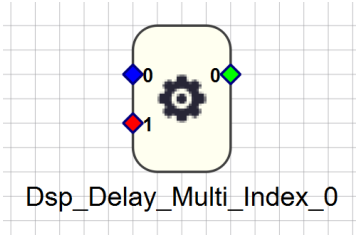


Figure 2.3.3-1 Multi Delay (Index Selectable) Control

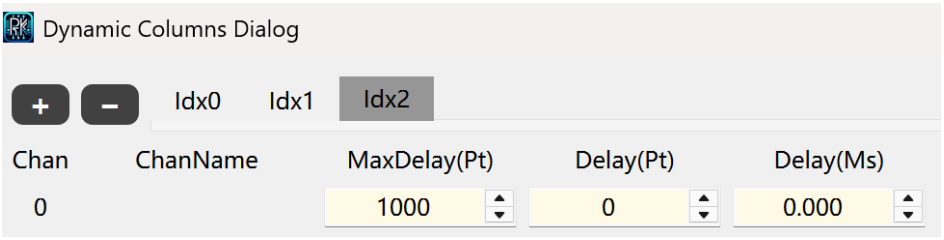


Figure 2.3.3-2 Multi Delay (Index Selectable) Settings Interface

2.3.4 Multi Delay Slew (Index Selectable)

Function: Indexed multi-channel delay with smooth transition. The function is similar to Multi Delay (Index Selectable), but with an added Slew feature. When the Index changes, a smoothing effect is applied to ensure the switching process is not abrupt.

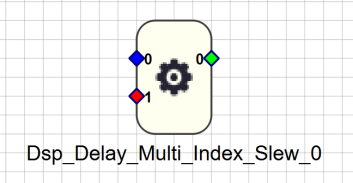


Figure 2.3.4-1 Multi Delay Slew(Index Selectable) Control

The parameter settings interface is the same as Multi Delay (Index Selectable).

2.3.5 DSP Feedback

Function: Feedback loop. It feeds the output signal back to serve as an input signal for a plugin. This can be the output signal of the current plugin or the output signal of another plugin.

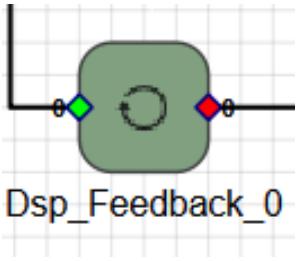


Figure 2.3.5-1 Feedback Control

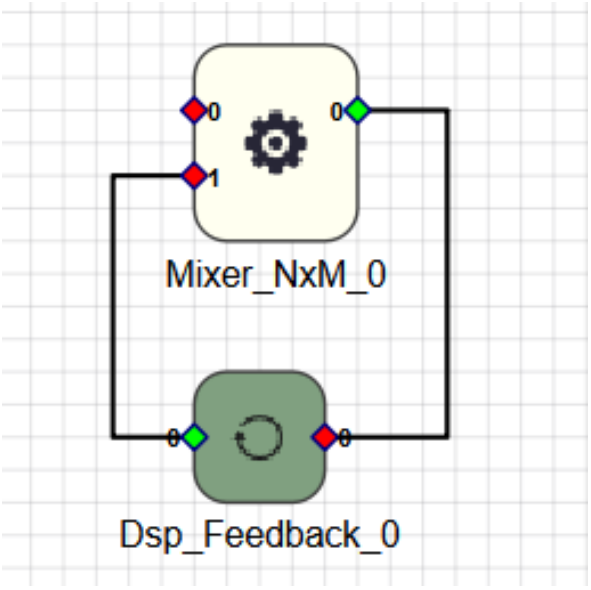


Figure 2.3.5-2 Feedback Connection Diagram

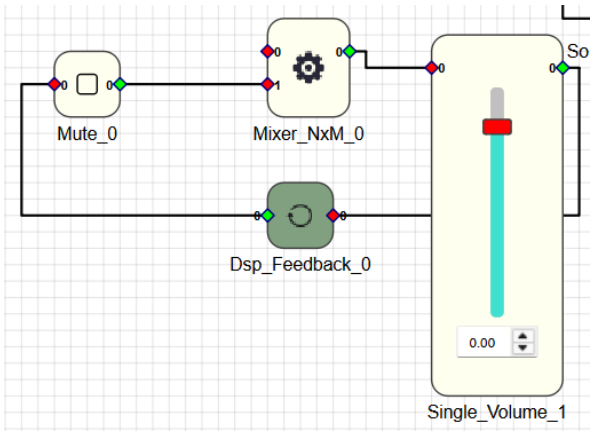


Figure 2.3.5-3 Feedback Connection Diagram

2.4 Logic

2.4.1 AbsMax

Function Description: Computes the maximum absolute value of all input channels.

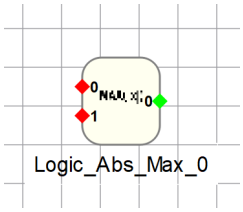


Figure 2.4.1-1 AbsMax Control

Parameters: None

2.4.2 Max

Function Description: Computes the maximum value of all input channels.

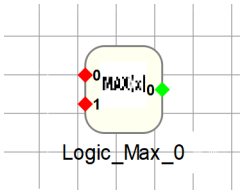


Figure 2.4.2-1 Max Control

Parameters: None

2.4.3 Min

Function Description: Computes the minimum value of all input channels.

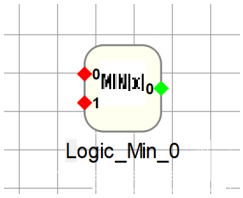


Figure 2.4.3-1 Min Control

Parameters: None

2.4.4 Logic AB

Function Description: Compares the values of channels 0 and 1 and outputs the corresponding values of channels 0 or 1 based on the following five conditions:

```
0 > 1  ?  0 : 1
0 < 1  ?  0 : 1
0 ≥ 1  ?  0 : 1
0 ≤ 1  ?  0 : 1
0 == 1  ?  0 : 1
```

For example, in the first case, if the data in channel 0 is greater than that in channel 1, the data from channel 0 is output; otherwise, the data from channel 1 is output.

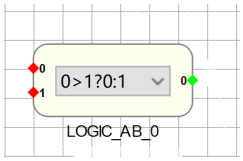


Figure 2.4.4-1 Logic AB Control

Parameters: None

2.4.5 Logic ABCD

Function Description: Compares the values of channels 0 and 1 and outputs the corresponding values of channels 2 or 3 based on the following five conditions:

```
0 > 1  ?  2 : 3
0 < 1  ?  2 : 3
0 ≥ 1  ?  2 : 3
0 ≤ 1  ?  2 : 3
0 == 1  ?  2 : 3
```

For example, in the first case, if the data in channel 0 is greater than that in channel 1, the data from channel 2 is output; otherwise, the data from channel 3 is output.

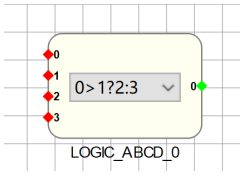


Figure 2.4.5-1 Logic ABCD Control

Parameters: None

2.4.6 Logic Calc

Function Description: Computes the logical relationship between two channel values and outputs the result. The supported logical operations are:
AND, NAND, OR, NOR, XOR

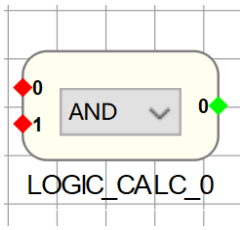


Figure 2.4.6-1 Logic Calc Control

Parameters: None

2.4.7 Zero Comparator

Function Description: If the input value is equal to 0, the set parameter value is output; otherwise, 0 is output.

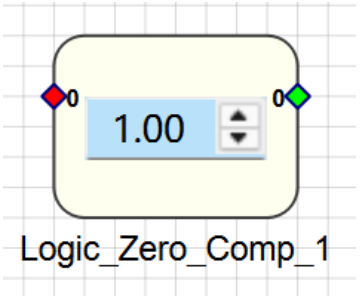


Figure 2.4.7-1 Logic Zero Comparator Control

Parameters: None

2.4.8 Value Hold

Function Description: Commonly used to "latch" or "sample and hold" the current input value. It can hold a momentary input signal for subsequent logical processing or observation.

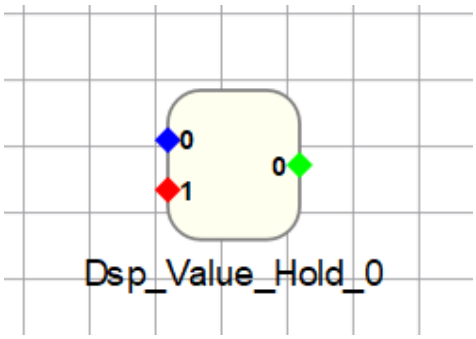


Figure 2.4.8-1 Value Hold Control

Parameters: None

2.4.9 AbsMax Value Hold

Function Description: Calculates and holds the historical maximum absolute value across all input channels.

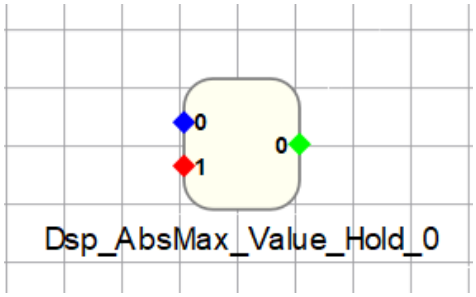


Figure 2.4.9-1 AbsMax Value Hold Control

Parameters: None

2.4.10 Max Value Hold

Function Description: Calculates and holds the historical maximum value across all input channels.

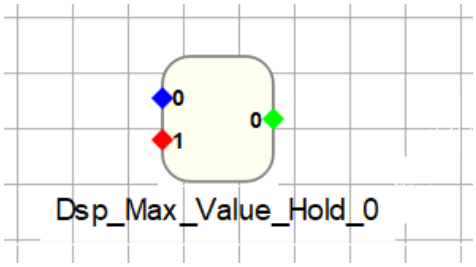


Figure 2.4.10-1 Max Value Hold Control

Parameters: None

2.4.11 Min Value Hold

Function Description: Calculates and holds the historical minimum value across all input channels.

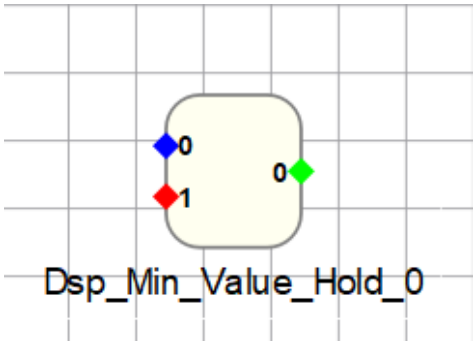


Figure 2.4.11-1 Min Value Hold Control

Parameters: None

3 Dynamics Processors

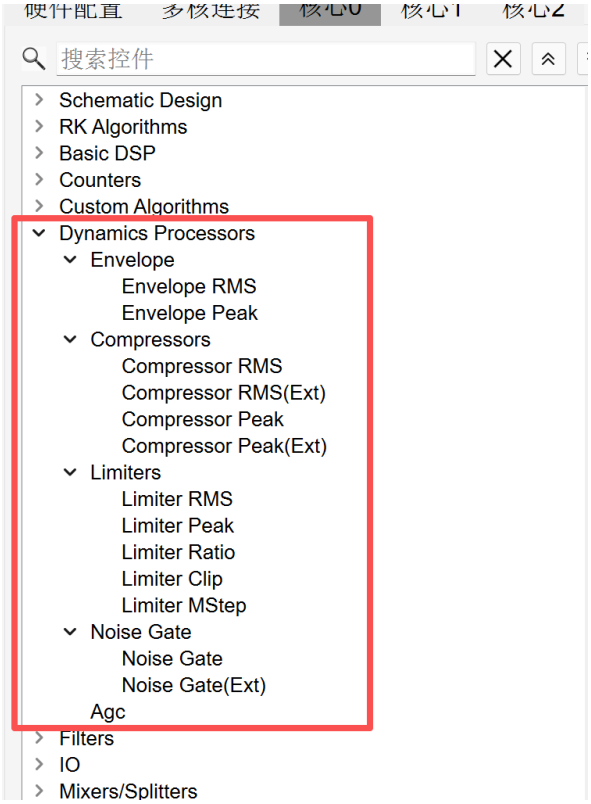


Figure 3-1 Location of Dynamics Processors in the tool

3.1 Envelope

3.1.1 Envelope RMS

Function: Smoothly tracks the RMS value of the data.

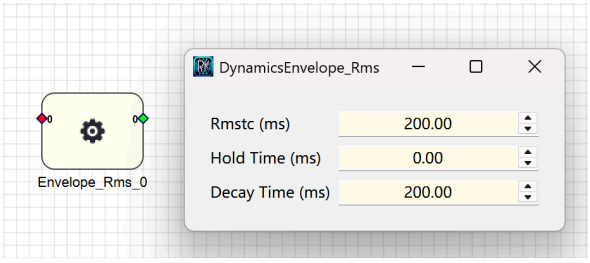


Figure 3.1.1-1 Envelope RMS control and settings interface

Number of Parameters: 3

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Rmstc (ms)	0	float	Response speed to signal changes	ms	0 ~ 2000	200

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Hold Time (ms)	1	float	Duration to hold the current peak before envelope decay	ms	0 ~ 2000	0
Decay Time (ms)	2	float	Speed of gain decay	ms	0 ~ 2000	200

Demonstration of actual effect:

Input data is white noise, with two typical conditions:

- Condition 1: Rmstc = 20, Hold Time = 10, Decay Time = 10;
- Condition 2: Rmstc = 50, Hold Time = 20, Decay Time = 20.

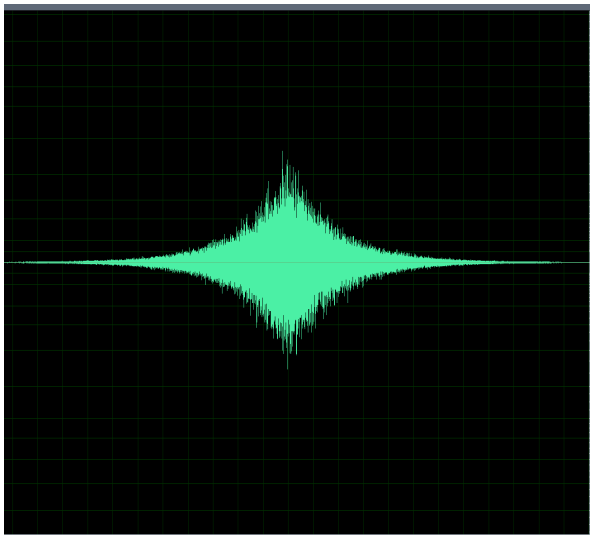


Figure 3.1.1-2 Input sound source

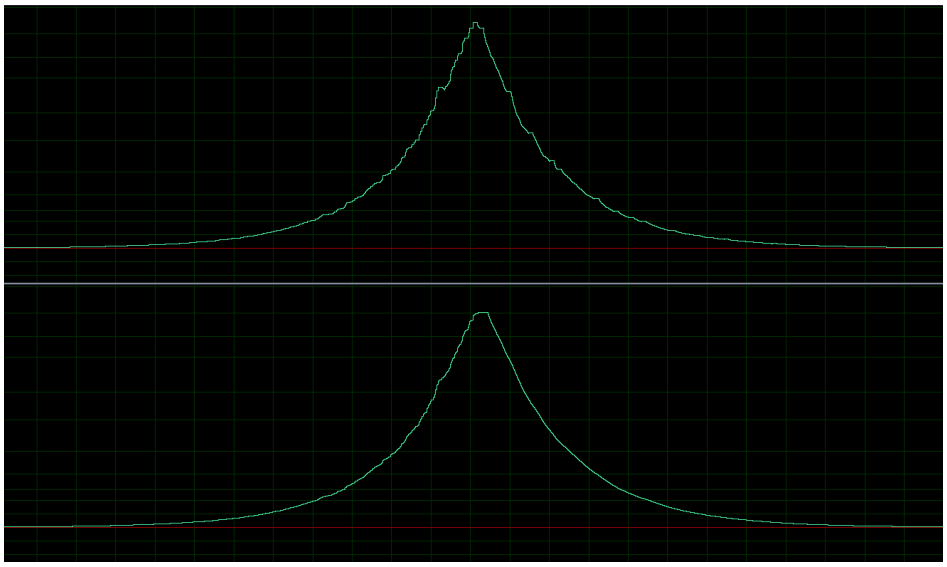


Figure 3.1.1-3 Output time-domain of Condition 1 (upper half) and Condition 2 (lower half)

3.1.2 Envelope Peak

Function: Smoothly tracks the Peak value of the data.

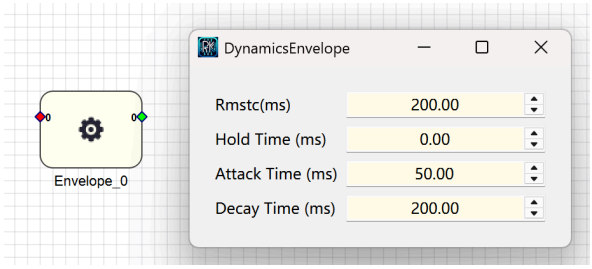


Figure 3.1.2-1 Envelope Peak control and settings interface

Number of Parameters: 4

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Rmstc (ms)	0	float	Response speed to signal changes	ms	0 ~ 2000	200
Hold Time (ms)	1	float	Duration to hold the current peak before envelope decay	ms	0 ~ 2000	0
Attack Time (ms)	2	float	Speed of envelope rise	ms	0 ~ 2000	50
Decay Time (ms)	3	float	Speed of envelope decay	ms	0 ~ 2000	200

Demonstration of actual effect:

Input data is white noise, with two typical conditions:

- Condition 1: Hold Time = 10, Decay Time = 10;
- Condition 2: Hold Time = 50, Decay Time = 20.

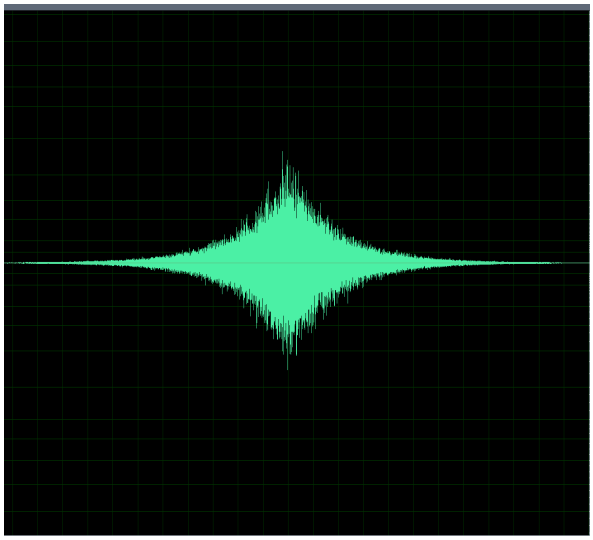


Figure 3.1.2-2 Input sound source

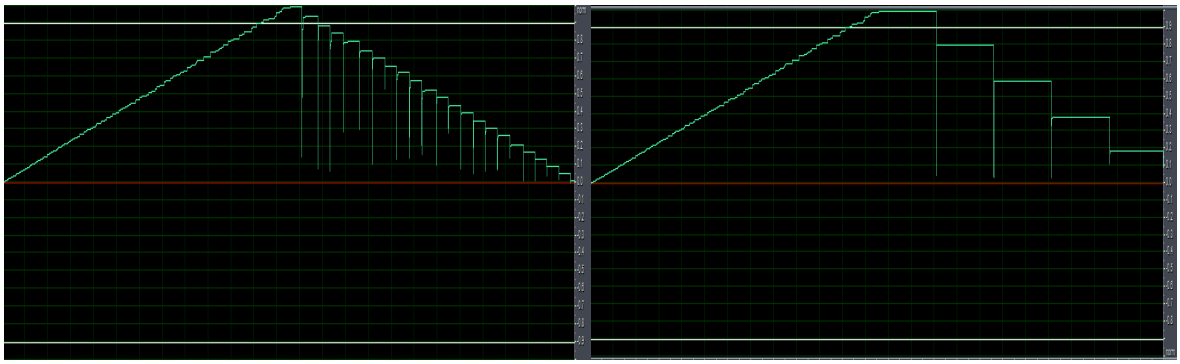


Figure 3.1.2-3 Output time-domain of Condition 1 (left) and Condition 2 (right)

3.2 Compressors

3.2.1 Compressor RMS

Function: Compressor that adjusts the audio output range based on a set mapping curve. The Compressor RMS module maps the curve based on energy. Double-click the blank area to set smooth parameters, and click the button to set the mapping curve.

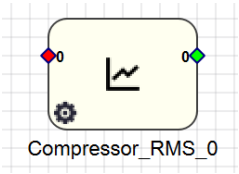


Figure 3.2.1-1 Compressor RMS control

Number of Parameters: 6

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Rmstc (ms)	0	float	Response speed to signal changes (higher values result in slower changes)	ms	0 ~ 2000	20
Hold Time (ms)	1	float	Duration to hold the current peak before envelope decay	ms	0 ~ 2000	0
Attack Time (ms)*	2	float	Speed of gain rise	ms	0 ~ 2000	200
Release Time (ms)*	3	float	Speed of gain decay	ms	0 ~ 2000	200
Soft Knee	4	int	Whether to use a soft knee at the curve connection point	None	0/1	0
Delay Time (ms)	5	float	Delay time	ms	0 ~ 100	0

Demonstration of actual effect:

Input data includes music and impulse sounds, with the following case configurations:

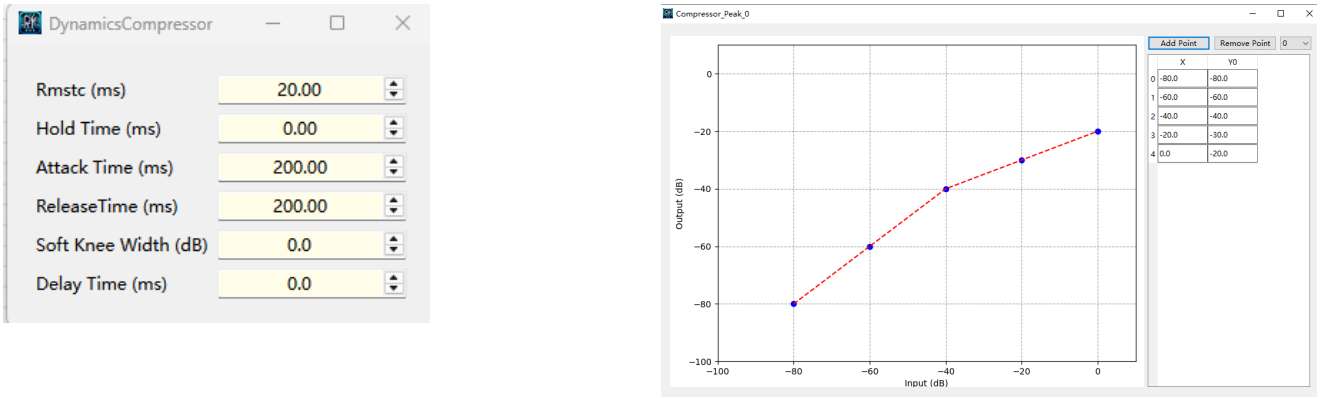
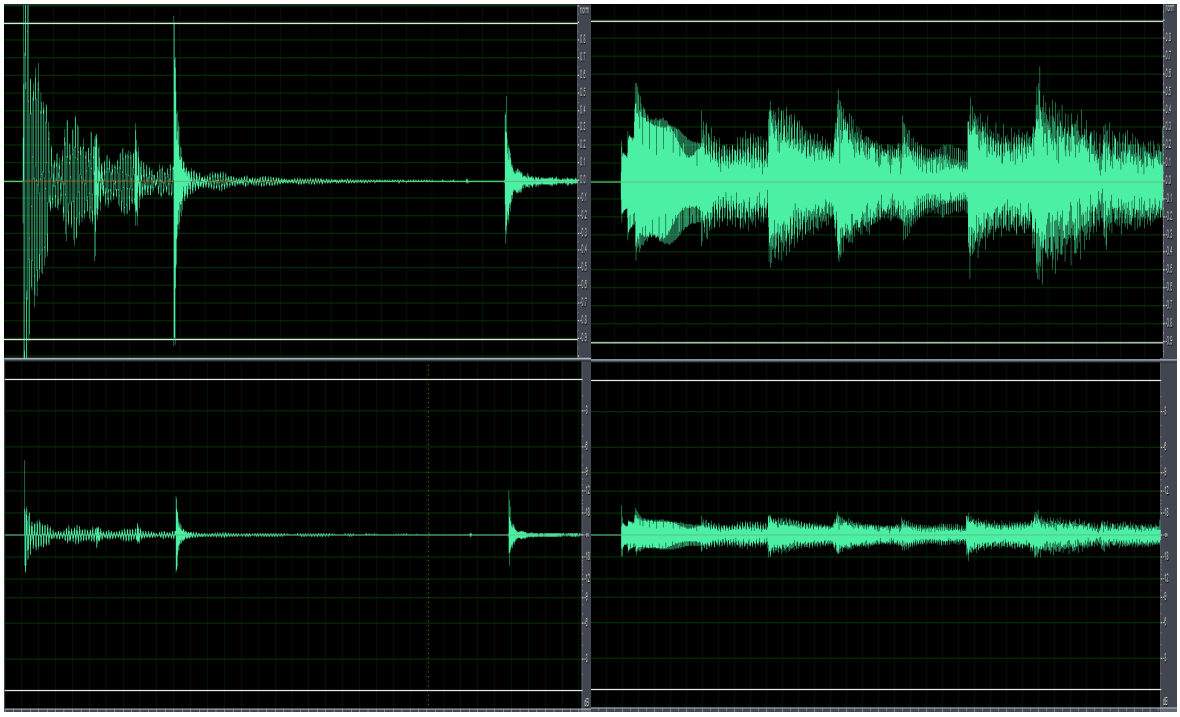


Figure 3.2.1-2 Compressor RMS parameters (left) and Compressor RMS curve (right)

Figure 3.2.1-3 Input to the algorithm: Impulse sound time-domain (left) and Music time-domain (right)

Figure 3.2.1-4 Output of the algorithm: Impulse sound output time-domain (left) and Music output time-domain (right)



From the right side of Figure 3.2.1-2, the curve we set is:

For signal strengths between (-80dB ~ -40dB), the output remains unchanged;

For signal strengths between (-40dB ~ 0dB), the output is mapped to (-40dB ~ -20dB).

From the time-domain results, low-amplitude waveforms remain unchanged, while high-amplitude waveforms are attenuated.

3.2.2 Compressor RMS External Input

Function: Compressor that adjusts the audio output range based on a set mapping curve. The Compressor Rms Ext module maps the curve based on the peak value of the external input. Double-click the blank area to set smooth parameters, and click the button to set the mapping curve.

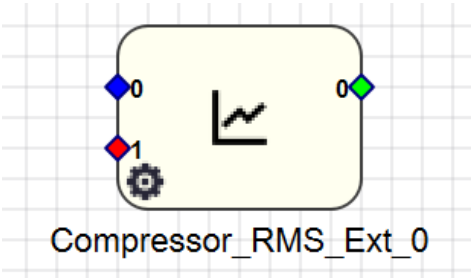


Figure 3.2.2-1 Compressor Ext Input RMS control

Number of Parameters: 6

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Rmstc (ms)	0	float	Response speed to signal changes (higher values result in slower changes)	ms	0 ~ 2000	20
Hold Time (ms)	1	float	Duration to hold the current peak before envelope decay	ms	0 ~ 2000	0
Attack Time (ms)*	2	float	Speed of gain rise	ms	0 ~ 2000	200
Release Time (ms)*	3	float	Speed of gain decay	ms	0 ~ 2000	200
Soft Knee	4	int	Whether to use a soft knee at the curve connection point	None	0/1	0
Delay Time (ms)	5	float	Delay time	ms	0 ~ 100	0

For specific effects, refer to 3.2.1 Compressor RMS.

3.2.3 Compressor Peak

Function: Compressor that adjusts the audio output range based on a set mapping curve. The Peak module maps the curve based on the peak value. Double-click the blank area to set smooth parameters, and click the button to set the mapping curve.

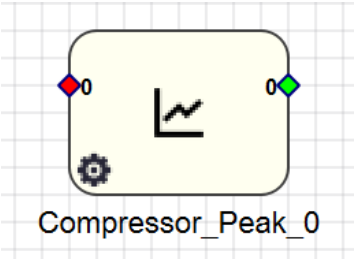


Figure 3.2.3-1 Compressor Peak control

Number of Parameters: 5

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Hold Time (ms)	0	float	Duration to hold the current peak before envelope decay	ms	0 ~ 2000	0
Attack Time (ms)*	1	float	Speed of gain rise	ms	0 ~ 2000	200
Release Time (ms)*	2	float	Speed of gain decay	ms	0 ~ 2000	200
Soft Knee	3	int	Whether to use a soft knee at the curve connection point	None	0/1	0
Delay Time (ms)	4	float	Delay time	ms	0 ~ 100	0

Demonstration of actual effect:

Input data includes music and impulse sounds, with the following case configurations:

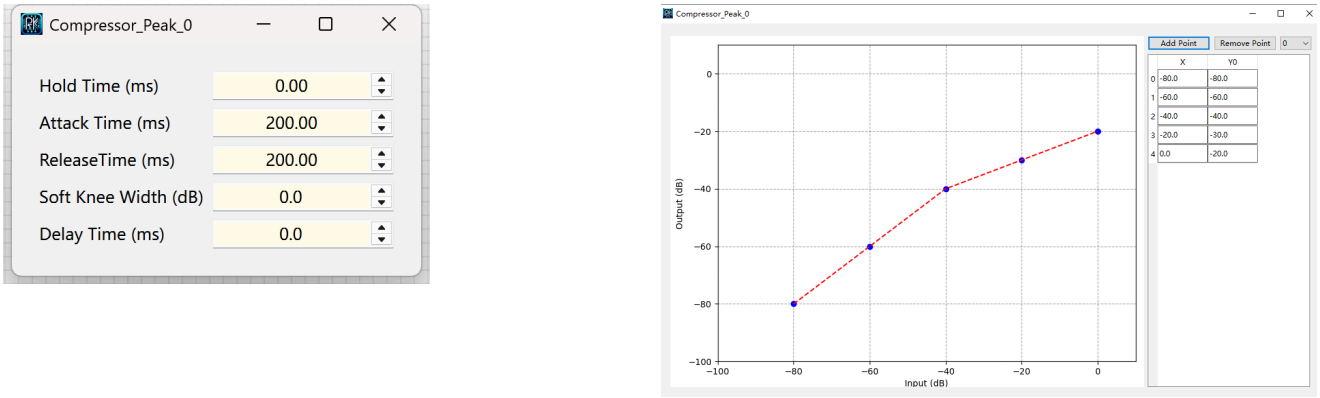


Figure 3.2.3-2 Compressor Peak parameters (left) and Compressor Peak curve (right)

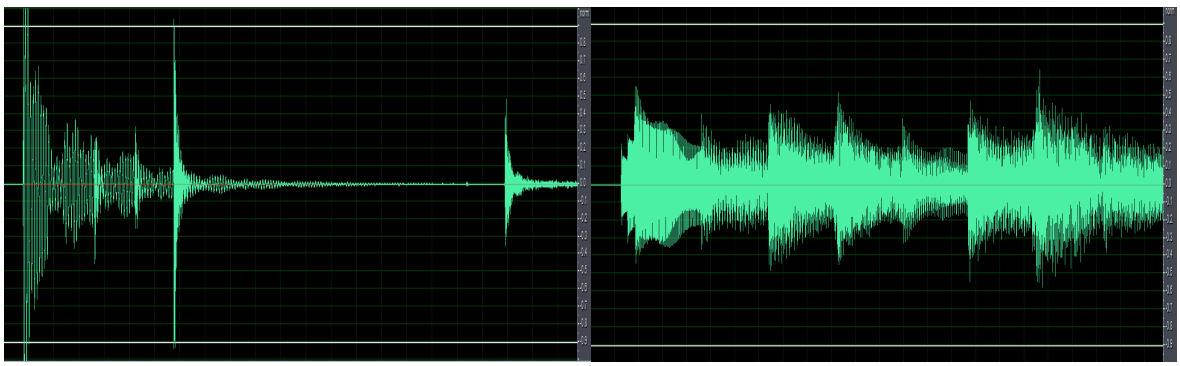


Figure 3.2.3-3 Input to the algorithm: Impulse sound time-domain (left) and Music time-domain (right)

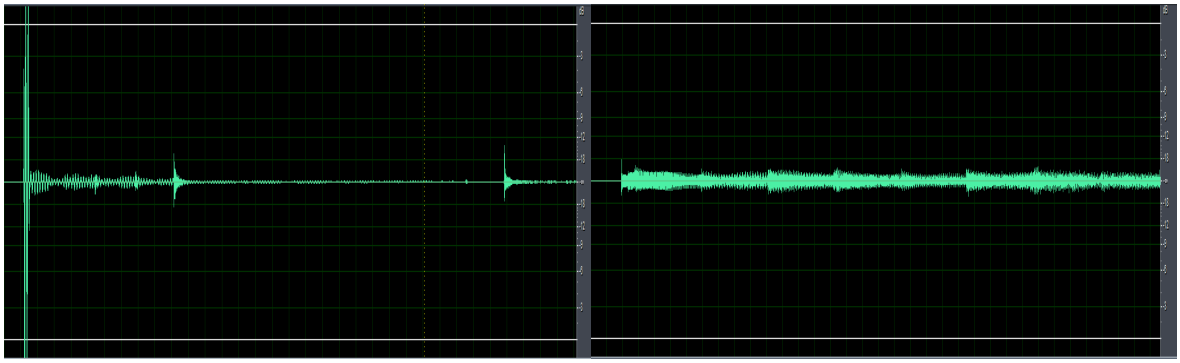


Figure 3.2.3-4 Output of the algorithm: Impulse sound output time-domain (left) and Music output time-domain (right)

From the right side of Figure 3.2.3-2, the curve we set is:

For signal peaks between (-80dB ~ -40dB), the output remains unchanged;

For signal peaks between (-40dB ~ 0dB), the output is mapped to (-40dB ~ -20dB).

From the time-domain results, low-amplitude waveforms remain unchanged, while high-amplitude waveforms are attenuated.

3.2.4 Compressor Peak External Input

Function: Compressor that adjusts the audio output range based on a set mapping curve. The Peak module maps the curve based on the peak value of the external input. Double-click the blank area to set smooth parameters, and click the button to set the mapping curve.

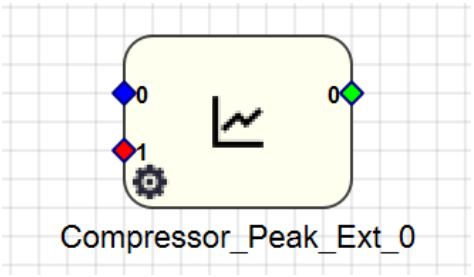


Figure 3.2.4-1 Compressor Peak EXT control

Number of Parameters: 5

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Hold Time (ms)	0	float	Duration to hold the current peak before envelope decay	ms	0 ~ 2000	0

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Attack Time (ms)*	1	float	Speed of gain rise	ms	0 ~ 2000	200
Release Time (ms)*	2	float	Speed of gain decay	ms	0 ~ 2000	200
Soft Knee	3	int	Whether to use a soft knee at the curve connection point	None	0/1	0
Delay Time (ms)	4	float	Delay time	ms	0 ~ 100	0

For specific effects, refer to 3.2.3 Compressor Peak.

3.3 Limiter

3.3.1 Limiter RMS

Function: Limiter that controls and adjusts the maximum audio output to be below a fixed threshold.

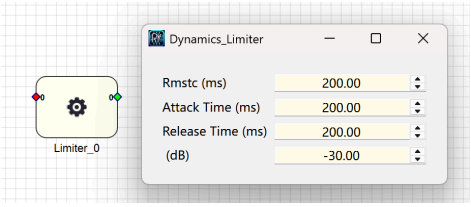


Figure 3.3.1-1 Limiter RMS control

Number of Parameters: 4

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Rmstc (ms)	0	float	Response speed to signal changes (higher values result in slower changes)	ms	0 ~ 2000	200
Attack Time (ms)*	1	float	Speed of gain rise	ms	0 ~ 2000	200

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Release Time (ms)*	2	float	Speed of gain decay	ms	0 ~ 2000	200
Threshold (dB)	3	float	Threshold, input above this value will be limited below it	dB	-80 ~ 0	-30

Demonstration of actual effect:

Input data includes music and impulse sounds, with the following case configurations:

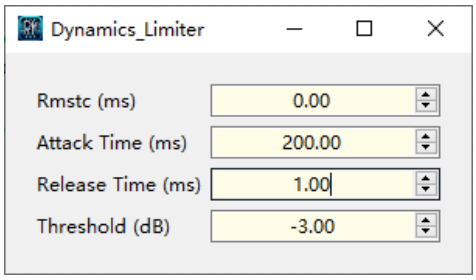


Figure 3.3.1-2 Limiter parameter settings

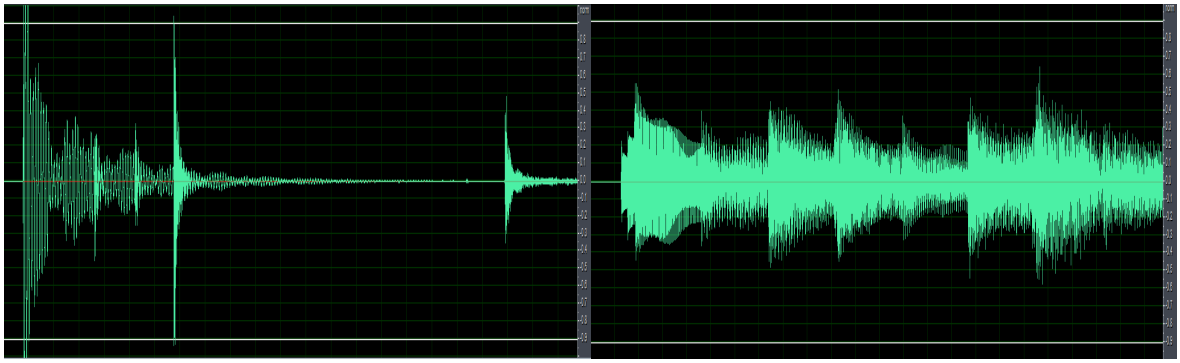


Figure 3.3.1-3 Input to the algorithm: Impulse sound time-domain (left) and Music time-domain (right)

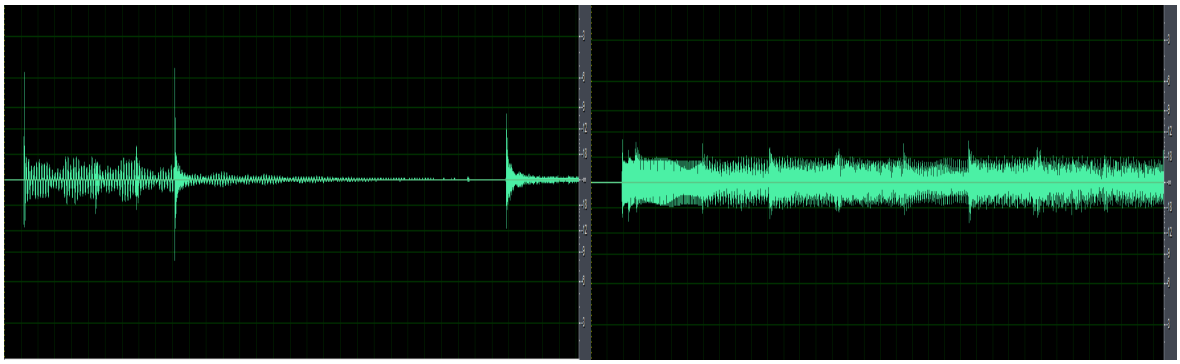


Figure 3.3.1-4 Output of the algorithm: Impulse sound output time-domain (left) and Music output time-domain (right)

3.3.2 Limiter Peak

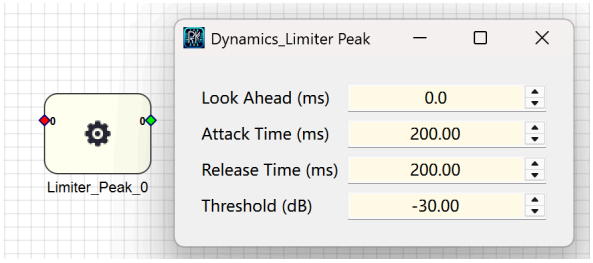


Figure 3.3.2-1 Limiter Peak control

Number of Parameters: 4

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Look Ahead (ms)	0	float	Observation time ahead of the current point	ms	0 ~ 100	10
Attack Time (ms)*	1	float	Speed of gain rise	ms	0 ~ 2000	200
Release Time (ms)*	2	float	Speed of gain decay	ms	0 ~ 2000	200
Threshold (dB)	3	float	Threshold, input above this value will be limited below it	dB	-80 ~ 0	-30

Note: To explain the significance of the parameters, the following examples are provided:

The input signal for the control is:

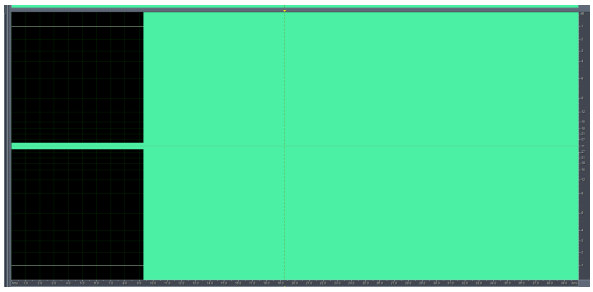


Figure 3.3.2-2 Input signal: A sine wave with amplitude jumping from -32.70dB to 0dB

- Look Ahead: Essentially a delay, observing the signal points after the current point. The larger the value, the smoother the output amplitude change, and the stronger the suppression of impulse signals.

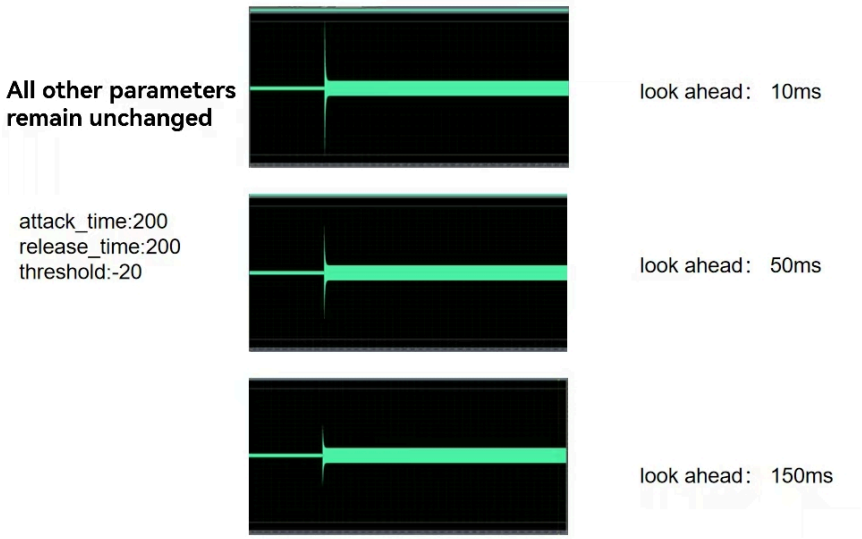


Figure 3.3.2-3 Effects of different look-ahead values on the result

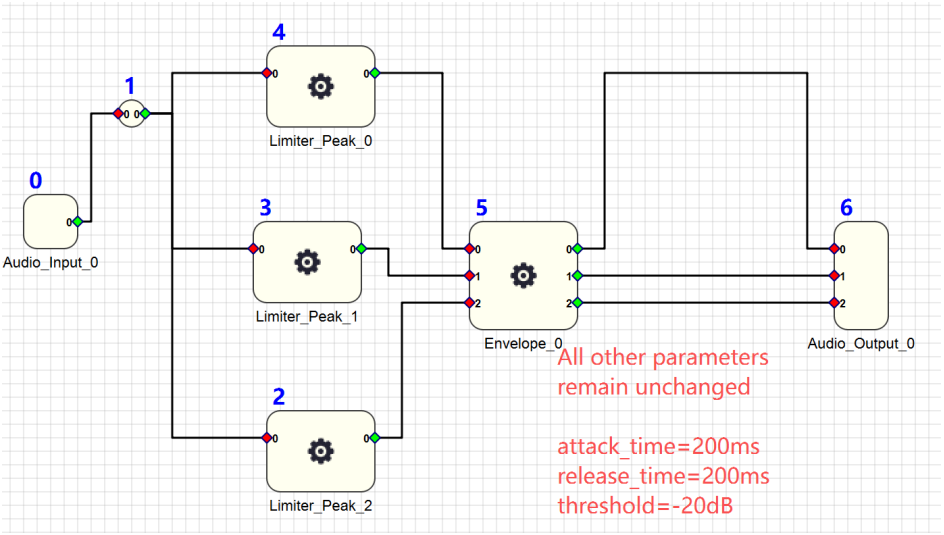


Figure 3.3.2-4 Schematic for look-ahead comparative testing

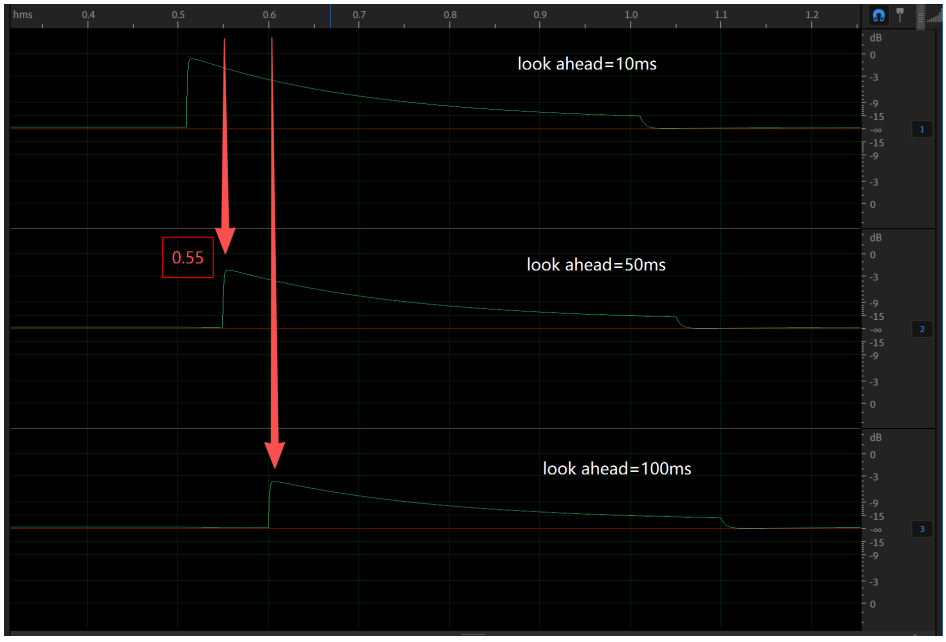


Figure 3.3.2-5 Effects of different look-ahead values on the result

- Attack Time: The time it takes for the output to transition from unrestricted to restricted. When a signal suddenly exceeds the threshold, the output will briefly exceed the threshold before being limited.

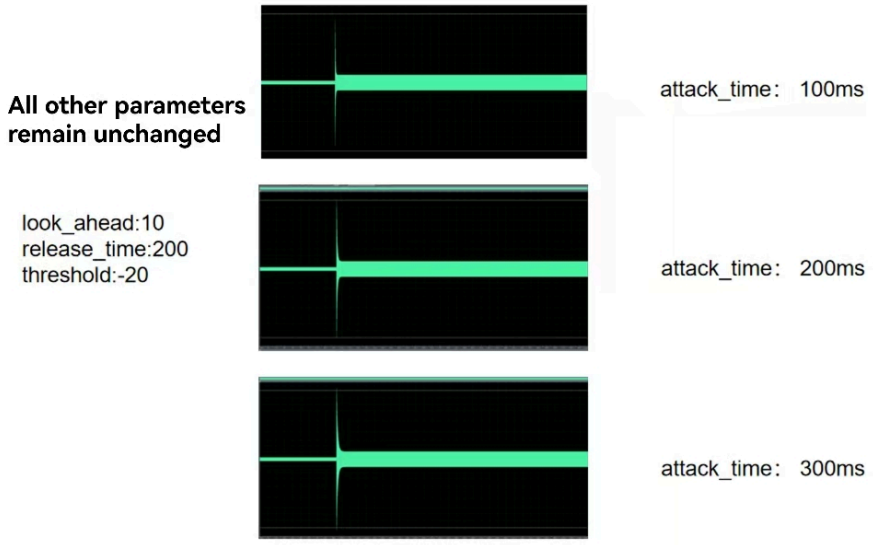


Figure 3.3.2-6 Effects of different attack times on the result

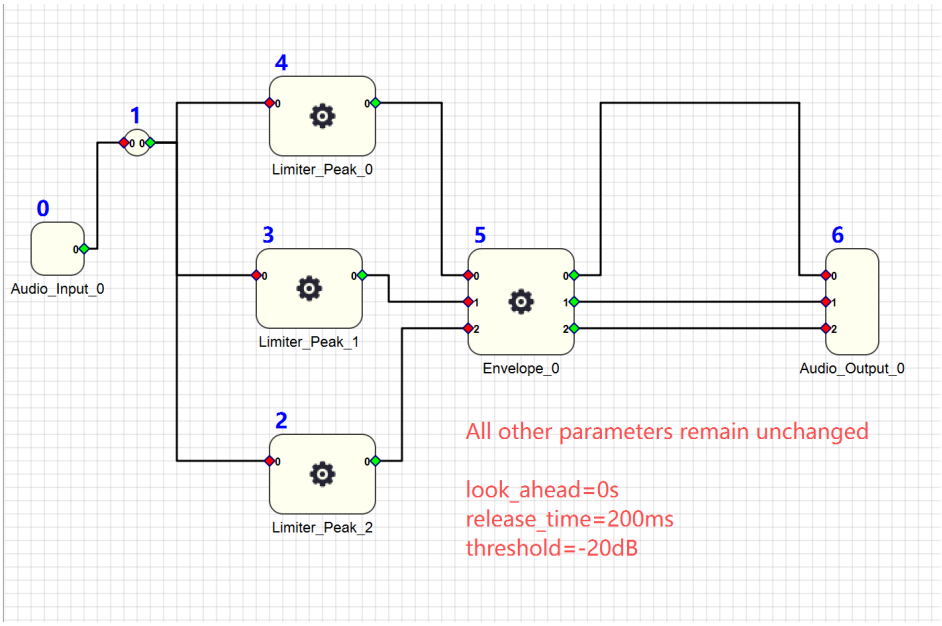


Figure 3.3.2-7 Schematic for attack time comparative testing

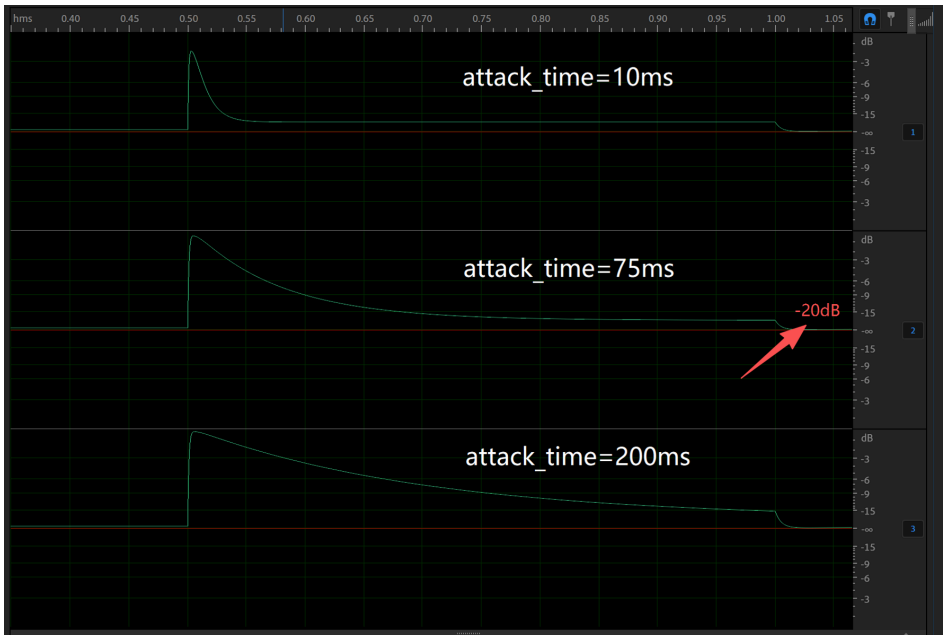


Figure 3.3.2-8 Effects of different attack times on the result

- Release Time: The time it takes for the output to transition from restricted to unrestricted. When the threshold suddenly changes from below to above the signal value, the output will slowly increase before matching the signal value.

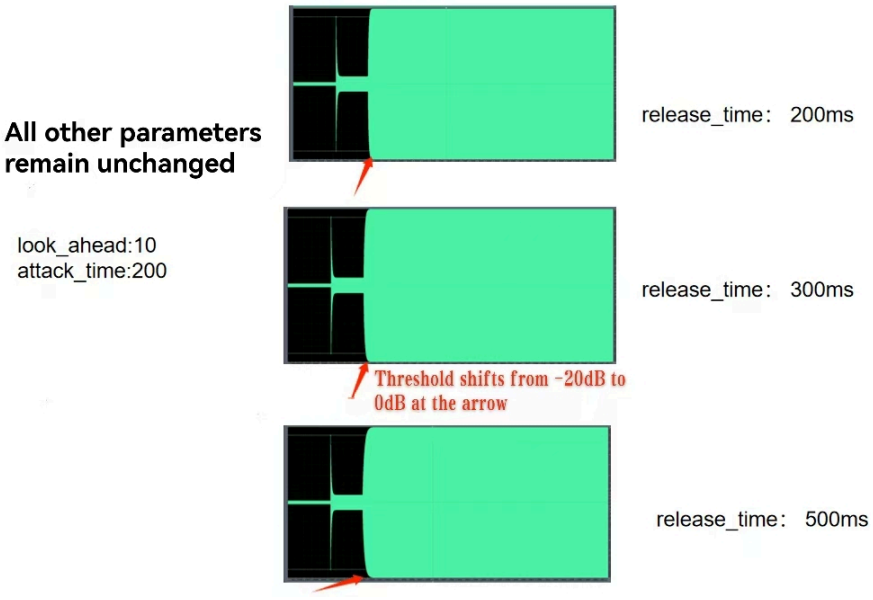


Figure 3.3.2-9 Effects of different release times on the result

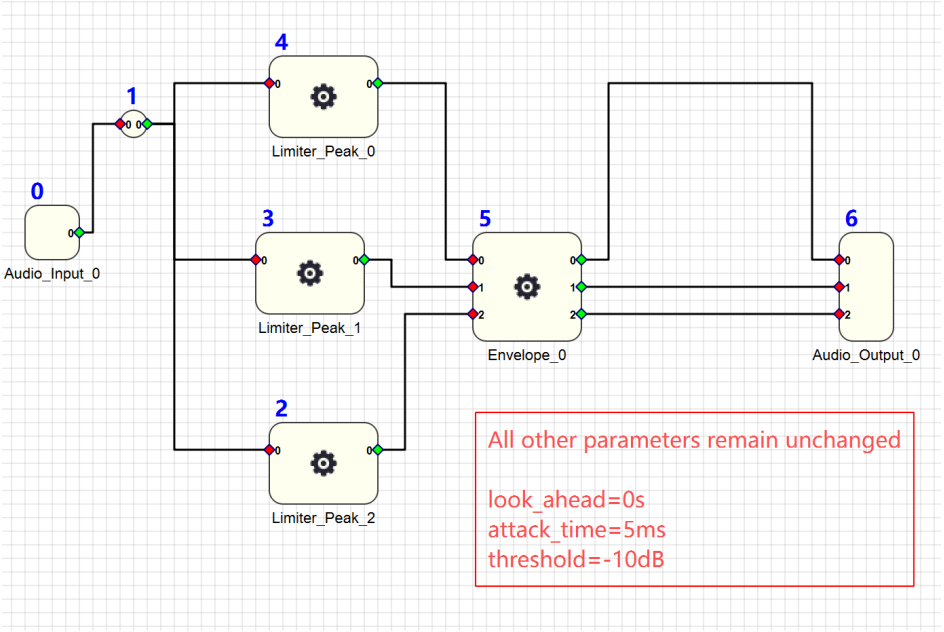


Figure 3.3.2-10 Schematic for release time comparative testing

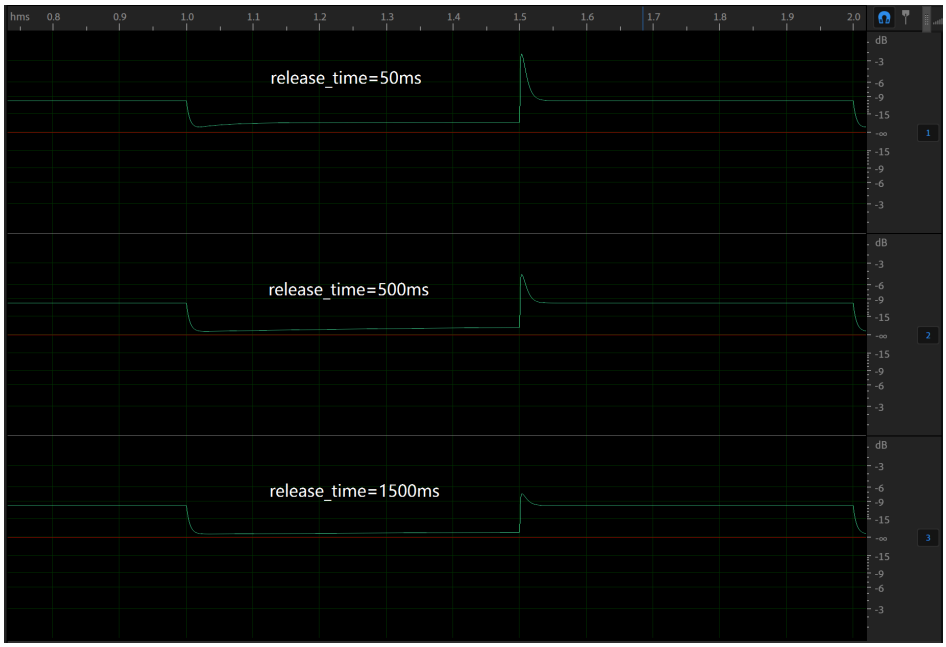


Figure 3.3.2-11 Effects of different release times on the result

3.3.3 Limiter Ratio

Function: Limiter that controls and adjusts the maximum audio output. After exceeding the threshold, the compression ratio (x/y) is set by the parameter Ratio.

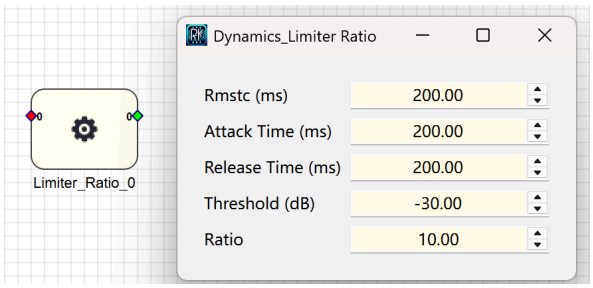


Figure 3.3.3-1 Limiter Ratio control

Number of Parameters: 4

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Rmstc (ms)	0	float	Response speed to signal changes (higher values result in slower changes)	ms	0 ~ 2000	200
Attack Time (ms)*	1	float	Speed of gain rise	ms	0 ~ 2000	200
Release Time (ms)*	2	float	Speed of gain decay	ms	0 ~ 2000	200
Threshold (dB)	3	float	Threshold, input above this value will be limited below it	dB	-80 ~ 0	-30
Ratio	4	float	Compression ratio (x/y); higher ratio results in better compression	None	1 ~ 10000	10

Demonstration of actual effect:

Input data includes music and impulse sounds, with the following case configurations:

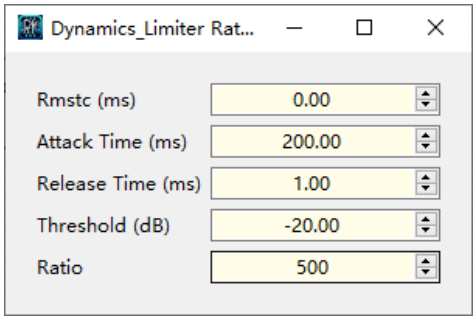
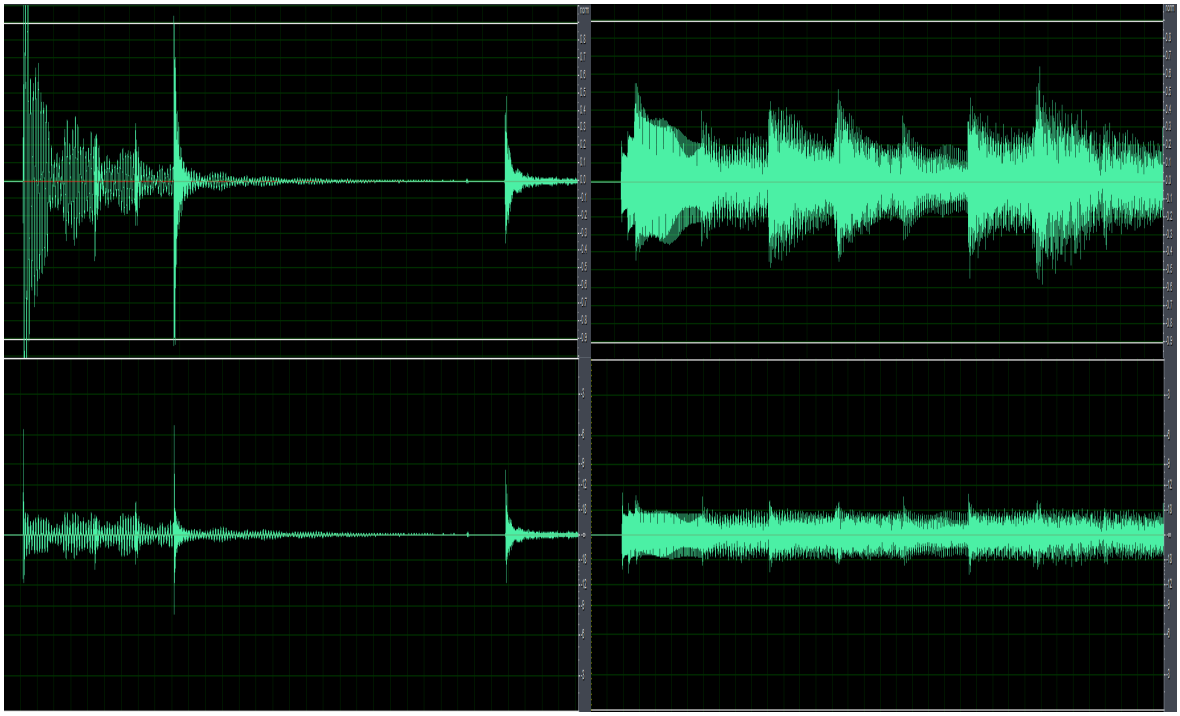


Figure 3.3.3-2 Limiter Ratio parameter settings

Figure 3.3.3-3 Input to the algorithm: Impulse sound time-domain (left) and Music time-domain (right)

Figure 3.3.3-4 Output of the algorithm: Impulse sound output time-domain (left) and Music output time-domain (right)



3.3.4 Limiter Clip

Function: When the input control signal is 1, the compression threshold slowly decreases until it reaches the threshold value; when the control signal is 0, the compression threshold slowly increases until it reaches 0.

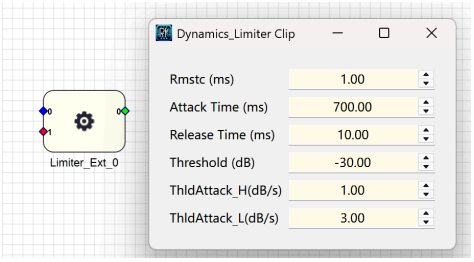


Figure 3.3.4-1 Limiter Clip control

Number of Parameters: 6

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Rmstc (ms)	0	float	Response speed to signal changes (higher values result in slower changes)	ms	0 ~ 2000	1
Attack Time (ms)*	1	float	Speed of gain rise	ms	0 ~ 2000	700

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Release Time (ms)*	2	float	Speed of gain decay	ms	0 ~ 2000	10
Threshold	3	float	Initial threshold, also the minimum threshold during threshold adjustment	dB	-80 ~ 0	-30
ThldAttack_H (dB/s)	4	float	Speed of threshold increase	dB/s	0 ~ 2000	1
ThldAttack_L (dB/s)	5	float	Speed of threshold decrease	dB/s	0 ~ 2000	3

For specific effects, refer to 3.3.1 Limiter RMS.

3.4 Noise Gate

3.4.1 Noise Gate

Function: Noise gate that compresses the signal when the input is below the threshold; when the input is above the threshold, the output is the same as the input.

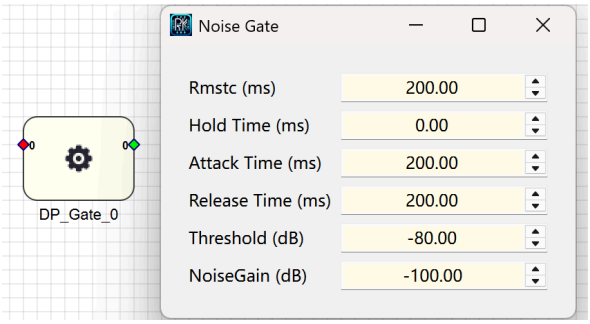


Figure 3.4-1 Noise Gate control and settings interface

Number of Parameters: 6

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Rmstc(ms)	0	float	Speed of smoothly tracking signal	ms	0 ~ 2000	20

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
			changes; larger values mean slower changes switch			
Hold Time(ms)	1	float	Duration to hold the current peak value before the envelope drops switch	ms	0 ~ 2000	0
Attack Time (ms)	2	float	Speed of gain rise	ms	0 ~ 2000	200
Release Time (ms)	3	float	Speed of gain decay	ms	0 ~ 2000	200
Threshold (dB)	4	float	Noise gate activation threshold	dB	-100 ~ 0	-80
Noise Gain (dB)	5	float	Actual attenuation value for noise	dB	-200 ~ 0	-100

Note: Unlike other dynamic processing modules that input raw dB parameters, here Threshold and Gain are converted to linear values. Threshold uses RMS dB, while Gain uses Peak dB.

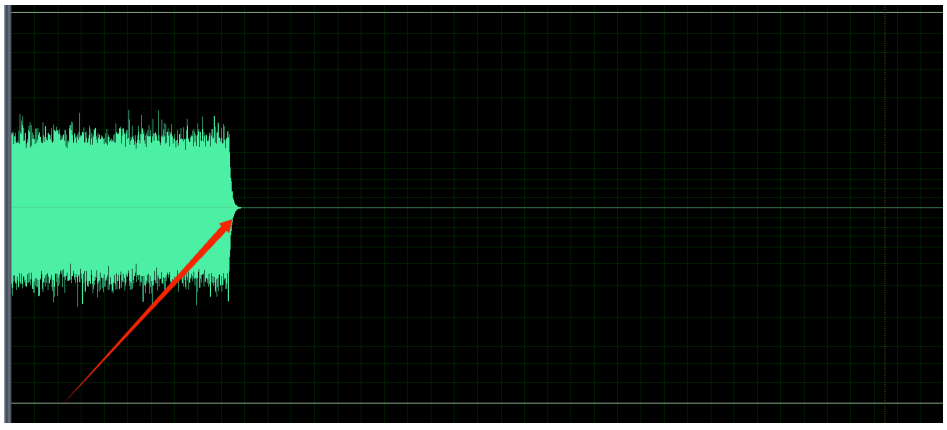


Figure 3.4.1-2 With default parameters, input is white noise, and the control is enabled at the arrowed moment

3.4.2 Noise Gate (Ext)

Function: Noise gate with an external sidechain input.

- Input 0：External control signal (Blue pin).
- Input 1：Main audio signal (Red pin).

Working Principle: The module determines whether to open or close the noise gate by detecting the level of Input 0.

- When the level of Input 0 is lower than the set threshold, the gate closes, and the signal from Input 1 is attenuated.
- When the level of Input 0 is higher than the threshold, the gate opens, and the signal from Input 1 passes through normally.

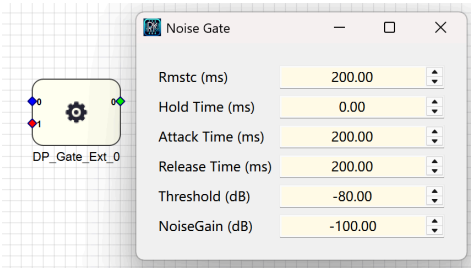


Figure 3.4.2-1 Noise Gate control and settings interface

Number of Parameters：6
The parameter definitions are consistent with the standard Noise Gate; please refer to the previous table.

3.5 Agc

Function: Automatic gain control. It controls the amplitude of the input signal to be near a target value.

Actual Gain = (Input Signal Amplitude - Target Value) / Ratio

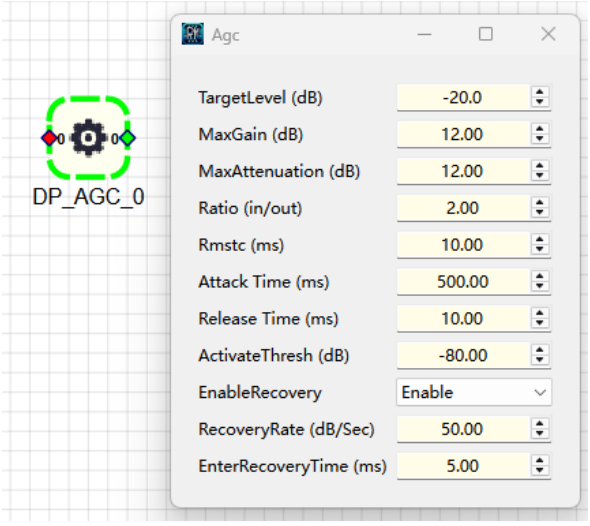


Figure 3.5-1 Agc control and settings interface

Number of Parameters：11

Parameter Name	Param Index	Type	Function	Unit	Recommended Value	Default Value
TargetLevel （dB）	0	float	Target control level	dB	-100 ~ 50	-20
MaxGain （dB）	1	float	Maximum gain value	dB	0 ~ 100	12
MaxAttenuation （dB）	2	float	Maximum attenuation value	dB	0 ~ 100	12
Ratio （in/out）	3	float	Gain change ratio	None	1 ~ 100	12
Rmstc（ms）*	2	float	Speed of smoothly tracking signal changes; larger values mean slower changes	ms	0 ~ 2000	10
Attack Time（ms）*	5	float	Gain increase speed	ms	0 ~ 2000	500
Release Time （ms）*	6	float	Gain decrease speed	ms	0 ~ 2000	10
ActivateThresh （dB）	7	float	Operation threshold; starts working when higher than this value	dB	-200 ~ 0	-80
EnableRecovery	8	float	Enable gain recovery. When enabled, if the input signal amplitude is lower than ActivateThresh, the gain will recover to 0dB; otherwise, the gain maintains the maximum gain value.	None	0/1	1
RecoveryRate (dB/Sec)*	9	float	Gain attenuation speed when the signal is lower than the threshold (during recovery)	db/Sec	1 ~ 200	50

Parameter Name	Param Index	Type	Function	Unit	Recommended Value	Default Value
EnterRecoveryTime*	10	float	During recovery, if the time the input signal amplitude is lower than ActivateThresh exceeds EnterRecoveryTime, the system is forced into the recovery state. This avoids sound anomalies caused by gain increasing when the signal drops too quickly.	ms	0 ~ 200	5

4 Filters

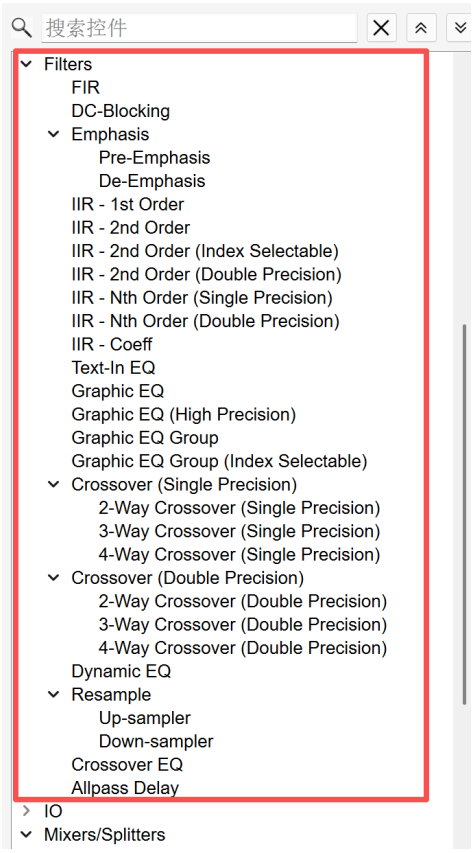


Figure 4-1 Location of the Filter in the Tool

4.1 FIR

Function Description: FIR Filter. The number 12 in the control box determines that the filter has 12 coefficients, and the filter order is the number of coefficients minus 1, which is 11. Double-clicking enters the coefficient configuration box, where you can import and export filter coefficients. You can also copy and paste by clicking on the table header.

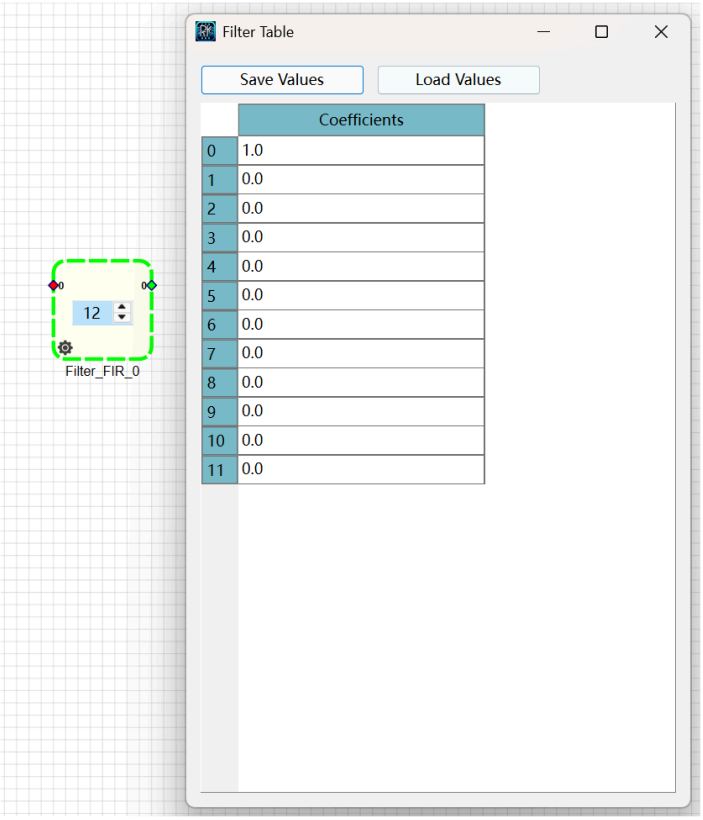


Figure 4.1-1 The number of filter coefficients is shown in the module icon

- **Tool Parameters:**
  Number of Parameters: Number of filter coefficients + 1

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Filter Order	0	int	Number of FIR filter coefficients			12
C0	1	float	First coefficient of the filter			1
C1	2	float	Second coefficient of the filter			0

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Cn...	...	float	...th coefficient of the filter			0

- Download Parameters *: Same as tool parameters

Note: Filter coefficients can be calculated in various ways. Two common methods are listed below:

- Calculated using FilterDesigner tools or other design tools. Specific methods can be found by the user; they are not listed here due to copyright reasons.
- Can also be calculated using the Python SciPy library function scipy.signal.firwin*.

4.2 DC-Blocking

Function Description: Removes DC signals using a first-order high-pass filter.

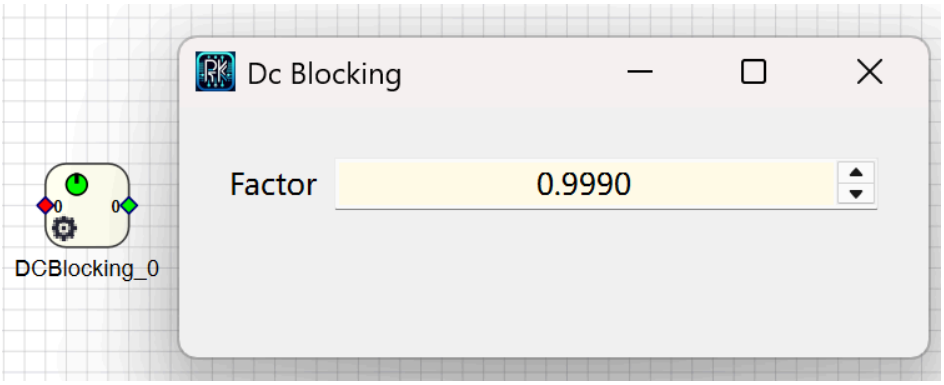


Figure 4.2-1 DC-Blocking Control

- **Tool Parameters:**
  Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	float	Algorithm enable switch		1	1
Factor	1	float	Filter coefficient; smaller values provide better filtering but may attenuate some low frequencies		0 ~ 1	0.9999

- Download Parameters *: Same as tool parameters

Note: The small button in the upper-left corner of the control is the enable switch. Green indicates 1 (enabled), and red indicates 0 (disabled). This applies to all controls below.

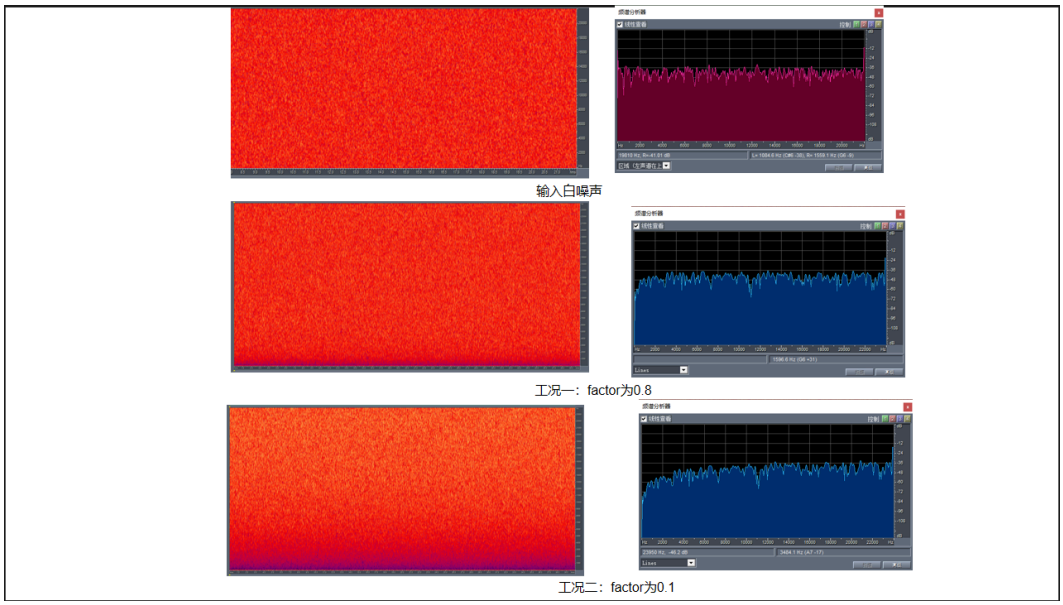


Figure 4.2-2 The left image shows the spectrum obtained by dividing the time domain into small segments for FFT, while the right image shows the spectrum over the entire time period. Comparing the effects of different parameters, it can be seen that a smaller factor results in greater attenuation of low frequencies.

4.3 Emphasis

4.3.1 Pre-Emphasis

Function Description: Pre-emphasis, which enhances high-frequency signals and attenuates low-frequency signals.

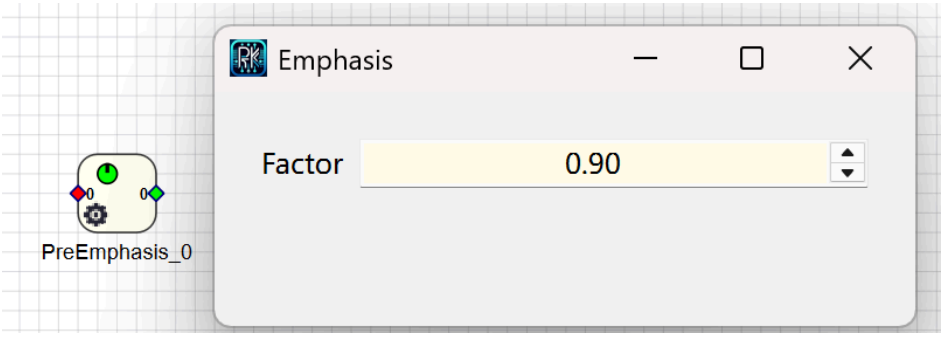


Figure 4.3.1-1 Pre-Emphasis Control and Settings Interface

- **Tool Parameters:**
  Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	float	Algorithm enable switch		1	1
Factor	1	float	Pre-emphasis coefficient; higher values provide better effect		0 ~ 1	0.9

- Download Parameters *: Same as tool parameters

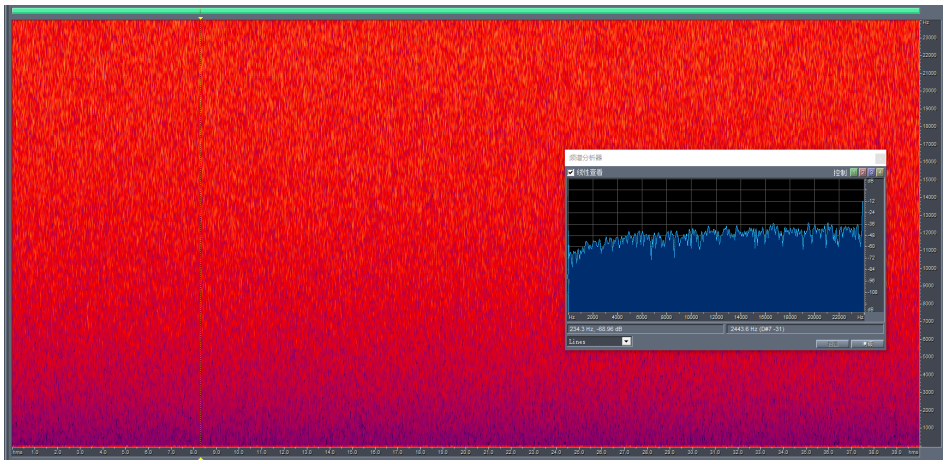


Figure 4.3.1-2 The effect when factor = 0.9, showing enhanced high frequencies and attenuated low frequencies.

4.3.2 De-Emphasis

Function Description: De-emphasis, which attenuates high-frequency signals and enhances low-frequency signals.

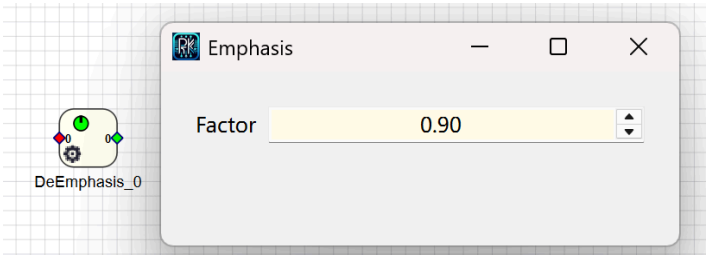


Figure 4.3.2-1 De-Emphasis Control and Settings Interface

- **Tool Parameters:**
  Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	float	Algorithm enable switch		1	1
Factor	1	float	De-emphasis coefficient; higher values provide better effect		0 ~ 1	0.9

- Download Parameters *: Same as tool parameters

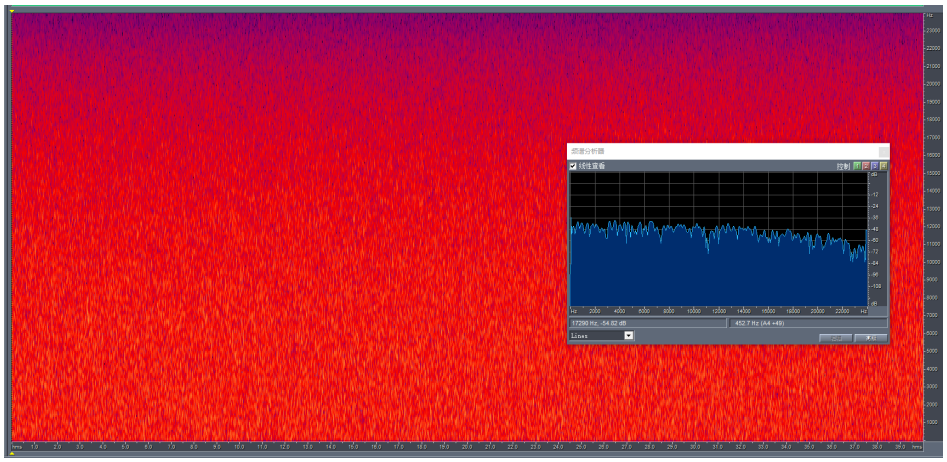


Figure 4.3.2-2 Input is white noise, with factor = 0.9, showing attenuated high frequencies and enhanced low frequencies.

4.4 IIR - 1st Order

Function Description: General first-order filter, with the following 5 filter types available:

Type	Description
LP1st	Low-pass filter
HP1st	High-pass filter
AP1st	All-pass filter
LS1st	Low-shelf filter
HS1st	High-shelf filter

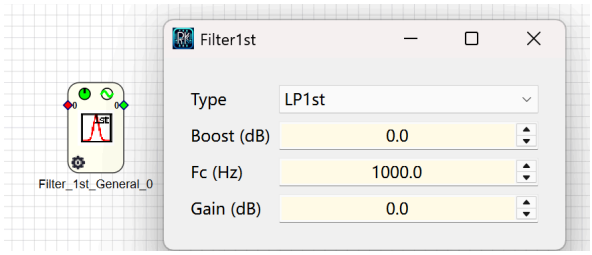


Figure 4.4-1 IIR-1st Order Control and Settings Interface

- **Tool Parameters:**
  Number of Parameters: 6

Tool Parameter	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	int	Enable switch		1	1
Polarity	1	int	Inversion switch		0	0
Type	2	int	Filter type, refer to function description		0 - 4	0
Boost(dB)*	3	float	Filter gain	dB	-20 ~ 20	0
Fc(Hz)	4	float	Filter center frequency/cutoff frequency	Hz	20 ~ 24000	1000
Gain(dB)	5	float	Overall output gain	dB	-20 ~ 20	0

Note: In this control, only the LS low-shelf filter and HS high-shelf filter can set the Boost parameter; the Boost parameter is invalid for other filters.

- Download Parameters *: Parameter conversion logic for this control*
  Number of Parameters: 3

Download Parameter	Parameter Index	Type	Function
b0	0	float	Filter numerator coefficient b0
b1	1	float	Filter numerator coefficient b1
a1	2	float	Filter denominator coefficient a1

Note: a0 is fixed at 1, so it is not listed in the parameter settings; this applies to all controls below.

The function of the first-order filter is consistent with the second-order filter described later, although their frequency response curves differ. We will provide comparisons below:

For the low-pass filter (LP), the comparison between first-order and second-order filters is as follows:

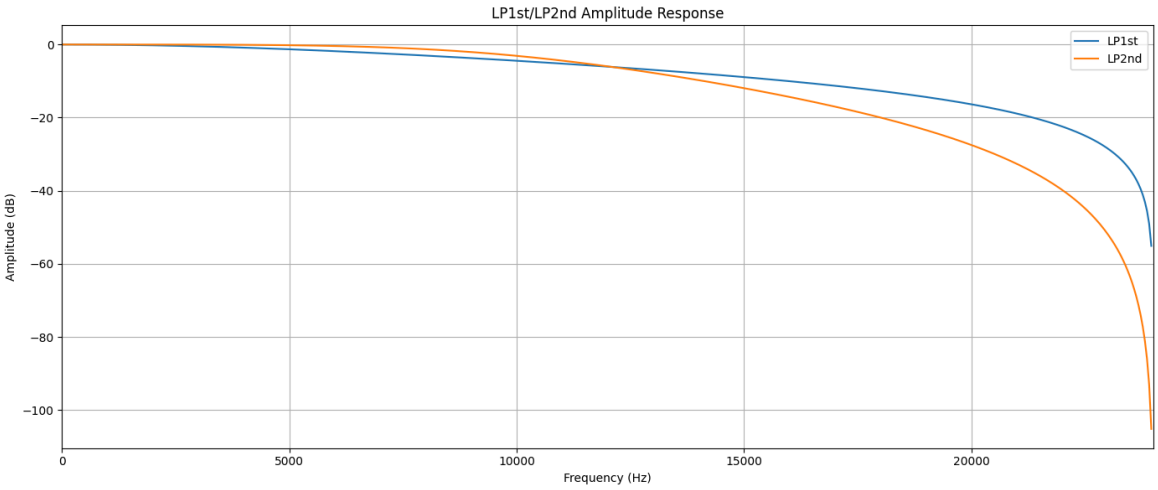


Figure 4.4-2 Theoretical frequency response curves of LP1st and LP2nd filters

For the high-pass filter (HP), the comparison between first-order and second-order filters is as follows:

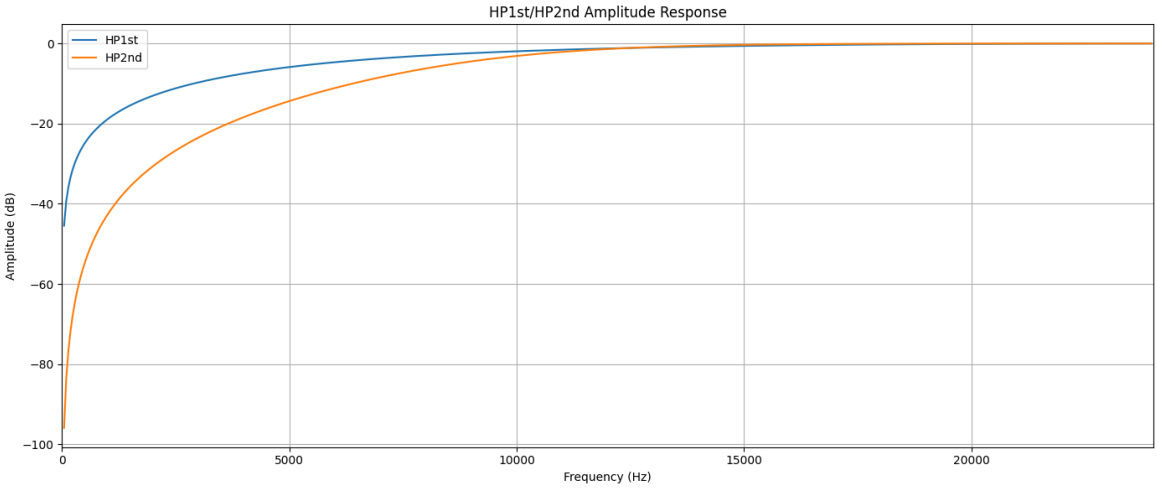


Figure 4.4-3 Theoretical frequency response curves of HP1st and HP2nd filters

For the all-pass filter (AP), the comparison between first-order and second-order filters is as follows:

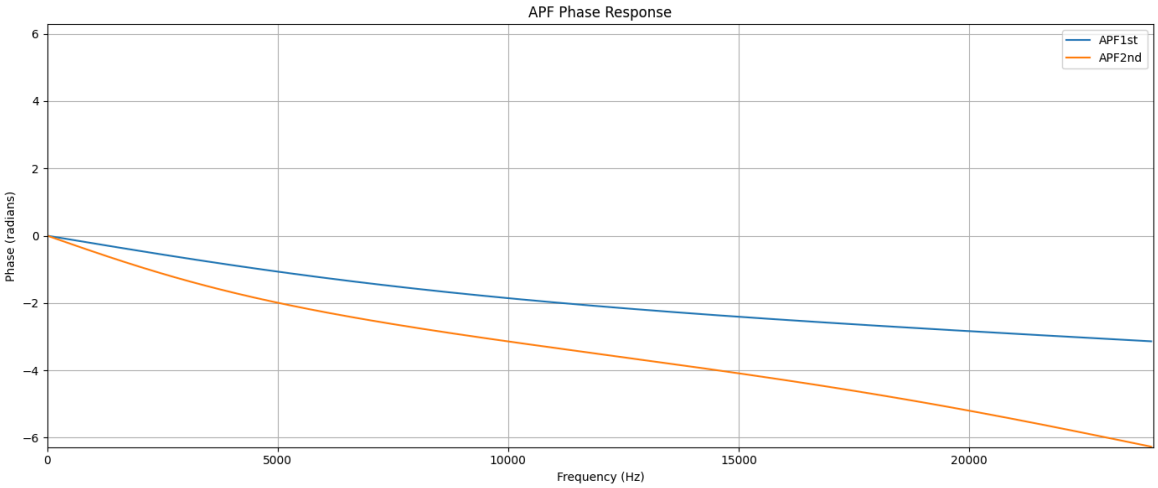


Figure 4.4-4 Theoretical frequency response curves of AP1st and AP2nd filters

For the low-shelf filter (LS), the comparison between first-order and second-order filters is as follows:

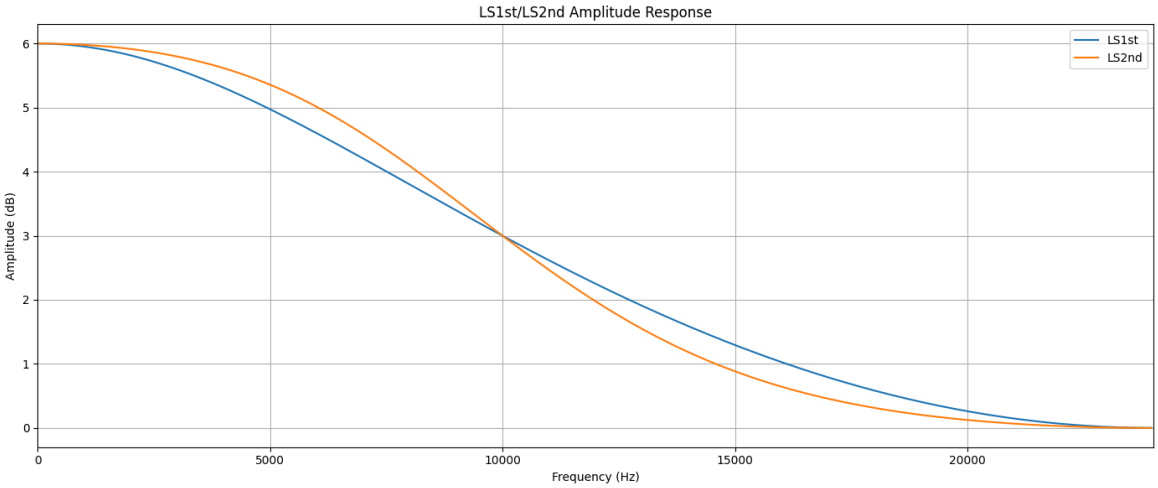


Figure 4.4-5 Theoretical frequency response curves of LS1st and LS2nd filters

For the high-shelf filter (HS), the comparison between first-order and second-order filters is as follows:

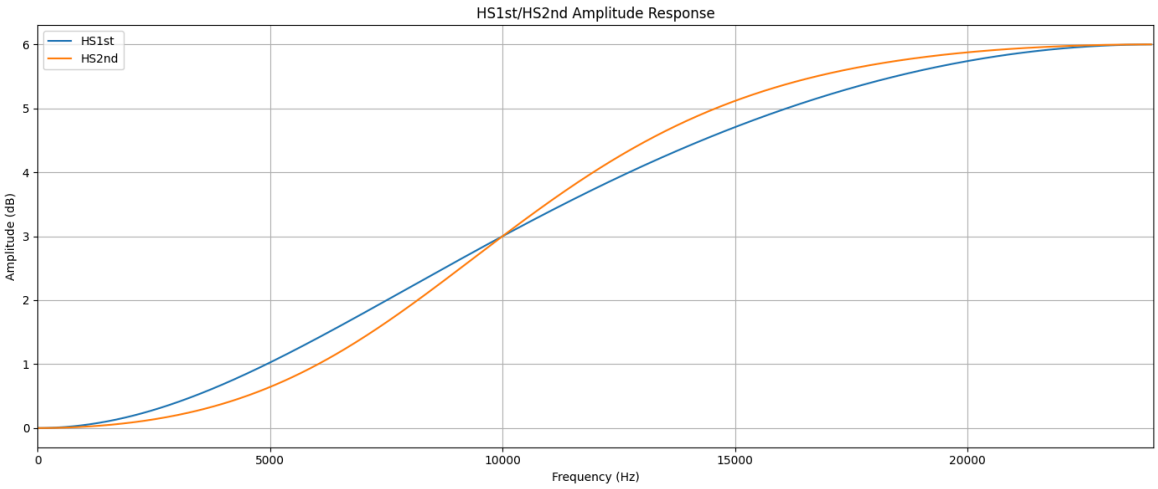


Figure 4.4-6 Theoretical frequency response curves of HS1st and HS2nd filters

4.5 IIR - 2nd Order

4.5.1 IIR - 2nd Order

Function Description: General second-order filter, with the following 16 filter types available:

Type	Description
Parametric	Parametric filter
LowShelf	Low-shelf filter
HighShelf	High-shelf filter
GeneralLP	Low-pass filter
GeneralHP	High-pass filter
GeneralBP	Band-pass filter
GeneralBS	Band-stop filter
AFP	All-pass filter
Peaking	Peaking filter
Notch	Notch filter
ChebyshevLP	Chebyshev low-pass filter
ChebyshevHP	Chebyshev high-pass filter
BesselLP	Bessel low-pass filter
BesselHP	Bessel high-pass filter
ButterLP	Butterworth low-pass filter
ButterHP	Butterworth high-pass filter

- **Tool Parameters:**
  Number of Parameters: 7 (Some filters do not require all parameters, and the tool will disable the corresponding input boxes)

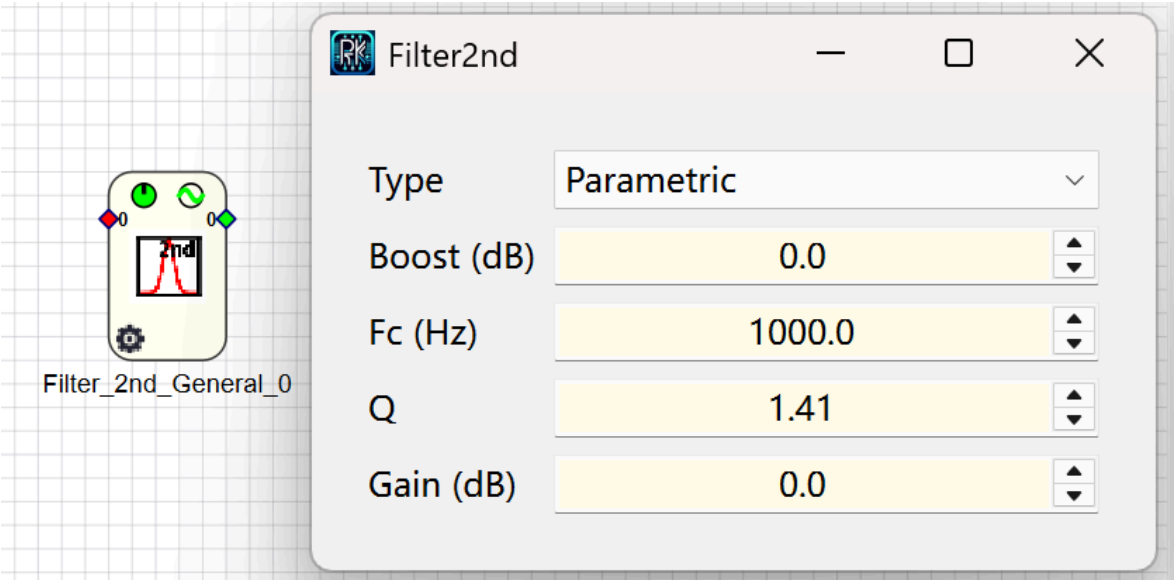


Figure 4.5.1-1 IIR_2nd Control and Parameter Settings

- Note the positions of Enable and Polarity.
  - Enable corresponds to the upper-left button; green indicates Enable=1, and red indicates Enable=0.
  - Polarity corresponds to the upper-right button; green indicates Polarity=0, and red indicates Polarity=1.

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	int	Enable switch		1	1
Polarity	1	int	Inversion switch		0	0
Type	2	int	Filter type, refer to function description		0 - 15	0
Boost(dB)*	3	float	Filter gain	dB	-20 ~ 20	0
Fc(Hz)	4	float	Filter center frequency	Hz	20 ~ 24000	1000
Q	5	float	Filter quality factor		0.01 ~ 16	1.41
Gain(dB)	6	float	Overall output gain	dB	-20 ~ 20	0

- Download Parameters *: Parameter conversion logic for this control*
  Number of Parameters: 5

Download Parameter	Parameter Index	Type	Function
b0	0	float	Filter numerator coefficient b0
b1	1	float	Filter numerator coefficient b1
b2	2	float	Filter numerator coefficient b2
a1	3	float	Filter denominator coefficient a1
a2	4	float	Filter denominator coefficient a2

Note: a0 is fixed at 1, so it is not listed in the parameter settings; this applies to all controls below.

You may notice two gain control parameters, Boost and Gain*. The former controls the center gain of the filter, while the latter is the overall gain. The latter can be understood as an additional gain applied to the filter output. The following figure intuitively shows the effect of the Gain parameter.

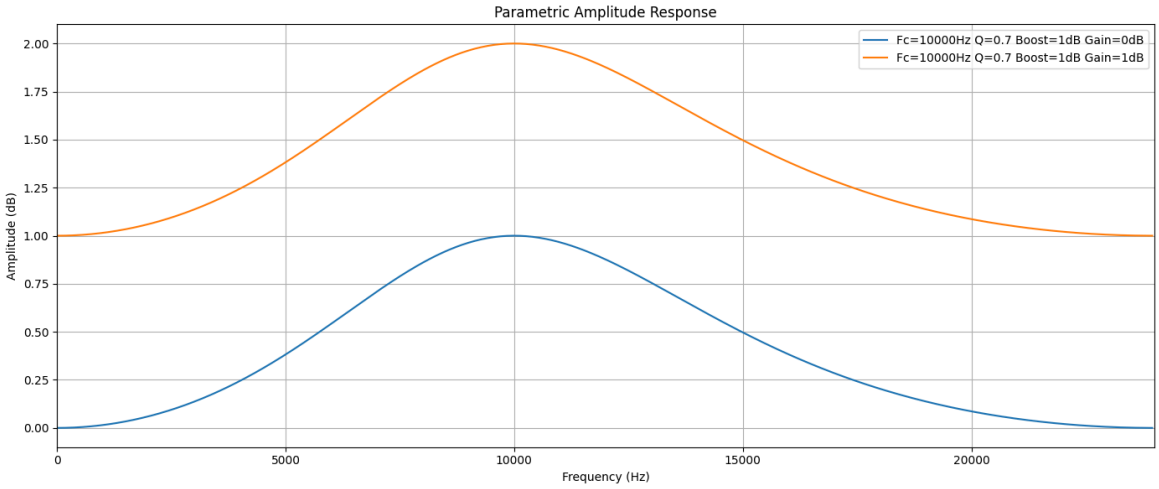


Figure 4.5.1-2 The Gain parameter has the effect of lifting the overall frequency response curve of the filter.

Theoretical frequency response curves are shown below. We will display the theoretical frequency response curves for each filter to help users intuitively understand the function of each filter and the effects of the Fc parameter, Q parameter, and Boost parameter on the amplitude-frequency response curve.

For Parametric Filter
Show the effects of the Fc parameter, Q parameter, and Boost parameter on the amplitude-frequency response curve.

- Fc: The Fc parameter determines the center frequency position affected by the filter.
- Boost: The Boost parameter controls the gain of the filter at the center frequency.

- Q: The Q parameter affects the range of frequencies near the center frequency that the filter affects. Higher Q values result in narrower affected frequency bands.

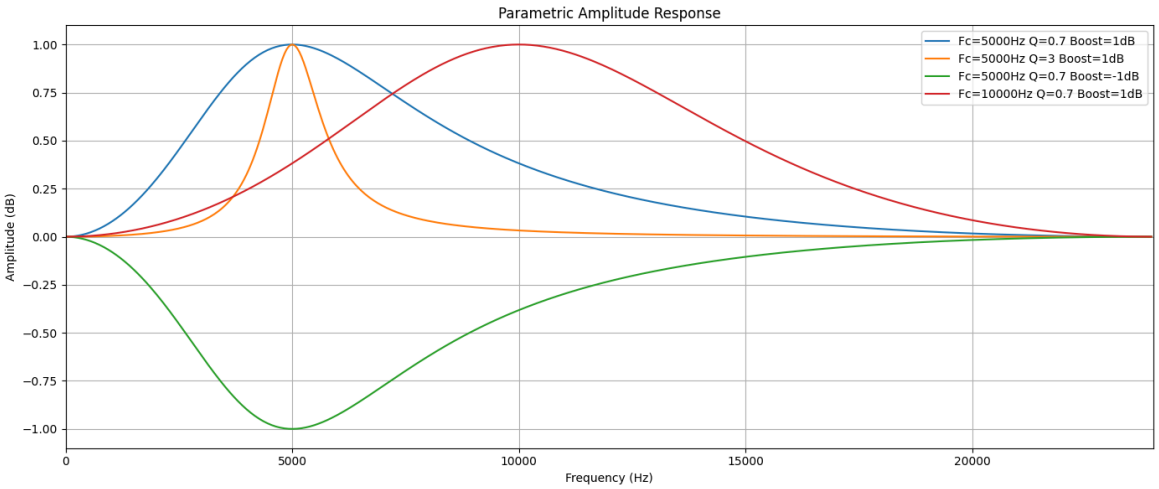


Figure 4.5.1-3 Theoretical frequency response curve of the Parametric filter

For LowShelf and HighShelf Filters

LowShelf affects the low-frequency range, while HighShelf affects the high-frequency range. Show the effects of the Fc parameter, Q parameter, and Boost parameter on the amplitude-frequency response curve.

- Boost parameter: This is the maximum gain of the filter.
- Fc parameter: This determines the frequency at which the gain drops to half.
- Q parameter: This affects the speed of gain change near Fc.

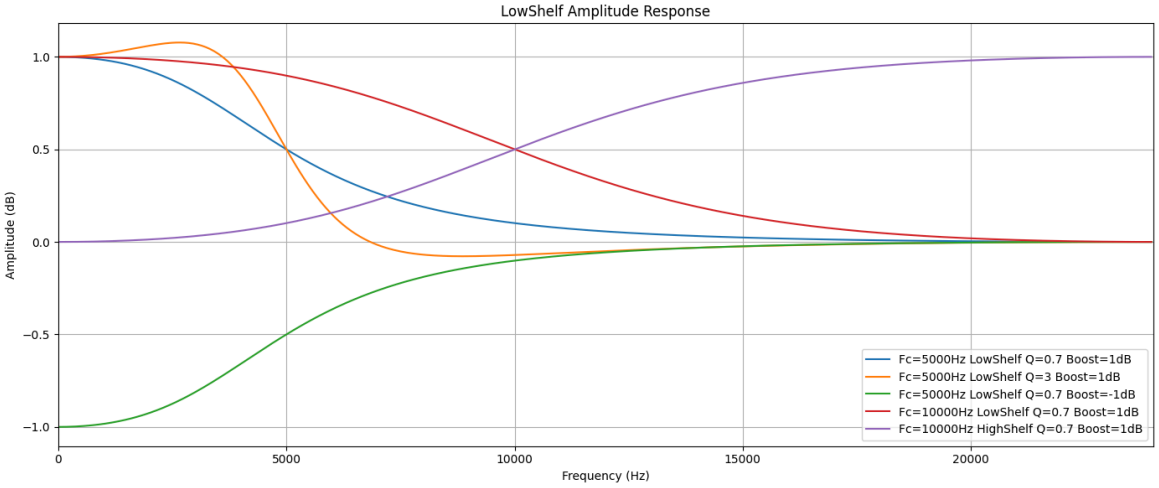


Figure 4.5.1-4 Theoretical frequency response curves of LowShelf and HighShelf filters

For GeneralLP and GeneralHP

Show the effects of the Fc parameter and Q parameter on the amplitude-frequency response curve.

- Fc parameter: This is the cutoff frequency.

- Q parameter: This affects the speed of gain change near Fc. When Q > $\sqrt{2}$, a peak will appear near Fc, and the curve is flattest when Q = $\sqrt{2}$.
- Boost: These controls have no Boost parameter.

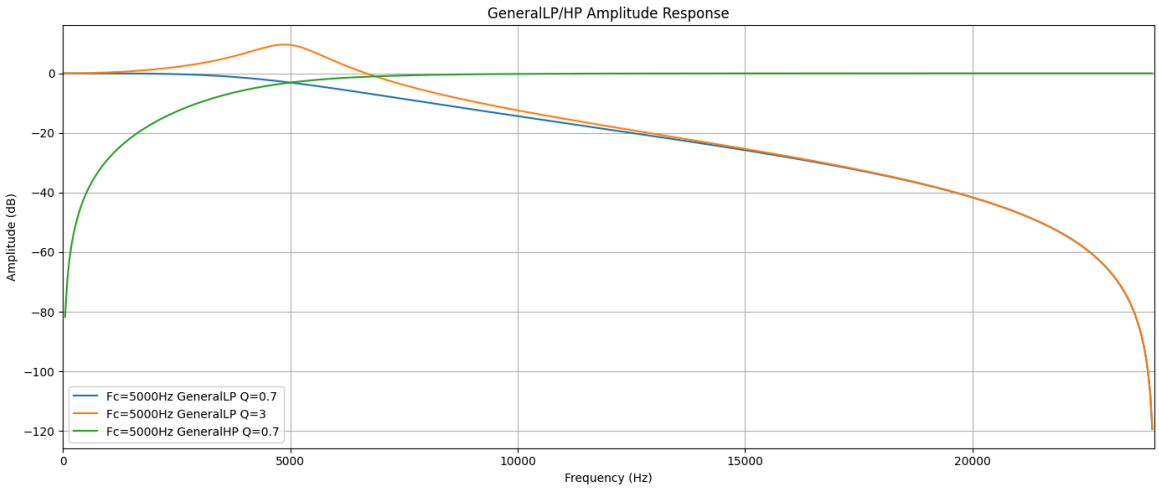


Figure 4.5.1-5 Theoretical frequency response curves of GeneralLP and GeneralHP filters

For GeneralBP and GeneralBS

Show the effects of the Fc parameter and Q parameter on the amplitude-frequency response curve. These filters have no Boost parameter.

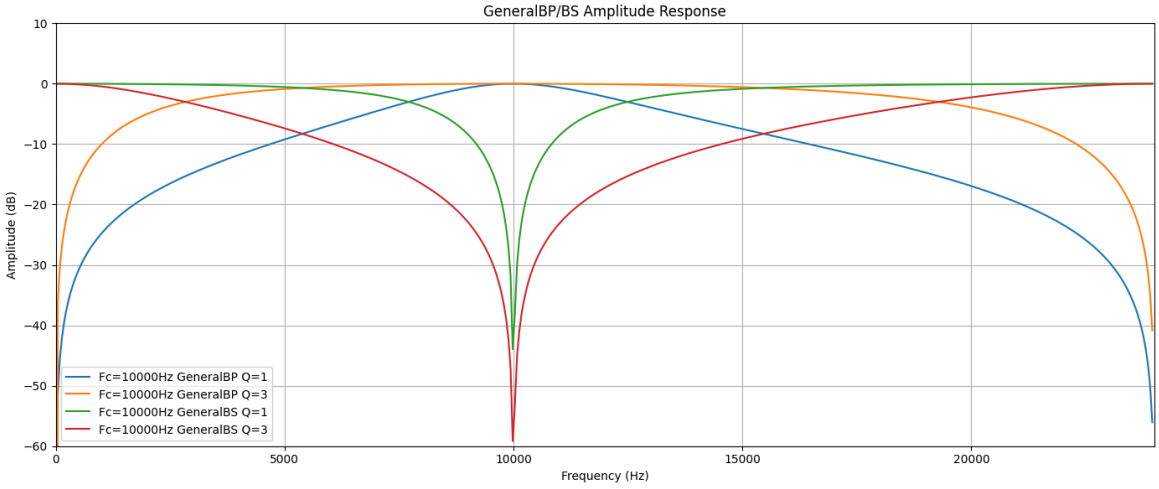


Figure 4.5.1-6 Theoretical frequency response curves of GeneralBP and GeneralBS filters

For APF (All-Pass Filter)

The APF does not affect the signal's amplitude but affects its phase. Show the effects of the Fc parameter and Q parameter on the phase-frequency response curve. These filters have no Boost parameter.

- Fc parameter: This determines the frequency at which the filter causes a -π phase shift.
- Q parameter: This affects the speed of phase change near the center frequency.

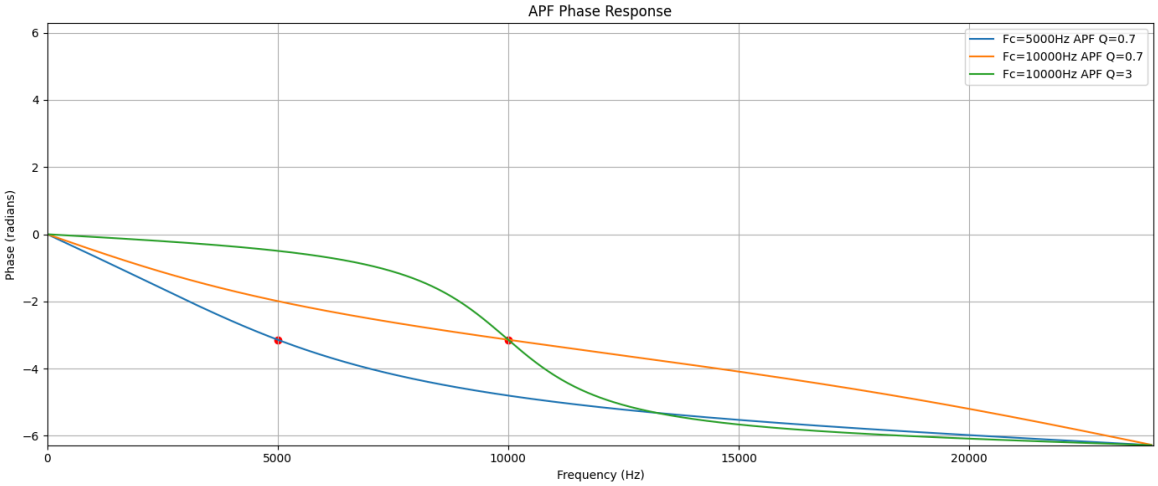


Figure 4.5.1-7 Theoretical frequency response curve of the APF filter

For Peaking and Notch Filters

Peaking filters are similar to GeneralBP, while Notch filters are similar to GeneralBS. However, the former has a more concentrated effect for the same Q value. Show the effects of the Fc parameter and Q parameter on the amplitude-frequency response curve. These filters have no Boost parameter.

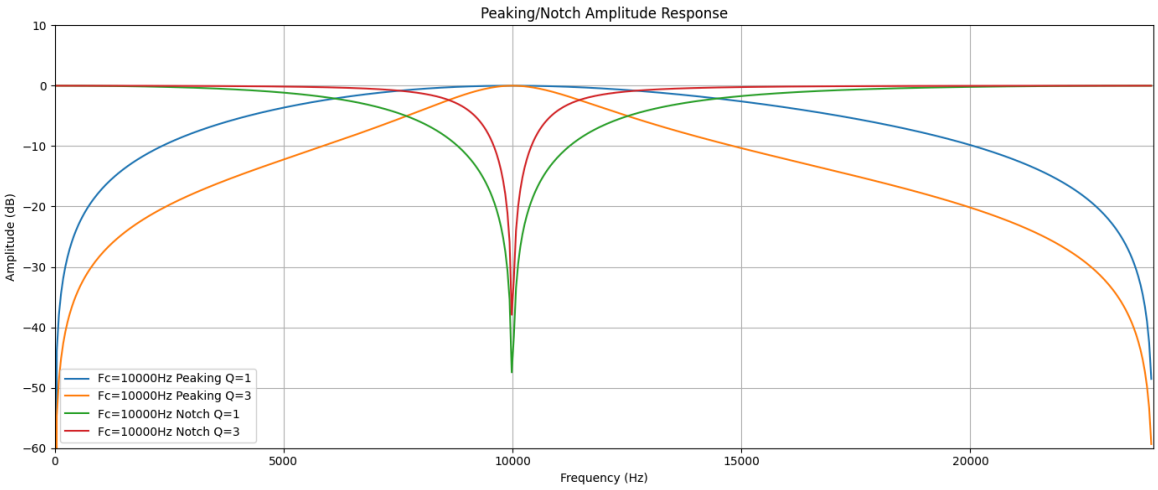


Figure 4.5.1-8 Theoretical frequency response curves of Peaking and Notch filters

For Butterworth and Bessel Filters

These filters have no Q or Boost parameters.

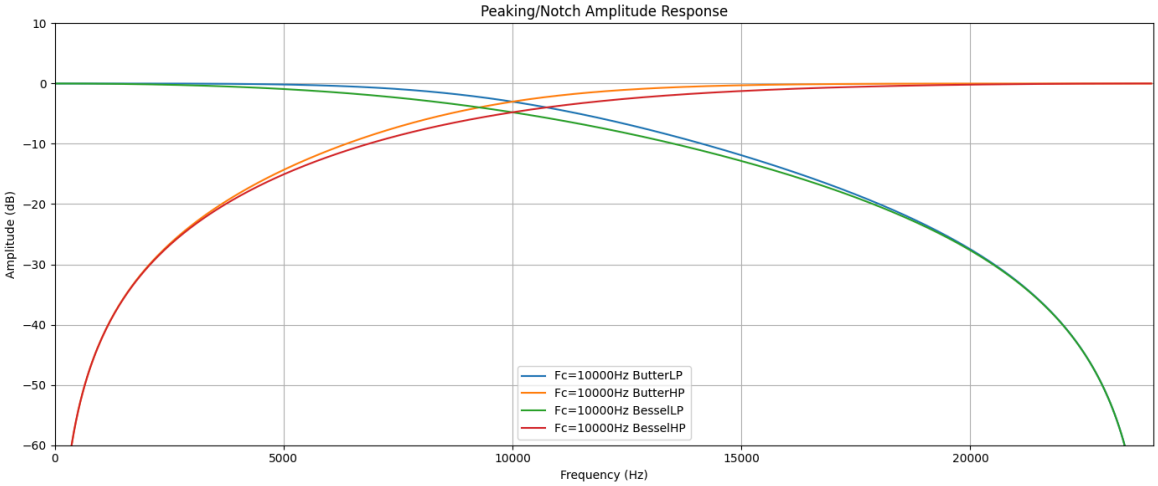


Figure 4.5.1-9 Theoretical frequency response curves of Butterworth and Bessel filters

Demonstration of Actual Effects:

Input two channels of white noise sources. Each channel of white noise passes through a filter separately to obtain the output. Depending on the parameters, there are the following typical cases: :

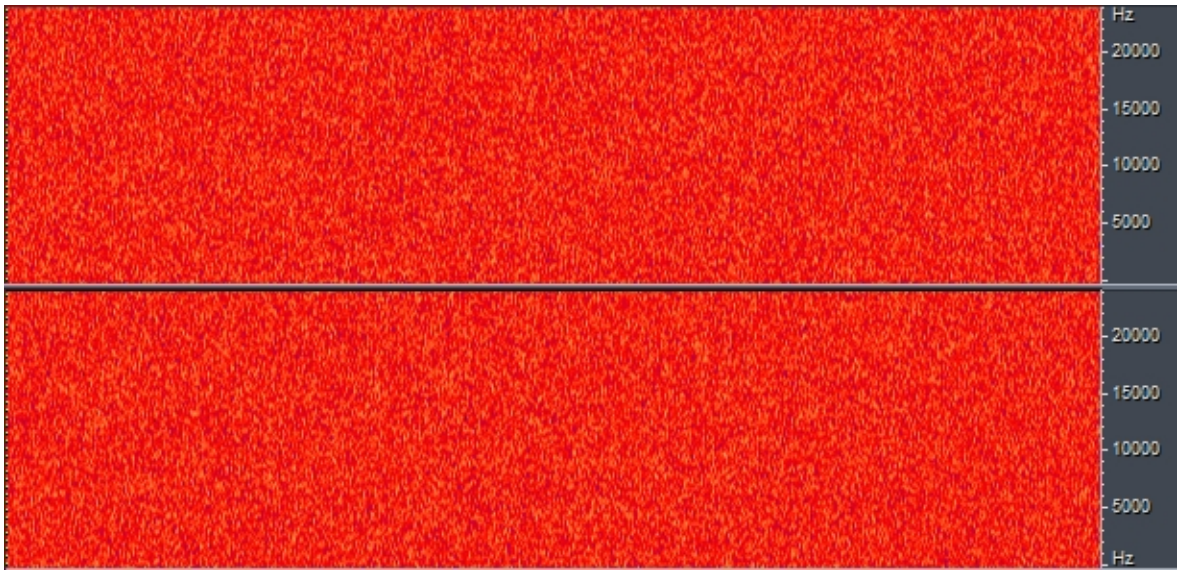


Figure 4.5.1-10 Input White Noise Spectrum

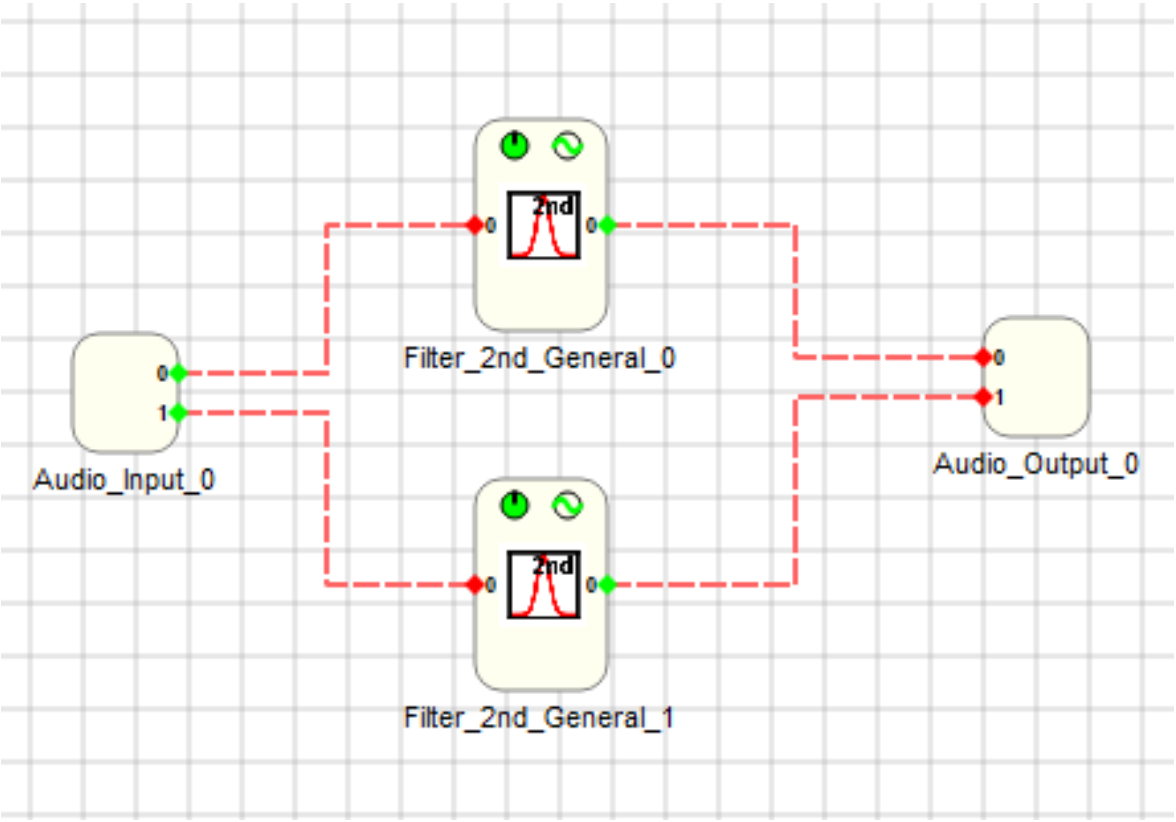


Figure 4.5.1-11 Case 1 General Audio Data Flow Diagram

Case 1: (Demonstrating the effect of the Q value)

- Channel 0 parameters configured in order: [1, 0, 0, -10, 5000, 0.7, 0] (Q set to 0.7)
- Channel 1 parameters configured in order: [1, 0, 0, -10, 5000, 15, 0] (Q set to 15)

The output is as follows:

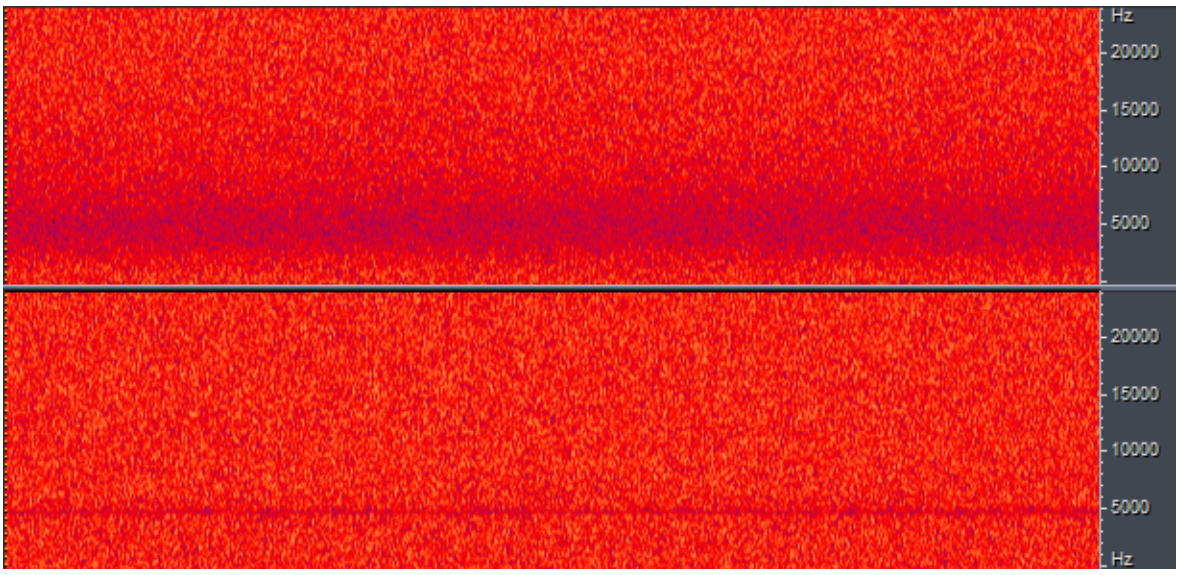


Figure 4.5.1-12 RKStudio Output Frequency Domain Diagram. The larger the Q value, the more concentrated the Parametric filter's effect, and the narrower the affected frequency band.

Case 2: (LowShelf Filter and HighShelf Filter)

- Channel 0 parameters configured in order: [1, 0, 1, 10, 5000, 1.41, 0] (The third parameter type is 1, which is a LowShelf filter)
- Channel 1 parameters configured in order: [1, 0, 2, 10, 5000, 1.41, 0] (The third parameter type is 2, which is a HighShelf filter)

The output is as follows:

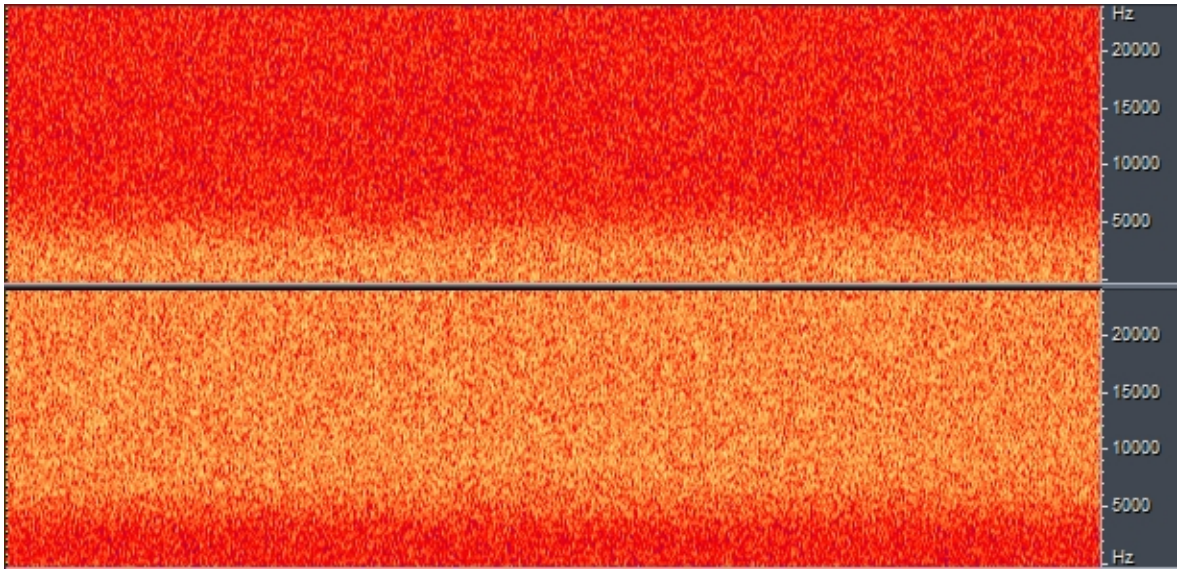


Figure 4.5.1-13 RKStudio Output Frequency Domain Diagram. LowShelf affects the part below the center frequency, and HighShelf affects the part above the center frequency

Case 3: (BP BandPass and BS BandStop)

- Channel 0 parameters configured in order: [1, 0, 5, 0, 5000, 1.41, 0] (The third parameter type is 5, which is a BandPass filter)
- Channel 1 parameters configured in order: [1, 0, 6, 0, 5000, 1.41, 0] (The third parameter type is 6, which is a BandStop filter)

The output is as follows:

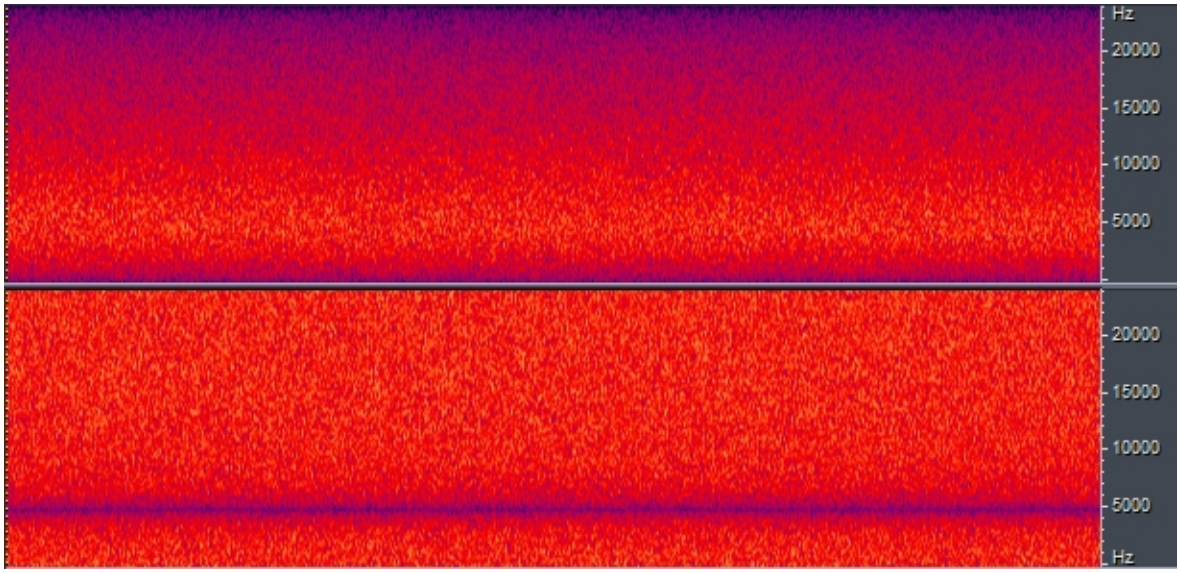


Figure 4.5.1-14 RKStudio Output Frequency Domain Diagram. GeneralBP preserves the part near the center frequency, while GeneralBS blocks the part near the center frequency. Adjusting the Q value affects the range of influence for both filters

Case 4: (Peaking Filter and Notch Filter)

- Channel 0 parameters configured in order: [1, 0, 8, 0, 5000, 1.41, 0] (The third parameter type is 8, which is a Peaking filter)
- Channel 1 parameters configured in order: [1, 0, 9, 0, 5000, 1.41, 0] (The third parameter type is 9, which is a Notch filter)

The output is as follows:

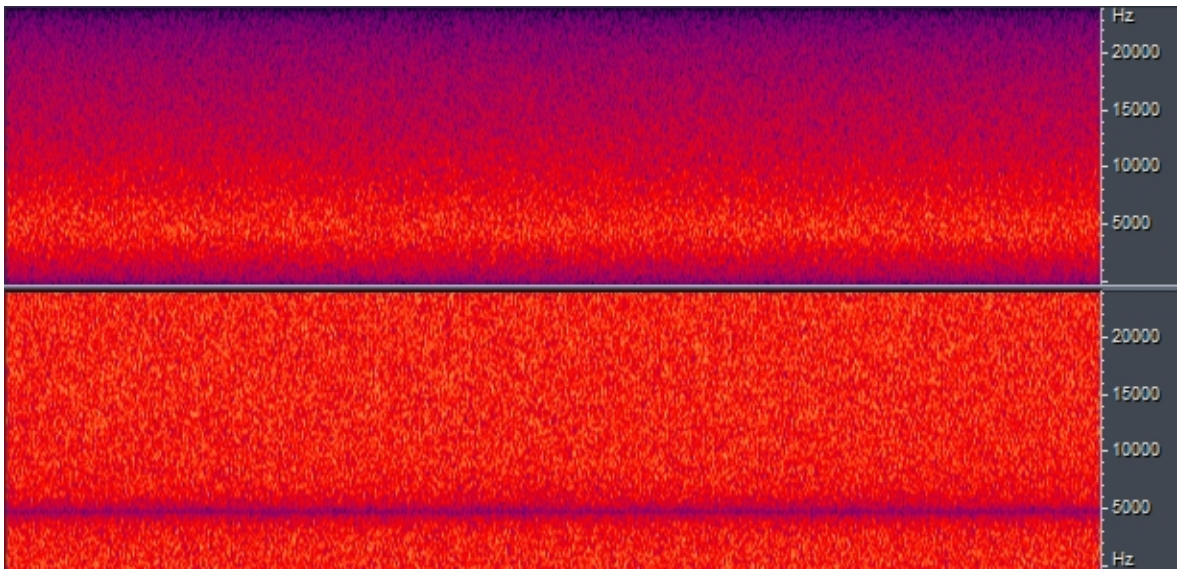


Figure 4.5.1-15 RKStudio Output Frequency Domain Diagram. The Peaking filter is similar to GeneralBP, and the Notch filter is similar to GeneralBS, but with the same Q value, the range of influence for the former is more concentrated

4.5.2 IIR - 2nd order (Index Selectable)

Function: Indexed second-order IIR filter. The function is identical to the standard IIR - 2nd Order, but adds an Index parameter. It allows users to pre-configure multiple sets of filter parameters (e.g., for different sound effect modes or scenes) and quickly switch between entire sets of filter parameters by changing the Index value. It is generally used for multi-level EQ configurations, bass/treble enhancement, and other scenarios.

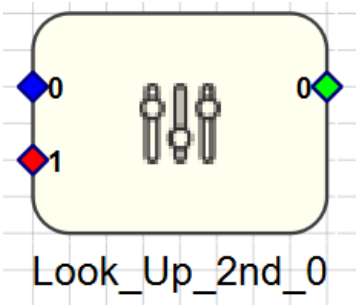


Figure 4.5.2-1 IIR - 2nd order (Index Selectable) Control

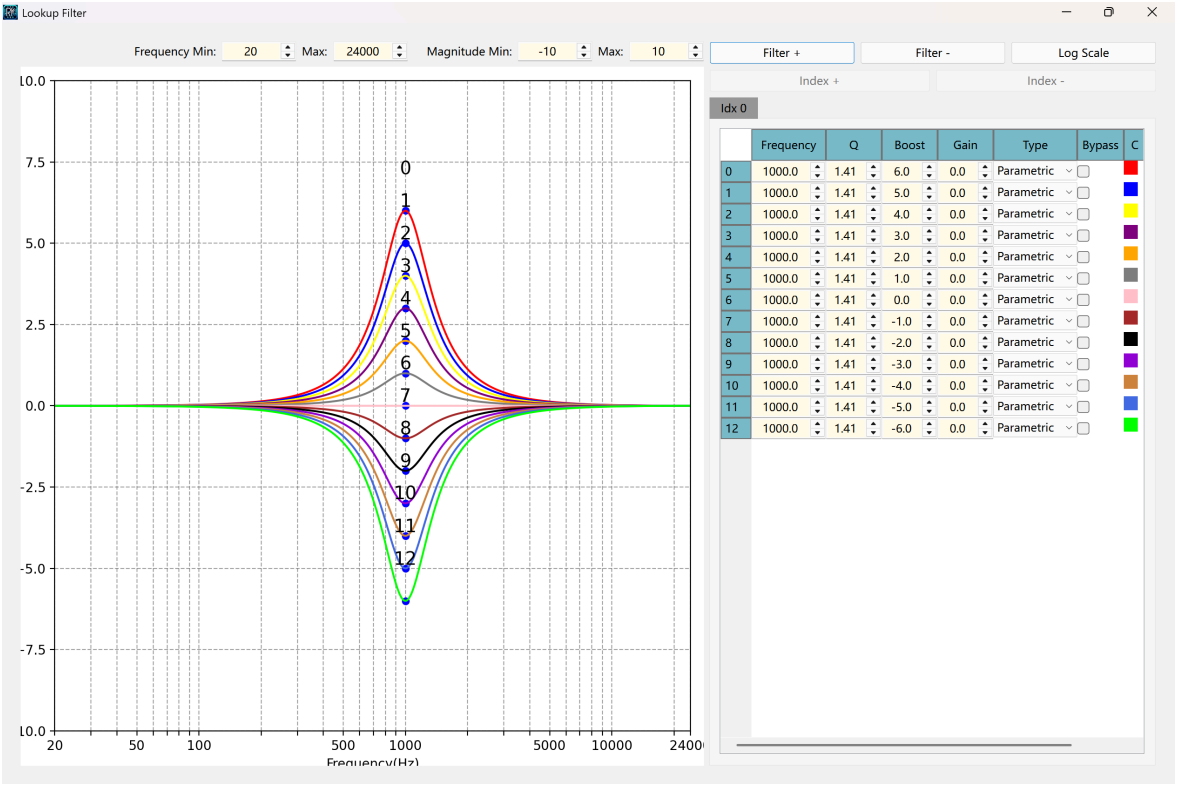


Figure 4.5.2-2 IIR - 2nd order (Index Selectable) Parameter Settings Interface

4.5.3 IIR - 2nd order (Double Precision)

Function: Double-precision second-order IIR filter. The algorithm logic and control parameters are completely consistent with the standard IIR - 2nd Order, but the calculation precision is higher, and the computational cost is also greater.

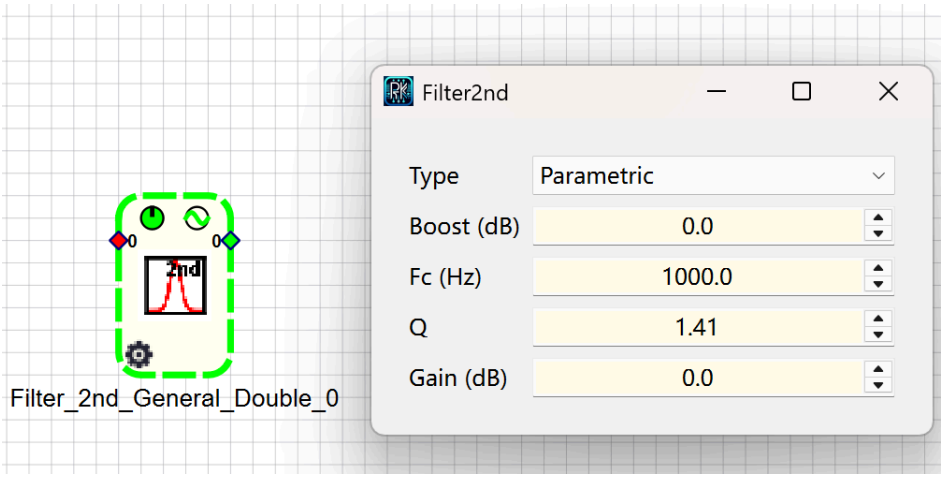


Figure 4.5.3-1 IIR - 2nd order (Double Precision) Control and Settings Interface

4.6 IIR Nth Order (Single Precision)

Function Description: High-order filter using single-precision floating-point arithmetic. It is actually implemented using cascaded second-order IIR filters biquads*. The following 8 filter types are available:

Type	Description
ButterLP	Butterworth low-pass filter
ButterHP	Butterworth high-pass filter
BesselLP	Bessel low-pass filter
BesselHP	Bessel high-pass filter
Cheby1LP	Chebyshev Type I low-pass filter
Cheby1HP	Chebyshev Type I high-pass filter
Cheby2LP	Chebyshev Type II low-pass filter
Cheby2HP	Chebyshev Type II high-pass filter

- **Tool Parameters:**
  Number of Parameters: 6 (Some filters do not require all parameters, and the tool will disable the corresponding input boxes)

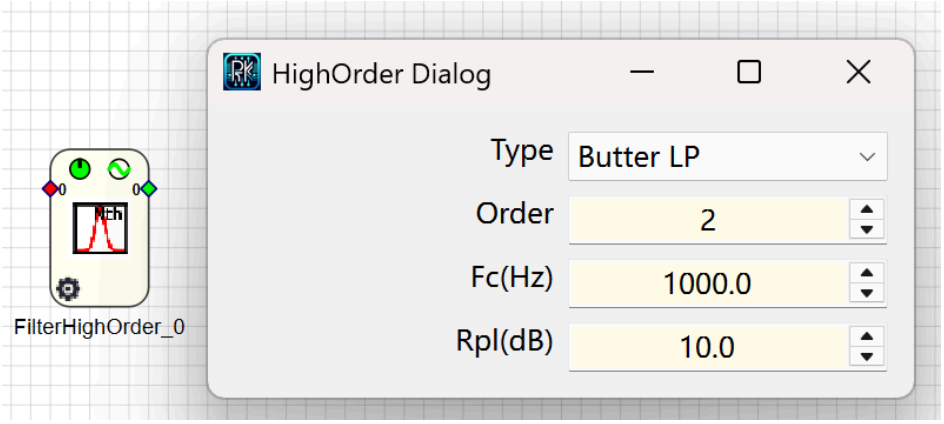


Figure 4.6-1 Locations of parameters in the module

Note: The positions of Enable and Polarity are as follows. The upper-left button is green when Enable=1 and red when Enable=0. The upper-right button is green when Polarity=0 and red when Polarity=1.

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	int	Enable switch		1	1
Polarity	1	int	Inversion switch		0	0
Type	2	int	Filter type, refer to function description		0 - 7	0
Order	3	int	Filter order (must be even due to biquad cascades)		2 - 10	2
Fc(Hz)	4	float	Filter center frequency	Hz	20 ~ 24000	1000
Rpl(dB)	5	float	Maximum gain within the passband	dB	0 ~ 100	10

- Download Parameters *: Parameter conversion logic for this control*
  Number of Parameters: (Filter Order / 2) * 5 + 1

Download Parameter	Parameter Index	Type	Function
NumSos	0	int	Number of biquad (second-order IIR) cascades
_0_b0	1	float	Numerator coefficient b0 of the first filter
_0_b1	2	float	Numerator coefficient b1 of the first filter

Download Parameter	Parameter Index	Type	Function
_0_b2	3	float	Numerator coefficient b2 of the first filter
_0_a1	4	float	Denominator coefficient a1 of the first filter
_0_a2	5	float	Denominator coefficient a2 of the first filter
_1_b0	6	float	Numerator coefficient b0 of the second filter
_x_yn	...	float	Numerator/Denominator coefficient n of the x-th filter

Note: a0 is fixed at 1, so it is not listed in the parameter settings; this applies to all controls below.

4.7 IIR Nth Order (Double Precision)

Function Description: High-order filter using double-precision floating-point arithmetic. It has the same functionality as 4.6 IIR Nth Order (Single Precision), but with higher precision and longer computation time. Single-precision floating-point arithmetic is generally sufficient for most applications.

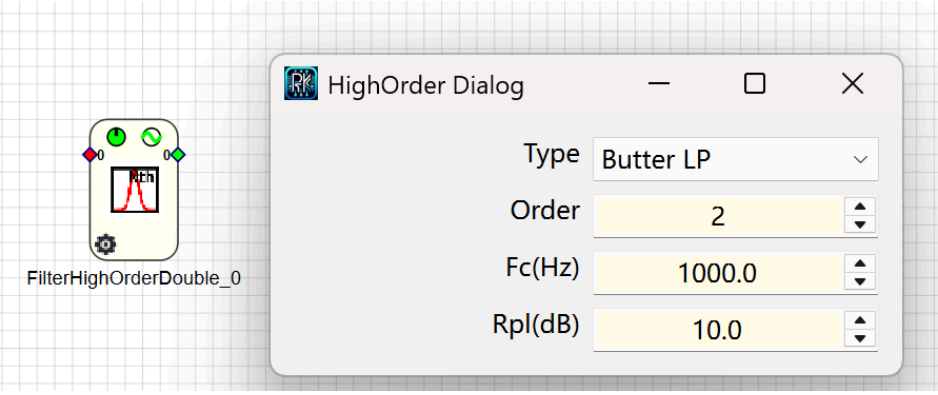


Figure 4.7-1 Parameter settings for high-order IIR filter with double-precision floating-point arithmetic

4.8 IIR - Coeff

Function Description: IIR filter that allows direct import of parameters, suitable for cases where the filter coefficients have already been designed. The control box indicates that the filter consists of 2 cascaded biquads*, each with 5 parameters (b0, b1, b2, a1, a2), resulting in a total of 10 coefficients. Double-clicking enters the coefficient configuration box, where you can import and export filter coefficients. You can also copy and paste by clicking on the table header.

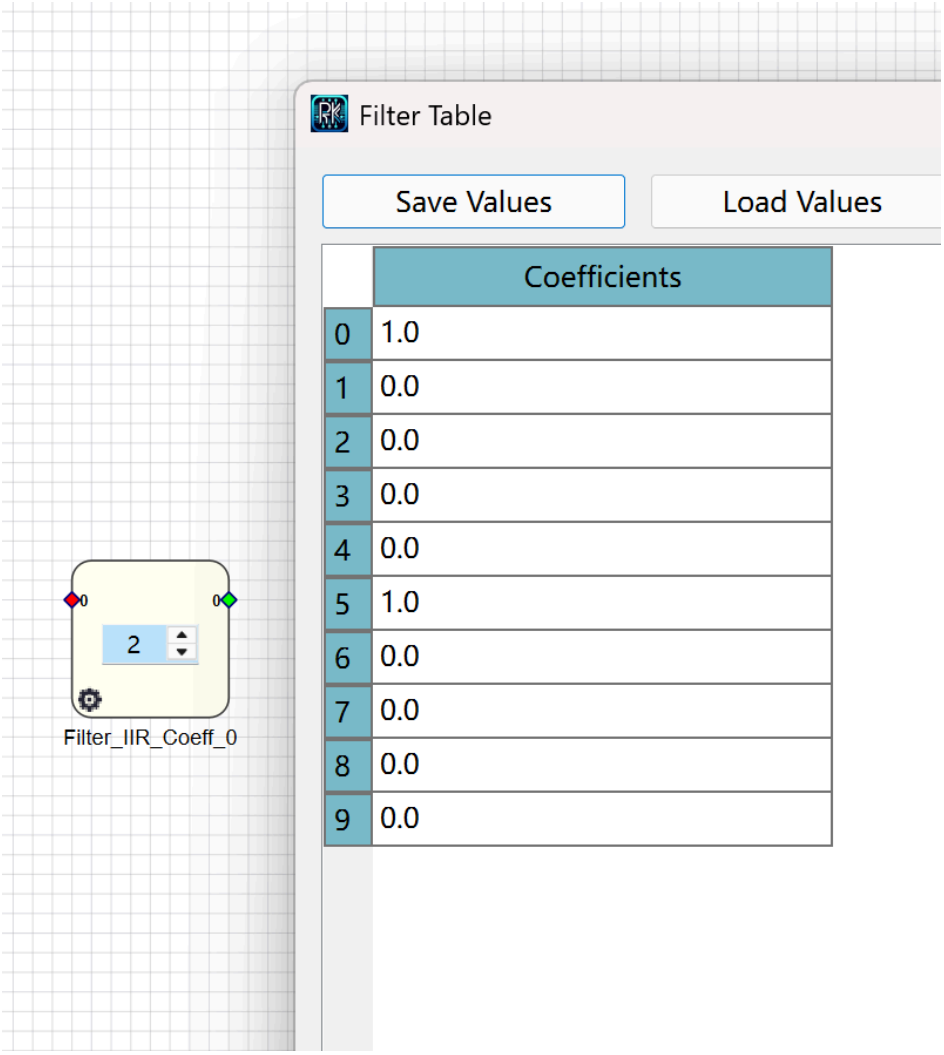


Figure 4.8-1 The number of cascaded biquads is shown in the module icon

- **Tool Parameters:**

  Number of Parameters: Number of cascaded biquads * 5 + 1

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
numCoeffs	0	int	Number of cascaded biquad filters		1	10
B0_0	1	float	First numerator coefficient of the first filter		-1000 ~ 1000	1
B1_0	2	float	Second numerator coefficient of the first filter		-1000 ~ 1000	0
B2_0	3	float	Third numerator coefficient of the first filter		-1000 ~ 1000	0

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
A1_0	4	float	First denominator coefficient of the first filter		-1000 ~ 1000	0
A2_0	5	float	Second denominator coefficient of the first filter		-1000 ~ 1000	0
B0_1	6	float	First numerator coefficient of the second filter		-1000 ~ 1000	1
B/A_x_y	...	float	Numerator/Denominator coefficient n of the x-th filter		-1000 ~ 1000	

- Download Parameters *: Same as tool parameters

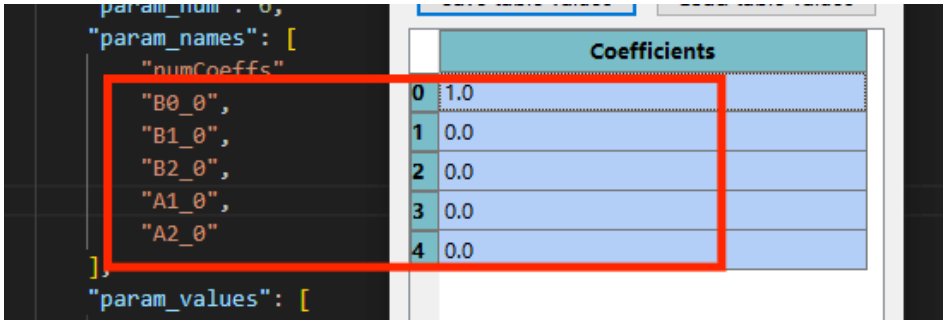


Figure 4.8-2 Correspondence between JSON file and tool settings parameters, using 1 cascaded biquad as an example

Note:

- The JSON file for the project is generally located in the project directory under the name `project_name_param.json` .

4.9 Graphical EQ

4.9.1 Graphical EQ

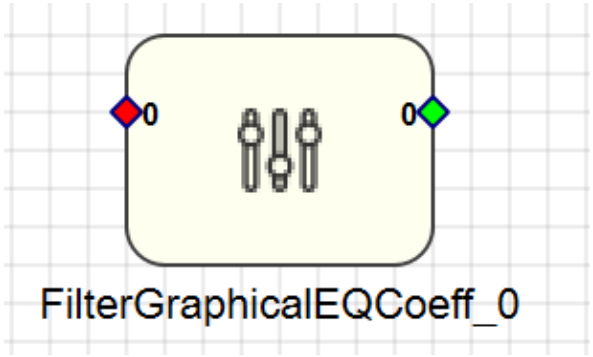


Figure 4.9.1-1 Graphical EQ Control

Function Description: Graphical EQ filter (equalizer), which uses multiple filters to attenuate or enhance signals in different frequency bands. Essentially, it is a series of filters working together to achieve the desired equalization effect by boosting desired frequency bands and attenuating undesired ones. The following filter types are available:

Type	Description
Parametric	Parametric filter
LowShelf	Low-shelf filter
HighShelf	High-shelf filter
GeneralLP	Low-pass filter
GeneralHP	High-pass filter
ButterLP	Butterworth low-pass filter
ButterHP	Butterworth high-pass filter
BesselLP	Bessel low-pass filter
BesselHP	Bessel high-pass filter
Allpass	All-pass filter

- **Tool Parameters:**
  Number of Parameters: n * 6 (6 refers to the number of parameters for each filter in the EQ) + 3

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
SmoothTime(ms)	0	float	Smoothing time	ms	20 ~ 22000	1000

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Reserved	1		Reserved			
NumFilter	2	int	Number of filters (add by clicking 'add', remove by clicking 'remove')			10
Freq	1*n + 2	float	Filter center frequency	Hz	20 ~ 22000	1000
Boost*	2*n + 2	float	Filter gain	dB	-20 ~ 20	0
Gain	3*n + 2	float	Overall output gain	dB	-20 ~ 20	0
Q	4*n + 2	float	Filter quality factor		0.01 ~ 16	0.71
Type	5*n + 2	int	Filter type, refer to function description		0 - 9	0
Bypass	6*n + 2	int	Bypass switch		0	0

- Download Parameters *: Parameter conversion logic for this control*

Each filter is implemented using a second-order filter, with 5 coefficients per filter.

Number of Parameters: n * 5 + 3

Download Parameter	Parameter Index	Type	Function
SmoothTime_ms	0	float	Smoothing time
Reserved	1		Reserved
NumFilter	2	int	Number of filters
_0_b0	3	float	Numerator coefficient b0 of the first filter

Download Parameter	Parameter Index	Type	Function
_0_b1	4	float	Numerator coefficient b1 of the first filter
_0_b2	5	float	Numerator coefficient b2 of the first filter
_0_a1	6	float	Denominator coefficient a1 of the first filter
_0_a2	7	float	Denominator coefficient a2 of the first filter
_1_b0	8	float	Numerator coefficient b0 of the second filter
_x_yn	...	float	Numerator/Denominator coefficient n of the x-th filter

Clicking the button in the middle of the module enters the EQ settings interface. Users can add or remove filters in the parameter box on the right, input the Fc, Boost, and Q parameters for each filter, and select different filter types. Users can also bypass certain filters, which will be marked in red on the frequency response curve on the left and will not participate in the calculation of the overall frequency response curve.

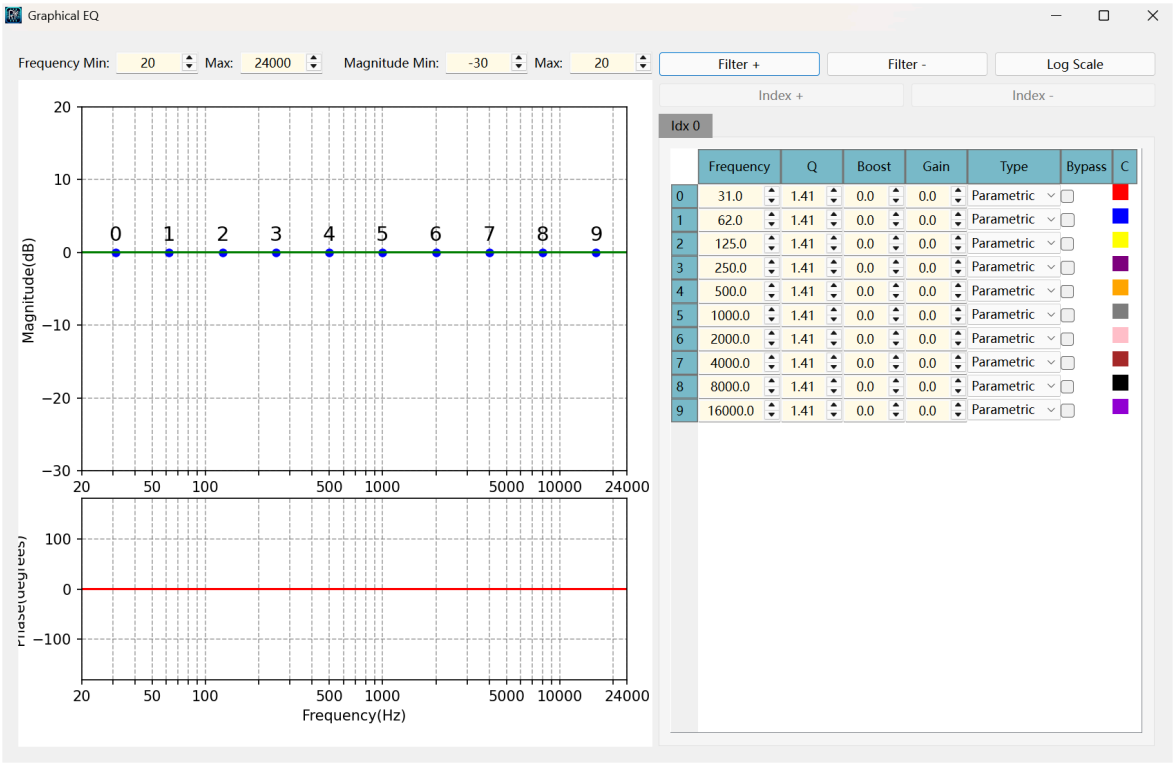


Figure 4.9.1-2 Frequency response curve of Graphical EQ

Function demonstration: In audio processing, it is often necessary to boost the low-frequency part to make the sound richer, highlight vocals, and attenuate high-frequency sharp sounds. For a more noticeable effect, we set the gain of the 2000 Hz band to 6 dB with a Q factor of 15; the gain of the 4000 Hz band to 6 dB with a Q factor of 7; and attenuate signals above 7000 Hz.

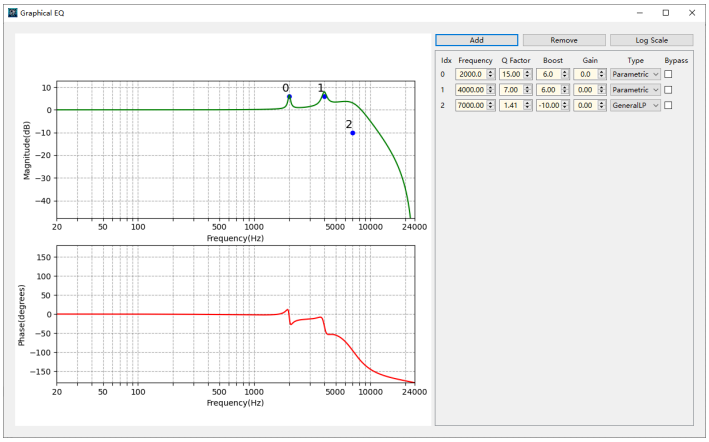


Figure 4.9.1-3 Graphical EQ parameter settings

Input is white noise.

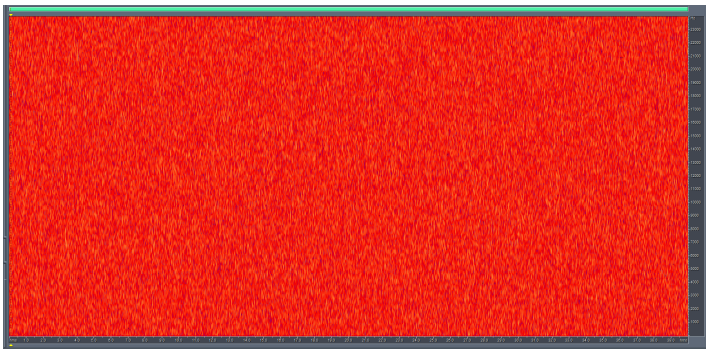


Figure 4.9.1-4 Input white noise spectrum

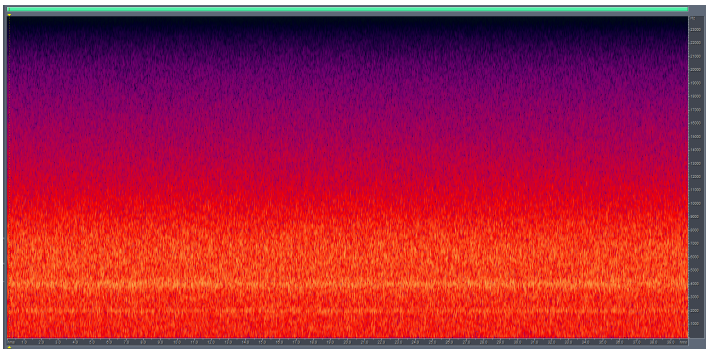


Figure 4.9.1-5 Output spectrum

From the figures, it can be seen that there is a significant boost at the 2000 Hz and 4000 Hz bands, and a significant attenuation above the 7000 Hz band. Additionally, comparing Figure 4.9-3 (Graphical EQ parameter settings) with the output spectrum, it is evident that higher Q values result in more concentrated effects and narrower affected frequency bands.

4.9.2 Graphic EQ (High Precision)

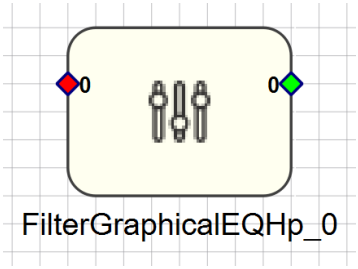


Figure 4.9.2-1 Graphic Equalizer (High Precision) Control

4.10 Graphical EQ Group

4.10.1 Graphical EQ Group

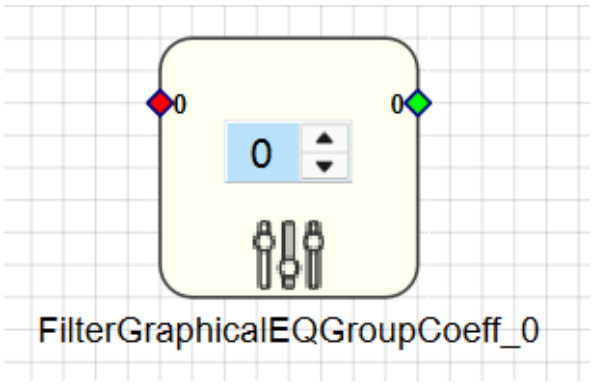


Figure 4.10.1-1 Graphical EQ Group Control

Function Description: Graphical EQ filter group, with the following filter types available:

Type	Description
Parametric	Parametric filter
LowShelf	Low-shelf filter
HighShelf	High-shelf filter
GeneralLP	Low-pass filter
GeneralHP	High-pass filter
ButterLP	Butterworth low-pass filter
ButterHP	Butterworth high-pass filter
BesselLP	Bessel low-pass filter
BesselHP	Bessel high-pass filter
Allpass	All-pass filter

Function Description: Graphical EQ filter, which has the same basic functionality as Graphical EQ but can set multiple filter groups. The active group can be switched, and only the settings of the selected group take effect. The Group=1 on the control indicates that the current filter group in use is the first one.

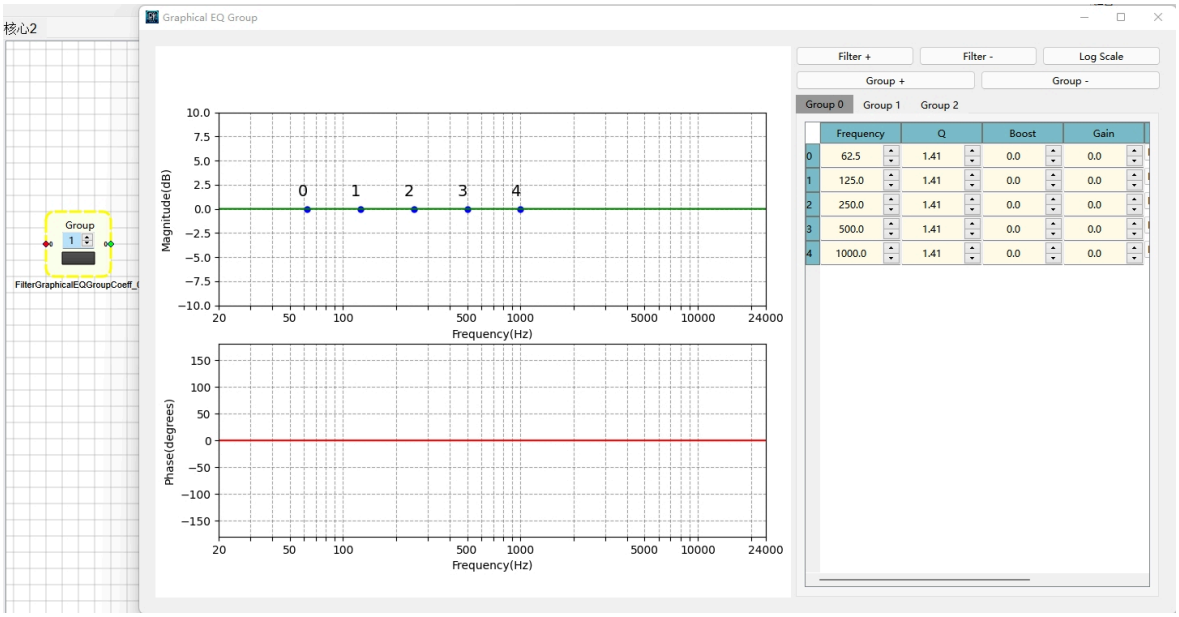


Figure 4.10.1-2 Frequency response curve of Graphical EQ

Note: Double-clicking the component allows you to set the smooth_time. The larger the value, the smoother the change.

4.10.2 Graphic EQ Group (Index Selectable)

The currently used Group ID is controlled via an external (Ext) control signal. The switching method is identical to that of the standard "Graphic EQ Group" except that control must be performed using an external signal. This control signal can be provided through a DC control input.

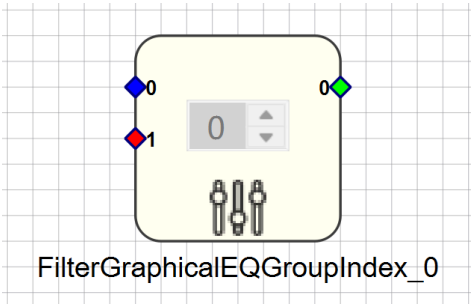


Figure 4.10.2-1 Graphic EQ Group (Index Selectable) Control

4.11 Crossover (Single Precision)

4.11.1 2-Way Crossover (Single Precision)

Function Description: 2-way crossover, which consists of two parallel filter groups responsible for filtering low-frequency and high-frequency signals. The module splits a single input signal into two paths, each passing through one of these filter groups to produce two output signals. As shown in the figure

below, when a single-channel input is used, output channel 0 contains the low-frequency signal, and output channel 1 contains the high-frequency signal. When multiple-channel inputs are used, the output channels are arranged in the order of low-frequency, high-frequency, low-frequency, high-frequency, etc.

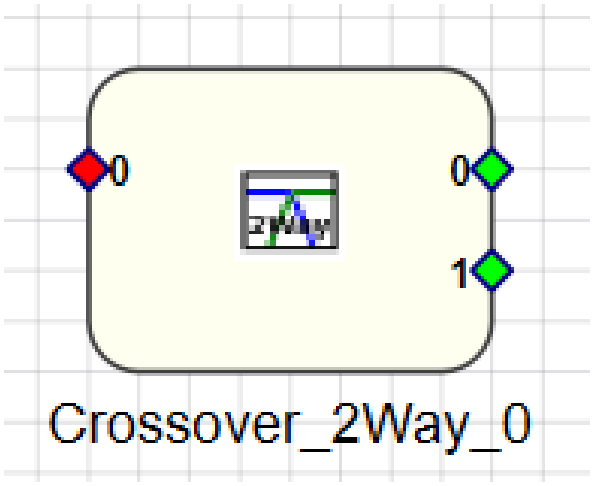


Figure 4.11.1-1 2-Way Crossover Module

Clicking the button enters the parameter configuration page, where you can see the frequency response curves of the low-frequency and high-frequency filters (green lines) and their combined frequency response curve (black line). Users can set the type, gain, and center frequency parameters for each filter below. The invert parameter determines whether the low-frequency part is output with inversion.

Available crossover filter types:

Type	Description
Bypass	Bypass, no processing
Linkwitz-Riley 12	Linkwitz-Riley 2nd order
Linkwitz-Riley 24	Linkwitz-Riley 4th order
Linkwitz-Riley 36	Linkwitz-Riley 6th order
Linkwitz-Riley 48	Linkwitz-Riley 8th order
Butterworth 12	Butterworth 2nd order
Butterworth 18	Butterworth 3rd order
Butterworth 24	Butterworth 4th order
Butterworth 36	Butterworth 6th order
Butterworth 48	Butterworth 8th order
Bessel 12	Bessel 2nd order
Bessel 18	Bessel 3rd order

Type	Description
Bessel 24	Bessel 4th order
Bessel 48	Bessel 8th order

Note: The number after the filter type indicates the roll-off rate in dB per octave. The higher this value, the steeper the frequency response curve in the transition band.

- 1st order: 6 dB per octave
- 2nd order: 12 dB per octave
- 3rd order: 18 dB per octave
- 4th order: 24 dB per octave
- 5th order: 30 dB per octave
- 6th order: 36 dB per octave
- 7th order: 42 dB per octave
- 8th order: 48 dB per octave

For example, comparing a low-pass Linkwitz-Riley 12 (2nd order) with a low-pass Linkwitz-Riley 24 (4th order):

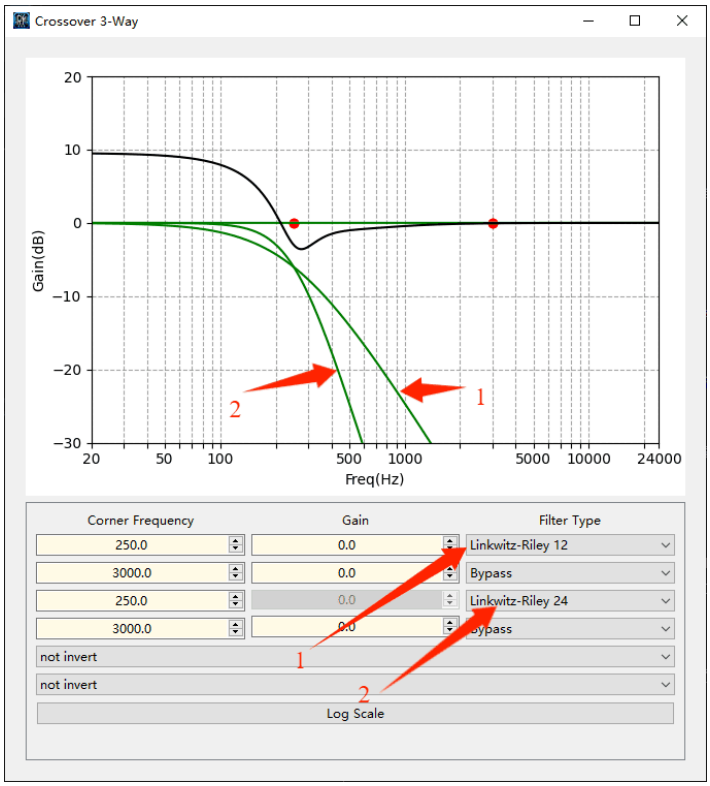


Figure 4.11.1-2 Frequency response curve comparison (the figure shows the settings interface for a 3-way crossover)

It can be seen that Linkwitz-Riley 24 has a steeper roll-off in the transition band. The higher the roll-off rate, the steeper the curve and the clearer the frequency band division.

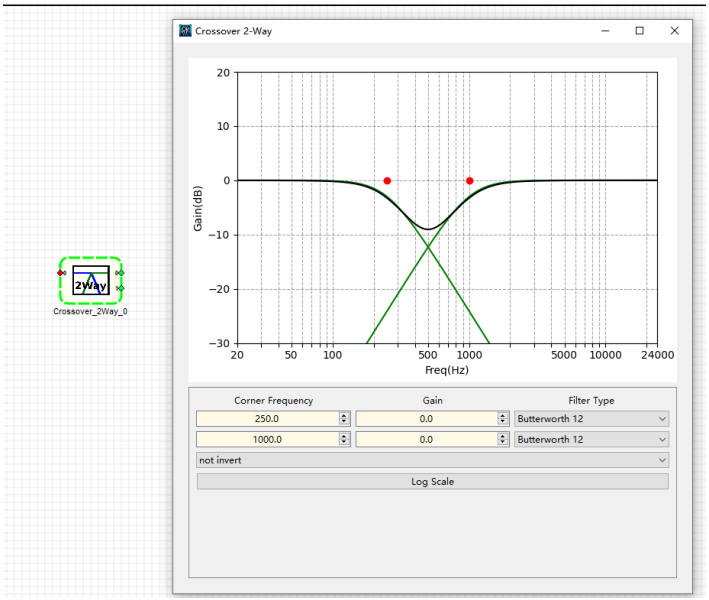


Figure 4.11.1-3 Crossover 2Way Control and Parameter Settings Interface

Note: The tool parameters correspond to the positions of the settings in the interface.

- The first row corresponds to the low-pass filter's LowFc, LowGainDB, and LowType.
- The second row corresponds to the high-pass filter's HighFc, HighGainDB, and HighType.
  The tool parameters are arranged according to the parameter sequence in the project's JSON file (`xxx_param.json`).

Tool Parameters:

Number of Parameters: 7

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
LowType	0	int	Low-pass filter type		0 ~ 13	5 (Butterworth 12)
LowFc	1	float	Low-pass filter cutoff frequency	HZ	20 ~ 24000	250
LowGainDB	2	int	Low-pass filter gain	dB	-30 ~ 30	0
HighType	3	int	High-pass filter type		0 ~ 13	5 (Butterworth 12)
HighFc	4	float	High-pass filter cutoff frequency	HZ	20 ~ 24000	1000

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
HighGainDB	5	float	High-pass filter gain	dB	-30 ~ 30	0
Polarity	6	int	Polarity inversion switch			0 (Disabled)

- Download Parameters *: Parameter conversion logic for this control*
  Number of Parameters: 12

Download Parameter	Parameter Index	Type	Function
SOSLow	0	int	Number of low-pass filters
SOSHigh	1	int	Number of high-pass filters
Low_B00	2	float	Numerator coefficient b0 of the low-pass filter
Low_B10	3	float	Numerator coefficient b1 of the low-pass filter
Low_B20	4	float	Numerator coefficient b2 of the low-pass filter
Low_A10	5	float	Denominator coefficient a1 of the low-pass filter
Low_A20	6	float	Denominator coefficient a2 of the low-pass filter
High_B00	7	float	Numerator coefficient b0 of the high-pass filter
High_B10	8	float	Numerator coefficient b1 of the high-pass filter
High_B20	9	float	Numerator coefficient b2 of the high-pass filter
High_A10	10	float	Denominator coefficient a1 of the high-pass filter
High_A20	11	float	Denominator coefficient a2 of the high-pass filter

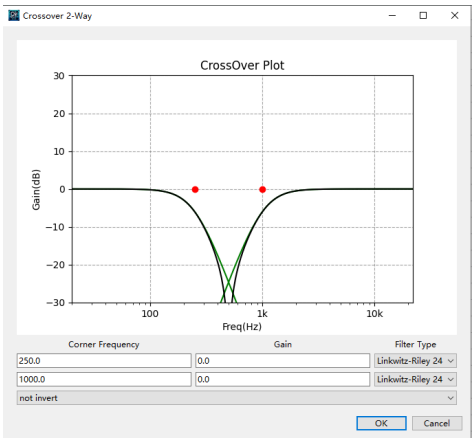


Figure 4.11.1-4 Crossover 2Way Configuration

The 2-way crossover has only two paths: low-frequency and high-frequency. The low-frequency component corresponds to the first filter (low-pass filter), while the high-frequency component corresponds to the second filter (high-pass filter).

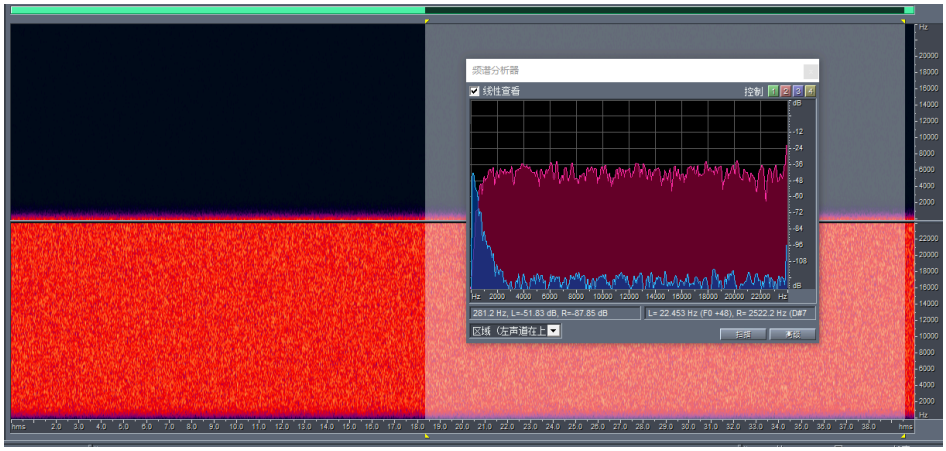


Figure 4.11.1-5 Output effect according to the configuration above. The input is white noise. The upper part of the middle horizontal line in the figure represents the low-frequency path, and the lower part represents the high-frequency path.

4.11.2 3-Way Crossover (Single Precision)

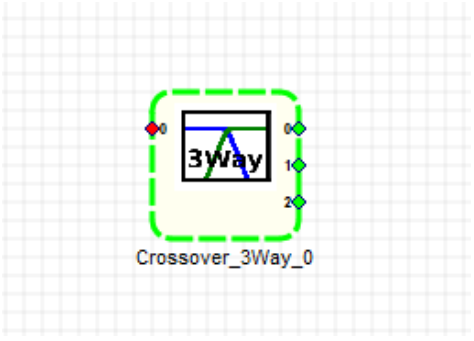


Figure 4.11.2-1 3-Way Crossover Configuration Interface

Function Description: In addition to the 2-way crossover, an extra filter group is added to filter out the mid-frequency band signal. Note that this filter is not a band-pass filter but is composed of a series connection of a high-pass filter and a low-pass filter. Available filter types* are the same as those in 4.11.1 Crossover 2Way.

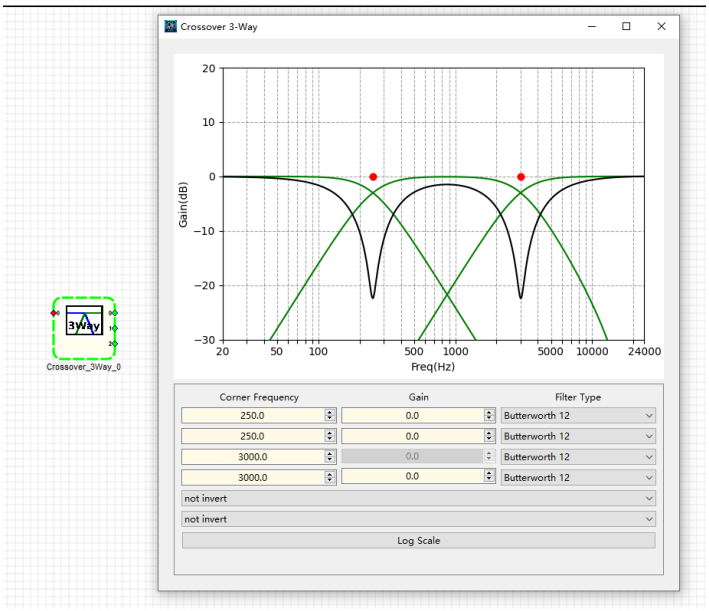


Figure 4.11.2-2 Crossover 3Way Control and Parameter Settings Interface

Note: The tool parameters correspond to the positions of the settings in the interface.

- The first row corresponds to the low-pass filter's LowFc, LowGainDB, and LowType for the low-frequency path.
- The second row corresponds to the low-pass filter's MedLPFc, MedLPGainDB, and MedLPType for the mid-frequency path.
- The third row corresponds to the high-pass filter's MedHpFc, MedHpGainDB, and MedHpType for the mid-frequency path.
- The fourth row corresponds to the high-pass filter's HighFc, HighGainDB, and HighType for the high-frequency path.

The tool parameters are arranged according to the parameter sequence in the project's JSON file (`xxx_param.json`).

Tool Parameters:

Number of Parameters: 13

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
LowType	0	int	Low-pass filter type for low-		0 ~ 13	5 (Butterworth 12)

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
			frequency component			
LowFc	1	float	Cutoff frequency of low-pass filter	HZ	20 ~ 24000	250
LowGainDB	2	int	Gain of low-pass filter	dB	-30 ~ 30	0
MedLpType	3	int	Low-pass filter type for mid-frequency component		0 ~ 13	5 (Butterworth 12)
MedLPFc	4	float	Cutoff frequency of low-pass filter for mid-frequency component	HZ	20 ~ 24000	250
MedHpType	5	int	High-pass filter type for mid-frequency component		0 ~ 13	5 (Butterworth 12)
MedHPFc	6	float	Cutoff frequency of high-pass filter for mid-frequency component	HZ	20 ~ 24000	3000
MedGainDB	7	float	Gain of low-pass and high-pass filters for mid-frequency component	dB	-30 ~ 30	0
HighType	8	int	High-pass filter type for high-		0 ~ 13	5 (Butterworth 12)

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
			frequency component			
HighFc	9	float	Cutoff frequency of high-pass filter	HZ	20 ~ 24000	3000
HighGainDB	10	float	Gain of high-pass filter	dB	-30 ~ 30	0
Polarity	11	int	Polarity inversion for low-frequency path			0 (Disabled)
Polarity	12	int	Polarity inversion for high-frequency path			0 (Disabled)

- Download Parameters *: Parameter conversion logic for this control*
  Number of Parameters: 24

Download Parameter	Parameter Index	Type	Function
SOSLow	0	int	Number of low-pass filters
SOSHigh	1	int	Number of high-pass filters
SOSMedLp	2	int	Number of low-pass filters for mid-frequency component
SOSMEDHp	3	int	Number of high-pass filters for mid-frequency component
Low_B00	4	float	Numerator coefficient b0 of the low-pass filter
Low_B10	5	float	Numerator coefficient b1 of the low-pass filter
Low_B20	6	float	Numerator coefficient b2 of the low-pass filter
Low_A10	7	float	Denominator coefficient a1 of the low-pass filter
Low_A20	8	float	Denominator coefficient a2 of the low-pass filter

Download Parameter	Parameter Index	Type	Function
MedLp_Xn0	9 ~ 13	float	Numerator/Denominator coefficients of the low-pass filter for mid-frequency component
MedHp_Xn0	14 ~ 18	float	Numerator/Denominator coefficients of the high-pass filter for mid-frequency component
High_Xn0	19 ~ 23	float	Numerator/Denominator coefficients of the high-pass filter

Clicking the button enters the parameter configuration page, which operates similarly to the 2-way crossover but with two additional sets of parameters that affect the high-pass and low-pass parts of the mid-frequency filter. The invert1 parameter determines whether the low-frequency part is output with inversion, and the invert2 parameter determines whether the high-frequency part is output with inversion.

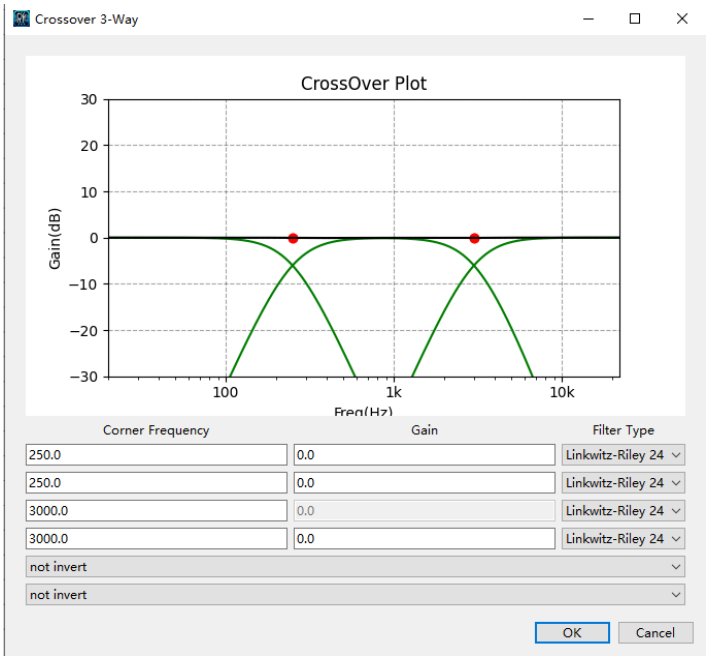
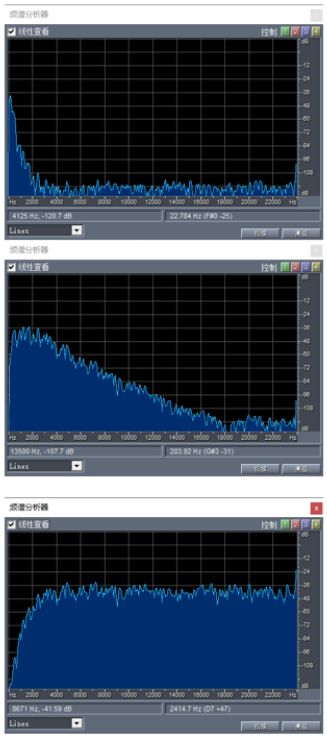


Figure 4.11.2-3 3-Way Crossover Configuration Interface

Note: The first filter (low-pass) outputs the low-frequency signal, the second (high-pass) and third (low-pass) filters in series output the mid-frequency signal, and the fourth (high-pass) filter outputs the high-frequency signal.



低频分量频谱

中频分量频谱

高频分量频谱

Figure 4.11.2-4 Output result according to the configuration above. The input is white noise.

4.11.3 4-Way Crossover(Single Precision)

Function Description: BBased on the 2-Way Crossover, two additional sets of filters are added to construct the mid-low-frequency and mid-high-frequency channels. The filters for the intermediate bands are not implemented as a single band-pass filter; instead, each intermediate band is formed by cascading a high-pass filter and a low-pass filter. The optional filter types* are the same as those in 4.11.1 crossover_2way.

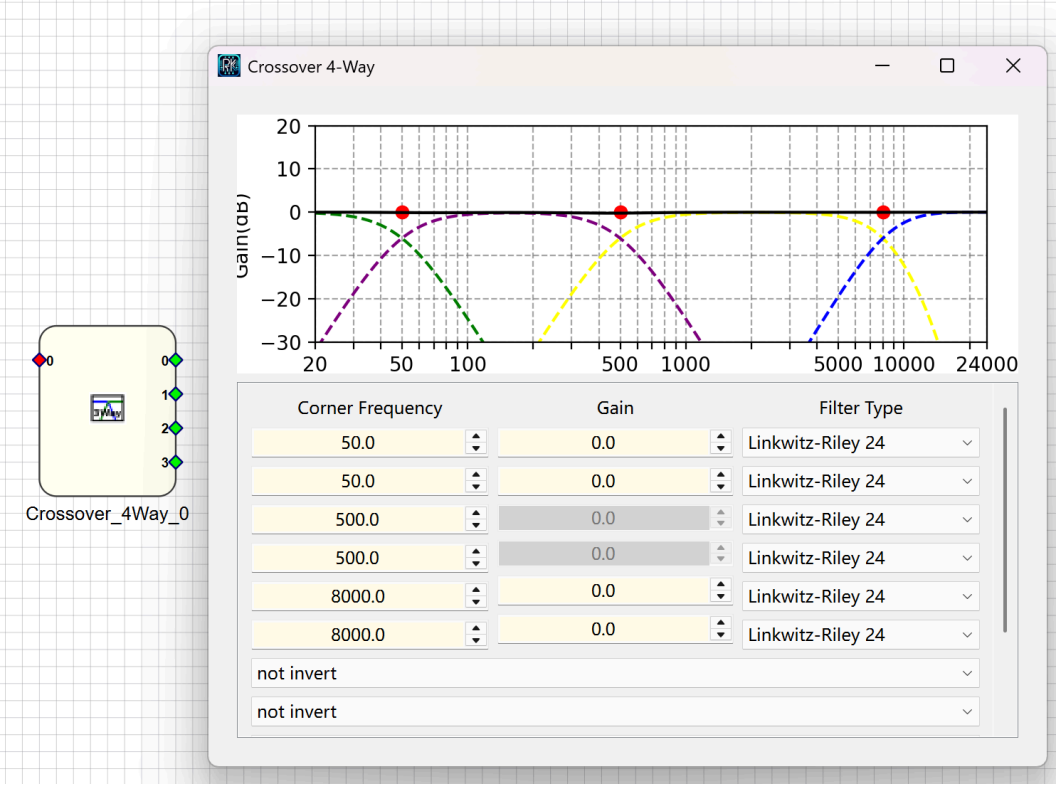


Figure 4.11.3-1 4-Way Crossover Control and Parameter Settings Interface

4.12 Crossover (Double Precision)

The 2-way，3-way and 4-Way crossover filters using double-precision floating-point arithmetic have the same functionality as their single-precision counterparts but offer higher precision at the cost of increased computational resources. Single-precision floating-point arithmetic is generally sufficient for most applications.

4.13 Dynamic EQ

Function Description: Dynamic EQ equalizer, which adjusts the filter based on the signal's amplitude. For example, when the intensity of a signal in a corresponding frequency band gradually increases and reaches the suppression threshold, it will be compressed at a set ratio and moved towards the target value, thereby achieving a dynamic equalization effect.

Actual boost = (Input signal amplitude − Target value) / ratio

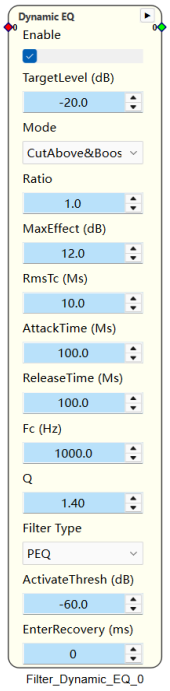


Figure 4.13-1 Dynamic EQ control and Settings interface

- **Tool Parameters:**
  Number of Parameters: 13

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	float	Module enable/disable switch	None	0/1	1

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
TargetLevel	1	float	Target control level	dB	-100 ~ 50	-20
Mode	2	float	Operating mode selection	None	0~4	4
Ratio (in/out)	3	float	Gain change ratio	None	1 ~ 100	2
MaxEffect	4	float	Maximum gain range / maximum boost amount	dB	0 ~ 100	12
Rmstc*	5	float	Smoothing time constant for tracking input signal changes; larger values = slower tracking	ms	0 ~ 2000	20
Attack Time*	6	float	Gain increase speed	ms	0 ~ 2000	200
Release Time*	7	float	Gain decrease speed	ms	0 ~ 2000	200
Fc	8	float	Center frequency	Hz	20~20000	1000
Q	9	float	Filter Q factor	None	0.1~16	1.4
FilterType	10	float	Filter type selection: Bypass/PEQ/LS/HS	None	0 ~ 3	1
ActivateThresh	11	float	If below this threshold, boost becomes 0 dB. If set to -120, the module always remains active and this threshold is disabled	dB	0 ~ -120	-120
EnterRecovery	12	float	If the current signal amplitude stays below ActivateThresh for	dB	0 ~ 200	0

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
			a period of time, the module is forced into /recovery state			

Operating Mode Description：

- Boost Above：When the signal amplitude is greater than the target value, the gain increases.
- Boost Below：When the signal amplitude is lower than the target value, the gain increases.
- Cut Above：When the signal amplitude is greater than the target value, the gain decreases.
- Cut Below：When the signal amplitude is lower than the target value, the gain decreases.
- CutAbove & Boost Below：When the signal amplitude is greater than the target value, the gain decreases; when the signal amplitude is lower than the target value, the gain increases.

FilterType说明：

Bypass：No filter applied.
PEQ：Parametric EQ filter.
LS：Low-shelf filter.
HS：High-shelf filter.

4.14 Text-In EQ

Function Description: A single-band EQ filter.

From the UI, the module behaves very similarly to a standard IIR 2nd-order filter section. It provides a standard biquad filter unit, including filter type selection, frequency, gain, and Q-factor settings.

Additionally, this module can be added via right-click → Add algo, making it convenient to invoke on the current page.

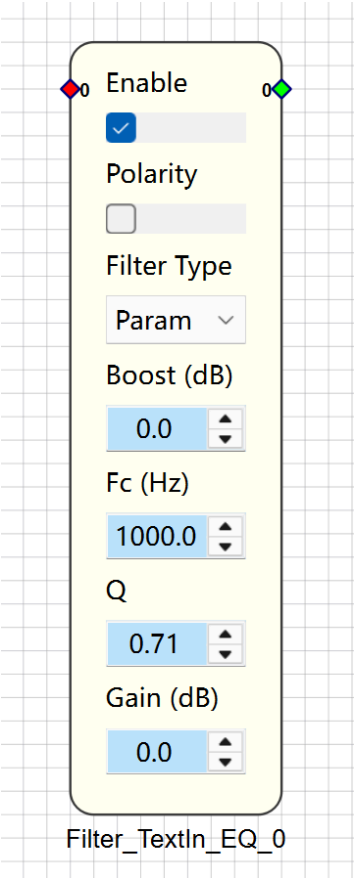


Figure 4.14-1 Text-In EQ Control and Settings Interface

Number of Parameters：7

4.15 Resample

4.15.1 Up-sampler

Function Description: An up-sampler (interpolator) used to increase the sampling rate of the audio signal.

Based on the configured Up Factor, the module inserts samples (typically zero-stuffing) between the original sampling points, thereby increasing the sampling rate. It is generally recommended to enable the Filter option to apply an interpolation filter (FIR) to remove imaging artifacts.

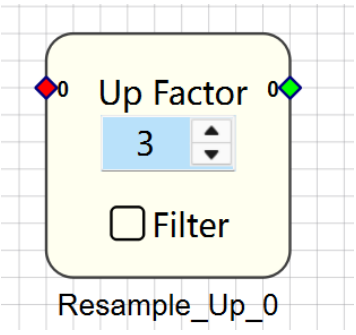


Figure 4.15.1-1 Up-sampler control and parameter settings interface

4.15.2 Down-sampler

Function Description: A down-sampler (decimator) used to reduce the sampling rate of the audio signal.

Based on the configured Dn Factor, the module keeps one sample every N samples, thereby reducing the data rate. It is recommended to enable the Filter option (FIR) as an anti-aliasing filter.

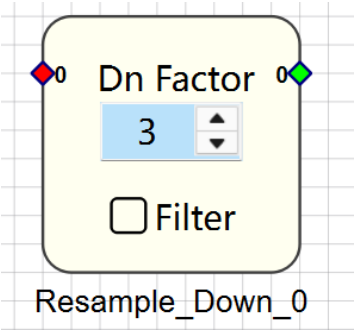


Figure 4.15.2-1 Down-sampler control and parameter settings interface

4.16 Crossover EQ

Function Description: Standalone crossover filter.

This module applies high-pass or low-pass filtering to a mono audio signal. It is commonly used to build custom crossover networks, or to impose independent band-limiting in loudspeaker/PA systems.

Users can freely select both the filter type and filter characteristics (e.g., Butterworth, Bessel, Linkwitz–Riley, as well as the corresponding slope). The filter selection and usage are consistent with the Crossover module, making it easier to apply.

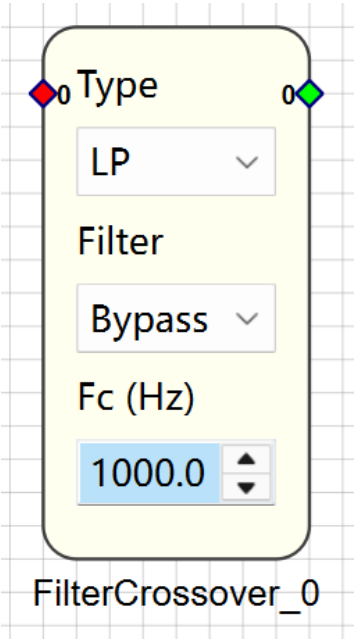


Figure 4.16.1-1 Crossover EQ control and parameter settings interface

4.17 Allpass Delay

Function Description: This module implements an all-pass filter with a delay line.

Key feature: It allows signals at all frequencies to pass with identical magnitude response, while introducing different phase delays as a function of frequency.

Typical Applications:

1、Artificial reverberation: As a core building block in Schroeder reverb structures, used to increase echo density without coloration.

2、Phase alignment: Used to align phase in multi-way crossover systems without changing timbre.

3、Decorrelation: Used to convert a mono signal into a pseudo-stereo signal, or to widen the soundstage.

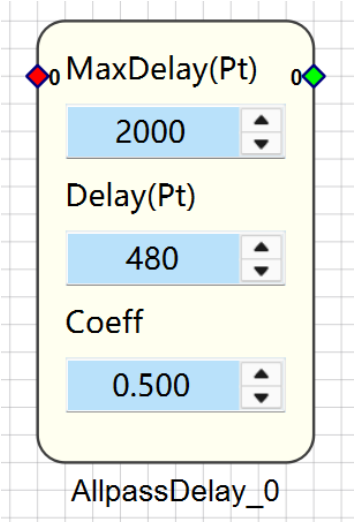


Figure 4.17.1-1 Allpass Delay control and parameter settings interface

Number of Parameters：3

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
MaxDelay(Pt)	0	float	Maximum delay	Point	1~20000	2000
Delay(Pt)	1	float	Current delay	Point	1~20000	480
Coeff	2	float	Feedback coefficient	None	-1~1	0.5

5 IO

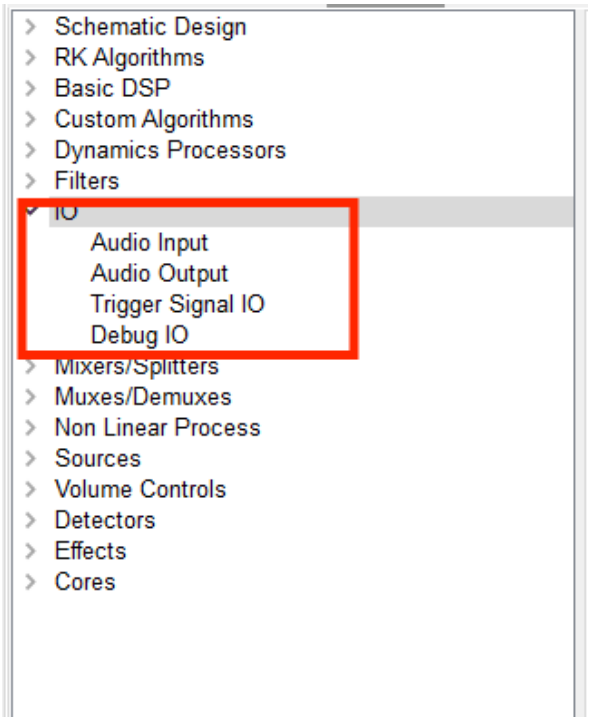


Figure 5-1 Location of IO in the tool

5.1 Audio Input

Function Description: Input.

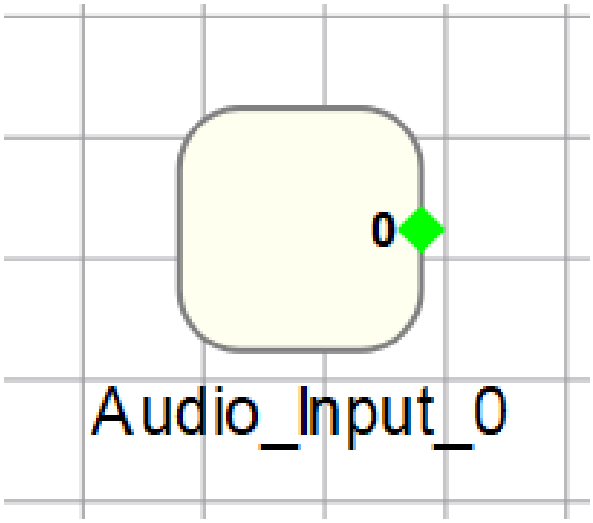


Figure 5.1-1 Audio Input Control

Parameters: None

5.2 Audio Output

Function Description: Output.

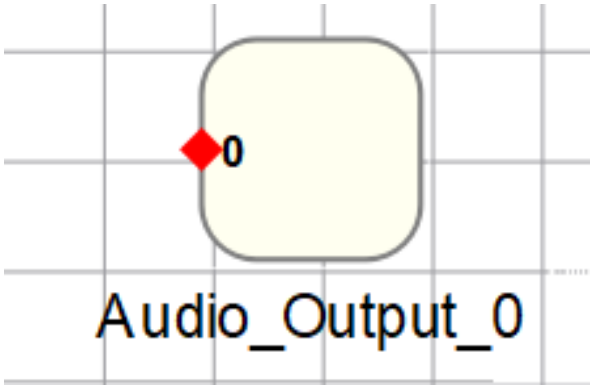


Figure 5.2-1 Audio Output Control

Parameters: None

5.3 Trigger Signal IO

Function Description: Signal triggering via GPIO.

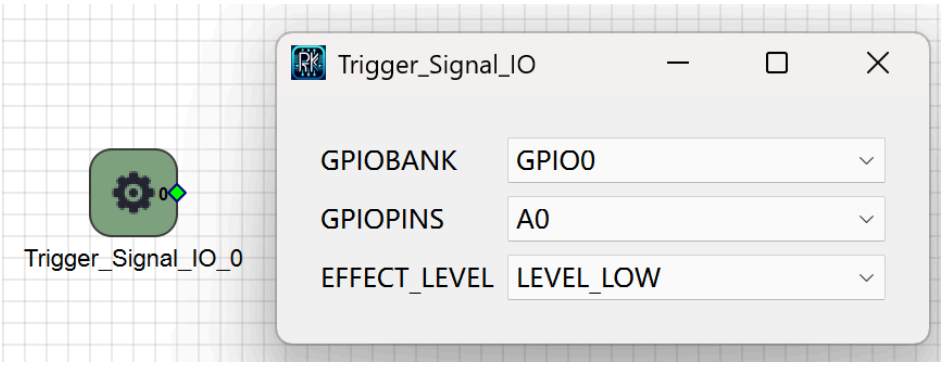


Figure 5.3-1 Trigger Signal IO Control and Settings Interface

Parameters: 3

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
GPIOBANK	0	int	BANK of the GPIO pin		0~4	0
GPIOPINS	1	int	GPIO pin number		0~31	0
EFFECT_LEVEL	2	int	Trigger method; LEVEL for level-triggered, INT for interrupt-triggered		0~6	0

5.4 Debug IO

Function Description: Connect Debug IO to output and configure it via the command line:

```
rkstudio_module_debugio 0 1 10 10 1
```

This allows routing the signal from core 0, plug-in output channel 10, to output channel 10.

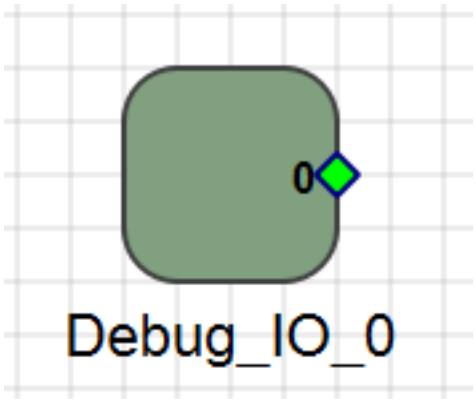


Figure 5.4-1 Debug IO Control

Parameters: None

6 Mixers/Splitters

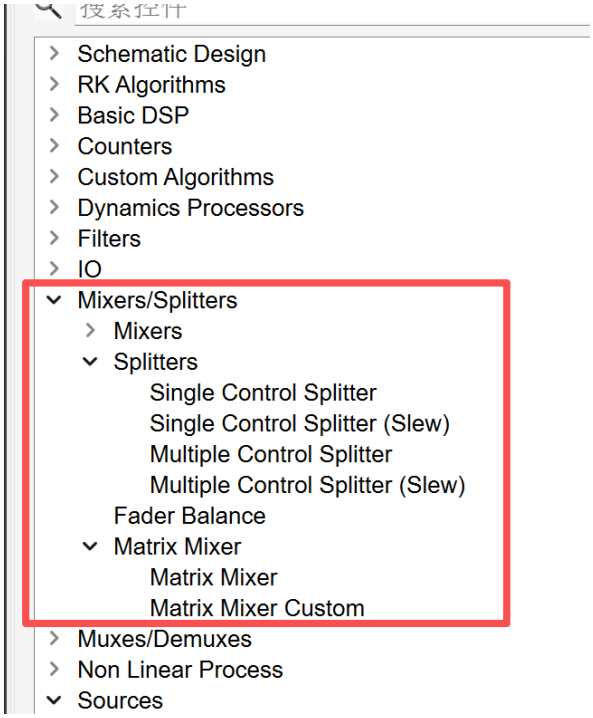


Figure 6-1 Location of Mixers/Splitters in the tool

6.1 Mixers

6.1.1 Signal Merge

Function Description: Multiple inputs are mixed into one output, divided by the number of channels.
Calculation formula: $output = (input0 + input1...)/channel$

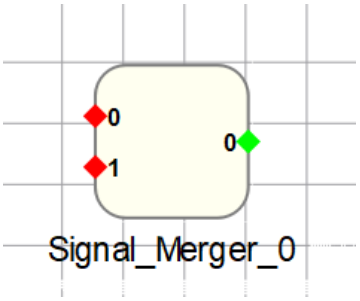


Figure 6.1.1-1 Signal Merge Control and Settings Interface

Parameters: None

6.1.2 Single Control Mixer

Function Description: Multiple inputs are mixed into one output, with all inputs corresponding to the same gain. The gain unit is dB (converted to linear gain* during calculation).

Calculation formula: $output = input0 * gain + input1 * gain...$

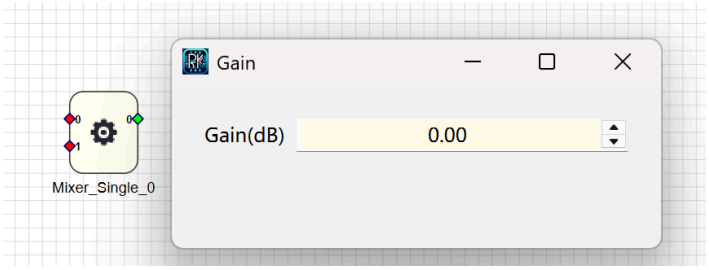


Figure 6.1.2-1 Single Control Mixer Control and Settings Interface

Number of Parameters: 1

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Gain (dB)	0	float	Gain	dB	-138~24	0

Effect Demonstration:
Refer to the Single Control Mixer* module; this module lacks the smoothing function.

Note: The formula for converting dB to linear gain is:

```python
def db_to_linear(db, en_rms=False):
    if en_rms:
        return 10**(db/10)
    else:
        return 10**(db/20)
```

In this control, linear gain is calculated using the else branch of the above program.

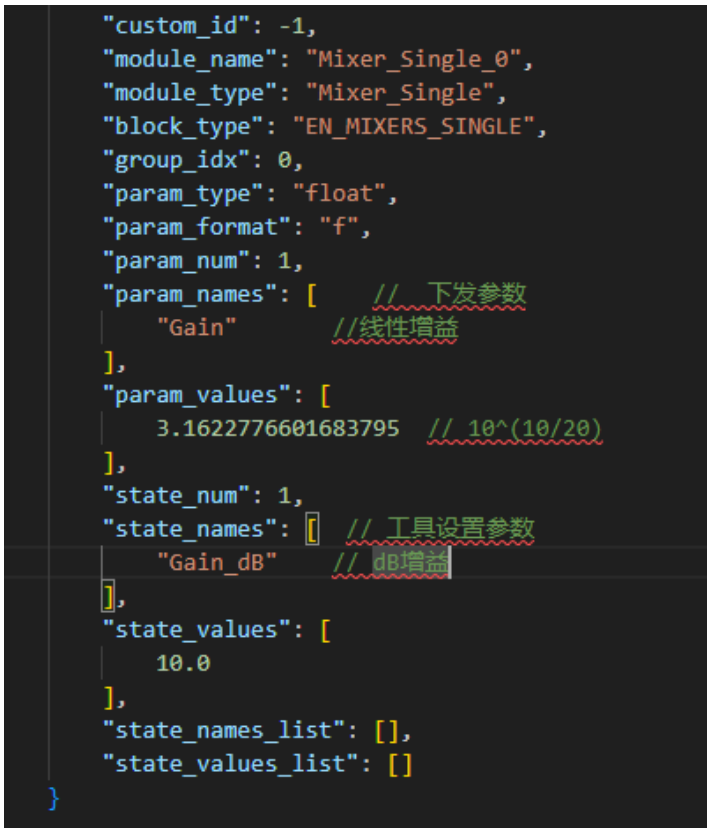


Figure 6.1.2-2 Project diagram JSON file

Parameters set in the software (referred to as tool parameters) may undergo certain conversions before being passed to the algorithm for calculation (referred to as download parameters). Tool parameters and download parameters can be viewed by opening the corresponding project diagram JSON file.

6.1.3 Single Control Mixer (Slew)

Function Description: Multiple inputs are mixed into one output, with all inputs corresponding to the same gain. The gain unit is dB (converted to linear gain* during calculation), and gain changes are smoothed according to the set slew rate.

Calculation formula: $output = input0 * gain + input1 * gain...$

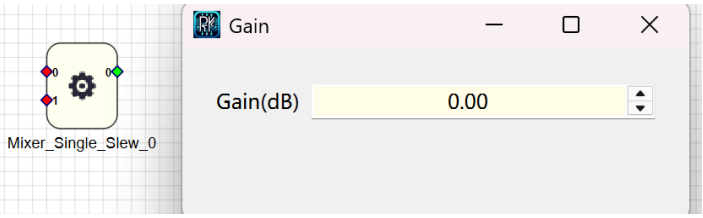


Figure 6.1.3-1 Single Control Mixer (Slew) Control and Settings Interface

Number of Parameters: 1

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Gain (dB)	0	float	Gain	dB	-138~24	0

Effect Demonstration:
Input data consists of two channels, with white noise on the left channel and a sine wave on the right channel. There are two typical cases:

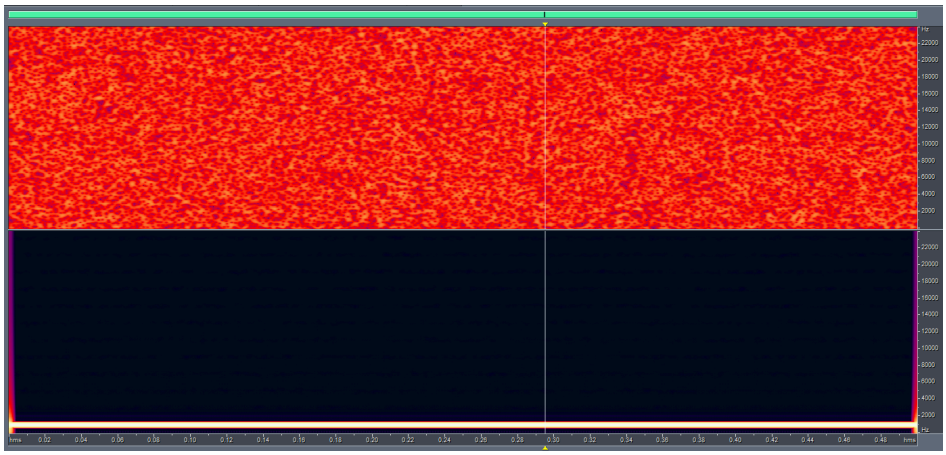


Figure 6.1.3-2 Input waveform frequency domain

- Case 1: Parameter set to 0 ； i.e., gain set to 0 dB;
- Case 2: Parameter set to -12 ； i.e., gain set to -12 dB.

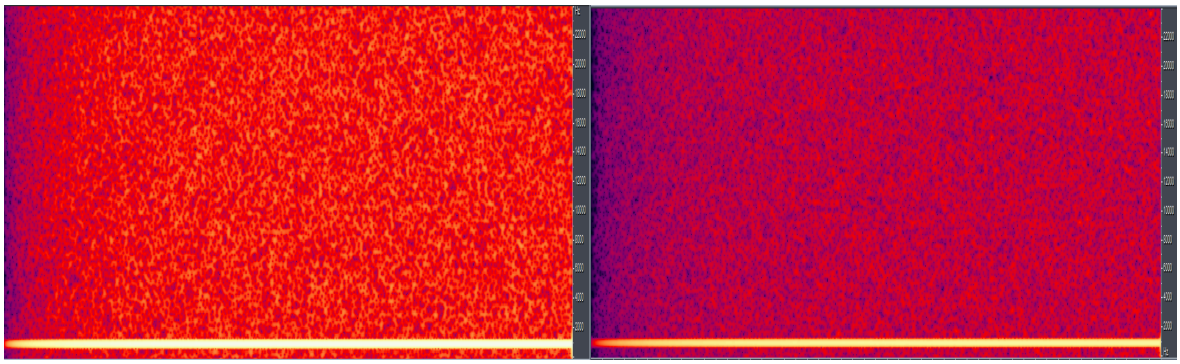


Figure 6.1.3-3 RKStudio output frequency domain; left figure for Case 1, right figure for Case 2

A smoothing effect can be observed at the beginning.

6.1.4 Multiple Control Mixer

Function Description: Multiple inputs are mixed into one output, with each input corresponding to a different gain. The gain unit is dB (converted to linear gain* during calculation).

Calculation formula: $output = input0 * gain0 + input1 * gain1...$

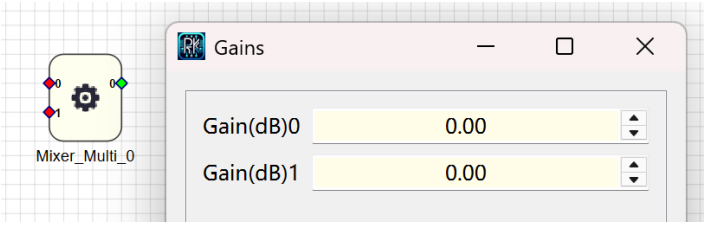


Figure 6.1.4-1 Multiple Control Mixer Control and Settings Interface

Number of Parameters: input_pin

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Gain(dB)0	0	float	Gain for channel 0	dB	-138~24	0
Gain(dB)1	1	float	Gain for channel 1	dB	-138~24	0

Effect Demonstration:
Similar to the Single Control Mixer*, this module allows individual gain control for each channel.

6.1.5 Multiple Control Mixer (Slew)

Function Description: Multiple inputs are mixed into one output, with each input corresponding to a different gain. The gain unit is dB (converted to linear gain* during calculation), and gain changes are smoothed according to the set slew rate.

Calculation formula: $output = input0 * gain0 + input1 * gain1...$

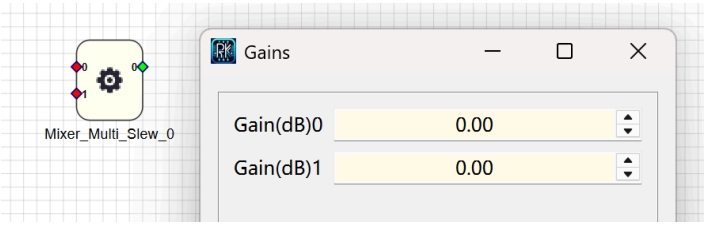


Figure 6.1.5-1 Multiple Control Mixer (Slew) Control and Settings Interface

Number of Parameters: input_pin

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Gain(dB)0	0	float	Gain for channel 0	dB	-138~24	0
Gain(dB)1	1	float	Gain for channel 1	dB	-138~24	0

Effect Demonstration:
Similar to the Single Control Mixer*, this module allows individual gain control for each channel.

6.1.6 Cross Mixer

Function Description: Multiple inputs are mixed into multiple outputs, with gains stored in parameters. If there are 2 inputs and 3 outputs, the gain order is:

- gain00, gain10
- gain01, gain11
- gain02, gain12

Calculation formula:
$output0 = input0 * gain00 + input1 * gain10$

$output1 = input0 * gain01 + input1 * gain11$

$output2 = input0 * gain02 + input1 * gain12$

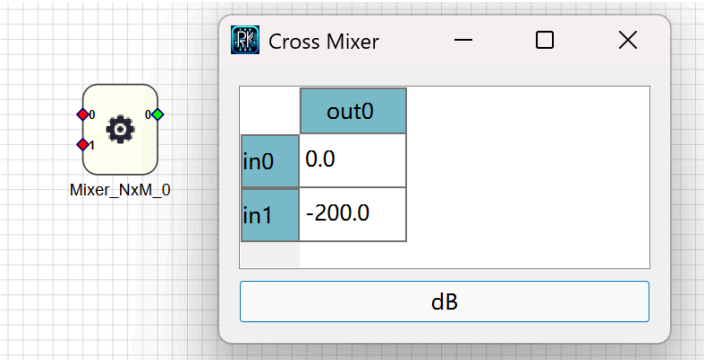


Figure 6.1.6-1 Cross Mixer Control and Settings Interface

The gain unit is displayed below the button and can be switched to dB by clicking the "linear" button. However, it is uniformly converted to linear gain when passed to the algorithm.

Number of Parameters: input_pin * output_pin

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Gain00	0	float	Gain	dB	-138~24	0
Gain01	1	float	Gain	dB	-138~24	0
Gain02	2	float	Gain	dB	-138~24	0
Gain10	3	float	Gain	dB	-138~24	0
Gain11	4	float	Gain	dB	-138~24	0
Gain12	5	float	Gain	dB	-138~24	0
Gain_x_y	x*output_pin+y	float	Gain	dB	-138~24	0

Effect Demonstration:
Similar to the Single Control Mixer*, this module allows multiple outputs and individual gain control for each channel.

6.1.7 Cross Mixer Fast

Function Description: This module performs multi-channel mixing to multi-channel outputs. Its basic functionality is similar to Cross Mixer*. The difference is that this module is more computationally efficient: channels with a gain of 0 are not processed. However, it requires more memory.

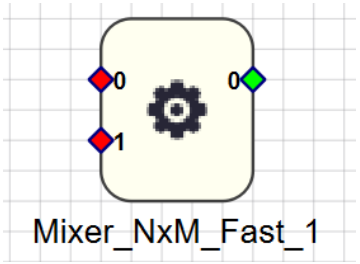


Figure 6.1.7-1 Cross Mixer Fast Control and Settings interface

6.1.8 Cross Mixer (Slew)

Function Description: Multiple inputs are mixed into multiple outputs, with gains stored in parameters. If there are 2 inputs and 3 outputs, the gain order is:

- gain00, gain01, gain02
- gain10, gain11, gain12

Calculation formula:
$$output0 = input0 * gain00 + input1 * gain10$$

$$output1 = input0 * gain01 + input1 * gain11$$

$$output2 = input0 * gain02 + input1 * gain12$$

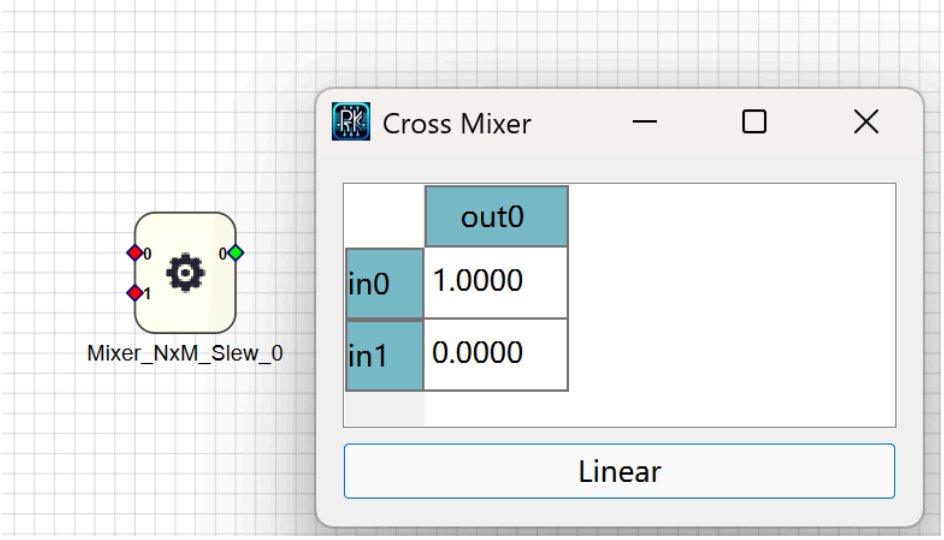


Figure 6.1.8-1 Cross Mixer (Slew) Control and Settings Interface

The gain unit is displayed below the button and can be switched to dB by clicking the "linear" button. However, it is uniformly converted to linear gain when passed to the algorithm.

Number of Parameters: input_pin * output_pin

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Gain00	0	float	Gain	dB	-138~24	0
Gain01	1	float	Gain	dB	-138~24	0
Gain02	2	float	Gain	dB	-138~24	0
Gain10	3	float	Gain	dB	-138~24	0
Gain11	4	float	Gain	dB	-138~24	0
Gain12	5	float	Gain	dB	-138~24	0
Gain_x_y	x*output_pin+y	float	Gain	dB	-138~24	0

Effect Demonstration:
Similar to the Single Control Mixer*, this module allows multiple outputs and individual gain control for each channel.

6.1.9 Cross Mixer Fast (Slew)

Function Description: This module performs multi-channel mixing to multi-channel outputs. Its basic functionality is similar to Cross Mixer (Slew)*. The difference is that this module is more computationally

efficient: channels with a gain of 0 are not processed. However, it requires more memory. In addition, when parameters change, the corresponding output channels are smoothed using fade-in/fade-out (slew) transitions.

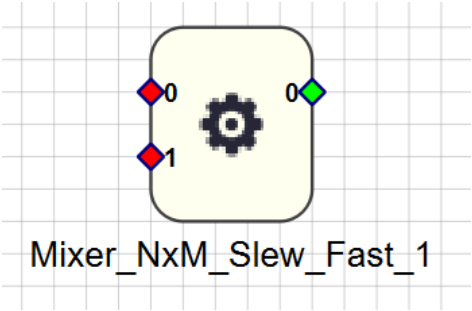


Figure 6.1.9-1 Cross Mixer Fast (Slew) Control and Settings interface

6.1.10 Cross Mixer Slew (Index)

Function Description: A slew-enabled mixing matrix with indexing functionality. Unlike the standard Cross Mixer, this module allows users to preset multiple sets of mixing gain configurations and switch between different Idx settings via control signals.

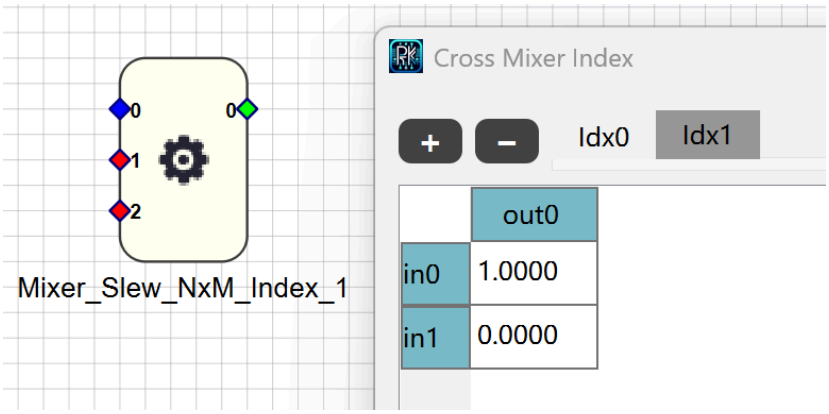


Figure 6.1.10-1 Cross Mixer Slew (Index) Control and Settings interface

6.2 Splitters

6.2.1 Single Control Splitter

Function Description: One input is split into multiple outputs, with all outputs corresponding to the same gain. The gain unit is dB (converted to linear gain* during calculation).

Calculation formula:
$output0 = input * gain$

$output1 = input * gain$

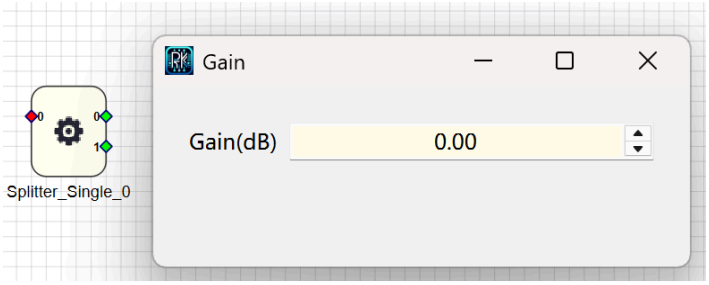


Figure 6.2.1-1 Single Control Splitter Control and Settings Interface

Number of Parameters: 1

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Gain (dB)	0	float	Gain	dB	-138~24	0

Effect Demonstration:
Similar to the Multiple Control Splitter (Slew)*, this module controls multiple output channels with a single gain.

6.2.2 Single Control Splitter (Slew)

Function Description: One input is split into multiple outputs, with all outputs corresponding to the same gain. The gain unit is dB (converted to linear gain* during calculation).

Calculation formula:
$output0 = input * gain$

$output1 = input * gain$

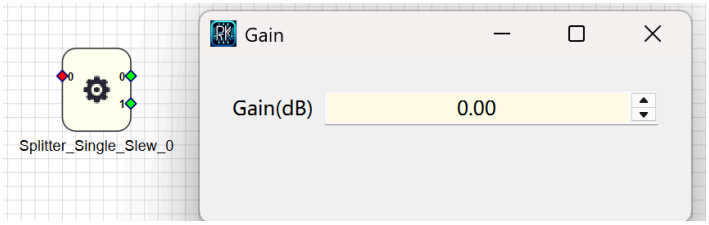


Figure 6.2.2-1 Single Control Splitter (Slew) Control and Settings Interface

Number of Parameters: 1

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Gain (dB)	0	float	Gain	dB	-138~24	0

Effect Demonstration:
Similar to the Multiple Control Splitter (Slew)*, this module controls multiple output channels with a single

gain.

6.2.3 Multiple Control Splitter

Function Description: One input is split into multiple outputs, with each output corresponding to a different gain. The gain unit is dB (converted to linear gain* during calculation).

Calculation formula:
$output0 = input * gain0...$

$output1 = input * gain1...$

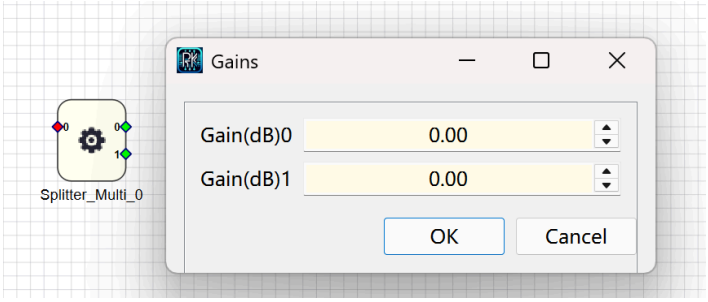


Figure 6.2.3-1 Multiple Control Splitter Control and Settings Interface

Number of Parameters: output_pin

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Gain (dB)0	0	float	Gain	dB	-138~24	0
Gain (dB)1	1	float	Gain	dB	-138~24	0

Effect Demonstration:
Similar to the Multiple Control Splitter (Slew)*, this module lacks the smoothing function.

6.2.4 Multiple Control Splitter (Slew)

Function Description: One input is split into multiple outputs, with each output corresponding to a different gain. The gain unit is dB (converted to linear gain* during calculation).

Calculation formula:
$output0 = input * gain0...$

$output1 = input * gain1...$

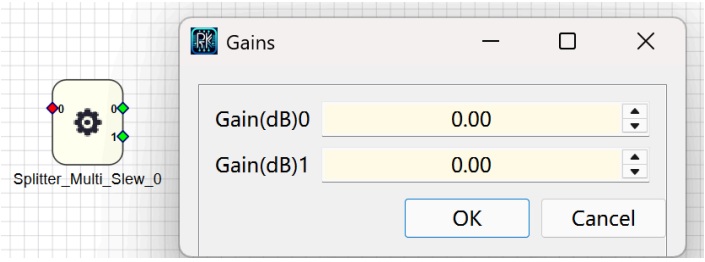


Figure 6.2.4-1 Multiple Control Splitter (Slew) Control and Settings Interface

Number of Parameters: output_pin

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Gain (dB)0	0	float	Gain	dB	-138~24	0
Gain (dB)1	1	float	Gain	dB	-138~24	0

Effect Demonstration:

Input is monaural white noise, and the output consists of two channels.

Parameters are set in order to -10, -20； i.e., gain0 is -10 dB and gain1 is -20 dB.

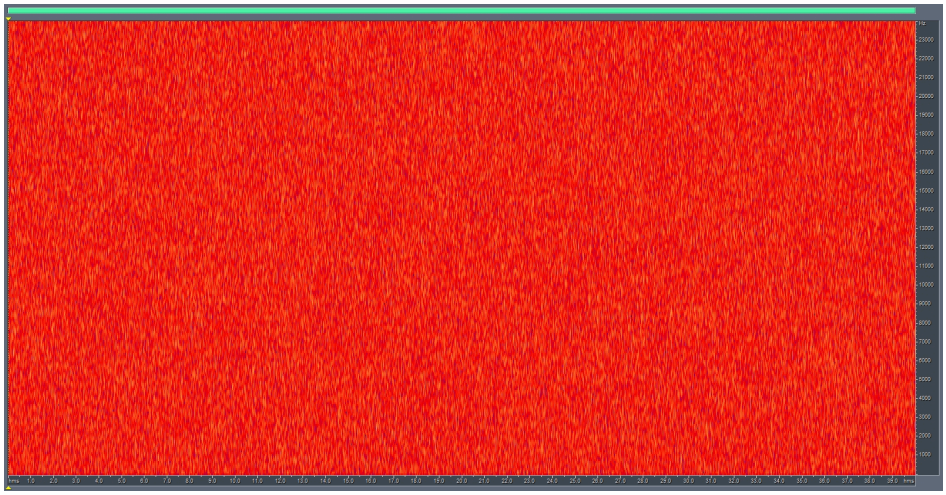


Figure 6.2.4-2 Input white noise frequency domain

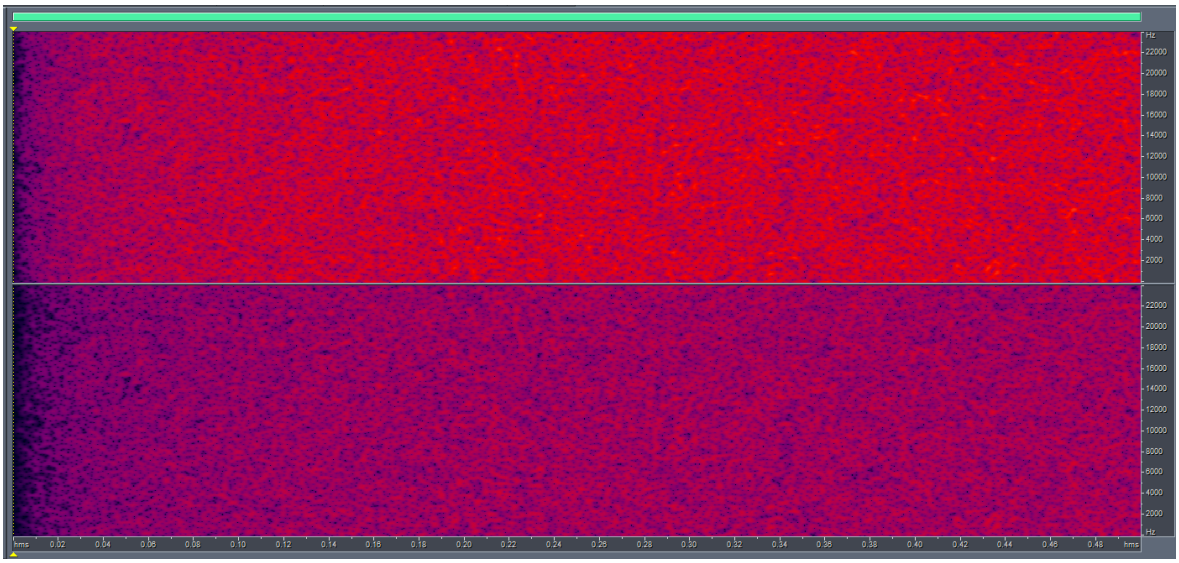
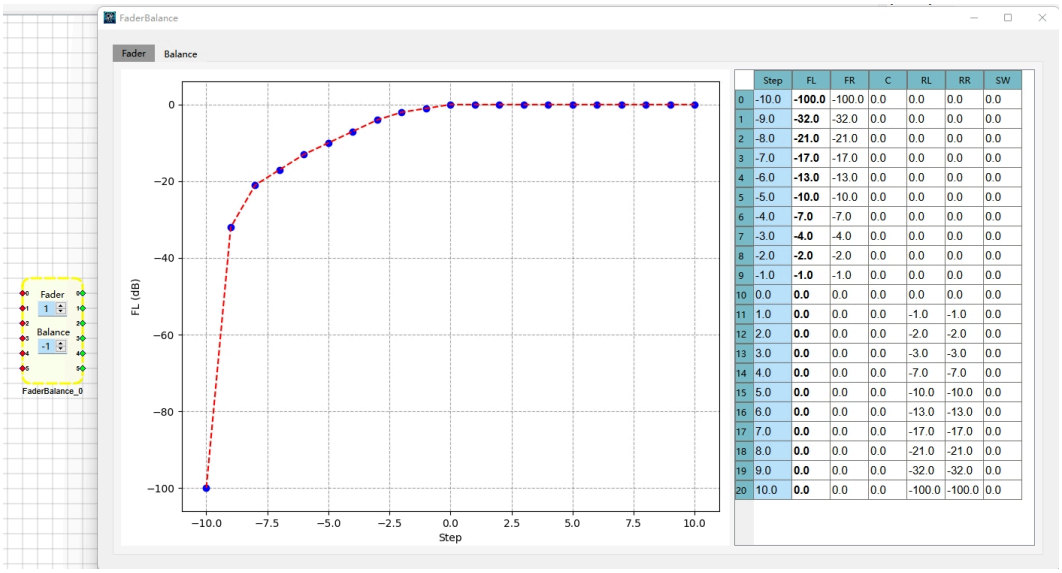


Figure 6.2.4-3 RKStudio output frequency domain (the upper part of the middle horizontal line corresponds to -10 dB gain, and the lower part corresponds to -20 dB gain)

6.3 Fader Balance

Function Description: Achieves sound field balance, where fader indicates front-back balance and balance indicates left-right balance. The step corresponds to the gain adjustment step. When step is 0, the sound field is in the center position, and the gain remains unchanged. In the control, the fader indicates that when step is 1, the fader's gain corresponds to [0/0/0/-1/-1/0]. At this time, FL and FR remain unchanged, but the gains of RL and RR are -1, which moves the sound field forward. The gain data in the table should be adjusted according to the actual situation.



	Step	FL	FR	C	RL	RR	SW
0	-10.0	**-100.0**	-100.0	0.0	0.0	0.0	0.0
1	-9.0	**-32.0**	-32.0	0.0	0.0	0.0	0.0
2	-8.0	**-21.0**	-21.0	0.0	0.0	0.0	0.0
3	-7.0	**-17.0**	-17.0	0.0	0.0	0.0	0.0
4	-6.0	**-13.0**	-13.0	0.0	0.0	0.0	0.0
5	-5.0	**-10.0**	-10.0	0.0	0.0	0.0	0.0
6	-4.0	**-7.0**	-7.0	0.0	0.0	0.0	0.0
7	-3.0	**-4.0**	-4.0	0.0	0.0	0.0	0.0
8	-2.0	**-2.0**	-2.0	0.0	0.0	0.0	0.0
9	-1.0	**-1.0**	-1.0	0.0	0.0	0.0	0.0
10	0.0	**0.0**	0.0	0.0	0.0	0.0	0.0
11	1.0	**0.0**	0.0	0.0	-1.0	-1.0	0.0
12	2.0	**0.0**	0.0	0.0	-2.0	-2.0	0.0
13	3.0	**0.0**	0.0	0.0	-3.0	-3.0	0.0
14	4.0	**0.0**	0.0	0.0	-7.0	-7.0	0.0
15	5.0	**0.0**	0.0	0.0	-10.0	-10.0	0.0
16	6.0	**0.0**	0.0	0.0	-13.0	-13.0	0.0
17	7.0	**0.0**	0.0	0.0	-17.0	-17.0	0.0
18	8.0	**0.0**	0.0	0.0	-21.0	-21.0	0.0
19	9.0	**0.0**	0.0	0.0	-32.0	-32.0	0.0
20	10.0	**0.0**	0.0	0.0	-100.0	-100.0	0.0

Figure 6.3-1 Fader Balance Control and Settings Interface

Since there are many parameters, the control parameters are listed below:
Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
FaderIdx	0	float	Fader step	None	-10 to 10	0
BalanceIdx	1	float	Balance step	None	-10 to 10	0

6.4 Matrix Mixer

Function Description: A digital matrix mixer that can handle variable multi-channel mixing. The number of input and output channels can be customized, typically used in Cases where the number of input channels changes while the number of output channels remains fixed. The input channels of the control should be set to the maximum number of variable channels. For example, if the number of input channels is 6, you can select which channels to mix in the layout, and the coefficient represents the mixing weight.

Currently, only one instance of this plug-in can be deployed, and it can only work with rkplayer for parameter switching. If multiple instances are required, or if you need the mixer gains to switch automatically, please use the Matrix Mixer Custom plug-in.

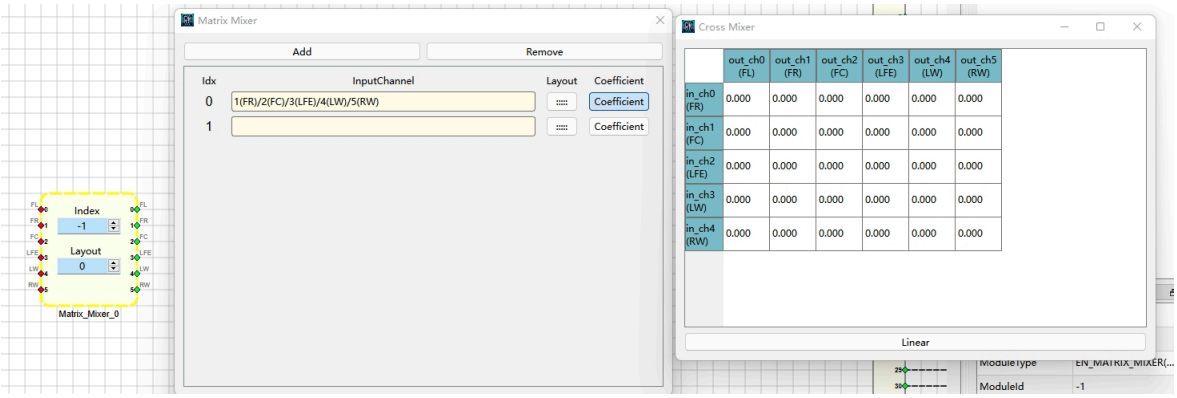


Figure 6.4-1 Matrix Mixer Control and Settings Interface

The Index in the control indicates which idx is currently selected, i.e., which set of configurations is being used. If the index is set to -1, the Layout will be used for selection. For example, in the figure below, Layout selects channels 2 and 4. The corresponding value of Layout is: 1 << 2 | 1 << 4 = 0b00010100 = 20 (a 1 in the corresponding bit position indicates selection).

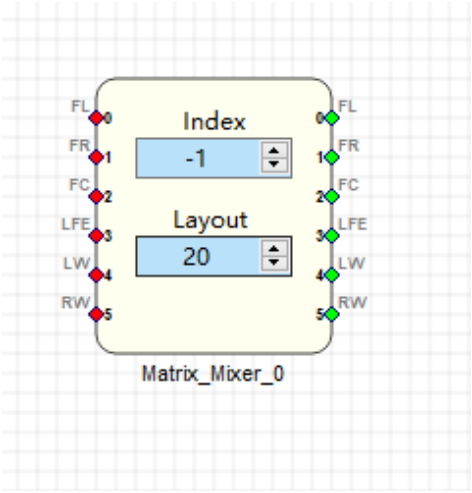


Figure 6.4-3 If idx is set to -1, the layout (20 = 0b00010100) will be used to select channels 2 and 4 (counting from the 0th bit on the right side of the binary number) for mixing; if idx is set to 0, the configuration for idx = 0 in Figure 6.4-3 will be used.

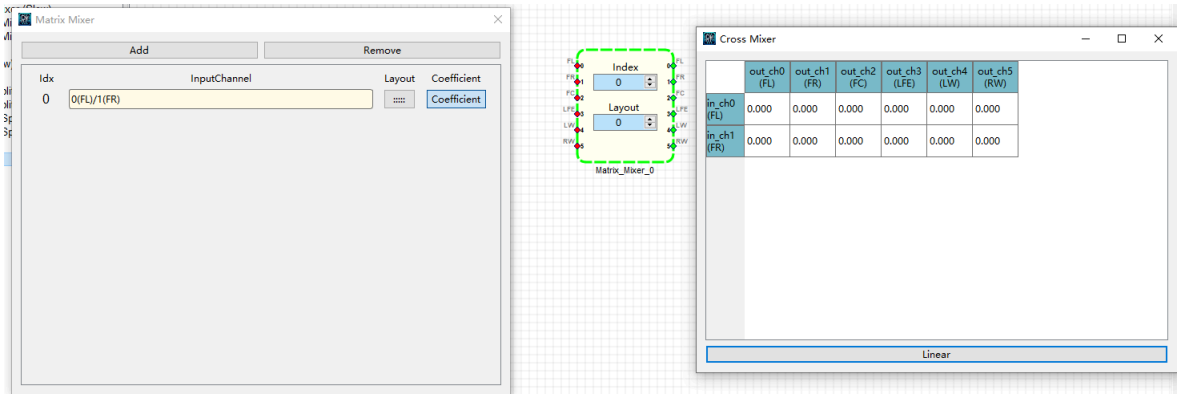


Figure 6.4-3 Click on coefficient to set the corresponding coeff weight

For example, we selected idx = 0 and chose FL and FR channels for mixing, with 6 output channels. The output is as follows:
out_fl = in_fl * coeff_0_0 + in_fr * coeff_0_1;
out_fr = in_fl * coeff_1_0 + in_fr * coeff_1_1;
out_fc = in_fl * coeff_2_0 + in_fr * coeff_2_1;
...and so on.

Since there are many parameters, the control parameters are listed below:
Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Index	0	float	Select which set of configurations to use	None	0 to 15	-1

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Layout	1	uint32_t	Select the current input channels in Hex	None	0 to 0xffffffff	0

6.5 Matrix Mixer Custom Plug-in

For functionality, refer to Section 6.4 Matrix Mixer. This plug-in is intended for user-defined parameter configuration and deployment.

7 Muxes/DeMuxes

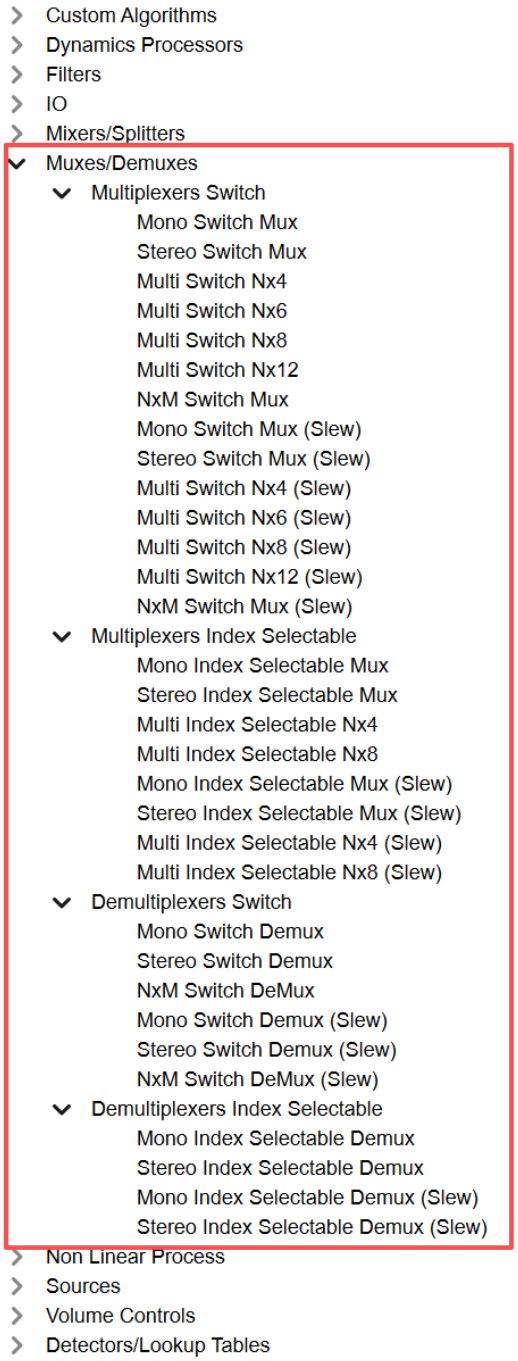


Figure 7-1 Location of Muxes/DeMuxes in the tool

First, let's briefly introduce the concept of Muxes/DeMuxes.

- Multiplexers (Muxes) select one group of inputs (each group can be mono, stereo, or multi-channel) based on the control signal and route it to the output. For example, as shown in Figure 7-2, the multiplexer has 4-channel input and output data. Depending on the control signal, the multiplexer selects either the first or the second group and routes it to the output.

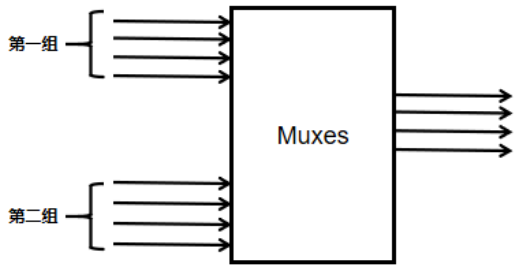


Figure 7-2 2-to-1 Multiplexer with 4-channel input and output

- Demultiplexers (DeMuxes) route a single input (which can be mono or stereo) to one of multiple output groups based on the control signal, while other groups output null signals. For example, as shown in Figure 7-3, the demultiplexer can route the input to the first output group and output 0 on the second group.

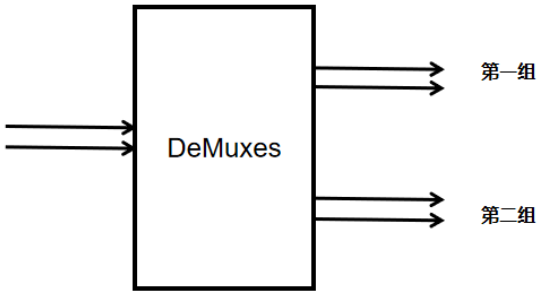


Figure 7-3 2-way Demultiplexer with 2-channel input and output

7.1 Multiplexers Switch

7.1.1 Mono Switch Mux

Function Description: Based on the button (Switch), it selects one group of inputs to output, with each input being mono. By default, it selects one from two groups, but users can add more input groups by right-clicking -> Add Point.

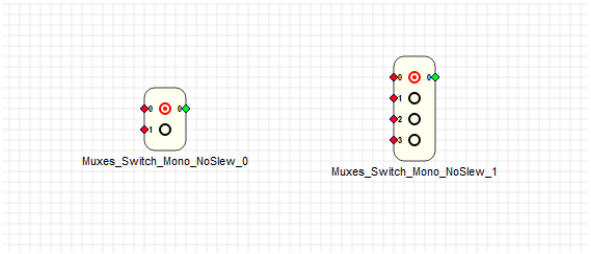


Figure 7.1.1-1 Mono Multiplexer

Demonstration of actual effects:
Input two-channel data, with Channel 1 and Channel 2 being different types of data. The output is mono, as shown in the following two typical Cases:

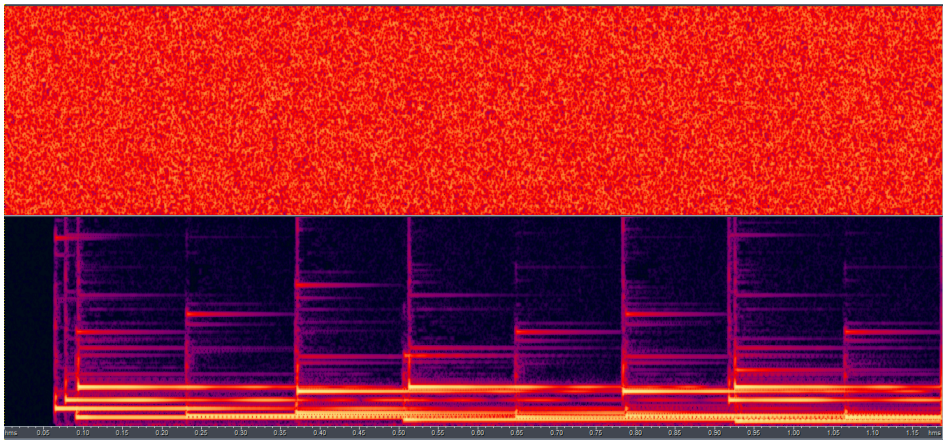


Figure 7.1.1-2 Input data, with Channel 1 as white noise and Channel 2 as music

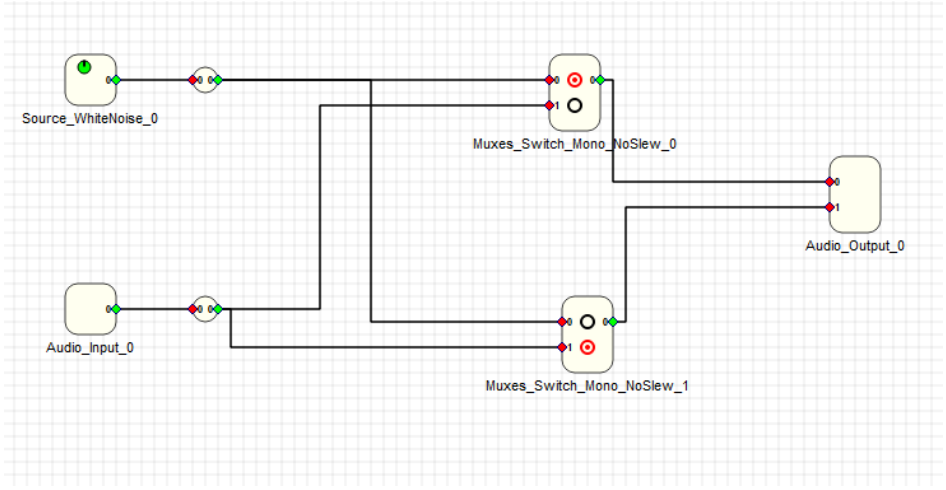


Figure 7.1.1-3 Case 1, selecting Channel 0; Case 2, selecting Channel 1

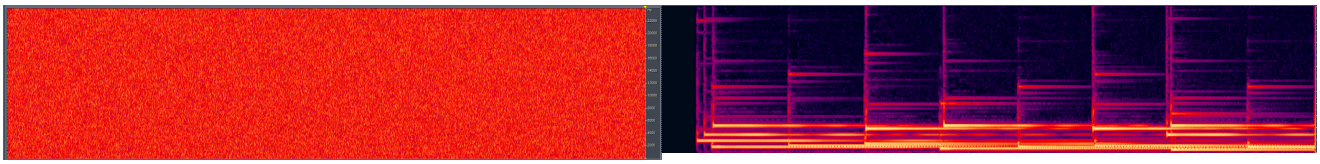


Figure 7.1.1-4 RKStudio output in the frequency domain, with the left image for Case 1 and the right image for Case 2

It can be seen that different switch selections result in different outputs.

7.1.2 Stereo Switch Mux

Function Description: Similar to 7.1 Mono Switch Mux, but each input group is stereo, and the output is stereo.

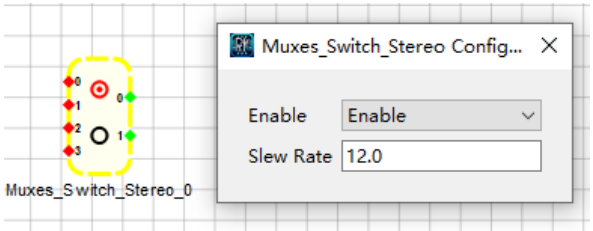


Figure 7.1.2-1 Stereo Multiplexer, which differs from 7.1 in that each input and output group consists of stereo data. For example, if the switch selects the first group, it outputs the left and right channels of the first group, i.e., Channels 0 and 1. If it selects the second group, it outputs the left and right channels of the second group, i.e., Channels 2 and 3.

7.1.3 Multi Switch Nx4

Function Description: Similar to 7.1.1 Mono Switch Mux, but each input group is 4-channel, and the output is 4-channel.

7.1.4 Multi Switch Nx6

Function Description: Similar to 7.1.1 Mono Switch Mux, but each input group is 6-channel, and the output is 6-channel.

7.1.5 Multi Switch Nx8

Function Description: Similar to 7.1.1 Mono Switch Mux, but each input group is 8-channel, and the output is 8-channel.

7.1.6 Multi Switch Nx12

Function Description: Similar to 7.1.1 Mono Switch Mux, but each input group is 12-channel, and the output is 12-channel.

7.1.7 Multi Switch NxM

Function Description: Similar to 7.1.1 Mono Switch Mux, this module supports user-defined numbers of input and output channels. The number of controls scales according to the input/output channel ratio. However, the number of input channels must be an integer multiple of the number of output channels.

7.1_ Multiplexers Switch (Slew)

The following controls are similar to the Multiplexer Switch above but include the Slew smoothing function.
The slewRate smoothing speed is converted into a smoothing coefficient and passed into the algorithm. The conversion code is as follows:

```
def slew_alpha_get(slew_rate, fs=48000, block_size=64):
    if slew_rate == 0:
        return 0
    tc = 0.0416 / block_size * math.pow(2, slew_rate - 1) / 1000
    alpha = math.exp(-1 / (tc * fs))
    return alpha
```

The following uses the 7.1.8 Mono Switch Mux (Slew) component as an example:

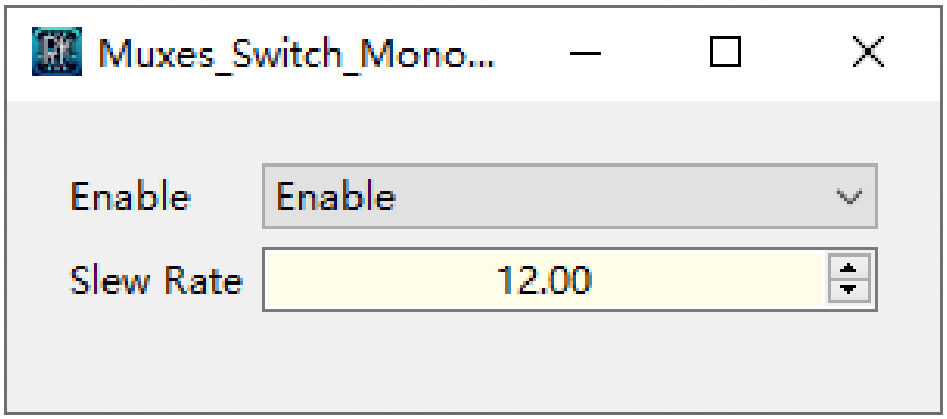


Figure 7.1_-1 Setting the Slew smoothing function; Parameter 0 is whether to enable it, and Parameter 1 is the smoothing speed. The higher the value, the better the smoothing effect, and the smoother the curve.

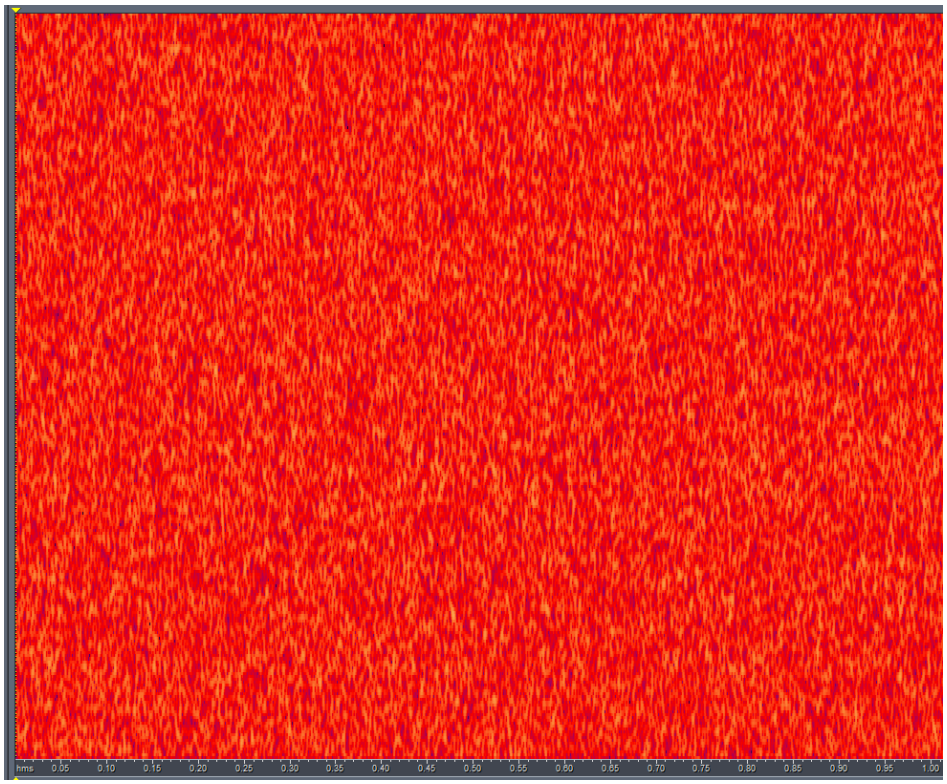


Figure 7.1_-2 Input as white noise (frequency domain)

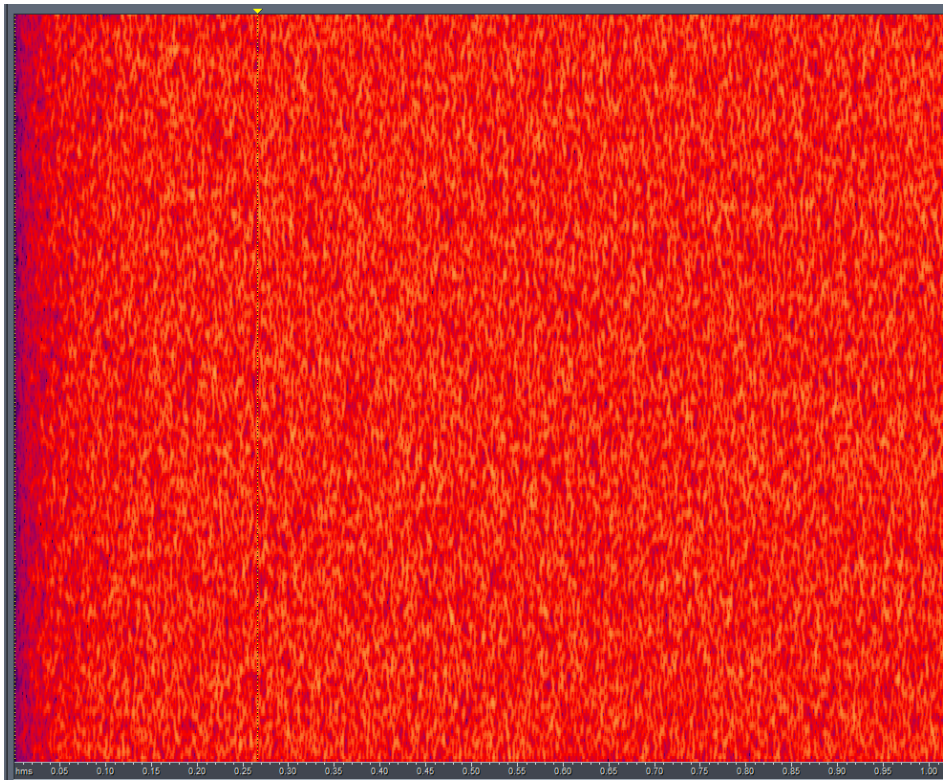


Figure 7.1_-3 Output frequency domain. At the beginning, the smoothing process can be observed, and the higher the Slew Rate, the better the smoothing effect.

7.1.8 Mono Switch Mux (Slew)

Function Description: Similar to 7.1.1 Mono Switch Mux, but with the added Slew smoothing* function.

7.1.9 Stereo Switch Mux (Slew)

Function Description: Similar to 7.1.1 Mono Switch Mux, but each input group is 4-channel, and it includes the Slew smoothing* function.

7.1.10 Multi Switch Nx4 (Slew)

Function Description: Similar to 7.1.1 Mono Switch Mux, but each input group is 4-channel, and the output is 4-channel, with the added Slew smoothing* function.

7.1.11 Multi Switch Nx6 (Slew)

Function Description: Similar to 7.1.1 Mono Switch Mux, but each input group is 6-channel, and the output is 6-channel, with the added Slew smoothing* function.

7.1.12 Multi Switch Nx8 (Slew)

Function Description: Similar to 7.1.1 Mono Switch Mux, but each input group is 8-channel, and the output is 8-channel, with the added Slew smoothing* function.

7.1.13 Multi Switch Nx12 (Slew)

Function Description: Similar to 7.1.1 Mono Switch Mux, but each input group is 12-channel, and the output is 12-channel, with the added Slew smoothing* function.

7.1.14 NxM Switch Mux (Slew)

Function Description: ： Similar to 7.1.1 Mono Switch Mux, this module supports user-defined numbers of input and output channels. The number of controls scales with the input/output channel ratio. The number of input channels must be an integer multiple of the number of output channels. In addition, this module provides Slew smoothing* functionality.

7.2 Multiplexers Index Selectable

7.2.1 Mono Index Selectable Mux

Function Description: Based on the control signal, it selects one group of inputs to output, with each input being mono. By default, it selects one from two groups, but users can add more input groups by right-clicking -> Add Point.

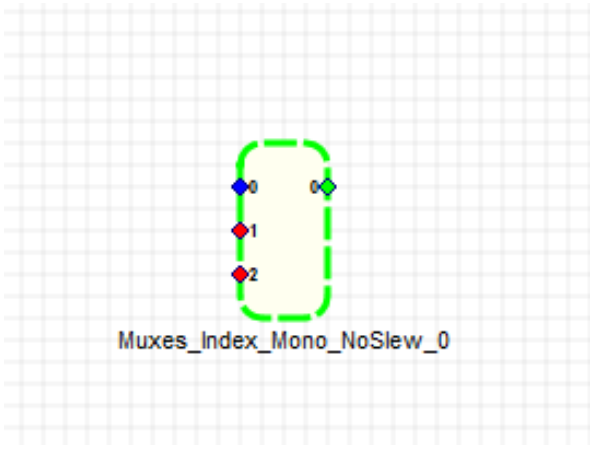


Figure 7.2.1-1 Note the difference from Figure 7.1.1. This module selects the output based on the control signal rather than a button. The control signal is a positive integer: 0 selects the first input group, 1 selects the second input group, and so on.

Demonstration of actual effects:

Input two-channel data, similar to the dual-channel data in 7.1.1, and then use the DC module to simulate the control signal, as shown in the following two typical Cases:

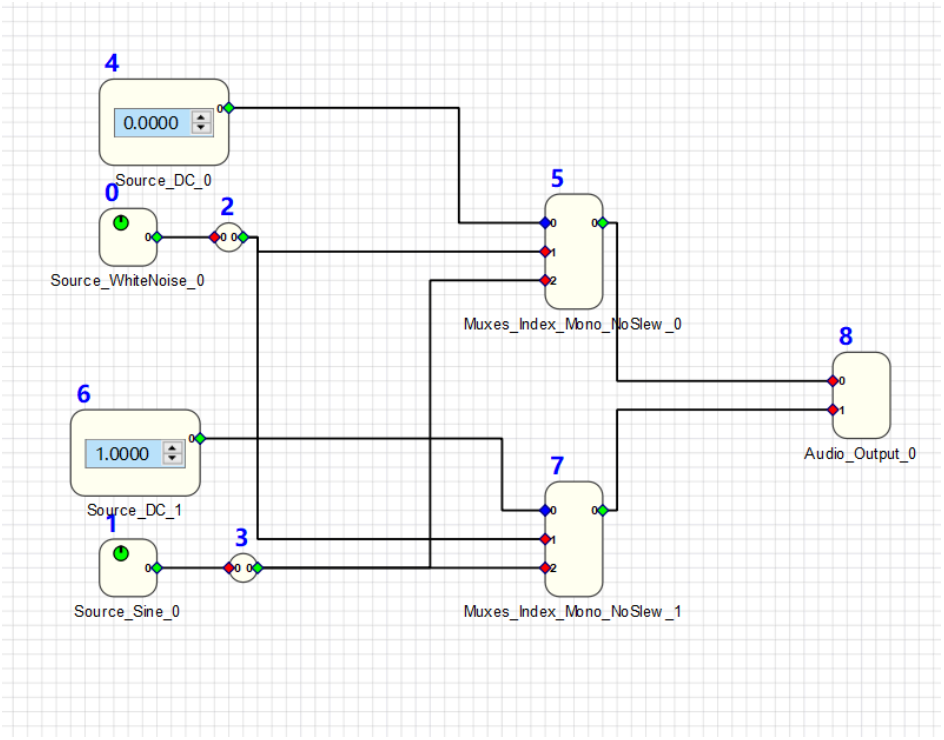


Figure 7.2.1-2 case 1, selecting Channel 0; case 2, selecting Channel 1

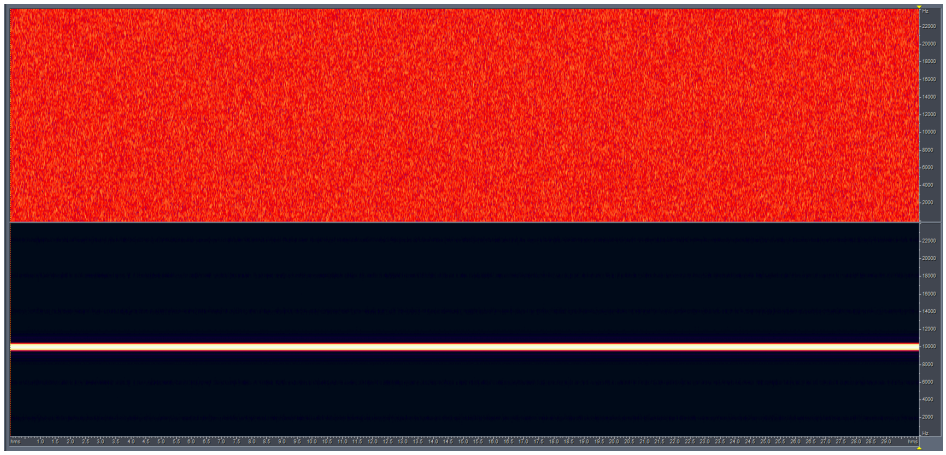


Figure 7.2.1-3 RKStudio output in the frequency domain, with the upper image for Case 1 and the lower image for Case 2

It can be seen that different external control signals result in different outputs.

7.2.2 Stereo Index Selectable Mux

Function Description: Similar to 7.2.1 Mono Index Mux, but each input group is stereo, and the output is stereo.

7.2.3 Multi Index Selectable Nx4

Function Description: Similar to 7.2.1 Mono Index Mux, but each input group is 4-channel, and the output is 4-channel.

7.2.4 Multi Index Selectable Nx8

Function Description: Similar to 7.2.1 Mono Index Mux, but each input group is 8-channel, and the output is 8-channel.

7.2_ Multiplexers Index Selectable (Slew)

The following controls are similar to the Multiplexer Index Selectable above but include the Slew smoothing function.

The Slew Rate smoothing speed is converted into a smoothing coefficient and passed into the algorithm. The conversion code is as follows:

```
def slew_alpha_get(slew_rate, fs=48000, block_size=64):
    if slew_rate == 0:
        return 0
    tc = 0.0416 / block_size * math.pow(2, slew_rate - 1) / 1000
    alpha = math.exp(-1 / (tc * fs))
    return alpha
```

The following uses the 7.2.5 Mono Index Selectable Mux (Slew) component as an example:

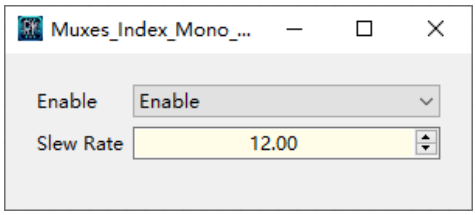


Figure 7.2_-1 Setting the Slew smoothing function; Parameter 0 is whether to enable it, and Parameter 1 is the smoothing speed. The higher the value, the better the smoothing effect, and the smoother the curve.

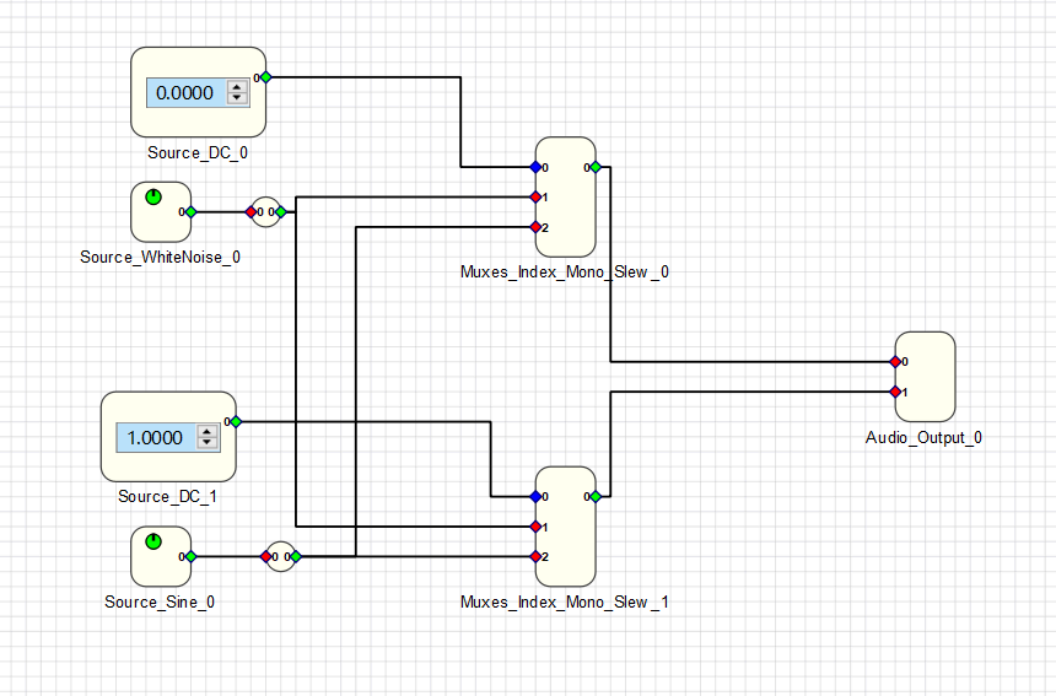


Figure 7.2_-2 Input as white noise and a 10kHz sine wave; Case 1, selecting Channel 0 with slew enabled, slew rate=23; Case 2, selecting Channel 1 with slew enabled, slew rate=23

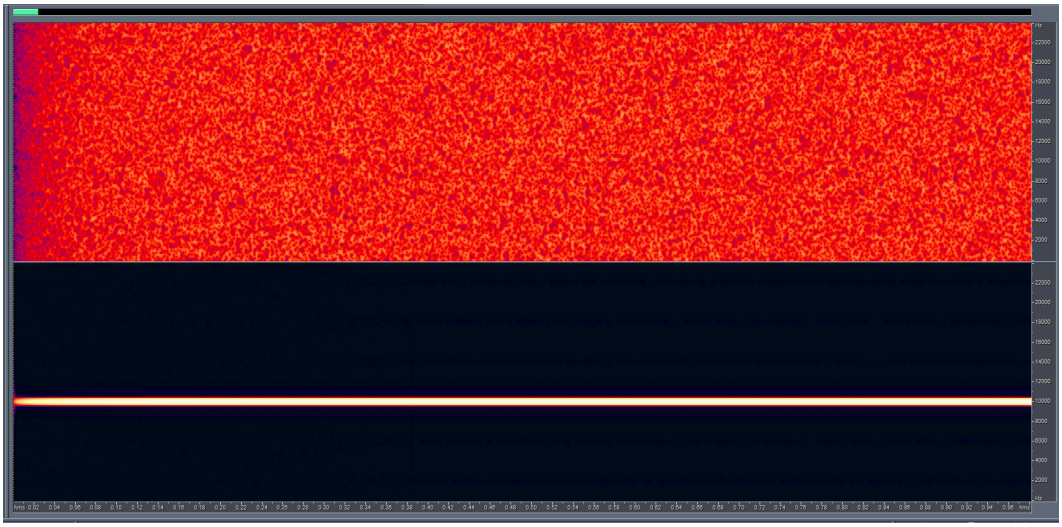


Figure 7.2_-3 Output frequency domain. The upper part of the figure shows the output for Case 1, and the lower part shows the output for Case 2. Compared to Figure 7.2.1-3, a noticeable smoothing effect can be observed at the beginning.

7.2.5 Mono Index Selectable Mux (Slew)

Function Description: Similar to 7.2.1 Mono Index Mux, but with the added Slew smoothing* function.

7.2.6 Stereo Index Selectable Mux (Slew)

Function Description: Similar to 7.2.1 Mono Index Mux, but each input group is stereo, and the output is stereo, with the added Slew smoothing* function.

7.2.7 Multi Index Selectable Nx4 (Slew)

Function Description: Similar to 7.2.1 Mono Index Mux, but each input group is 4-channel, and the output is 4-channel, with the added Slew smoothing* function.

7.2.8 Multi Index Selectable Nx8 (Slew)

Function Description: Similar to 7.2.1 Mono Index Mux, but each input group is 8-channel, and the output is 8-channel, with the added Slew smoothing* function.

7.3 DeMultiplexers Switch

7.3.1 Mono Switch Demux

Function Description: Based on the button (Switch), it routes one group of inputs to one of multiple output groups. Users can add more output groups by right-clicking -> Add Point.

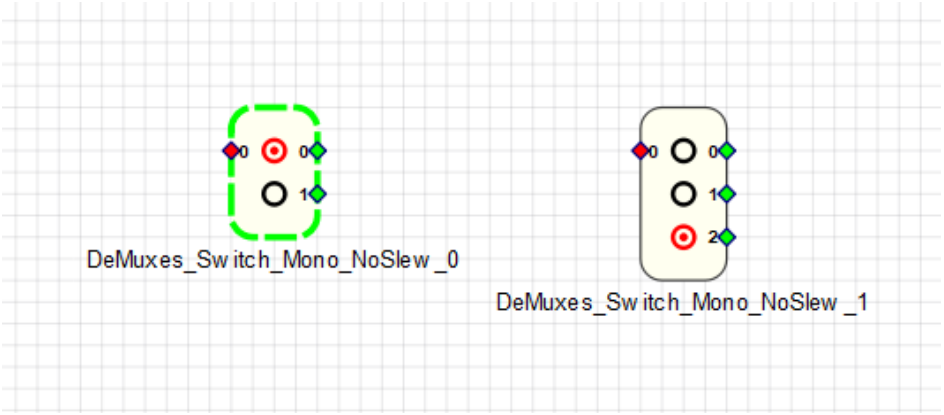


Figure 7.3.1-1 Mono Demultiplexer

The following are two common Cases: Input is white noise

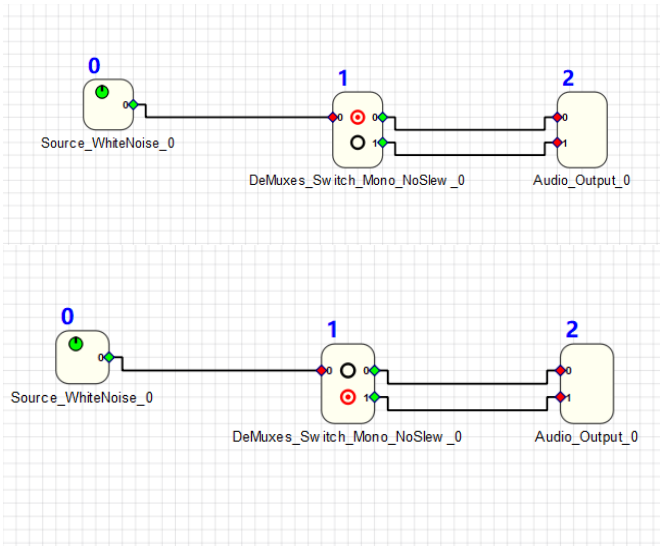


Figure 7.3.1-2 Upper image: Case 1, selecting Channel 0 for output; Lower image: Case 2, selecting Channel 1 for output

Figure 7.3.1-3 Upper image: Output result for Case 1; Lower image: Output result for Case 2

7.3.2 Stereo Switch Demux

Function Description: Similar to 7.3.1 Mono Switch Demux, but the input is stereo, and each output group is stereo.

7.3.3 NxM Switch Demux

Function Description: Similar to 7.3.1 Mono Switch Demux, this module supports user-defined numbers of input and output channels. The number of controls scales with the input/output channel ratio. However, the number of output channels must be an integer multiple of the number of input channels.

7.3.4 Mono Switch Demux (Slew)

Function Description: Similar to 7.3.1 Mono Switch Demux, but with the added Slew smoothing function.

The Slew Rate smoothing speed is converted into a smoothing coefficient and passed into the algorithm. The conversion code is as follows:

```
def slew_alpha_get(slew_rate, fs=48000, block_size=64):
    if slew_rate == 0:
        return 0
    tc = 0.0416 / block_size * math.pow(2, slew_rate - 1) / 1000
    alpha = math.exp(-1 / (tc * fs))
    return alpha
```

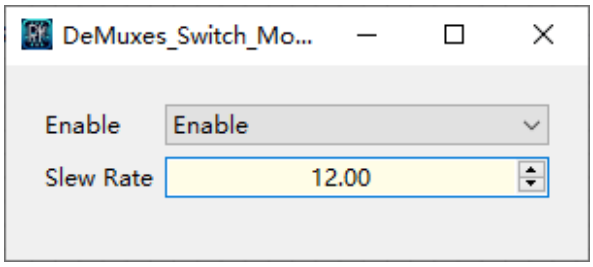


Figure 7.3.4-1 Setting the Slew smoothing function; Parameter 0 is whether to enable it, and Parameter 1 is the smoothing speed. The higher the value, the better the smoothing effect, and the smoother the curve.

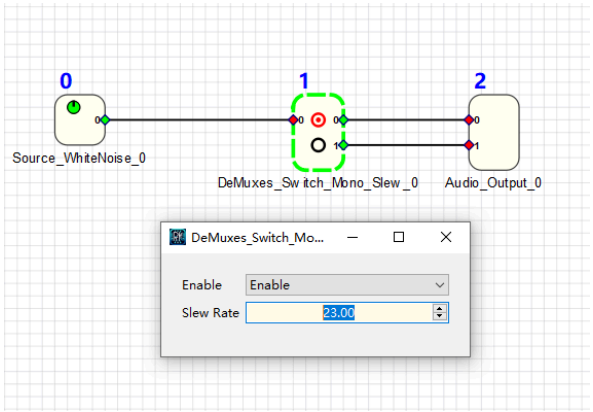


Figure 7.3.4-2 Enabling slew with slew rate=23

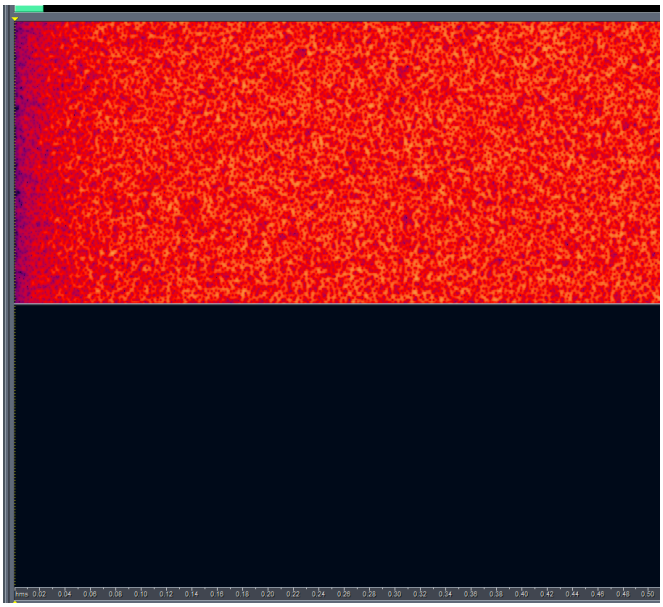


Figure 7.3.4-3 Output frequency domain. Compared to the output spectrum of white noise, a noticeable smoothing process can be observed at the beginning.

7.3.5 Stereo Switch Demux (Slew)

Function Description: Similar to 7.3.1 Mono Switch Demux, but the input is stereo, and each output group is stereo, with the added Slew smoothing* function (refer to 7.3.4 Mono Switch Demux (Slew) for the smoothing function).

7.3.6 NxM Switch Demux (Slew)

Function Description: Similar to 7.3.1 Mono Switch Demux, this module supports user-defined numbers of input and output channels. The number of controls scales with the input/output channel ratio. However, the number of output channels must be an integer multiple of the number of input channels. In addition, it provides the Slew smoothing* function (for details on smoothing behavior, refer to 7.3.3 Mono Switch Demux (Slew)).

7.4 DeMultiplexers Index Selectable

7.4.1 Mono Index Selectable Demux

Function Description: Based on the control signal, it routes one group of inputs to one of multiple output groups. Users can add more output groups by right-clicking -> Add Point.

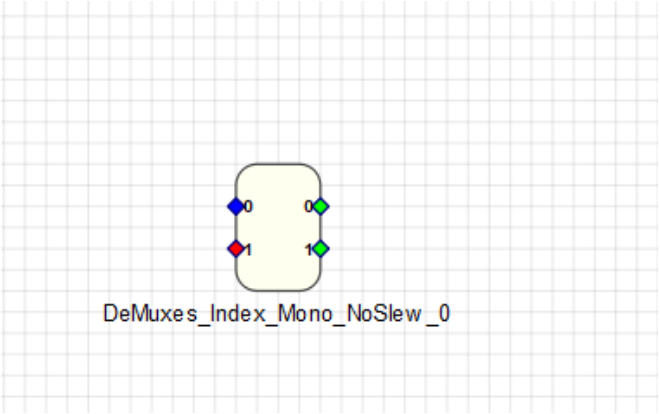


Figure 7.4.1-1 Note the difference from Figure 7.3.1. This module selects the output based on the control signal rather than a button. The control signal is a positive integer: 0 selects the first input group, 1 selects the second input group, and so on.

The following are two common Cases:

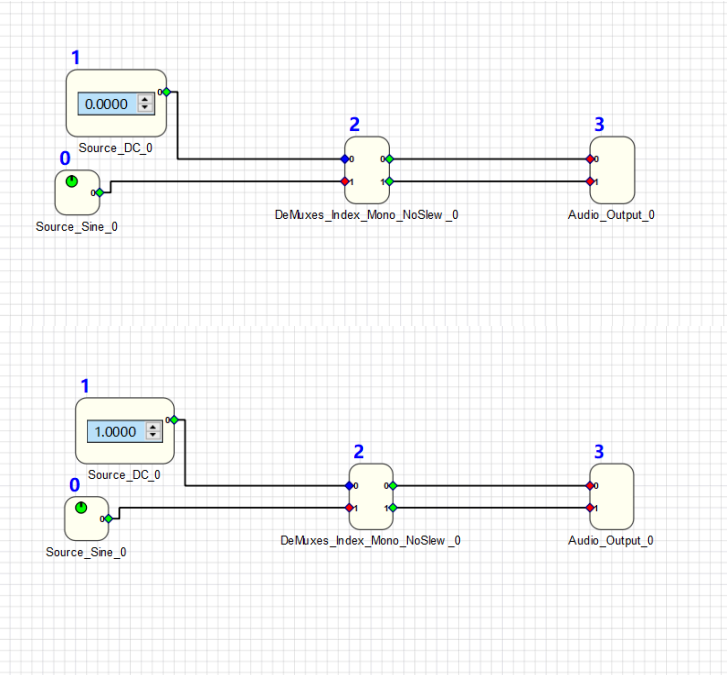


Figure 7.4.1-2 Upper image: Case 1, ext input=0 selects Channel 0 for output; Lower image: Case 2, ext input=1 selects Channel 1 for output

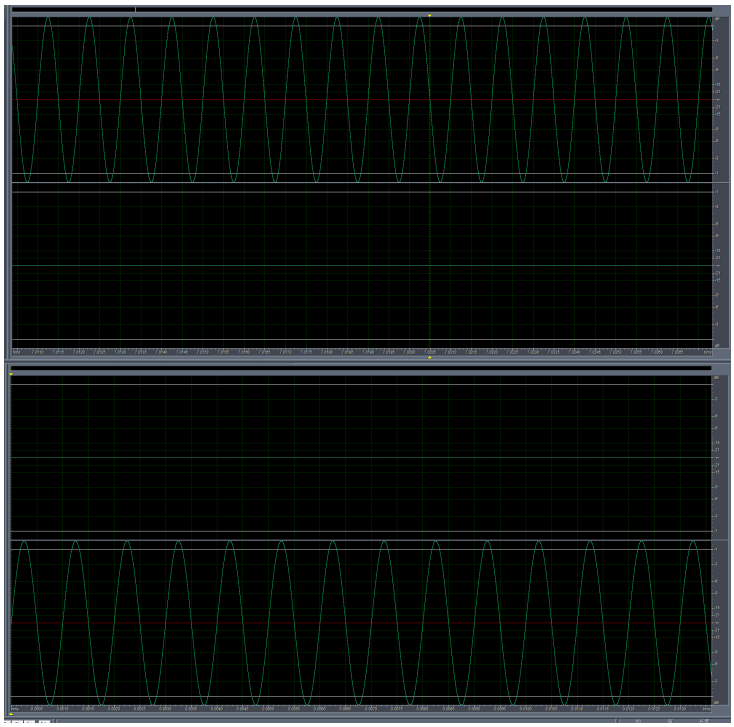


Figure 7.4.1-3 Output time-domain plot. Upper image: Output result for Case 1; Lower image: Output result for Case 2

7.4.2 Stereo Index Selectable Demux

Function Description: Similar to 7.4.1 Mono Index Selectable Demux, but the input is stereo, and each output group is stereo.

7.4.3 Mono Index Selectable Demux (Slew)

Function Description: Similar to 7.4.1 Mono Index Selectable Demux, but with the added Slew smoothing function.

The Slew Rate smoothing speed is converted into a smoothing coefficient and passed into the algorithm. The conversion code is as follows:

```
def slew_alpha_get(slew_rate, fs=48000, block_size=64):
    if slew_rate == 0:
        return 0
    tc = 0.0416 / block_size * math.pow(2, slew_rate - 1) / 1000
    alpha = math.exp(-1 / (tc * fs))
    return alpha
```

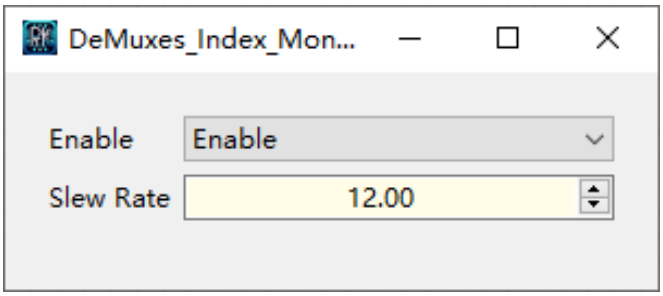


Figure 7.4.3 -1 Setting the Slew smoothing function; Parameter 0 is whether to enable it, and Parameter 1 is the smoothing speed. The higher the value, the better the smoothing effect, and the smoother the curve.

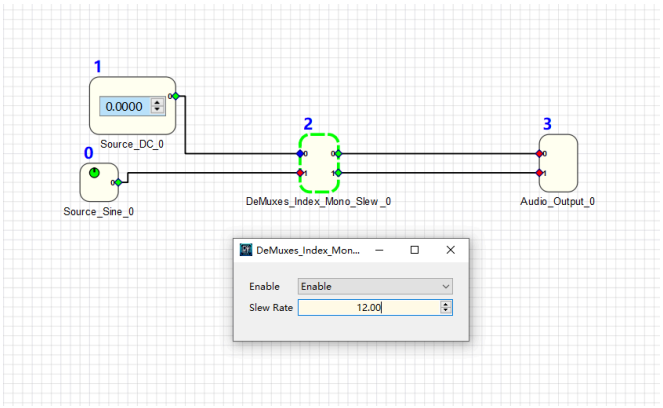


Figure 7.4.3-2 Input as a sine wave, with slew enabled and slew rate=12

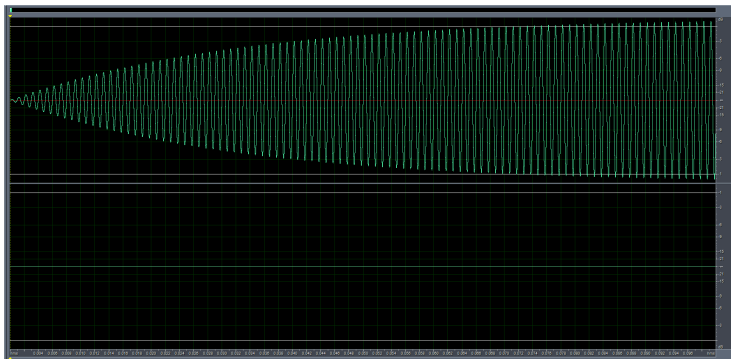


Figure 7.4.3-3 Output time-domain plot. Compared to the output sine wave, a noticeable smoothing process can be observed at the beginning.

7.4.4 Stereo Index Selectable Demux (Slew)

Function Description: Similar to 7.4.1 Mono Index Selectable Demux, but the input is stereo, and each output group is stereo, with the added Slew smoothing* function (refer to 7.4.3 Mono Index Selectable Demux (Slew) for the smoothing function).

8 Source

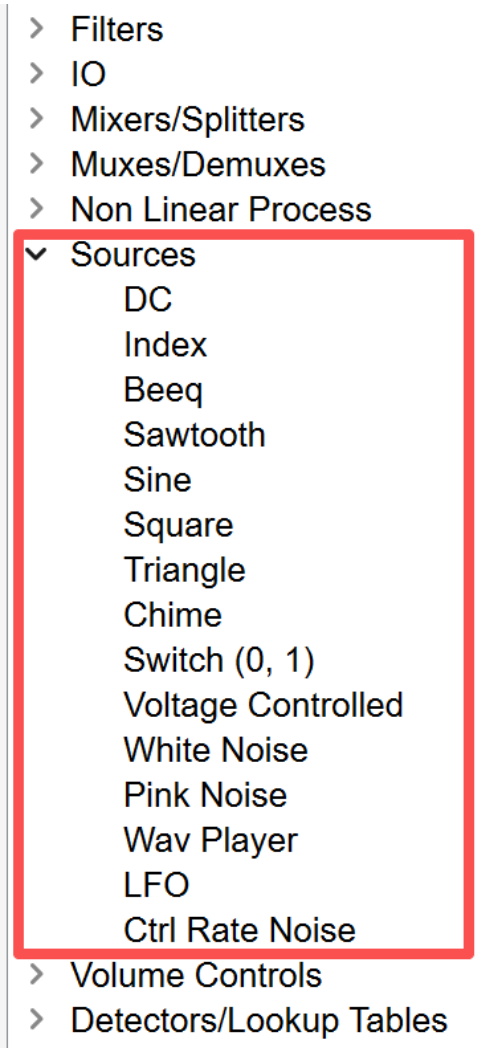


Figure 8-1 Location of the source in the tool

8.1 DC

Function Description: The DC control is used to generate a DC signal (a constant value). By modifying the value in the input box, you can adjust the magnitude of the DC signal.

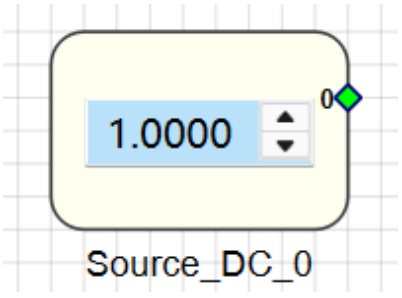


Figure 8.1-1 DC Control and Settings Interface

Number of Parameters: 1

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Value	0	float	DC signal magnitude		-1.0 ~ 1.0	1.0

Typical Case: The parameter is set to 0.01, and the corresponding input result is as follows:

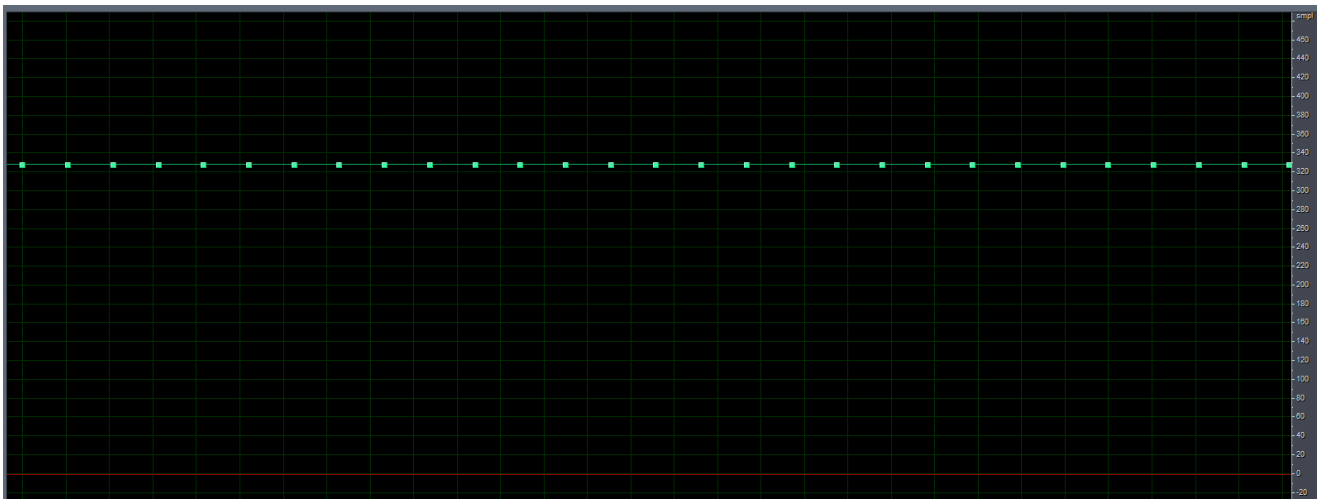


Figure 8.1-2 RKStudio DC Source Output Time Domain

8.2 Beeq

Function Description: The oscillator algorithm is used to generate a beep signal. Modify the Frequency to obtain the desired frequency signal.

Figure 8.2-1 Beeq Control and Settings Interface

Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	int	Enable	None	0/1	1
Frequency(Hz)	1	float	Frequency magnitude	Hz	Less than half the sampling rate	1000

Typical Case: The parameter is set to 1200, and the corresponding input result is as follows:

Figure 8.2-2 RKStudio Beep Source Output Frequency Domain

8.3 Sawtooth

Function Description: The oscillator algorithm is used to generate a sawtooth wave signal. Modify the Frequency to obtain the desired frequency signal.

Figure 8.3-1 Sawtooth Control and Settings Interface

Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	float	Enable	None	0/1	1
Frequency(Hz)	1	float	Frequency magnitude	Hz	Less than half the sampling rate	1000

Typical Case: The parameter is set to 1200, and the corresponding input result is as follows:

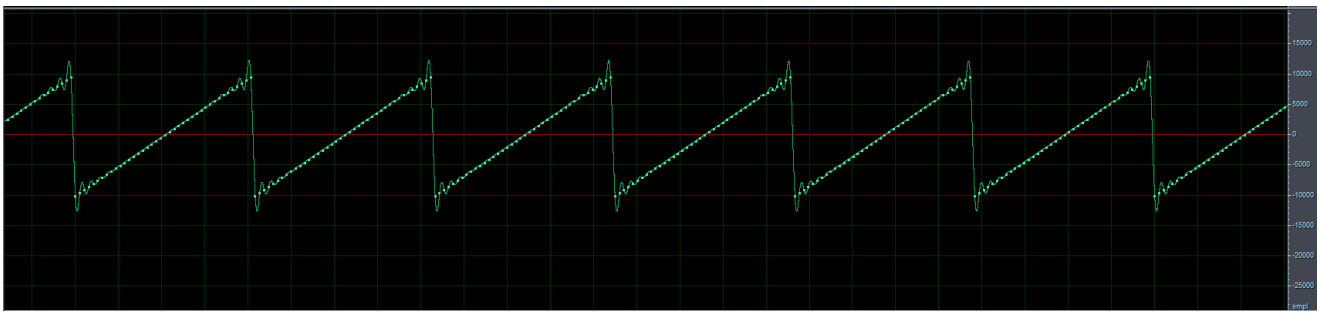


Figure 8.3-2 RKStudio Sawtooth Wave Source Output Time Domain

8.4 Sine

Function Description: The oscillator algorithm is used to generate a sine wave signal. Modify the Frequency to obtain the desired frequency signal.

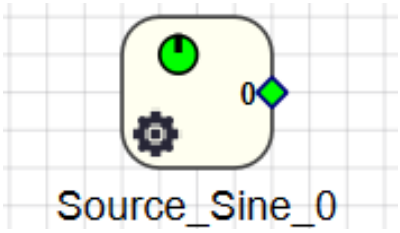


Figure 8.4-1 Sine Control and Settings Interface

Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	int	Enable	None	0/1	1
Frequency(Hz)	1	float	Frequency magnitude	Hz	Less than half the sampling rate	1000

Typical Case: The parameter is set to 1200, and the corresponding input result is as follows:

Figure 8.4-2 RKStudio Sine Wave Source Output Frequency Domain

8.5 Square

Function Description: The oscillator algorithm is used to generate a square wave signal. Modify the Frequency to obtain the desired frequency signal.

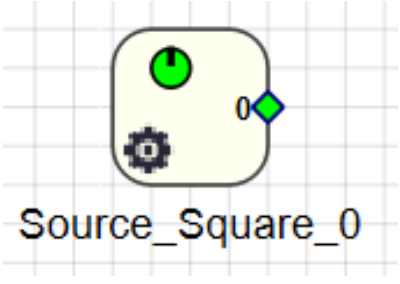


Figure 8.5-1 Square Control and Settings Interface

Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	int	Enable	None	0/1	1
Frequency(Hz)	1	float	Frequency magnitude	Hz	Less than half the sampling rate	1000

Typical Case: The parameter is set to 1200, and the corresponding input result is as follows:

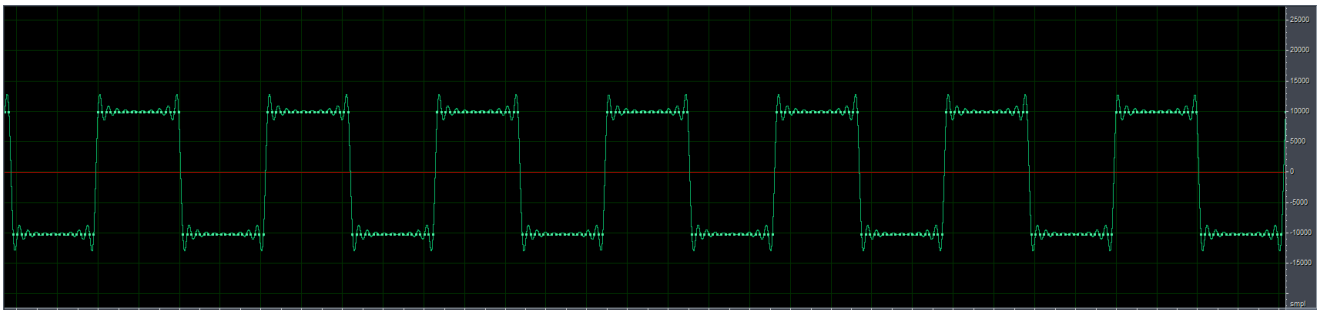


Figure 8.5-2 RKStudio Square Wave Source Output Time Domain

8.6 Triangle

Function Description: The oscillator algorithm is used to generate a triangle wave signal. Modify the Frequency to obtain the desired frequency signal.

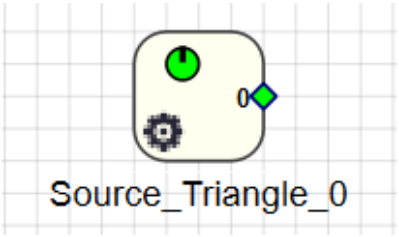


Figure 8.6-1 Triangle Control and Settings Interface

Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	int	Enable	None	0/1	1
Frequency(Hz)	1	float	Frequency magnitude	Hz	Less than half the sampling rate	1000

Typical Case: The parameter is set to 1200, and the corresponding input result is as follows:

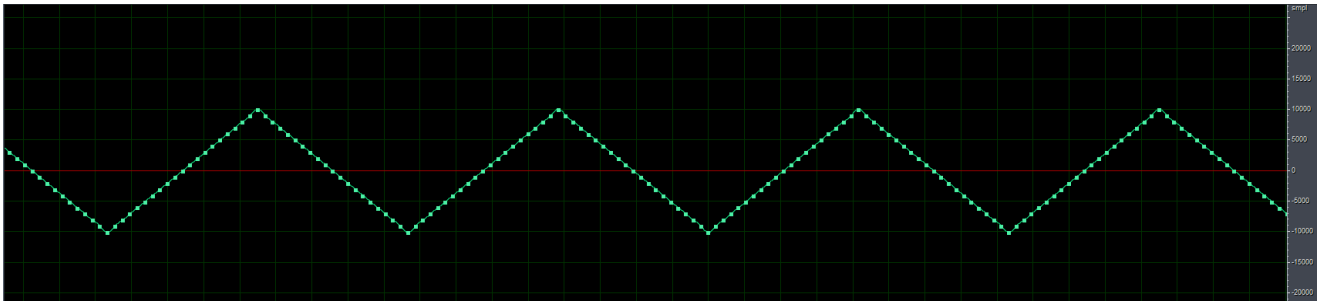


Figure 8.6-2 RKStudio Triangle Wave Source Output Time Domain

8.7 Chime

Function Description: By setting the time and frequency changes, a variable frequency audio signal can be generated, which can be used to create a frequency-sweep signal. The green button in the control is used to start/stop the frequency sweep, and the circle is used to set whether to loop playback.

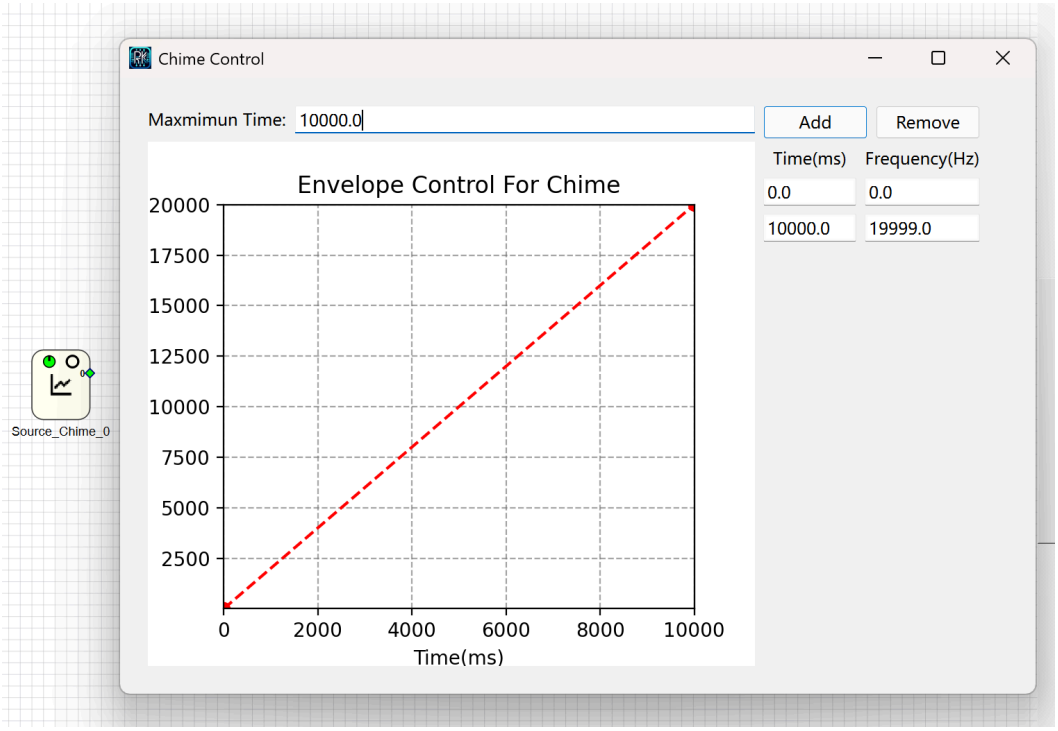


Figure 8.7-1 Chime Control and Settings Interface

Typical Case: When configured with the parameters shown in Figure 8.7-1, the output spectrum is as follows:

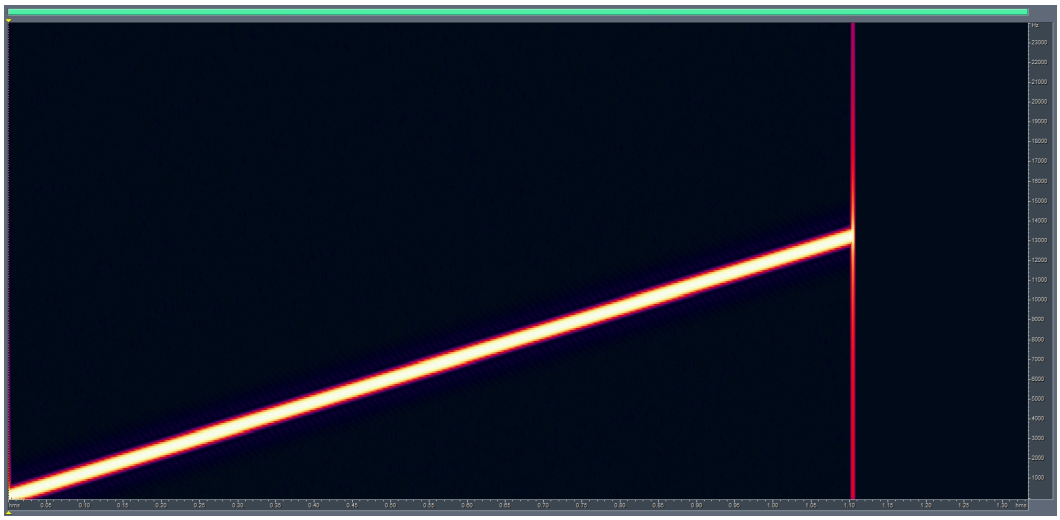


Figure 8.7-2 Spectrum of the Chime Output Waveform

8.8 Switch(0,1)

Function Description: Outputs a signal through a button. Green represents 1, and red represents 0.

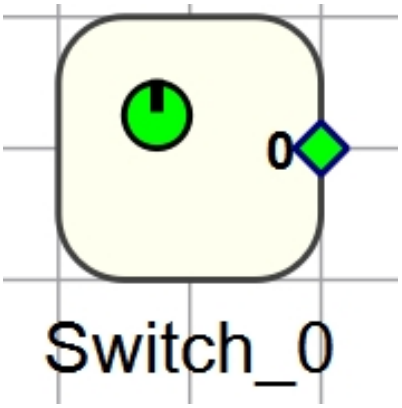


Figure 8.8-1 Switch Control and Settings Interface

Number of Parameters: 1

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Switch State	0	int	Switch state	None	0/1	1

8.9 Voltage Controlled

Function Description: Voltage-controlled oscillator algorithm. The output frequency is adjusted by the input size (input * samplerate / 2). If the input is greater than or equal to 1, the output frequency corresponds to half the sample rate; if less than or equal to 0, the output frequency is 0.

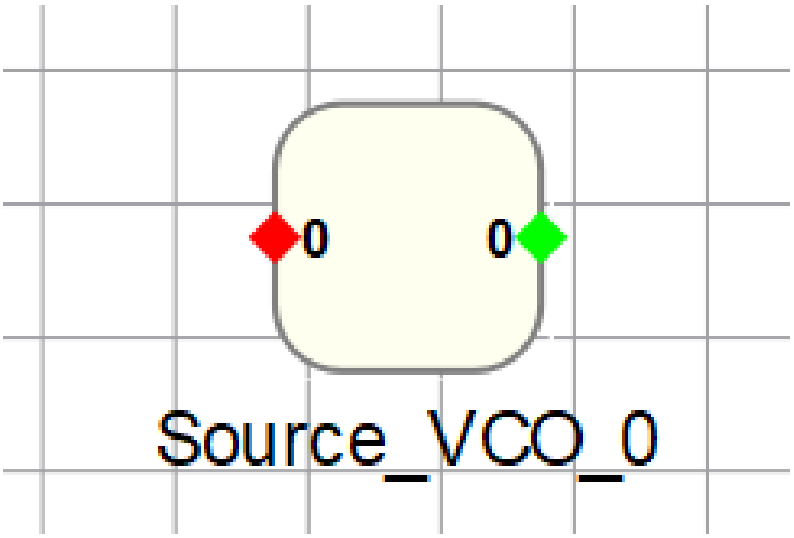


Figure 8.9-1 VCO Control and Settings Interface

Number of Parameters: 0

Typical Case: Add a DC module in front and set it to 0.5, connected to the VCO module.

Figure 8.9-2 RKStudio VCO Source Output Spectrum

8.10 White Noise

Function Description: Gaussian white noise algorithm. Modify the mean and variance to adjust the magnitude of the white noise.

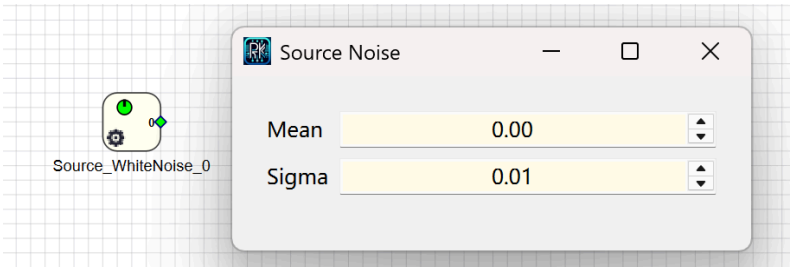


Figure 8.11-1 White Noise Control and Settings Interface

Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Mean	0	float	Mean, corresponding to DC bias		0 ~ 1	0
Sigma	1	float	Variance, corresponding to output amplitude		0 ~ 1	0.01

Typical Case: Set parameters to Mean = 0, Sigma = 0.1:

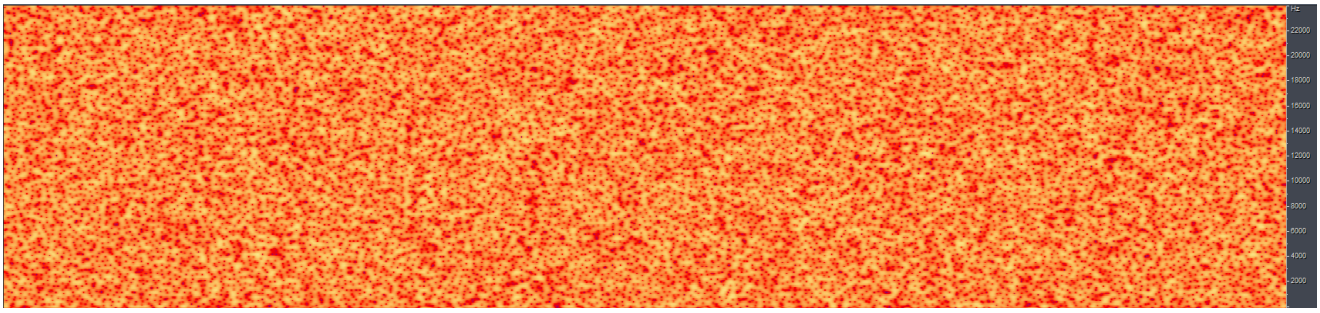


Figure 8.10-1 RKStudio White Noise Source Output Frequency Domain

8.11 Pink Noise

Function Description: Pink noise algorithm. Modify the mean and variance to adjust the magnitude of the pink noise.

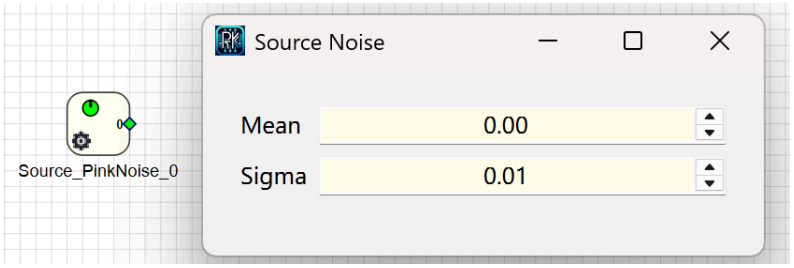


Figure 8.11-1 White Noise Control and Settings Interface

Number of Parameters: 2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Mean	0	float	Mean, corresponding to DC bias		0 ~ 1	0
Sigma	1	float	Variance, corresponding to output amplitude		0 ~ 1	0.01

Typical Case: Set parameters to Mean = 0, Sigma = 0.1:

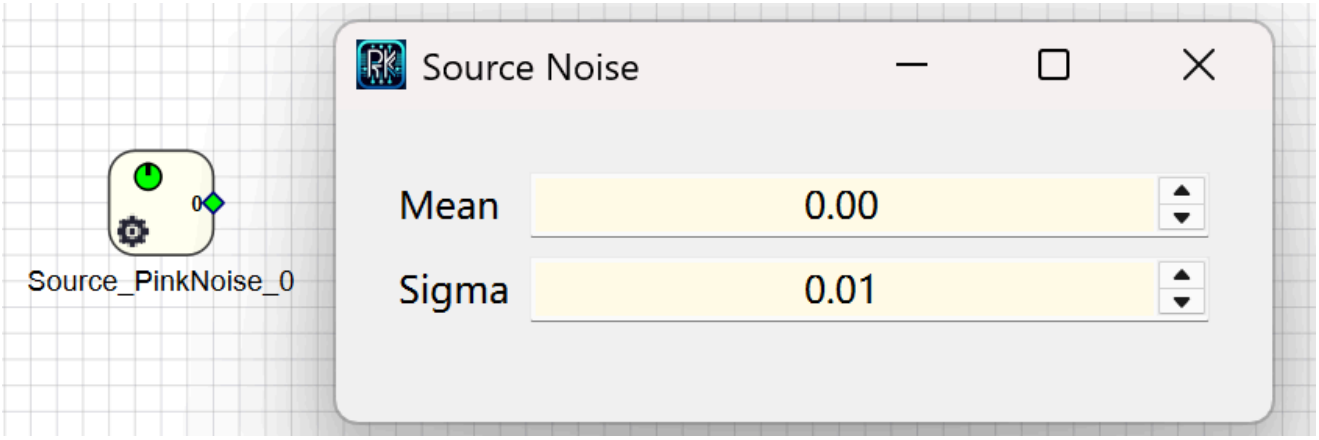


Figure 8.11-1 RKStudio Pink Noise Source Output Frequency Domain

8.12 Wav Player

Function Description: Plays WAV or PCM audio files. If it is a WAV file, the output channel will automatically adjust according to the actual format and supports resampling. The input box indicates the current file idx. The triangle represents the play button, the square is the stop button, and the circle is the loop playback button. Double-click to see the other parameter configuration windows.

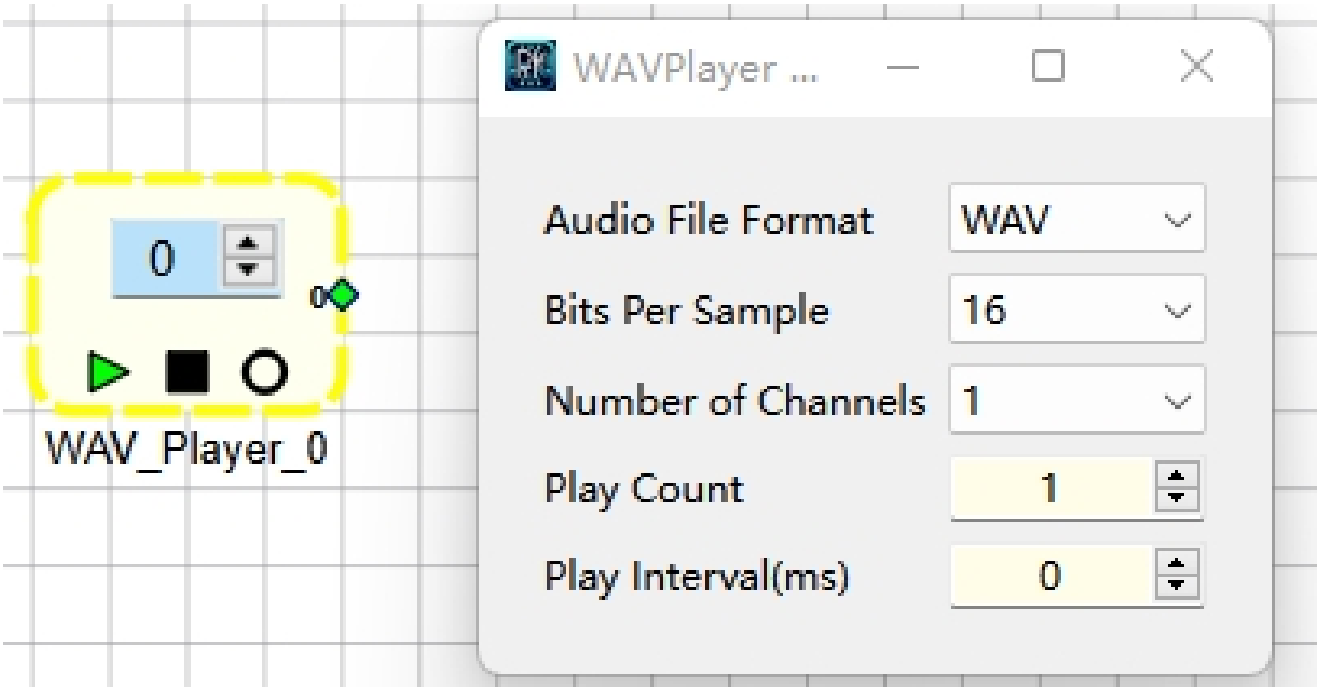


Figure 8.12-1 WavPlayer Control Settings Interface

Number of Parameters: 8

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Index	0	int	Current playing file idx	None	0 ~ 127	0

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
PlayState	1	int	Playback state, 0 for stop, 1 for pause, 2 for play. This parameter is both a status display and can control the playback state (green triangle area in the figure)	None	0/1/2	0
Circular Mode	2	int	Loop mode, 0 for single playback, 1 for loop (circle area in the figure, black for 0, green for 1)	None	0/1	0
Format	3	int	Current audio file format, 0 for PCM, 1 for WAV	None	0/1	0
Bits	4	int	Current audio sample quantization bit number, only for PCM format	None	0/1/2/3 (0 for 8, 1 for 16, 2 for 24, 3 for 32)	1
Chan	5	int	Current audio channel number, only for PCM format	None	0/1 (0 for 1 channel, 1 for 2 channels)	0
Play Count	6	int	Number of plays, only for single playback mode	None	0 ~ 1000	1
Play Interval	7	int	Playback interval	ms	0 ~ 10000	0

The files in the wavPlayer also depend on the burned files in the rkwav.img partition, which can be seen in the partition configuration and burning tool. This partition file contains WAV files, which can be played directly through idx. The configuration entry is in Tools -> File Partition Configuration. After clicking OK, an rkwav.img file will be generated automatically and needs to be burned to the corresponding partition.

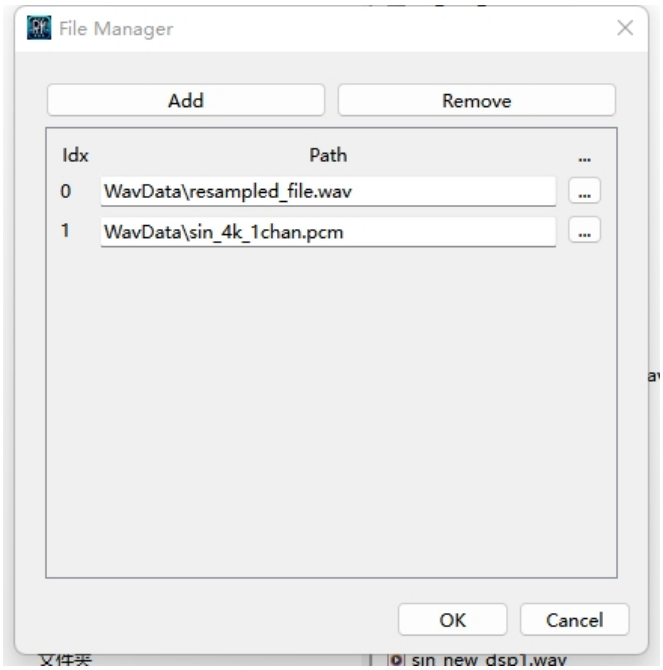


Figure 8.12-2 WAV Partition Management Settings Interface

8.13 LFO

Function Description: Low Frequency Oscillator (LFO).

This module is used to generate periodic waveform signals below the audio frequency range (typically < 20 Hz). The LFO itself is generally not used to produce audible sound directly; instead, it serves as a modulation source to control parameters of other audio processing modules. For example, modulating amplitude can produce tremolo, modulating frequency can produce vibrato, and it can also be used for effects such as automatic panning.

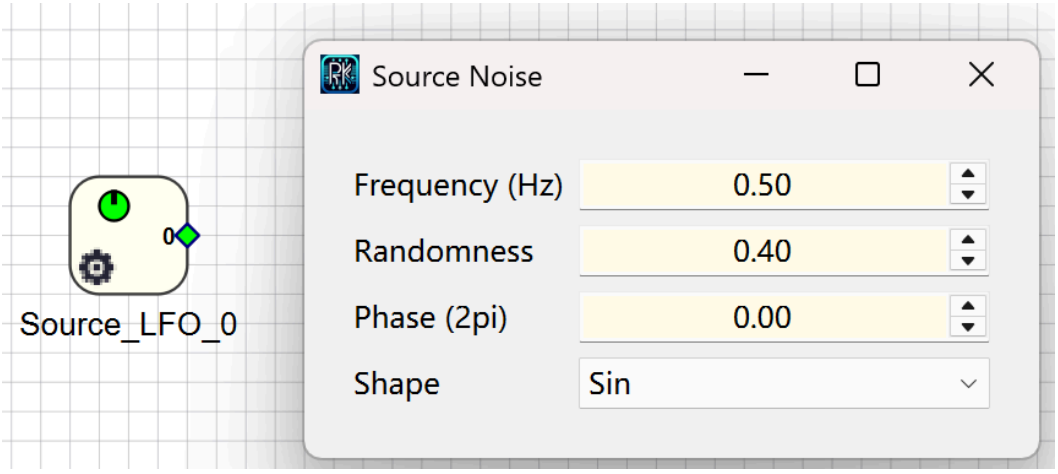


Figure 8.13-1 LFO control and settings interface

8.14 Ctrl Rate Noise

Function Description: Control-rate noise generator.

This module generates a low-frequency, randomly varying control signal. Unlike audio noise, its frequency is typically low and the signal changes smoothly over time. It is primarily used as a modulation source to control parameters of other modules, producing non-periodic and more organic-sounding effects.

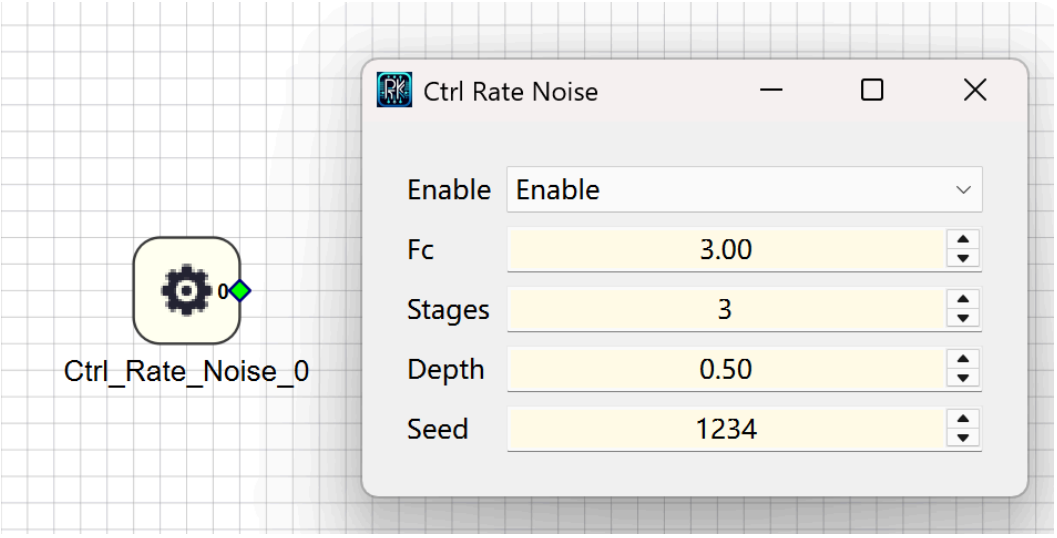


Figure 8.14-1 Ctrl Rate Noise control and settings interface

9 Volume Controls

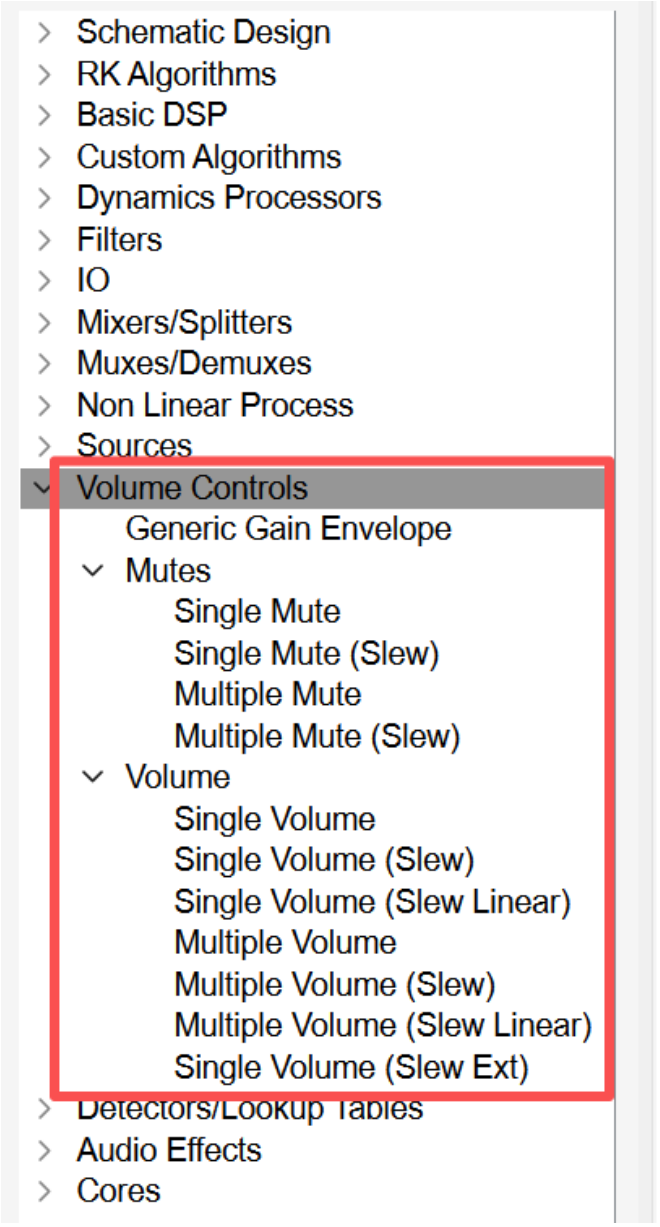


Figure 9-1 Location of VolumeControl in the algorithm list

9.1 Generic Gain Envelope

This module is designed for gain that changes over time. Two channels of white noise with an RMS of -20 dB are used for testing. The control points of the module have x-coordinates: {0, 2, 502, 504, 1004, 1104} and y-coordinates: {-100, 0, -100, 0, -70, 0}, as shown in the figure below:

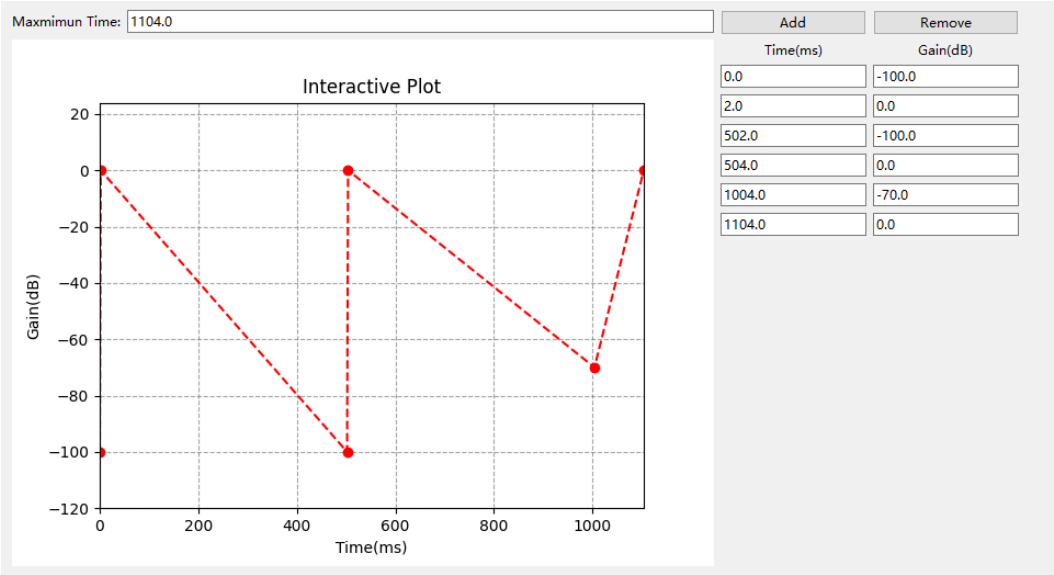


Figure 9.1-1 Generic Gain Envelope parameter settings

The output from RKStudio is as follows:

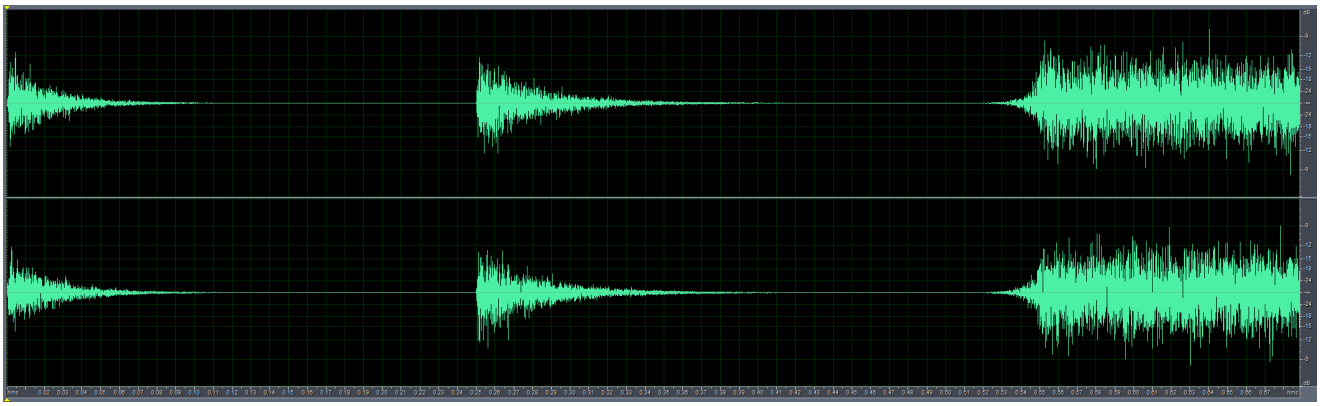


Figure 9.1-2 Generic Gain Envelope waveform output

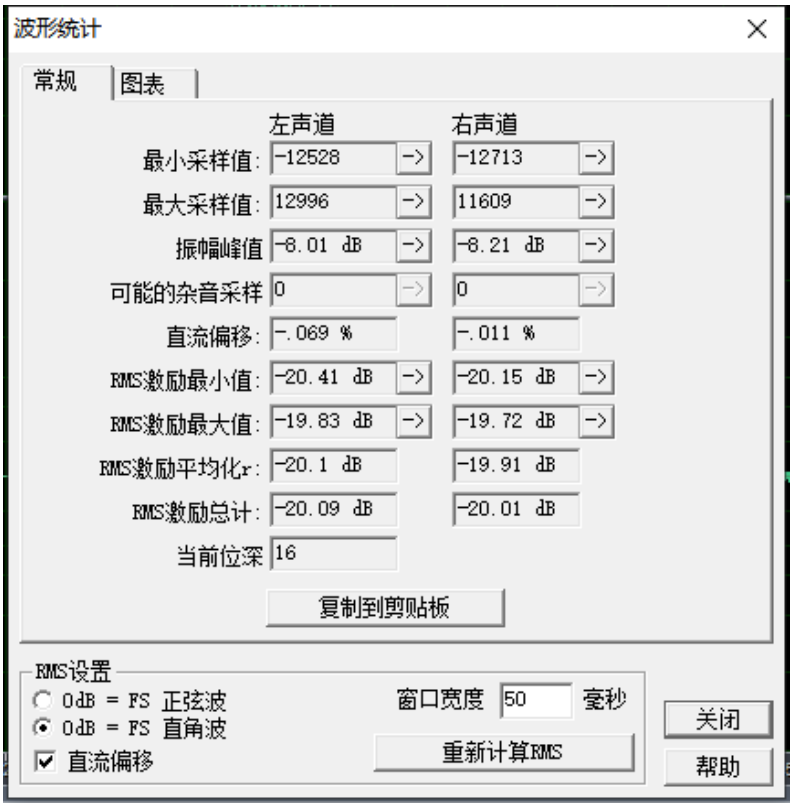


Figure 9.1-3 Generic Gain Envelope waveform statistics

9.2 Mute

9.2.1 Mute

Function Description: Mute.

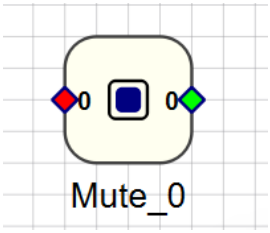


Figure 9.2-1 Mute Control

Parameter Description: Enable mute when selected.

9.2.2 Mute (Slew)

Function Description: Mute with smoothing.

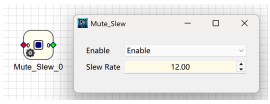


Figure 9.2-2 Mute (Slew) Control

Parameter Description: Enable mute when selected.

9.2.3 Multiple Mute

Function Description: Mute.

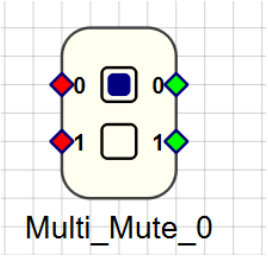


Figure 9.2-3 Multiple Mute Control

Parameter Description: Enable mute when selected.

9.2.4 Multiple Mute (Slew)

Function Description: Mute with smoothing.

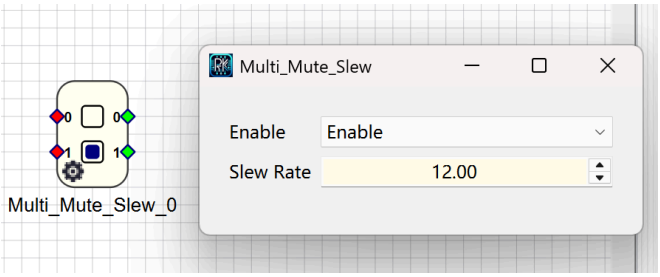


Figure 9.2-4 Multiple Mute (Slew) Control

Parameter Description: Enable mute when selected.

9.3 Volume

9.3.1 Single Volume

Function Description: Input * Fixed Gain = Output.

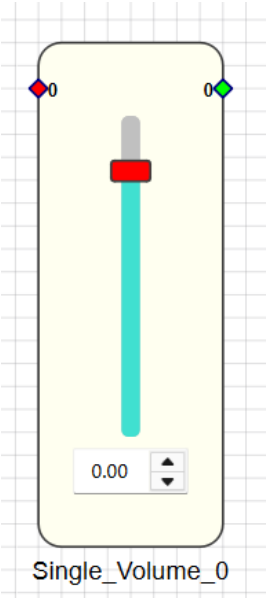


Figure 9.3.1-1 Single Volume Control

Number of Parameters: 1

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Gain	0	float	Gain for all channels	dB	-80 ~ 10	0

Effect Demonstration: Input data is white noise with a gain set to -10 dB:

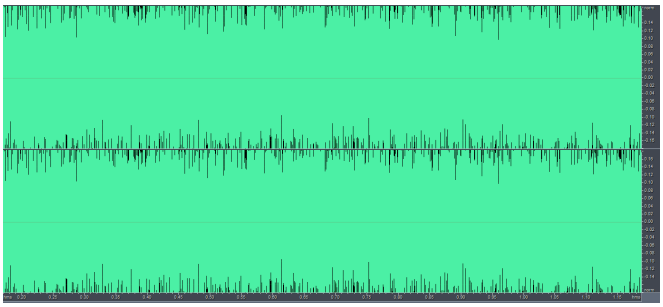


Figure 9.3.2-2 Input audio source

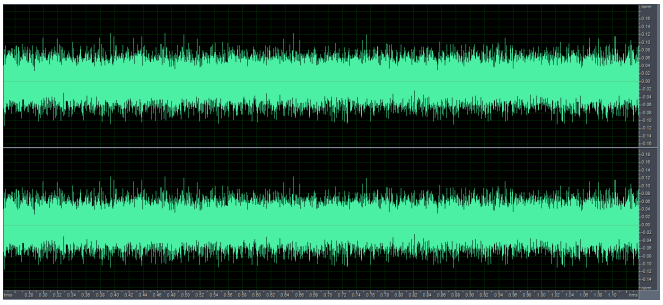


Figure 9.3.2-3 Single Volume Control output time domain

9.3.2 Single Volume (Slew)

Function Description: Output = Input * Fixed Gain , with smoothing between frames.

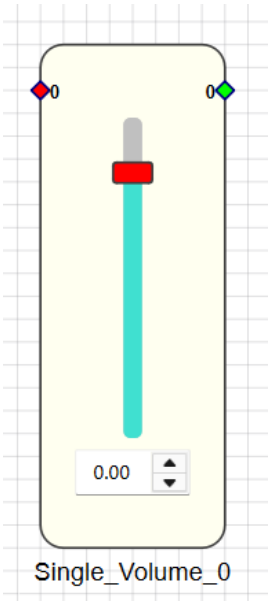


Figure 9.3.2-1 Single Volume (Slew) Control

Number of Parameters: 1

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Gain	0	float	Gain for all channels	dB	-80 ~ 10	0

Effect Demonstration: Input data is white noise with a gain set to -10 dB:

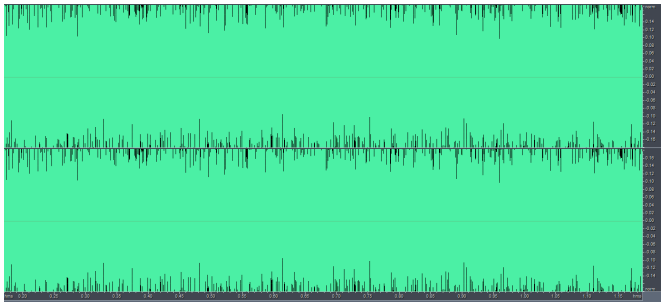


Figure 9.3.2-2 Input audio source

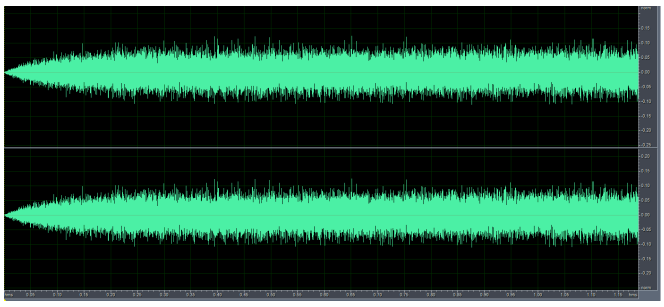


Figure 9.3.2-3 Single Volume Control (Slew) output time domain

9.3.3 Multiple Volume

Function Description: Each channel output = Each channel input * Each channel fixed gain .

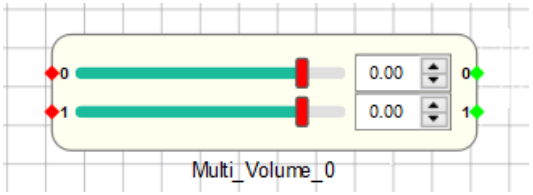


Figure 9.3.3-1 Multiple Volume Control

Number of Parameters: input_bin

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Gain	0 ~ input_bin-1	float	Gain for each channel	dB	-80 ~ 10	0

Effect Demonstration: Input data is white noise with a gain set to -10 dB, and parameters are configured as follows:

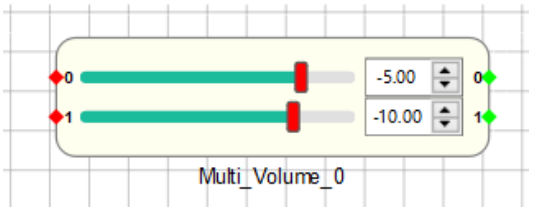


Figure 9.3.3-2 Multiple Volume parameters

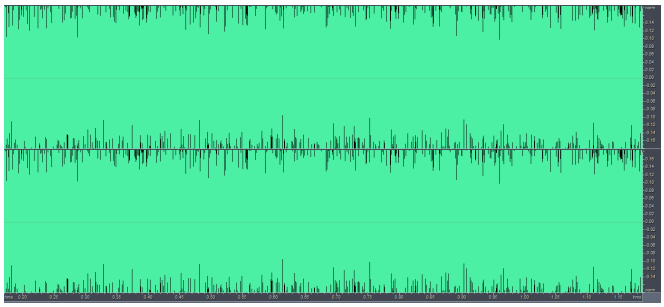


Figure 9.3.3-3 Input audio source

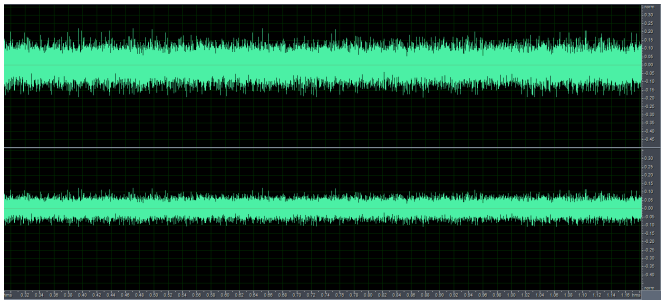


Figure 9.3.3-4 Multiple Volume Control output time domain

9.3.4 Multiple Volume (Slew)

Function Description: Each channel output= Each channel input * Each channel fixed gain , with smoothing between frames.

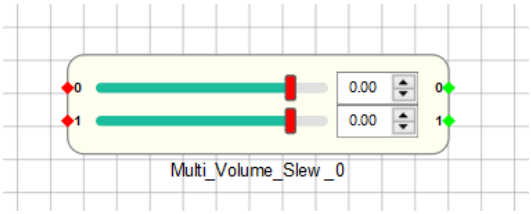


Figure 9.3.4-1 Multiple Volume (Slew) Control

Number of Parameters: input_bin

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Gain	0 ~ input_bin-1	float	Gain for each channel	dB	-80 ~ 10	0

Effect Demonstration: Input data is white noise, and parameters are configured as follows:

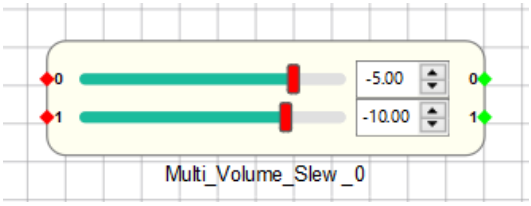


Figure 9.3.4-2 Multiple Volume (Slew) parameters

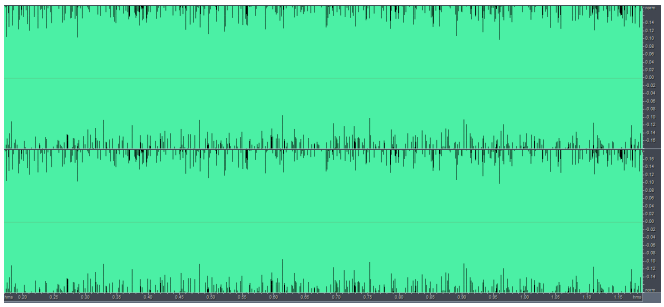


Figure 9.3.4-3 Input audio source

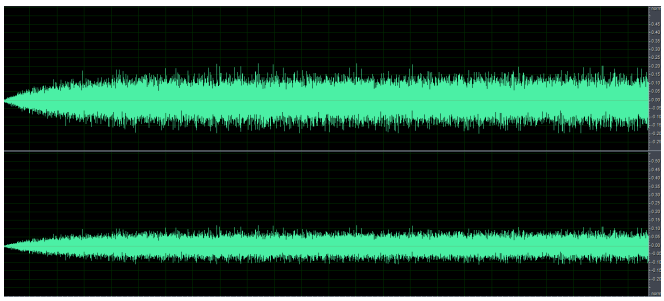


Figure 9.3.4-4 Multiple Volume output time domain

9.3.5 Single Volume(Slew Linear)

Function Description: Linear slew volume control.

Based on the standard volume adjustment function, this module adds a smoothing-time control parameter. When the user changes the gain, the module performs a linear ramp from the current gain to the target gain within the specified time.

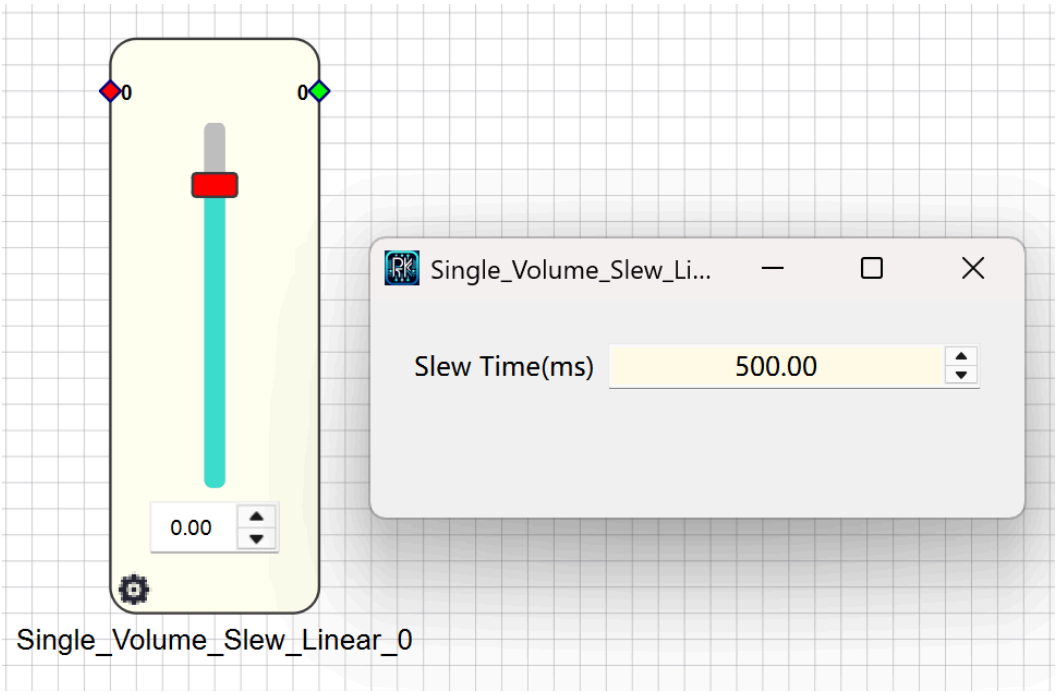


Figure 9.3.5-1 Single Volume (Slew Linear) control and settings interface

9.3.6 Multiple Volume(Slew Linear)

Function Description: Multi-channel linear slew volume control.

This module combines the multi-channel independent control capability of Multiple Volume with the user-configurable smoothing-time feature of Slew Linear.

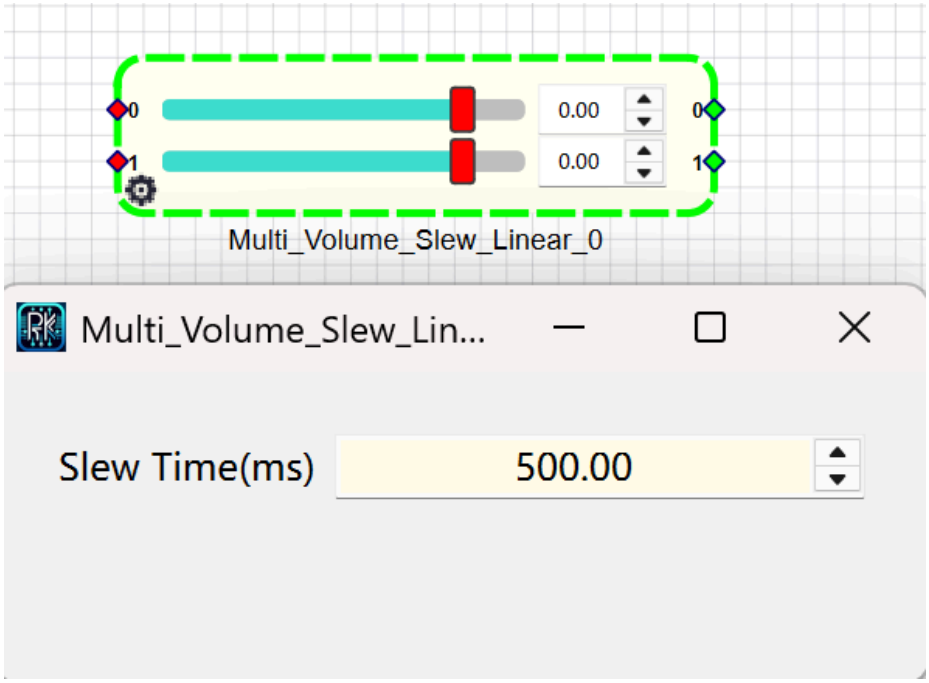


Figure 9.3.6-1 Multiple Volume (Slew Linear) control and settings interface

9.3.7 Single Volume(Slew Ext)

Function Description: Slew-enabled single-channel volume with external control.

- Signal routing: Input 0 (upper blue pin) is the control-signal input, and Input 1 (lower red pin) is the audio-signal input.
- Operating principle: The module uses the value from Input 0 as the gain coefficient applied to Input 1.
- Smoothing: When the control value at Input 0 changes, the module does not apply an immediate gain jump. Instead, it performs a smooth transition based on the configured Slew Rate, which is commonly used for envelope-following volume control or ducking effects.

10 Detectors

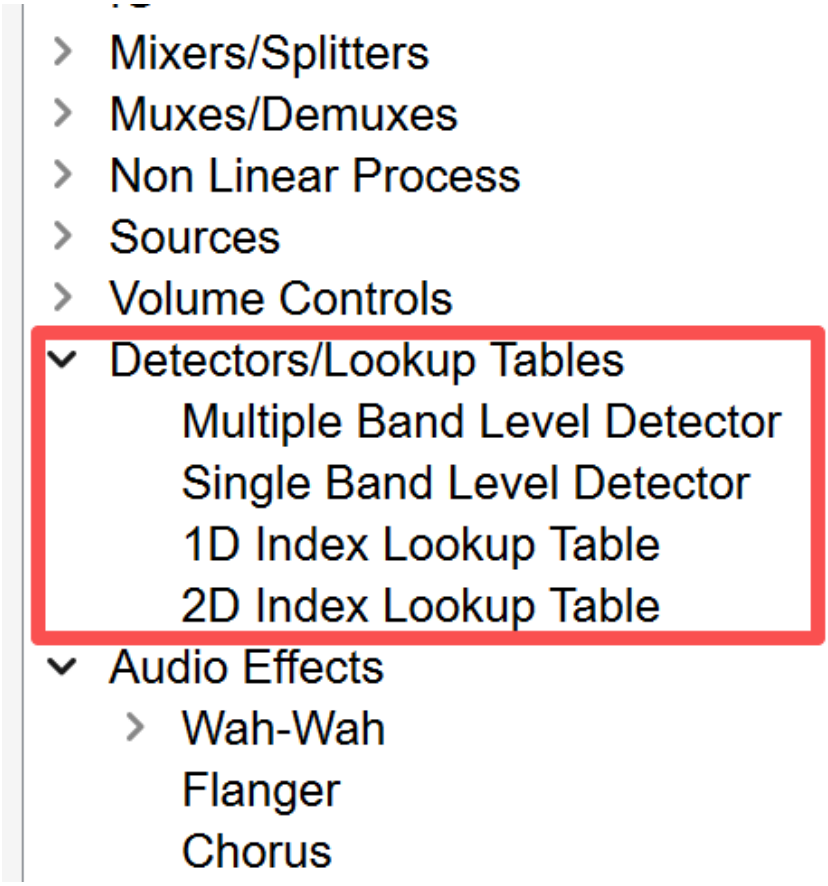


Figure 10-1 Location of Detectors in the tool

10.1 Multiple Band Level Detector

Function Description: Detects energy across multiple frequency bands and retrieves it through parameter reading. The principle involves setting multiple parallel filters, with each filter corresponding to the detection of a specific frequency band.

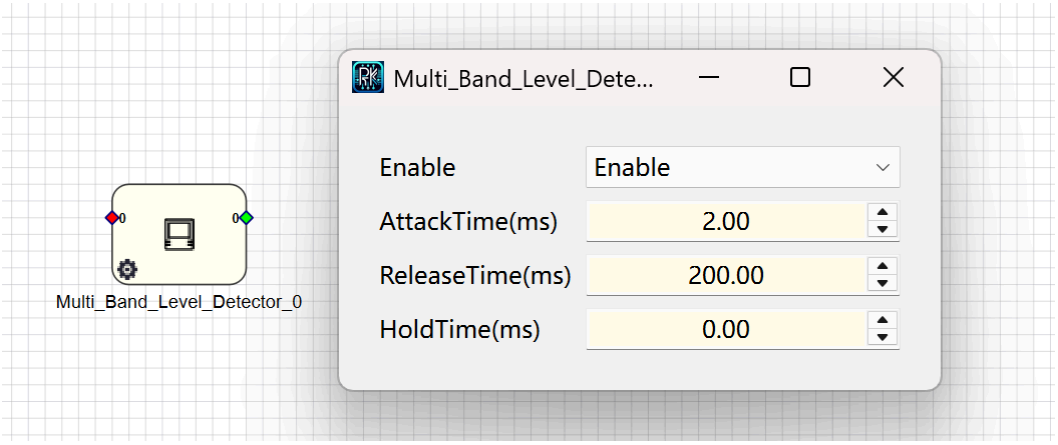


Figure 10.1-1 Control and general parameter settings interface

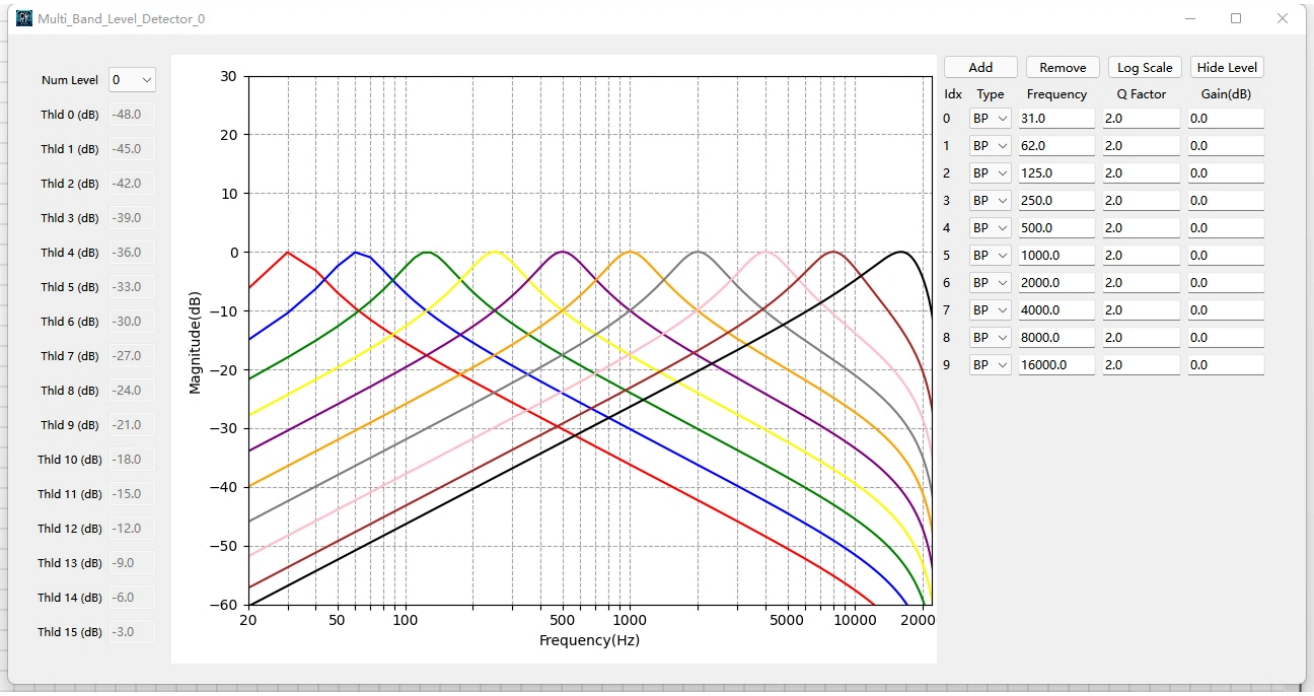


Figure 10.1-2 Frequency band settings interface

Figure 10.1-2 shows the parameter settings interface for the multi-band detector (click the middle button to access). Clicking "Show Level" in the upper right corner will display the band level classification function, which is generally not necessary. The "Thld" on the left is used to set the threshold range, which returns the corresponding energy level.

General parameter count: 6

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	int	Whether to enable		0/1	1
Type	1	int	Detection type (Peak/Rms)		0/1	0
Attack Time (ms)	2	float	Attack time for energy statistics	ms	0~1000	2
Release Time (ms)	3	float	Decay time for energy statistics	ms	0~1000	200
Detect Threshold (dB)	4	float	Overall detection threshold	dB	-138 ~ 24	0
Band Num	5	int	Number of frequency bands		1~32	10
Thld Num	6	int	Number of levels		1~16	0

Parameter settings for each filter:
Parameter count: 4

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
idx	...	int	Filter index			
Type	...	int	Filter type		0/1/2	0
Frequency	...	float	Center frequency of the band	Hz	20 ~ 23760	0
Q_Factor	...	int	Quality factor Q		0.01 ~ 16	2
Gain (dB)	...	float	Gain	dB	0.01 ~ 16	0

If 10 bands and 16 Thld Levels are defined, the following parameters are (starting from 7 because one Parameter Index is reserved):

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Coeff	7~(7 + 5 * 10)	float	Filter coefficients			
Thld	57~(57 + 16)	float	Threshold levels; if Thld Num is 0, it is not stored			
Rms	73 ~ (73 + 10)	float	Energy of each band; read using application parameters	dB		
Level	83 ~ (83 + 10)	float	Level corresponding to the energy of each band; read using application parameters			

10.2 Single Band Level Detector

Function Description: Detects the energy of an audio signal and retrieves it through parameter reading.

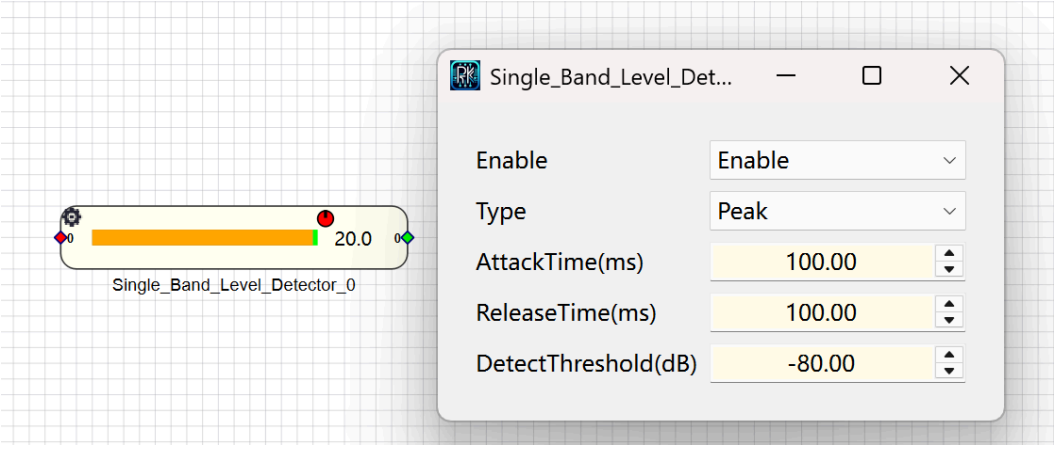


Figure 10.2-1 Control and parameter settings interface

Parameter count: 5

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	int	Whether to enable		0/1	1
Type	1	int	Detection type (Peak/Rms)		0/1	0
Attack Time (ms)	2	float	Attack time for energy statistics	ms	0~1000	100
Release Time (ms)	3	float	Decay time for energy statistics	ms	0~1000	100
Detect Threshold (dB)	4	float	Detection threshold	dB		-80
Rms	5	float	Energy of the audio signal; read using application parameters	dB		

10.3 1D Index Lookup Table

Function Description: One-dimensional index lookup table.

This module treats the input signal as an index and looks up the corresponding value in an internally predefined table, then outputs the retrieved value.

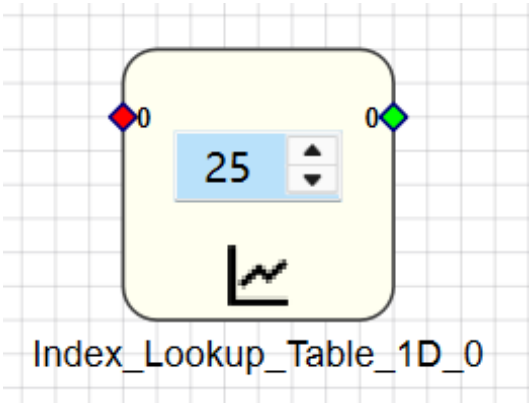
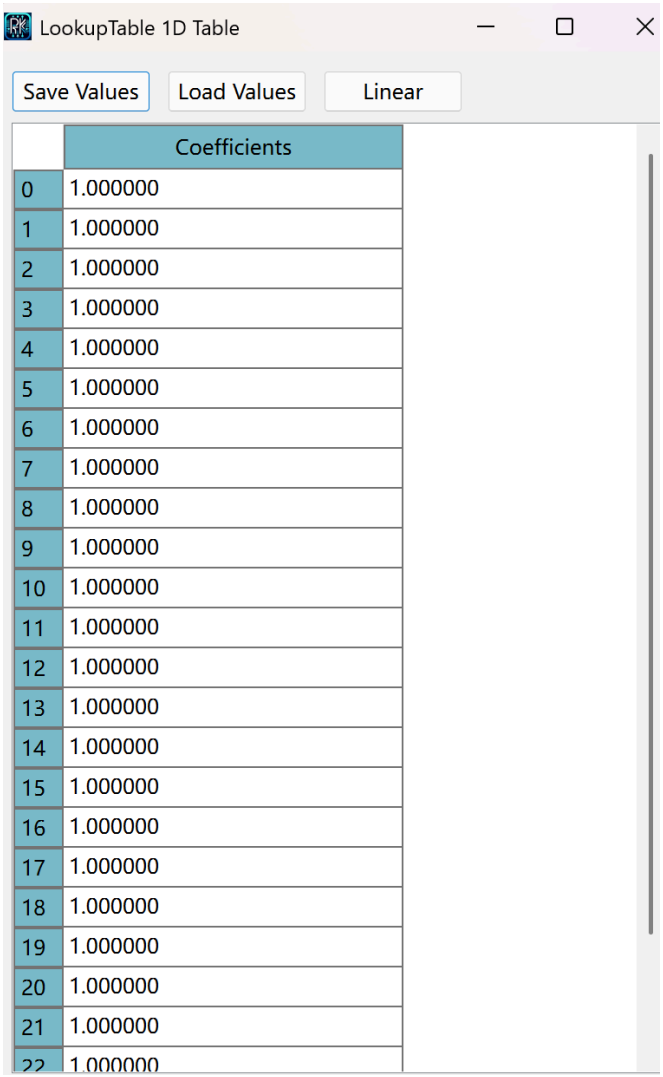


Figure 10.3-1 1D Index Lookup Table Control



10.3-2 1D Index Lookup Table Parameter settings interface

10.4 2D Index Lookup Table

Function Description: Two-dimensional index lookup table.

This module accepts two input signals (Input 0 corresponds to the X-axis and Input 1 corresponds to the Y-axis). The module treats them as coordinate indices and looks up the corresponding value in an internally defined 2D matrix (table), then outputs the retrieved value.

- X index: Determines the table column.
- Y index: Determines the table row.

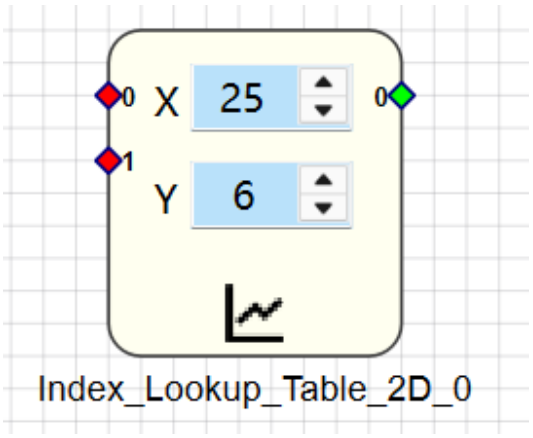
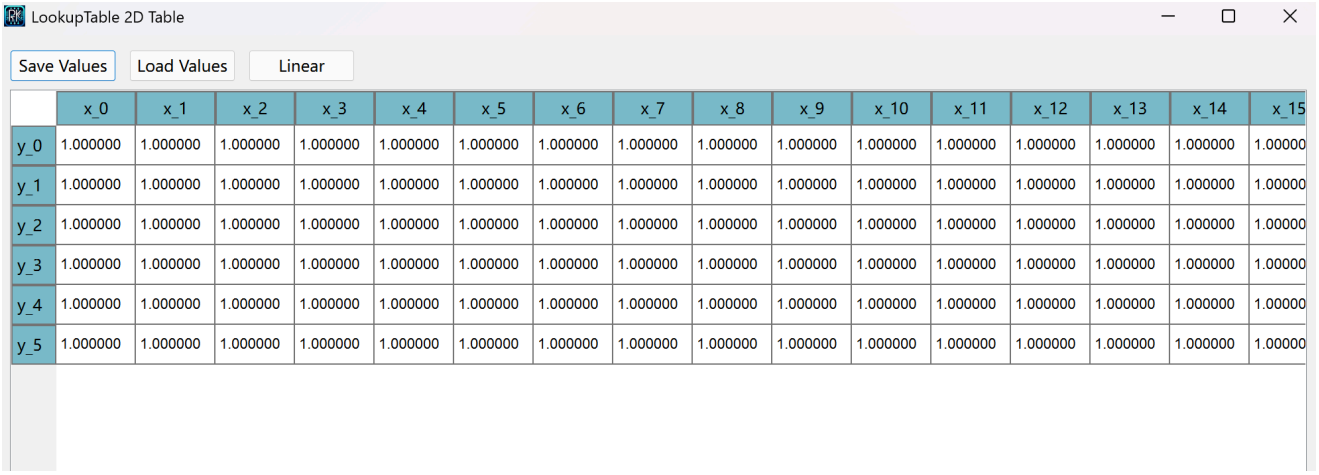


Figure 10.4-1 2D Index Lookup Table Control



	x_0	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_10	x_11	x_12	x_13	x_14	x_15
y_0	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.00000
y_1	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.00000
y_2	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.00000
y_3	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.00000
y_4	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.00000
y_5	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.00000

Figure 10.4-2 2D Index Lookup Table Parameter settings interface

11 Audio Effects

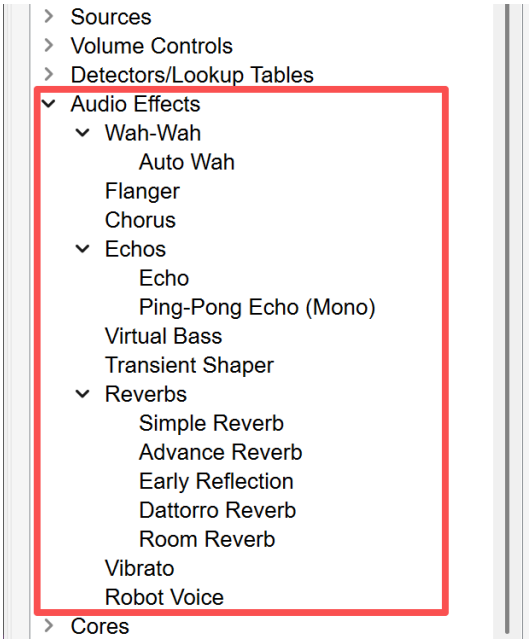


Figure 11-1 Location of Effects in the tool

11.1 Wah-Wah

11.1.1 Auto Wah

Function Description: An audio effect control used to simulate the traditional wah-wah pedal effect.

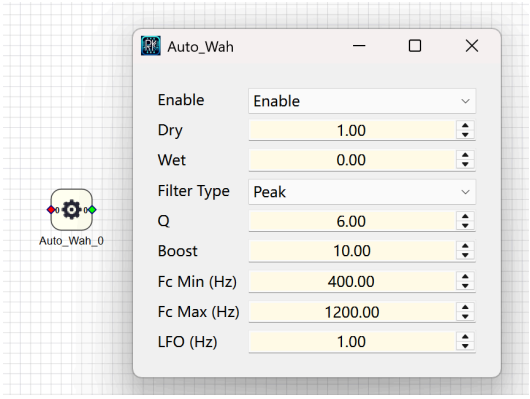


Figure 11.1.1-1 Auto Wah control and parameter settings

Number of parameters: 9

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	int	Whether to enable		0/1	1
Dry	1	float				1

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Wet	2	float				0
Filter Type	3	int	Filter type		0/1	0
Q	4	float	Quality factor			6
Boost	5	float				10
Fc Min (HZ)	6	float	Minimum cutoff frequency	HZ		400
Fc Max (HZ)	7	float	Maximum cutoff frequency	HZ		1200
LFO (HZ)	8	float		HZ		1

11.2 Flanger

Function description: By combining LFO with delay, special sound effects can be achieved. The structural diagram is shown below.

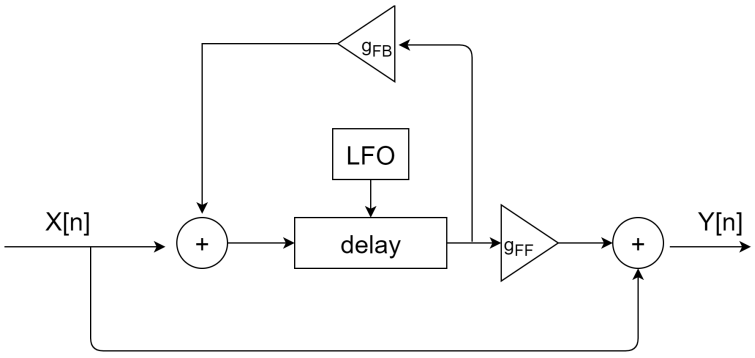


Figure 11.2-1 Flanger Control Block Diagram

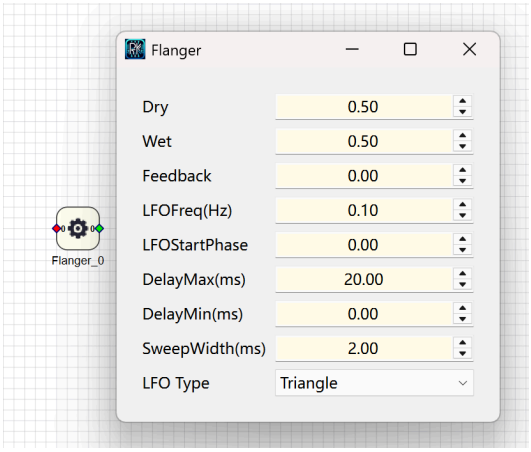


Figure 11.2-2 Flanger Control and Parameter Settings

Number of parameters: 9

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Dry	0	float	Dry signal coefficient (Original signal volume)	-	0.0 ~ 1.0	0.5
Wet	1	float	Wet signal coefficient (Effect signal volume)	-	0.0 ~ 1.0	0.5
Feedback	2	float	Feedback coefficient (Corresponds to gFB in Figure 11.3-1)	-	0.0 ~ 0.99	0
LFOFreq(Hz)	3	float	LFO frequency	Hz	0.1 ~ 10.0	0.10
LFOStartPhase	4	float	Start phase	degree	0 ~ 360	0.00
DelayMax(ms)	5	float	Maximum delay buffer length	ms	1.0 ~ 50.0	20.00
DelayMin(ms)	6	float	Minimum delay time	ms	0.0 ~ 10.0	0.00
SweepWidth(ms)	7	float	Delay sweep width	ms	0.1 ~ 10.0	2.00
LFO Type	8	int	LFO waveform type	-	0 ~ 3	Triangle

11.3 Echo

11.3.1 Echo

Function description: Implement echo simulation based on magnetic tape. The structural diagram is shown below. This module will extract multiple data from the delay buffer and multiply them by different coefficients to obtain the output result. At the same time, the output result will be fed back after passing through a low-pass filter and stored in the delay buffer together with the input data.

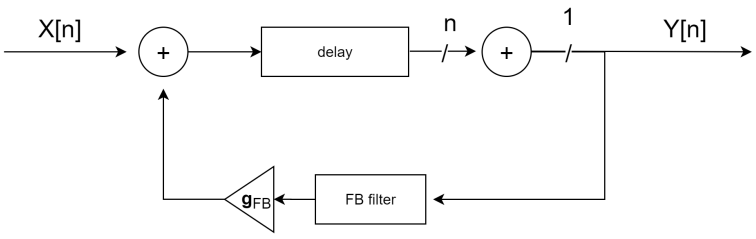


Figure 11.3-1 Echo Control Block Diagram

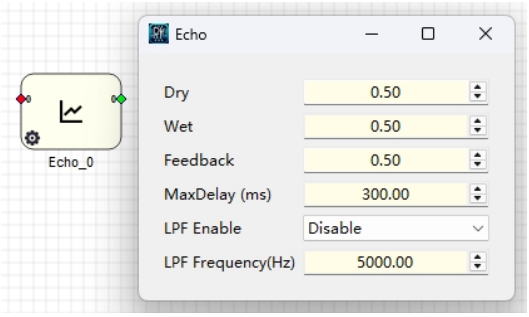


Figure 11.3-2 Click the button in the bottom left corner of the module to configure Echo control parameters

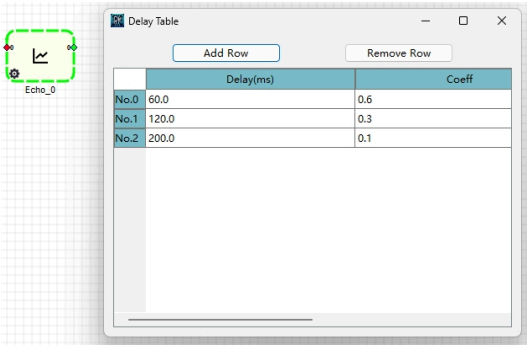


Figure 11.3-3 Click the middle button of the module to configure Echo delay parameters

Number of parameters: 3+1+2 * n (where n is the number of delay points)

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Dry	0	float	Dry signal gain coefficient			0.5
Wet	1	float	Wet signal gain coefficient			0.5
Feedback	2	float	Feedback coefficient, corresponding to gFB in Figure 11.3-1			0.5
MaxDelay	3	float	Maximum delay buffer length (real-time parameter tuning is supported when tap delays are within this range)	ms		300
LPF Enable	4	float	低Low-pass filter enable switch	Hz		Disable
LPF Frequency	5	float	Low-pass filter cutoff frequency	Hz		5000

Tap Parameters (Tap i, i = 1, 2, …, n)

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Delay	11+i-1	float	Delay time for tap i	ms		
Coeff	11+n+i	float	Gain coefficient for tap i			

Note: Using more than one tap may make the echo sound cluttered. For initial debugging, it is recommended to use one tap only.

11.3.2 Ping-Pong Echo (Mono)

Function description: Mono ping-pong echo.
This module accepts a mono input signal and processes it as two channels (left and right). By setting different delay times for the left and right channels (Delay L and Delay R) and introducing cross-feedback between the two channels, the echoes bounce back and forth between L and R, creating a wide stereo spatial effect.

- Input: Mono.
- Output: Stereo, including L and R channels.

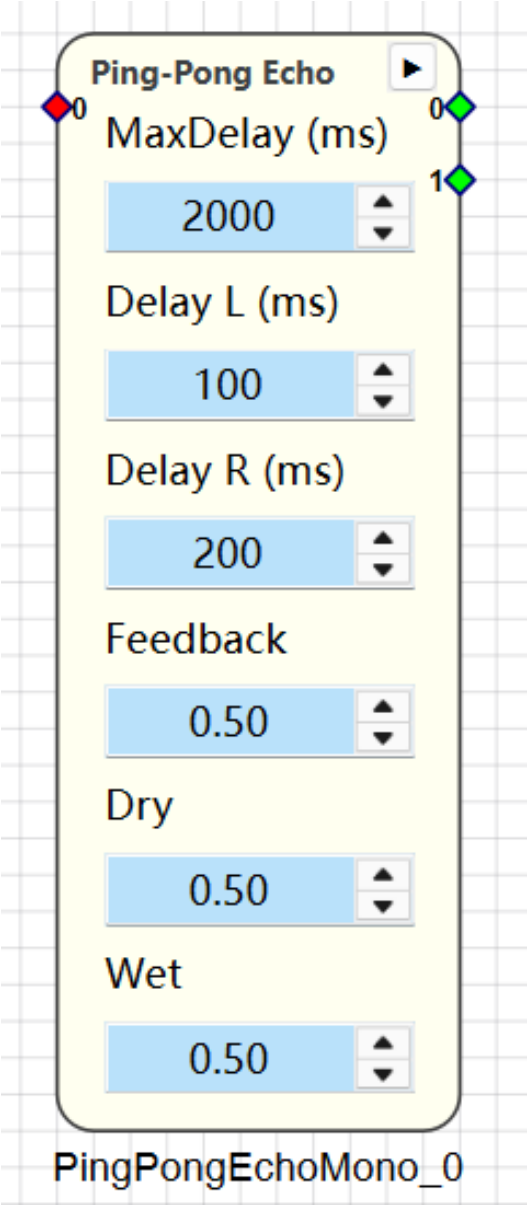


Figure 11.3.2-1 Ping-Pong Echo (Mono) control block diagram

11.4 Virtual Bass

Function description: Using the missing fundamental principle, virtual bass effect is achieved. The logical diagram is as follows, where HFIL is a fourth-order link witz high pass filter, FIL1 is a fourth-order link witz low-pass filter+fourth-order link witz high pass filter, FIL2 is a fourth-order link witz low-pass filter+fourth-order link witz high pass filter.

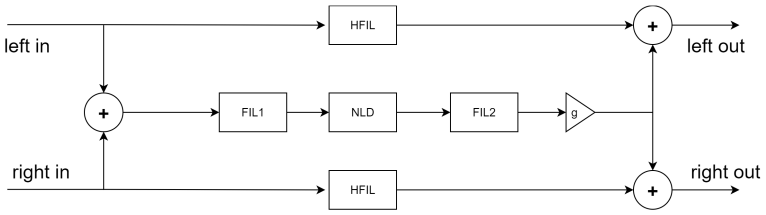


Figure 11.4-1 Virtual Bass Diagram

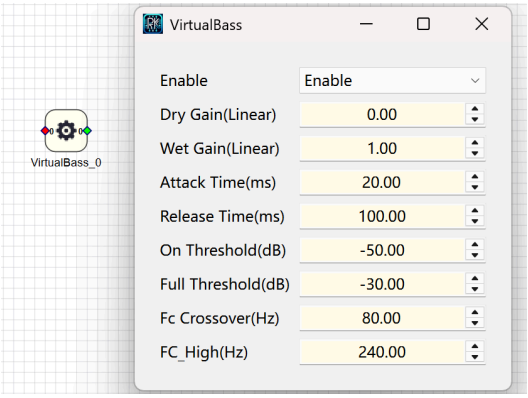


Figure 11.4-2 Virtual Bass parameter settings

Number of parameters: 9

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	float	Module enable switch	-	0 / 1	1.0
Dry Gain(Linear)	1	float	Dry signal gain (linear value)	-	0.0 ~ 2.0	0.00
Wet Gain(Linear)	2	float	Wet signal gain (linear value for enhanced signal)	-	0.0 ~ 2.0	1.00
Attack Time(ms)	3	float	Detector attack time	ms	1 ~ 100	20.00
Release Time(ms)	4	float	Detector release time	ms	10 ~ 1000	100.00
On Threshold(dB)	5	float	Activation threshold; enhancement starts above this level	dB	-90 ~ 0	-50.00
Full Threshold(dB)	6	float	Full enhancement threshold; maximum enhancement reached	dB	-90 ~ 0	-30.00
Fc Crossover(Hz)	7	float	Crossover frequency for bass processing	Hz	40 ~ 200	80.00
FC_High(Hz)	8	float	Cutoff frequency for generated harmonics	Hz	100 ~ 500	240.00

In addition to using this module to achieve the Virtual Bass effect, the tool can also achieve the same effect through a combination of filters and NLP. The schematic diagram is as follows:

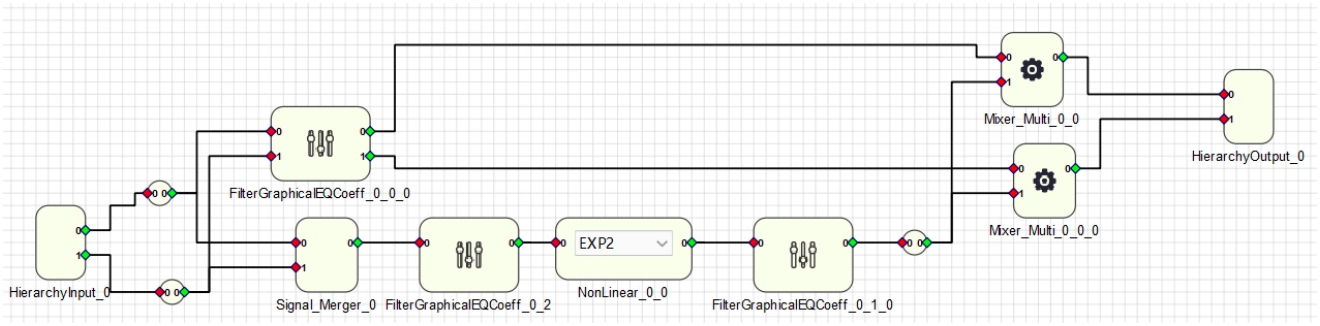


Figure 11.4-3 Implementing Virtual Bass with Filter+NLP

11.5 Reverbs

11.5.1 Simple Reverb

Function Description: Implements a simple reverb effect, supporting both mono and stereo.

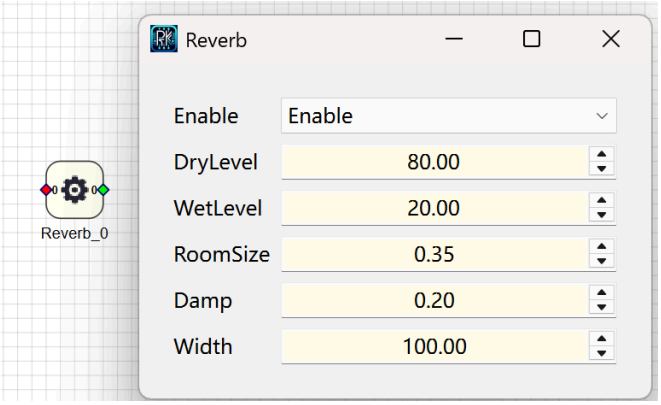


Figure 11.5.1-1 Simple Reverb control and settings interface

Number of Parameters: 6

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	int	Enable	-	0/1 (0: Disable / 1: Enable)	1
DryLevel*	1	float	Dry signal (original sound) ratio	%	0 ~ 100	80
WetLevel*	2	float	Wet signal (processed reverb sound) ratio	%	0 ~ 100	20
RoomSize	3	float	Reverb intensity coefficient	-	0 ~ 1	0.35
Damp	4	float	Damping coefficient	-	0 ~ 1	0.2
Width	5	float	Stereo width	%	0 ~ 100	100

Demonstration of actual effect: Input stereo music, with the following case configuration:

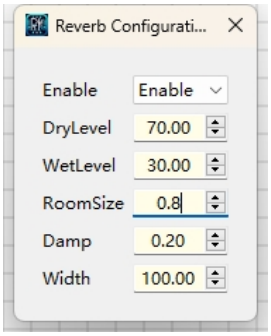


Figure 11.5.1-2 Simple Reverb parameters

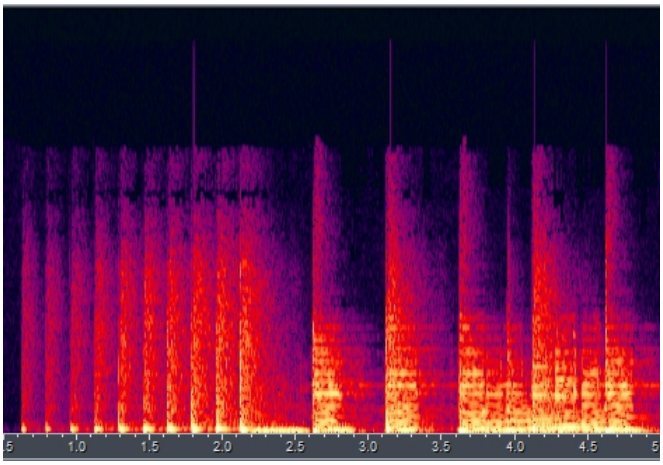


Figure 11.5.1-3 Input music spectrum

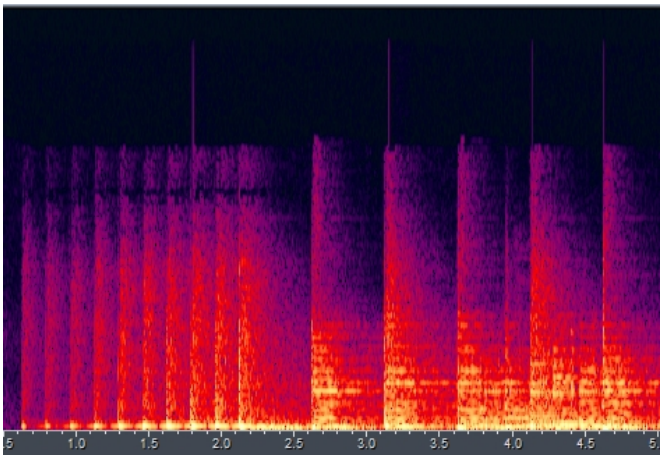


Figure 11.5.1-4 Output music spectrum

The figure below shows the difference between Damp = 0.2 and Damp = 0.5, with all other parameters kept the same (DryLevel = 0, WetLevel = 100, RoomSize = 0.8, Width = 100).

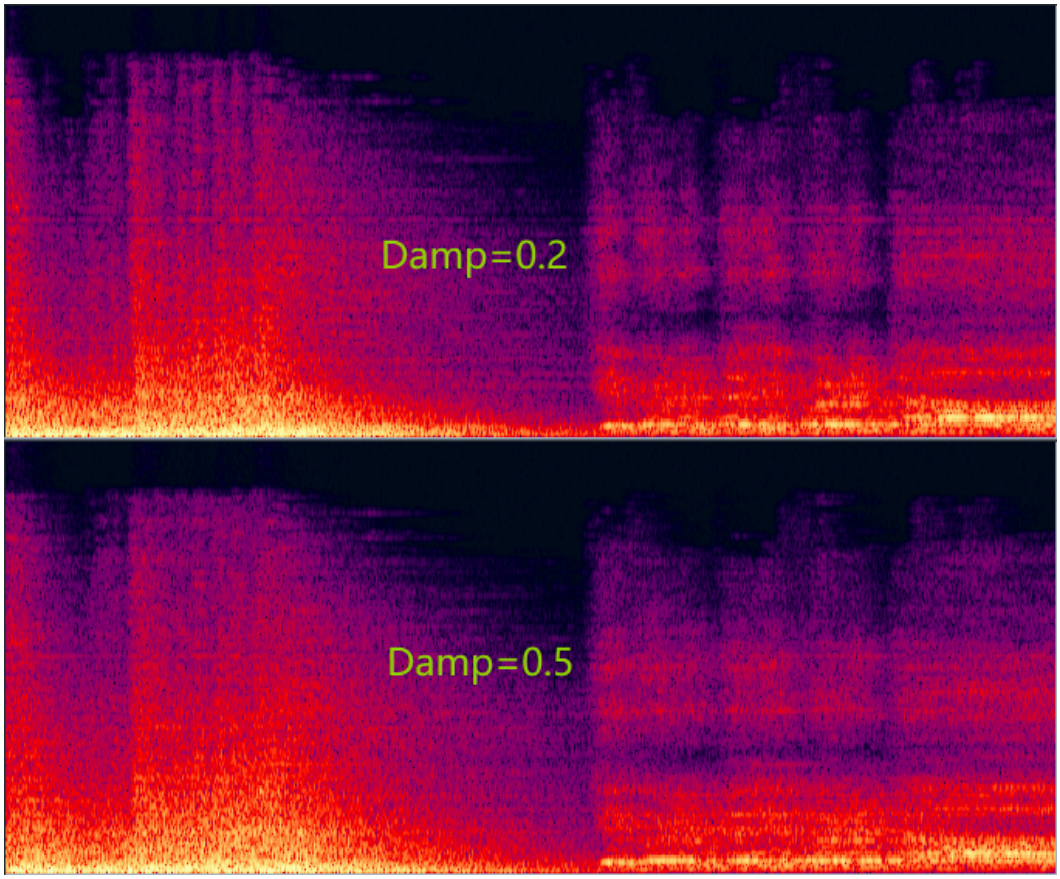


Figure 11.5.1-5 Effect of damp on the reverberation

11.5.2 Dattorr Reverb

Function Description: A reverb module designed based on the reference Effect Design Part 1: Reverberator and Other Filters. For the detailed structure of this module and the meaning of each parameter, please refer to the above reference.

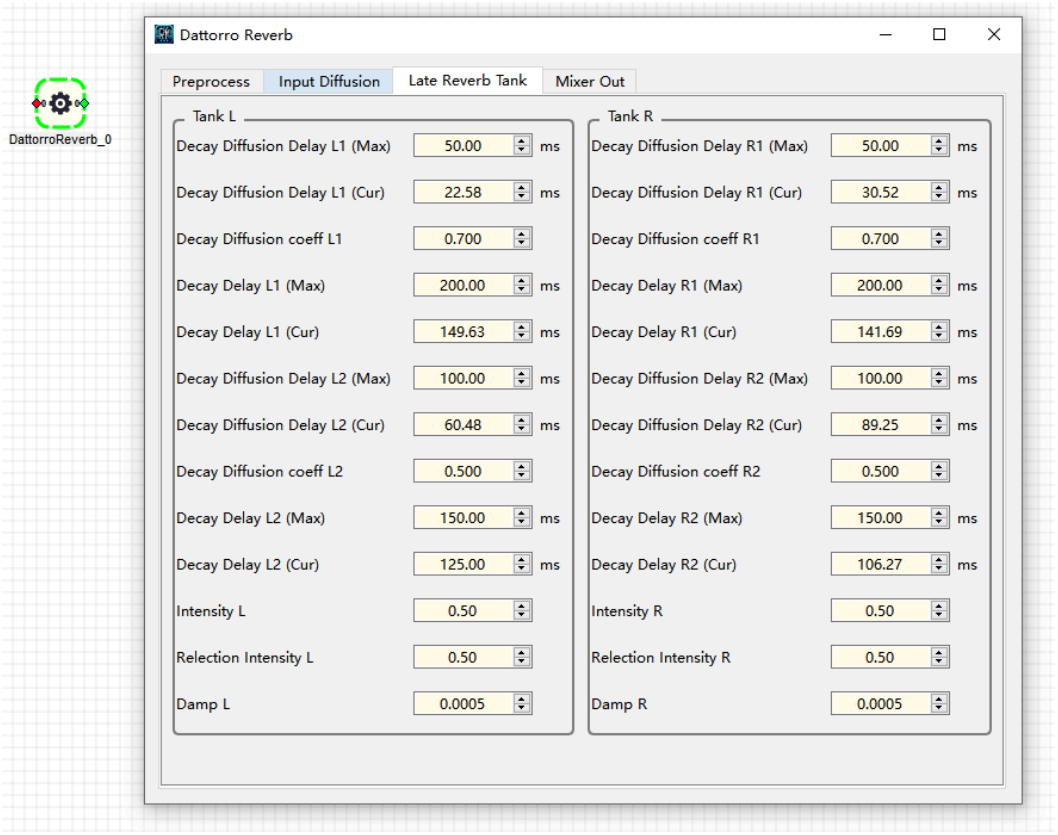


Figure 11.5.2-1 Dattorr Reverb parameters

Note 1: The default parameters in the original reference are calculated based on a sampling rate of 29761 Hz. In this tool, they have been converted to 48000 Hz.

Note 2: To adjust the reverb intensity in a simple way, try modifying Intensity L/R and Reflection Intensity L/R in the Late Reverb Tank section.

11.5.3 Advance Reverb

Function Description: Advanced reverb.
Compared with Simple Reverb, this module provides more parameter controls, allowing finer adjustment of reverb diffusion, decay characteristics, and stereo width.

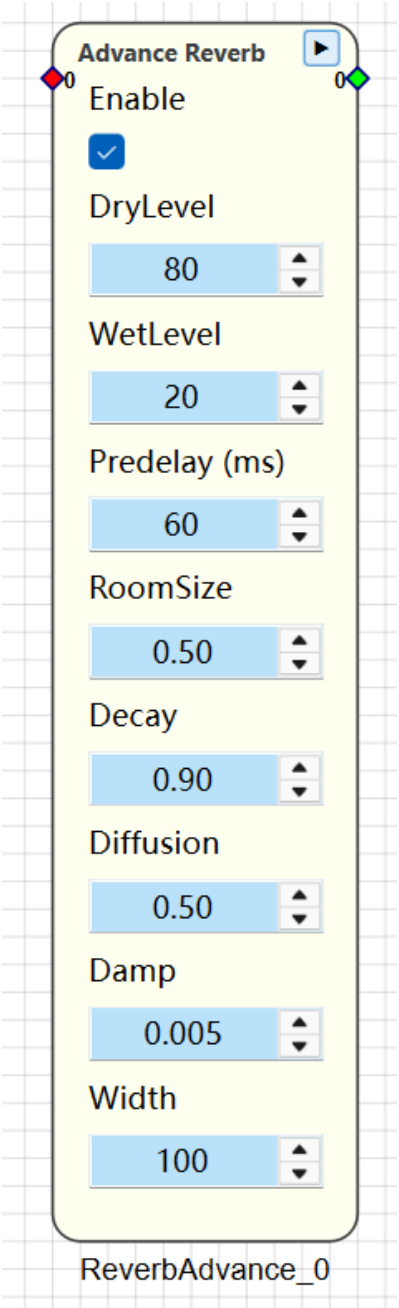


Figure 11.5.3-1 Advance Reverb parameters

Number of Parameters：9

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	float	Module enable switch	-	0/1	1
DryLevel	1	float	Dry signal (direct sound) level	-	0 ~ 100	80
WetLevel	2	float	Wet signal (reverberant sound) level	-	0 ~ 100	20
Predelay (ms)	3	float	Pre-delay: time difference between the arrival of direct	ms	0 ~ 200	60

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
			sound and the onset of reflections.			
RoomSize	4	float	Room size factor. Affects internal delay-line lengths; larger values create a larger perceived space.	-	0 ~ 1.0	0.50
Decay	5	float	Decay-time factor. Determines the length of the reverb tail.	-	0 ~ 1.0	0.90
Diffusion	6	float	Diffusion. Higher values produce denser reflections (e.g., smooth walls); lower values create a more granular echo texture.	-	0 ~ 1.0	0.50
Damp	7	float	High-frequency damping. Simulates HF absorption by air/walls; larger values result in faster HF decay and a “darker” sound.	-	0 ~ 1.0	0.005
Width	8	float	Stereo width	%	0 ~ 100	100

11.5.4 Early Reflection

Function Description: Early reflection simulator.
This module is specifically designed to simulate room reflection models in order to enhance the sense of space.

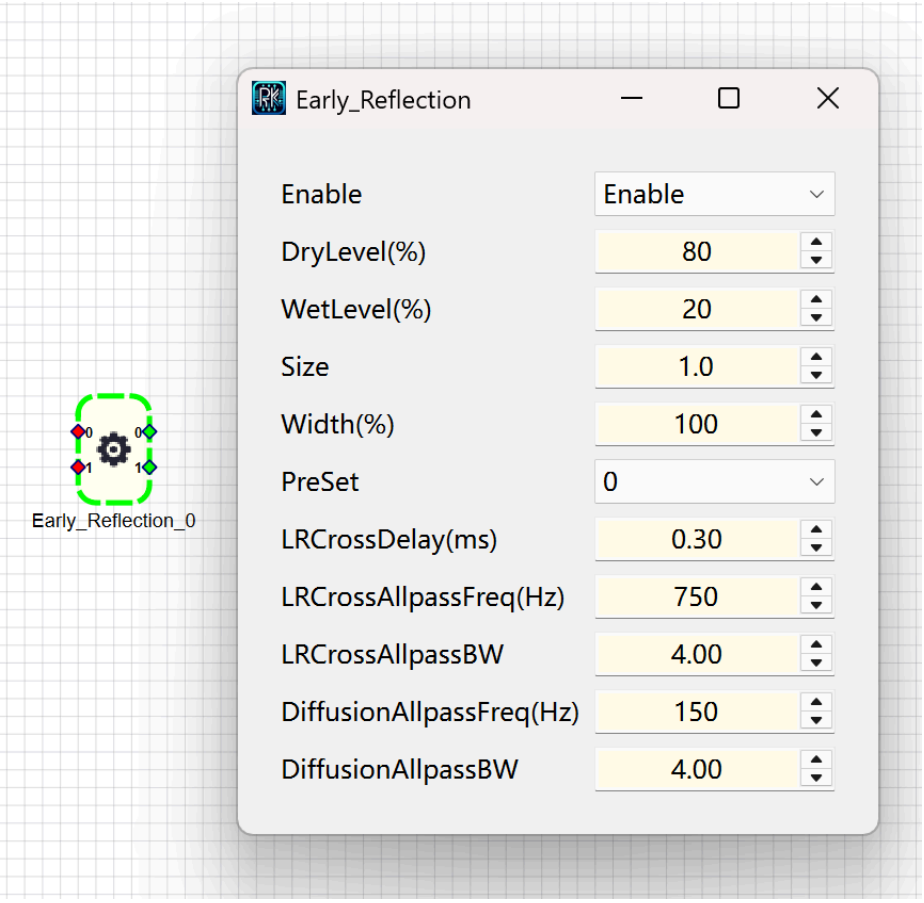


Figure 11.5.4-1 Early Reflection parameters

Number of Parameters：11

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	float	Module enable switch	-	0/1	Enable
DryLevel(%)	1	float	Dry-signal ratio	%	0 ~ 100	80
WetLevel(%)	2	float	Wet-signal ratio	%	0 ~ 100	20
Size	3	float	Reflection-space size factor	-	0 ~ 2.0	1.0
Width(%)	4	float	Stereo width	%	0 ~ 100	100
PreSet	5	float	Preset mode (select different reflection models)	-	0 ~ 32	0

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
LRCrossDelay(ms)	6	float	Left/right cross-feedback delay	ms	0 ~ 10	0.30
LRCrossAllpassFreq(Hz)	7	float	Cross-feedback all-pass filter frequency	Hz	20 ~ 20k	750
LRCrossAllpassBW	8	float	Cross-feedback all-pass filter bandwidth	-	0.1 ~ 10	4.00
DiffusionAllpassFreq(Hz)	9	float	Diffusion all-pass filter frequency	Hz	20 ~ 20k	150
DiffusionAllpassBW	10	float	Diffusion all-pass filter bandwidth	-	0.1 ~ 10	4.00

11.5.5 Room Reverb

Function Description: Room reverb.
This is a frequency-dependent reverb algorithm that allows users to independently adjust the reverb time for low frequencies and for mid/high frequencies. By setting the crossover frequency, the module can simulate acoustic characteristics of rooms with different materials.

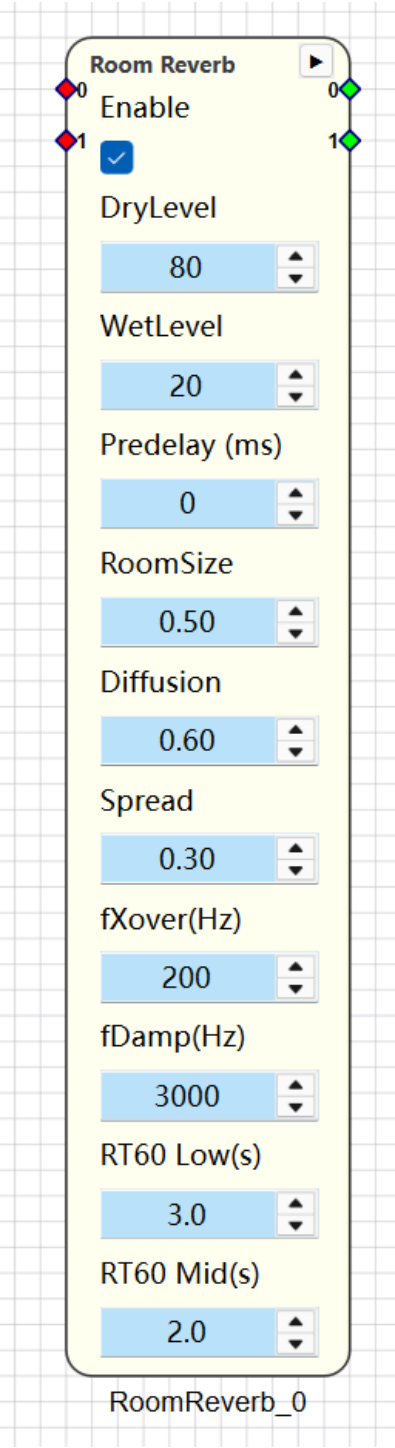


Figure 11.5.5-1 Room Reverb parameters

Number of Parameters：11

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	float	Enable switch	-	0/1	Check
DryLevel	1	float	Dry-signal level	-	0 ~ 100	80
WetLevel	2	float	Wet-signal level	-	0 ~ 100	20

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Predelay (ms)	3	float	Pre-delay	ms	0 ~ 200	0
RoomSize	4	float	Room size factor	-	0 ~ 1.0	0.50
Diffusion	5	float	Diffusion	-	0 ~ 1.0	0.60
Spread	6	float	Spatial spread (stereo enhancement)	-	0 ~ 1.0	0.30
fXover(Hz)	7	float	LF/MF crossover frequency. Defines the operating range of RT60 Low and RT60 Mid	Hz	20 ~ 1000	200
fDamp(Hz)	8	float	HF damping cutoff frequency. Signals above this frequency decay faster	Hz	1000 ~ 20k	3000
RT60 Low(s)	9	float	Low-frequency reverb time (time to decay by 60 dB)	s	0.1 ~ 10.0	3.0
RT60 Mid(s)	10	float	Mid-frequency reverb time (time to decay by 60 dB)	s	0.1 ~ 10.0	2.0

11.6 Chorus

Function Description: Chorus effect.

This module duplicates the original signal, applies a short and continuously varying delay to the copy, and then mixes it back with the dry signal. This processing simulates the thick and wide sound produced when multiple singers perform the same melody, where slight differences in pitch and timing create a natural ensemble effect.

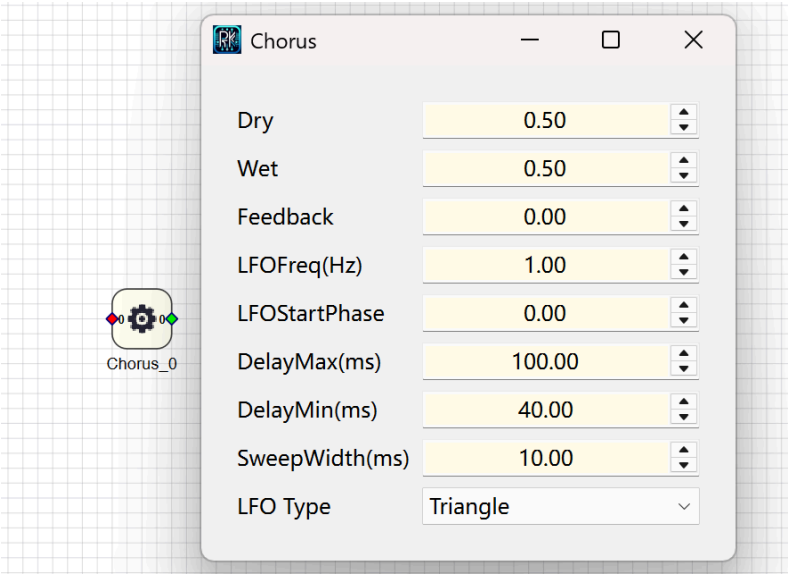


Figure 11.6-1 Chorus parameters

Number of Parameters：9

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Dry	0	float	Dry signal (direct sound) ratio	-	0 ~ 1.0	0.50
Wet	1	float	Wet signal (effect sound) ratio	-	0 ~ 1.0	0.50
Feedback	2	float	Feedback coefficient. Feeds the output back into the input to add a metallic character (flanger-like behavior). For a pure chorus, this is typically set to 0.	-	0 ~ 0.99	0.00
LFOFreq(Hz)	3	float	LFO frequency. Determines how fast the delay is modulated (chorus modulation rate).	Hz	0.1 ~ 5.0	1.00
LFOStartPhase	4	float	Initial phase of the LFO	-	0 ~ 1.0	0.00
DelayMax(ms)	5	float	Maximum delay buffer length (memory allocation limit)	ms	> DelayMin+Sweep	100.00
DelayMin(ms)	6	float	Minimum delay time (base delay)	ms	10 ~ 50	40.00
SweepWidth(ms)	7	float	Sweep width (modulation depth). Delay time varies between `DelayMin` and `DelayMin + SweepWidth`	ms	0 ~ 20	10.00
LFO Type	8	float	LFO waveform type (Triangle/Sine, etc.)	-	0~1	1

11.7 Transient Shaper

Function Description: Transient shaper.
This module reshapes the dynamic envelope of the audio signal, particularly focusing on the attack portion.

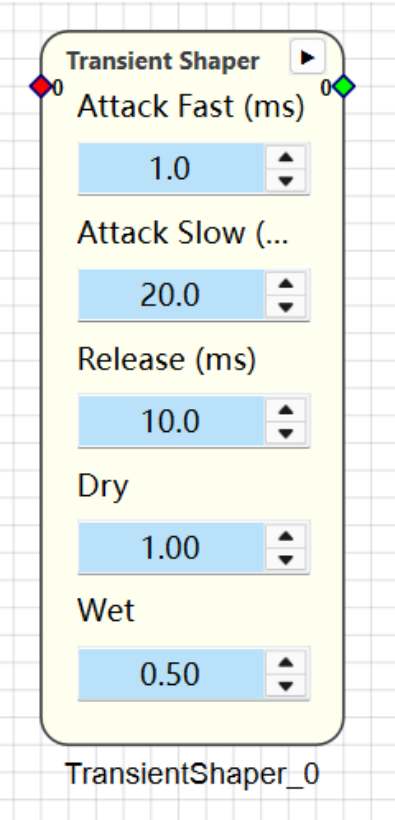


Figure 11.7-1 Transient Shaper parameters

Number of Parameters：5

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Attack Fast (ms)	0	float	Attack time for the fast envelope, used to capture instantaneous level peaks	ms	0.1 ~ 10	1.0
Attack Slow (ms)	1	float	Attack time for the slow envelope, used to estimate average level	ms	10 ~ 100	20.0
Release (ms)	2	float	Release time for envelope detection	ms	10 ~ 500	10.0
Dry	3	float	Dry signal gain (original signal)	-	0 ~ 2.0	1.00
Wet	4	float	Wet signal gain (extracted transient component). Increasing this value enhances punch/impact.	-	-2.0 ~ 2.0	0.50

11.8 Vibrato

Function Description: Vibrato effect.

This module produces periodic pitch modulation by periodically varying the delay time. Unlike Chorus, Vibrato typically outputs only the wet signal, and the delay time is very short. It emphasizes pitch fluctuation rather than spatial thickening.

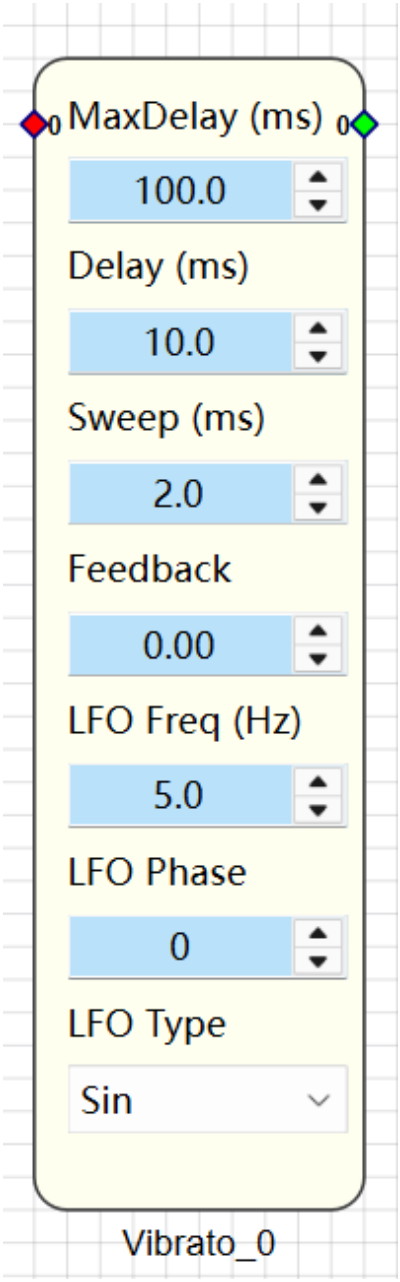


Figure 11.8-1 Vibrato parameters

Number of Parameters：7

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
MaxDelay	0	float	Maximum delay buffer	ms	> Delay+Sweep	100.0

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
(ms)			allocation			
Delay (ms)	1	float	Base delay time	ms	1 ~ 20	10.0
Sweep (ms)	2	float	Modulation depth. Larger values produce stronger pitch deviation.	ms	0.1 ~ 5.0	2.0
Feedback	3	float	Feedback coefficient	-	0 ~ 0.99	0.00
LFO Freq (Hz)	4	float	Vibrato rate	Hz	1.0 ~ 10.0	5.0
LFO Phase	5	float	Initial LFO phase	-	0 ~ 1.0	0
LFO Type	6	float	LFO waveform type (Sine/Triangle, etc.)	-		Sin

11.9 Robot Voice

Function Description: Robot voice effect (voice changer).
This module generates a robot-like voice effect using WOLA (Weighted Overlap-Add).

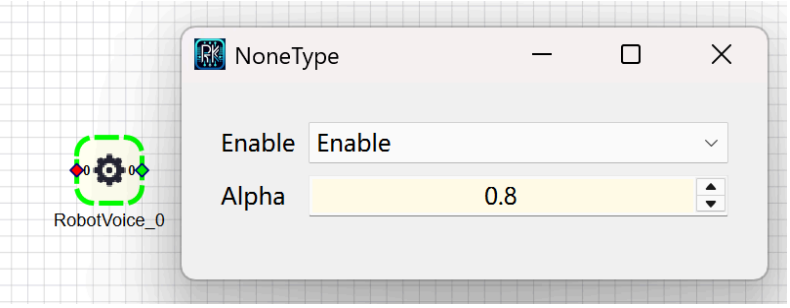


Figure 11.9-1 Robot Voice parameters

Number of Parameters：2

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
Enable	0	float	模Module enable/disable switch	-	0/1	1
Alpha	1	float	Effect coefficient. Typically controls the pitch-shift intensity, grain size, or modulation frequency. A larger value may produce a stronger	-	0 ~ 1.0	0.8

Parameter Name	Parameter Index	Type	Function	Unit	Recommended Value	Default Value
			“mechanical” or more glitch-like character.			

12 Cores

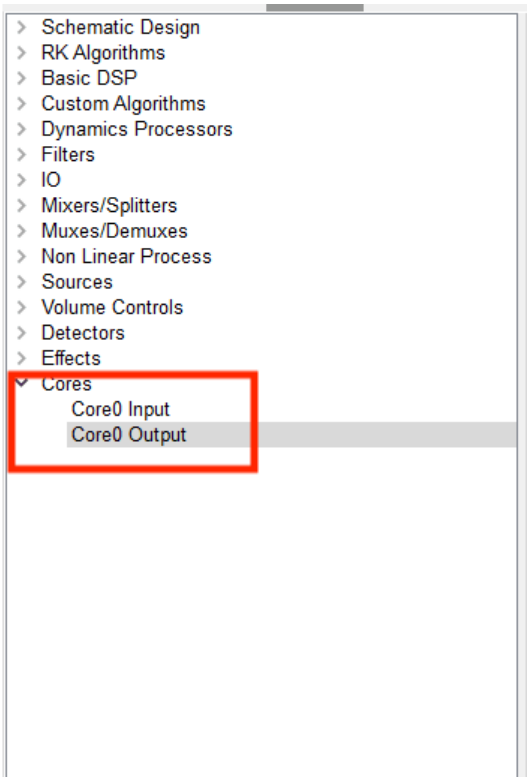


Figure 12-1 Location of Cores in the tool

This module is used for data interaction between different processing cores of the DSP chip. Each core can have one or more inputs and outputs.

For example, if we want to use the output of Core 0 as the input of Core 1.

We drag out the Core0_output control on the Core 0 editing page.

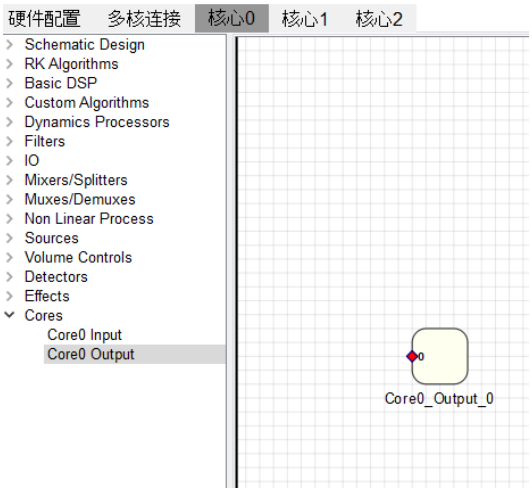


Figure 12-2 Core 0 editing interface

Click on Core 1 in the upper-left corner to enter the diagram editing interface of Core 1, and drag out the Core1_input control on the Core 1 editing page.

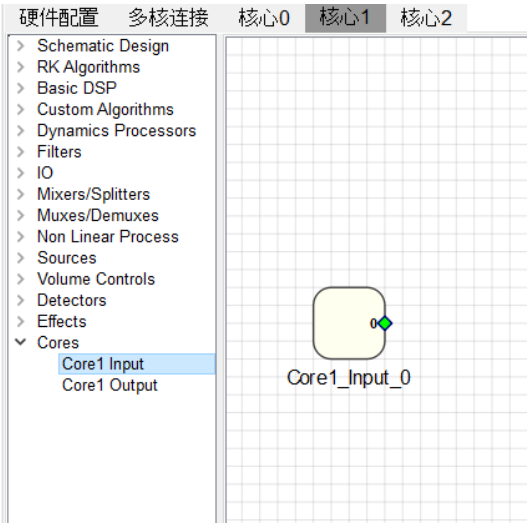


Figure 12-3 Core 1 editing interface

Click on the multicore connection in the upper-left corner to connect the inputs and outputs of the two controls.

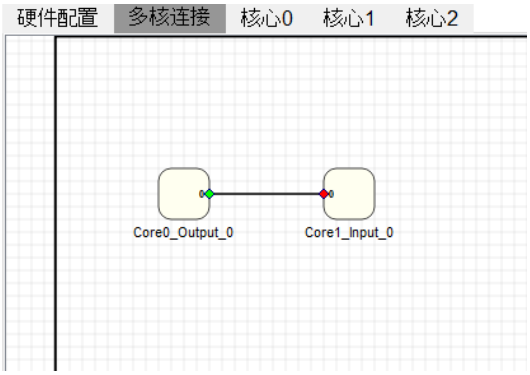


Figure 12-4 Multicore connection editing interface

In this way, we have connected the output of Core 0 to the input of Core 1.

12.1 Core_ Input

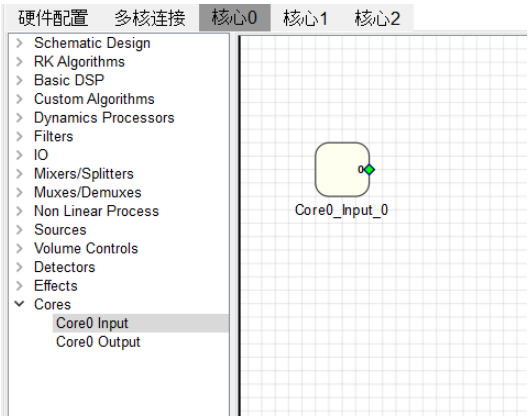


Figure 12.1-1 Core_Input Control

In the core editing interface, the input control has an output port that can be connected to the input of other controls within the same core.

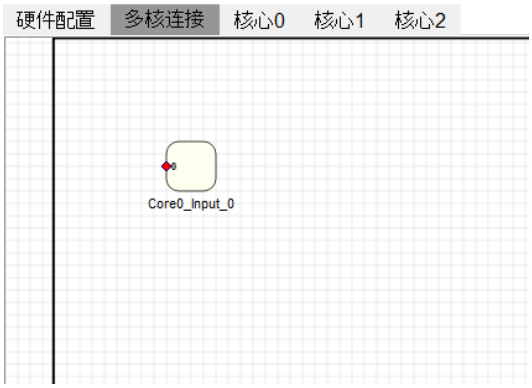


Figure 12.1-2 Core_Input Control

In the multicore connection editing interface, the input control has an input port that can be connected to the output of other cores.

12.2 Core_ Output

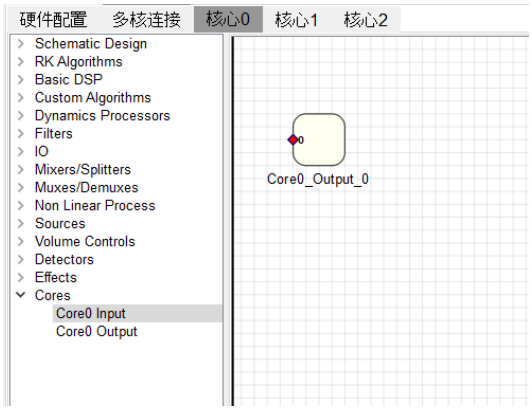


Figure 12.2-1 Core_Output Control

In the core editing interface, the output control has an input port that can be connected to the output of other controls within the same core.

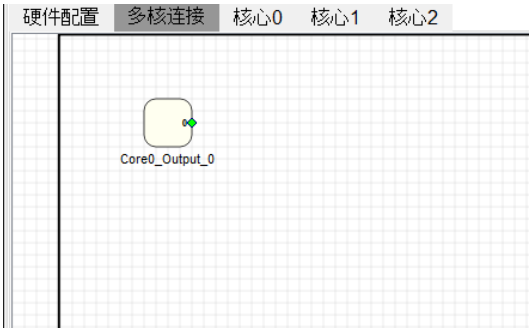


Figure 12.2-2 Core_Output Control

In the multicore connection editing interface, the output control has an output port that can be connected to the input of other cores.

13 Custom Algorithms

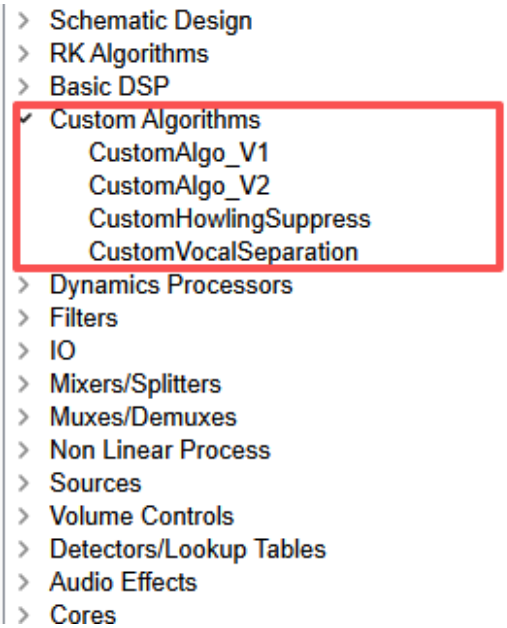


Figure 13-1 Location of Custom Algorithms in the tool

This section mainly includes custom algorithm modules generated using third-party algorithm templates. CustomAlgo_V1 and CustomAlgo_V2 are two example algorithms. For details on customization and integration, please refer to `Rockchip_RKStudio_CustomAlgo_Guide_EN.pdf` .

13.1 CustomHowlingSupporess

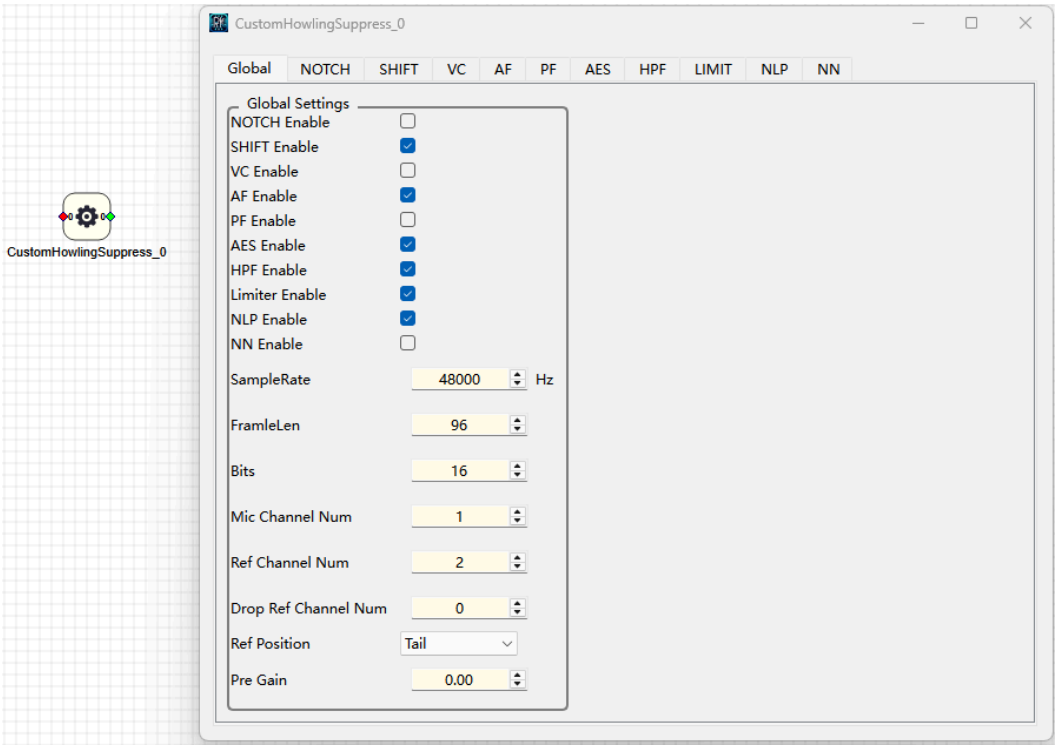


Figure 13.1-1 Howling suppression module algorithm

This module implements howling (feedback) suppression, suitable for karaoke, voice calls, and similar scenarios. For algorithm details, refer to `Rockchip_Developer_Guide_Howling_Suppresion_Tuning_EN.pdf`.

Note: To use the howling suppression algorithm, the algorithm library must be integrated separately. For specific usage instructions, please contact the relevant engineer(s).

13.2 CustomVocalSeparation

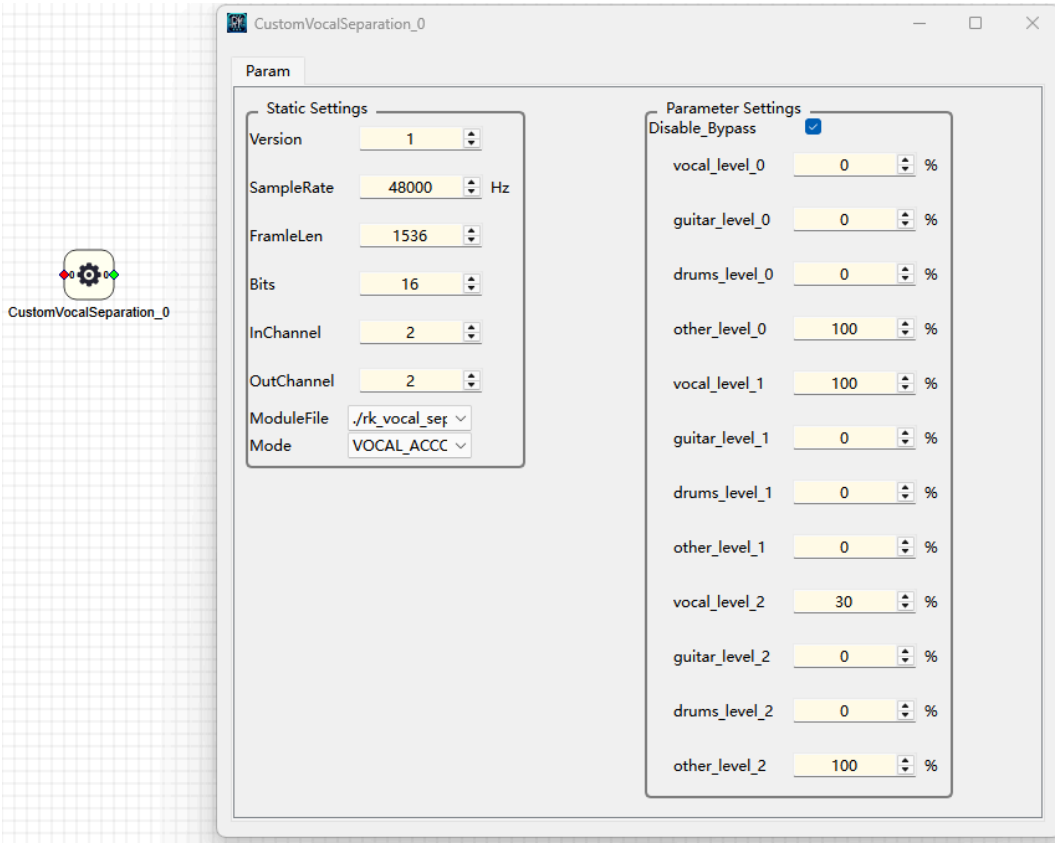


Figure 13.2-1 Vocal separation module algorithm

This module uses AI-based processing to perform vocal separation, suitable for karaoke, virtual surround, and similar scenarios.

Note: To use the vocal separation algorithm, the algorithm library must be integrated separately. For specific usage instructions, please contact the relevant engineer(s).

13.3 CustomAlgo_V1

Function Description: Custom algorithm template V1 (Basic).

This is an example module demonstrating how to create a custom algorithm UI with basic parameters (dropdown menu, float input, integer input). Developers can use this template to package simple DSP algorithms.

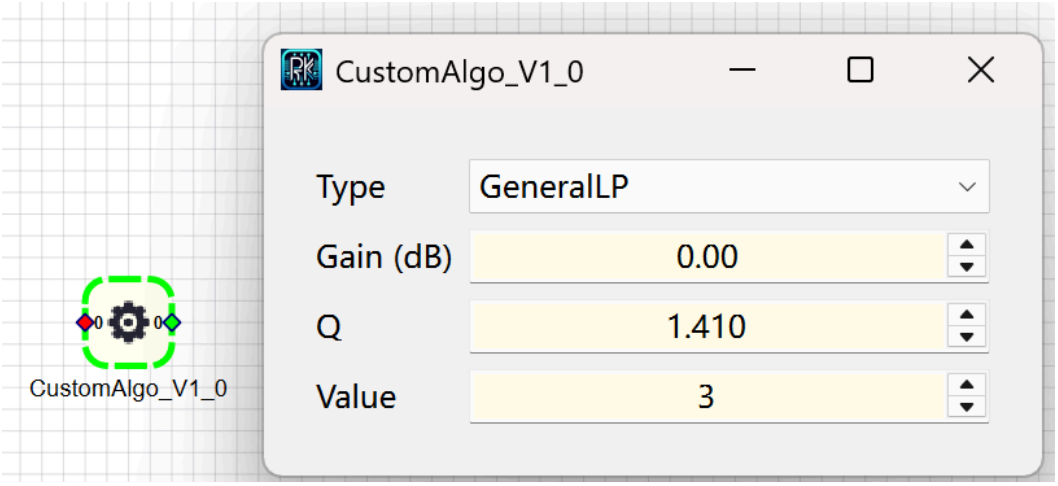


Figure 13.3-1 CustomAlgo_V1 control and parameter settings

13.4 CustomAlgo_V2

Function Description: Custom algorithm template V2 (Advanced).

This is a more complex example module that demonstrates how to build an algorithm UI with multiple tabs, checkboxes, file-loading buttons, and more advanced configuration logic. It is typically used for packaging complex algorithms such as AEC and LMS adaptive filtering.

Figure 13.4-1 CustomAlgo_V2 control

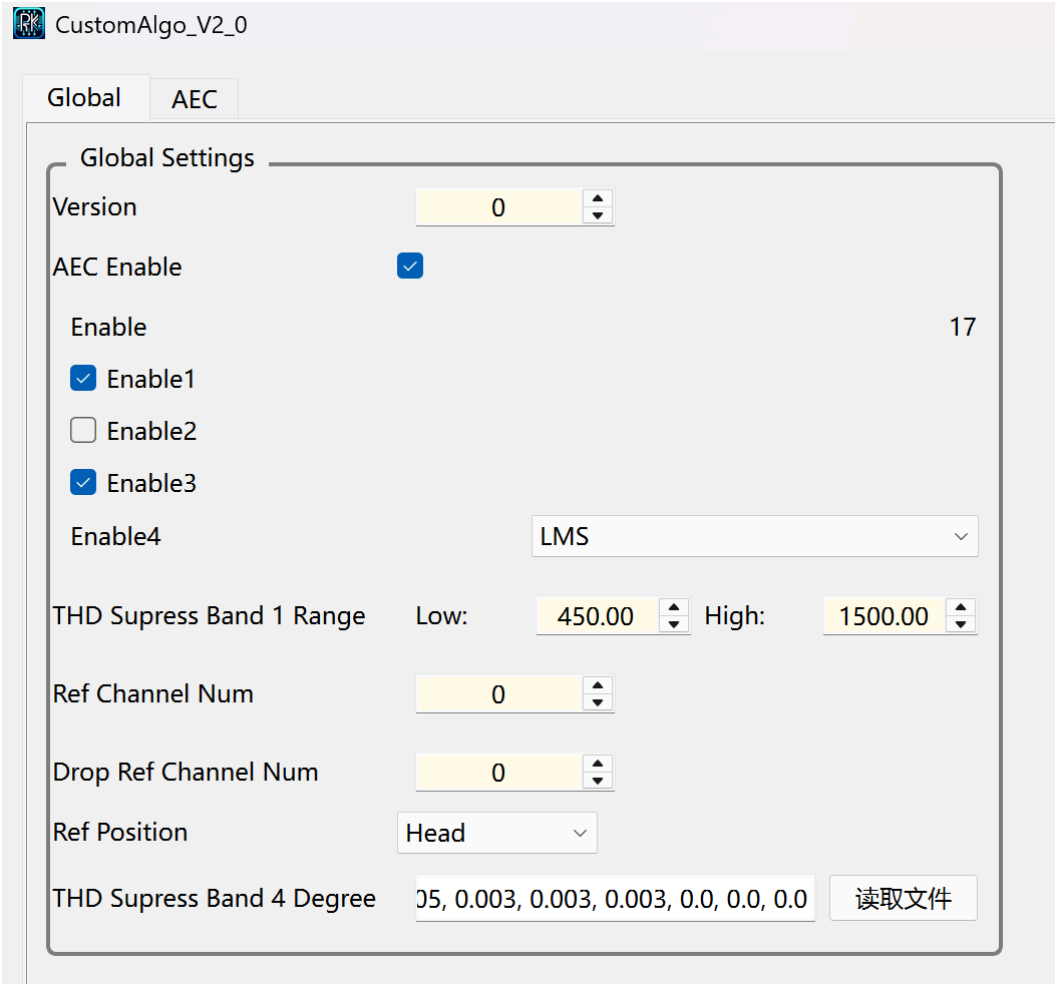


Figure 13.4-2 CustomAlgo_V2 parameter settings

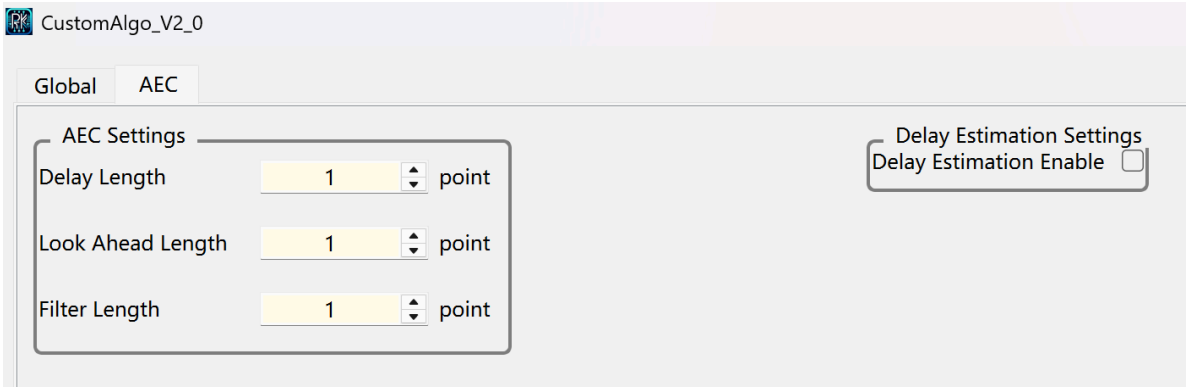


Figure 13.4-3 CustomAlgo_V2 parameter settings

14 Counters

14.1 Counter

Function Description: Basic counter (timer).
This module is used to measure a duration of time.

- When Enable is checked (set to 1), the module starts counting from 0 and outputs 0.

- When the count reaches the duration specified by MaxCount (in milliseconds), the counter stops (the output typically remains at its maximum value or triggers a logic state, depending on the DSP logic definition), and the output becomes 1.
- When Enable is unchecked (set to 0), the counter is reset.

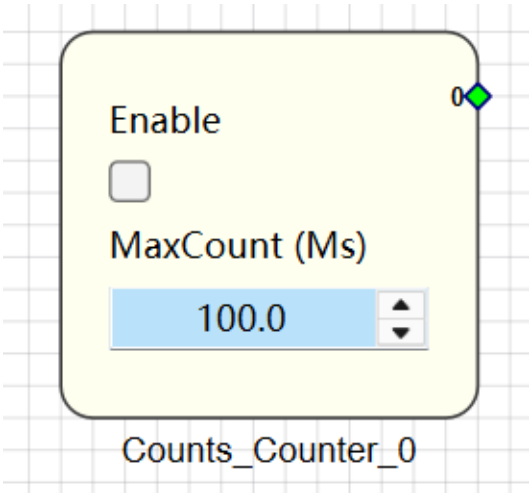


Figure 14.1-1 Counter control

14.2 Counter_Ext

Function Description: Externally controlled timer.
This module is similar to the standard Counter, except that the Enable button is removed from the UI. Instead, the counting state is controlled by the signal connected to Input 0: counting is disabled when the input is 0 and enabled when the input is 1.

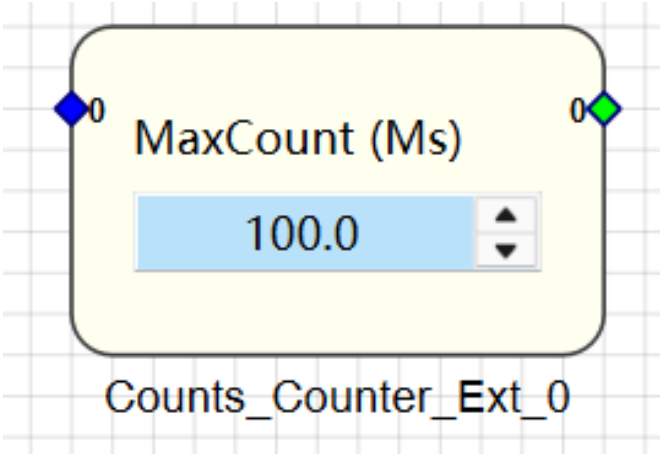


Figure 14.2-1 Counter_Ext control

15 Non Linear Process

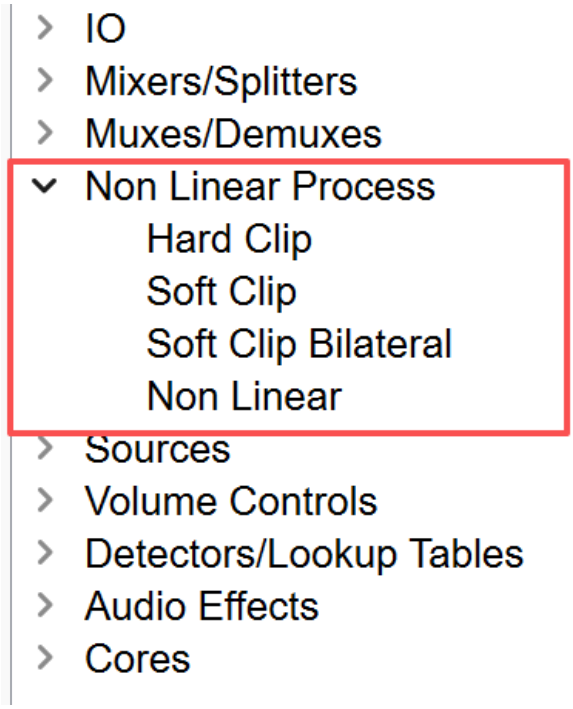


Figure 15-1 Location of Non Linear Process in the tool

15.1 Hard Clip

Function Description: Hard clipping forcibly truncates the audio signal using a fixed threshold. When the absolute value of the input exceeds the preset limit, the exceeding portion is discarded and the output is held at the boundary level.

- Transfer function

$$y = \begin{cases} L & \text{if } x > L \\ -L & \text{if } x < -L \\ x & \text{otherwise} \end{cases}$$

  (where x is the input and L is the clipping threshold)
- Characteristics
  - Waveform: Completely flattened peaks; sharp 90° corners at turning points.
  - Harmonics: Introduces strong high-order odd harmonics, resulting in a harsh and aggressive distortion.
- Applications
  - Acts as the “last line of defense” at the end of a digital signal chain to prevent severe artifacts caused by sample overflow.
  - A fundamental building block for industrial-style or extreme distortion effects.

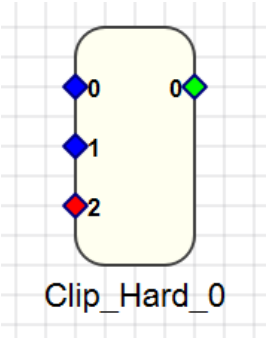


Figure 15.1-1 Hard Clip control

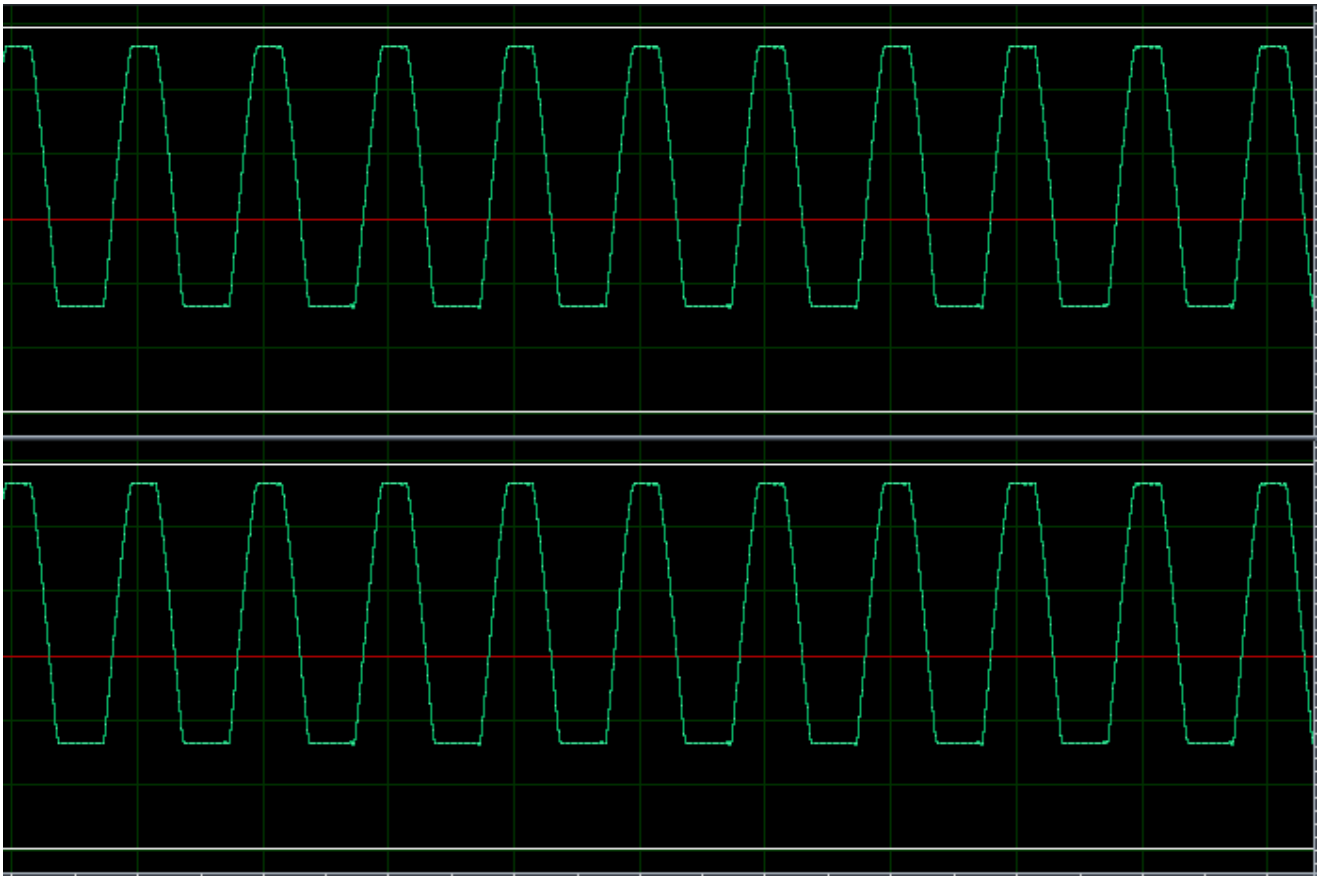


Figure 15.1-2 Hard Clip waveform

15.2 Soft Clip

Function Description: Soft clipping simulates the physical saturation behavior of analog circuits. Before reaching the clipping limit, it performs gradual gain compression using a nonlinear curve.

- Key parameters
  - Limit (dB)： Absolute upper bound of the output level.
  - Turn On (dB)： Nonlinear processing start point. Above this level, curve distortion begins.
- Characteristics
  - Waveform: Smooth arc-like transition at peaks.
  - Harmonics: Primarily produces low-order harmonics, resulting in a warmer and softer sound.
- Applications

- Increases perceived loudness without introducing obvious hard clipping artifacts.
- Adds “analog-like” coloration to digital audio.

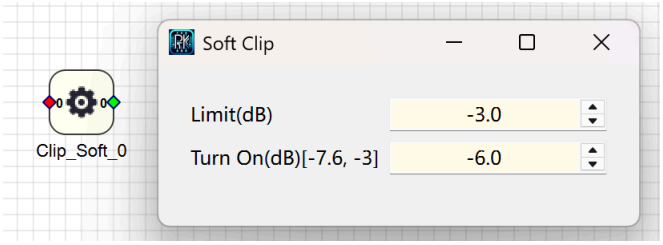


Figure 15.2-1 Soft Clip control

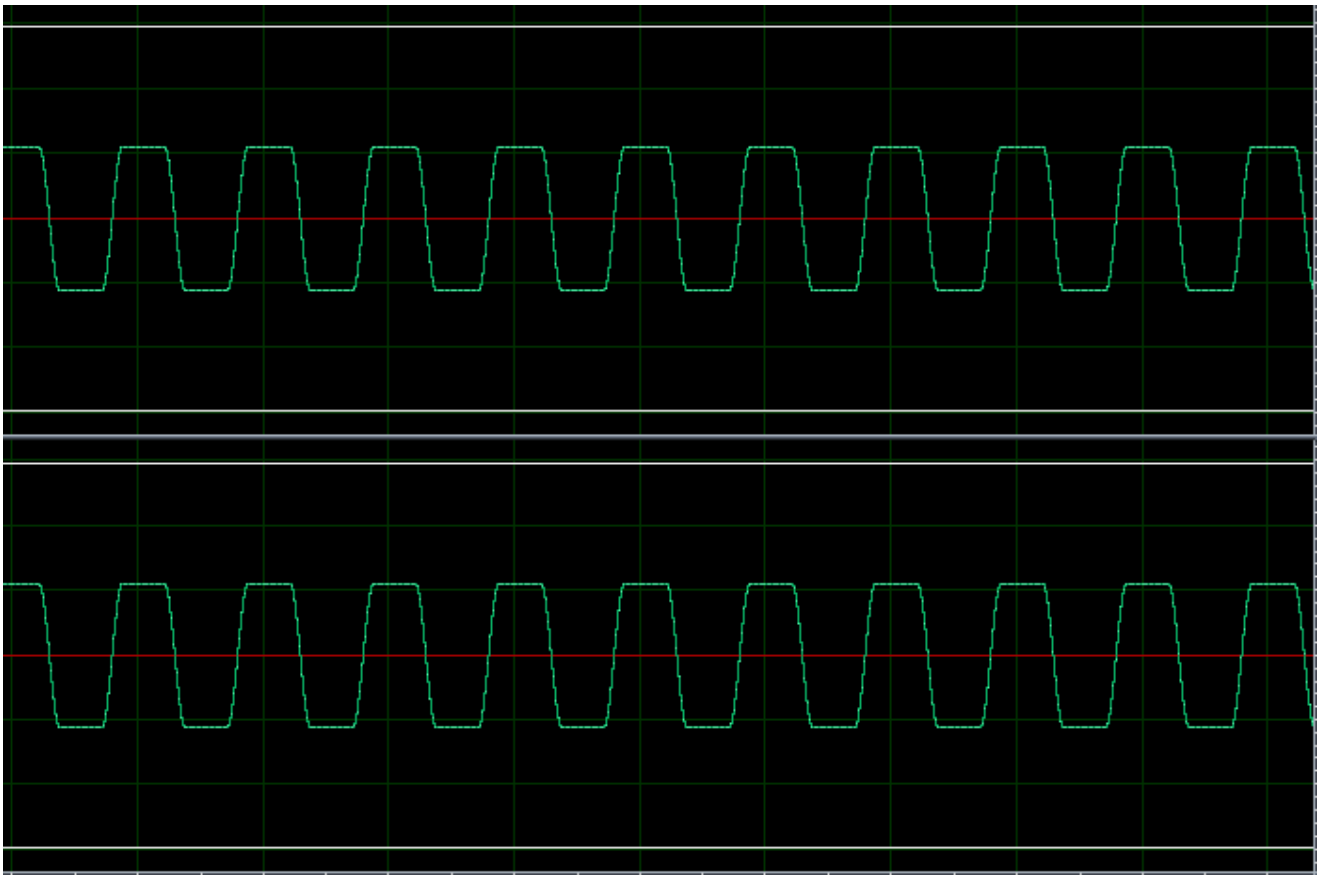


Figure 15.2-2 Soft Clip waveform

15.3 Soft Clip Bilateral

Function Description: The bilateral version is an advanced form of soft clipping, allowing independent algorithm configuration for the positive and negative half-cycles of the audio waveform.

- Key parameters
  - Limit Up / Turn On Up: Clipping and transition settings for the upper boundary.
  - Limit Down / Turn On Down: Clipping and transition settings for the lower boundary.
- Characteristics
  - Core advantage: Asymmetric distortion can be created by applying different settings, which is essential for emulating specific hardware tonal characteristics.

    - Harmonic shaping: By controlling asymmetry between positive and negative half-cycles, the distribution of even and odd harmonics can be finely shaped.
- Applications
    - High-end speaker emulation, hi-fi dynamics processors, and characteristic timbre algorithm development.

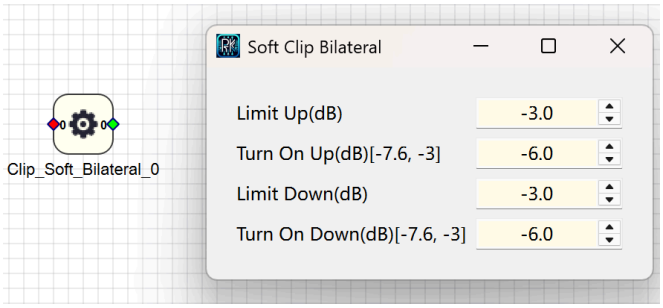


Figure 15.3-1 Soft Clip Bilateral control

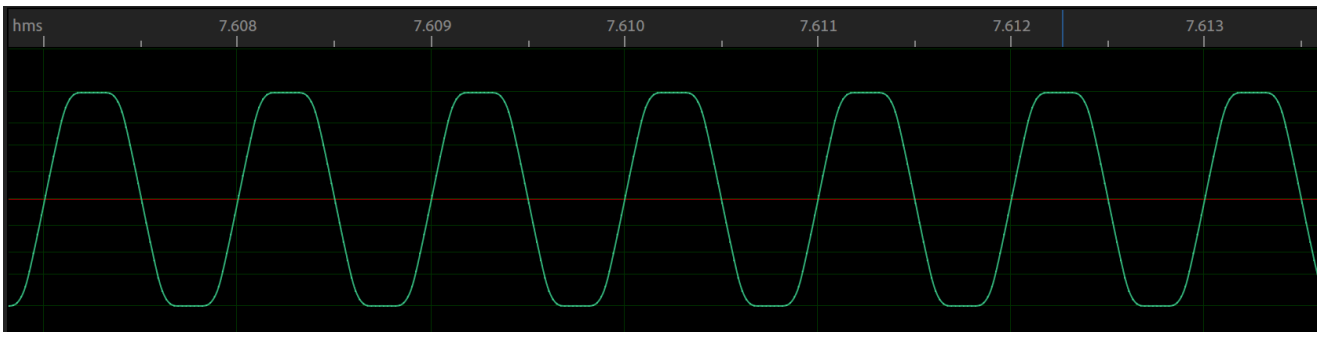


Figure 15.3-2 Soft Clip Bilateral waveform

15.4 Non Linear

Function Description: The Non Linear module performs full-range signal reshaping using a predefined mathematical transfer function. Unlike clipping algorithms that mainly affect peaks, this module changes the overall input–output transfer relationship.

- Common modes
    - EXP (Exponential): Exponential mapping, used to increase transient contrast or dynamic impact.
- Applications
    - Wave-shaping, dynamic envelope reconstruction, and creative sound design.

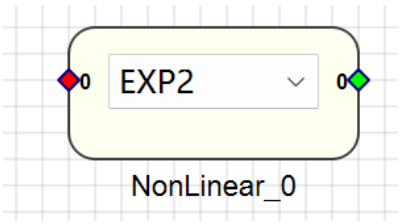


Figure 15.4-1 Non Linear control

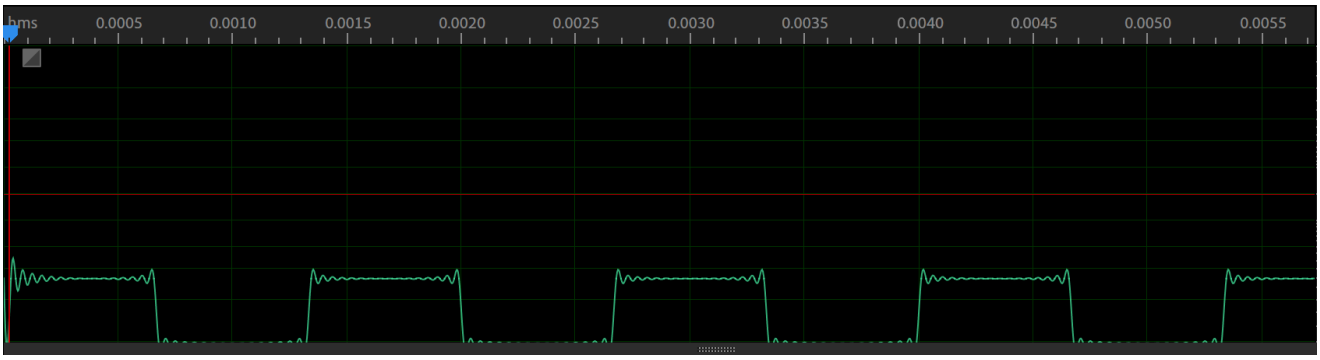


Figure 15.4-2 Output waveform when using a sine wave as input

Appendix 1 Schematic Design

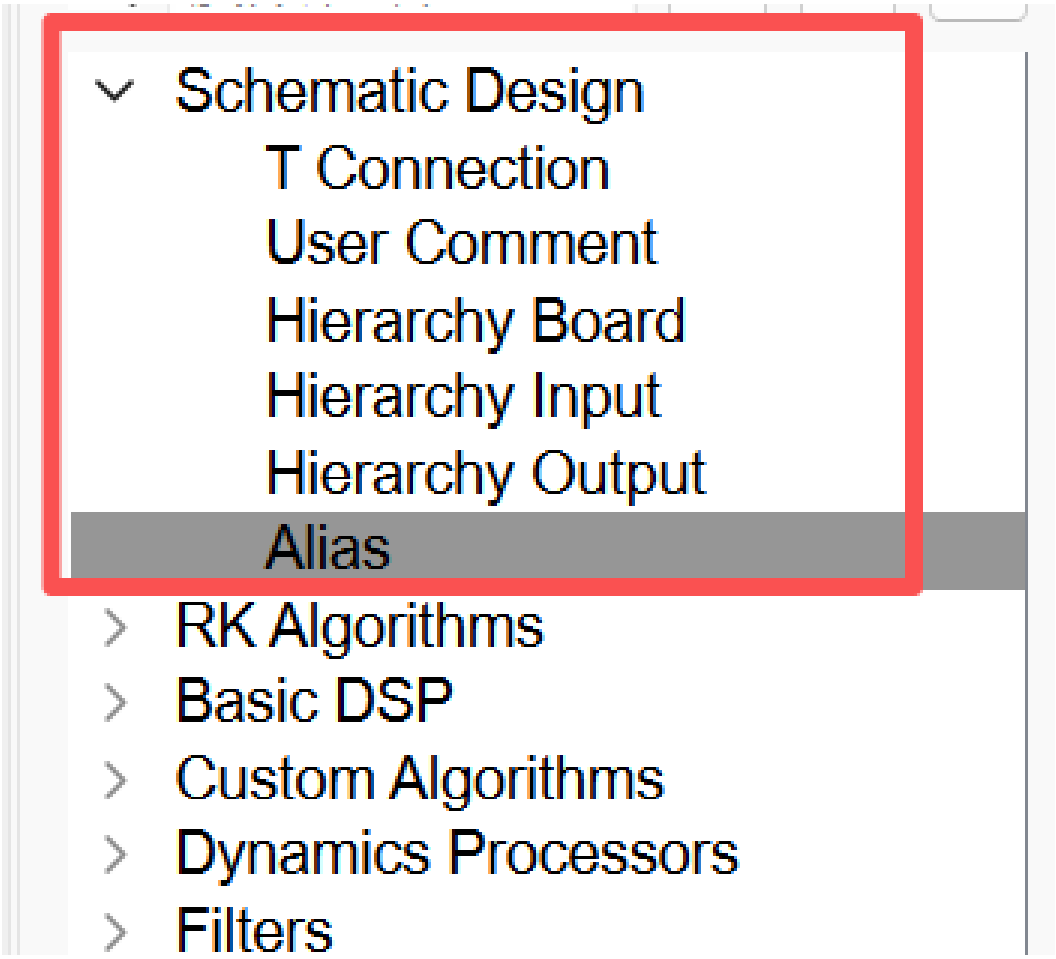


Figure Appendix 1-1 Location of General Function Components in the tool

Appendix 1.1 T Connection

Function Description: Single-channel input, with outputs that can be connected to multiple components. It essentially duplicates one input to multiple outputs, with no limit on the number of output channels.

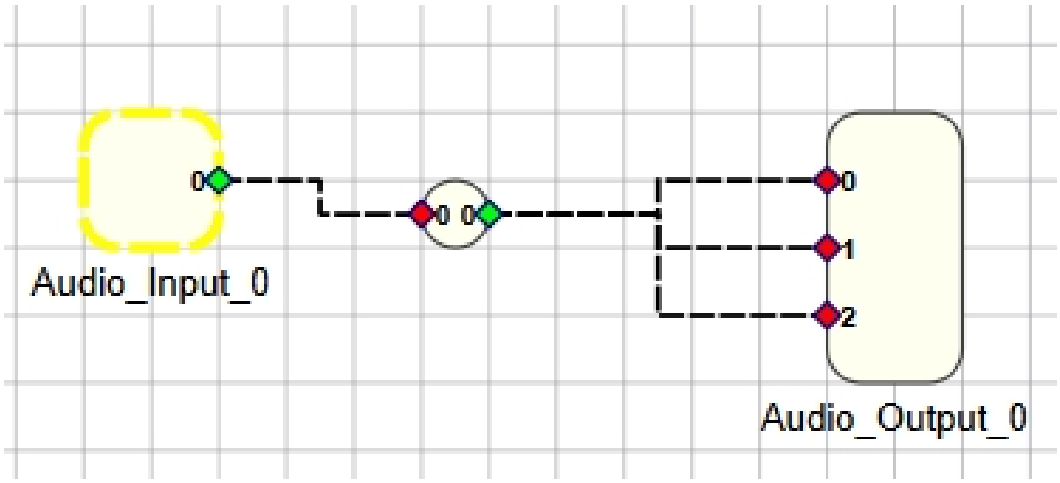


Figure Appendix 1.1-1 T Connection component connection

Appendix 1.2 User Comment

Function Description: User label function, which allows annotations on components to facilitate understanding.

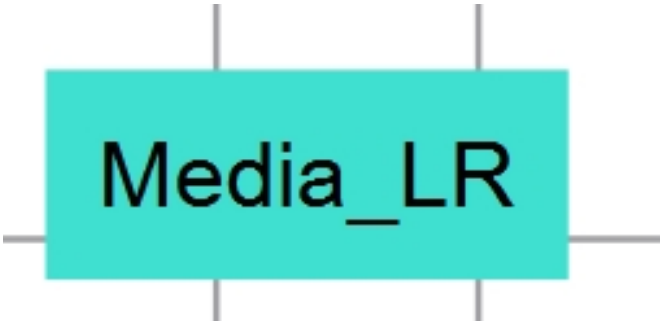


Figure Appendix 1.2-1 User Comment component

Appendix 1.3 Hierarchy Board

Function Description: Sub-diagram function, which allows multiple components to be deployed within a sub-diagram for easier management and portability. The sub-diagram supports encryption/decryption, import/export, and other functions.

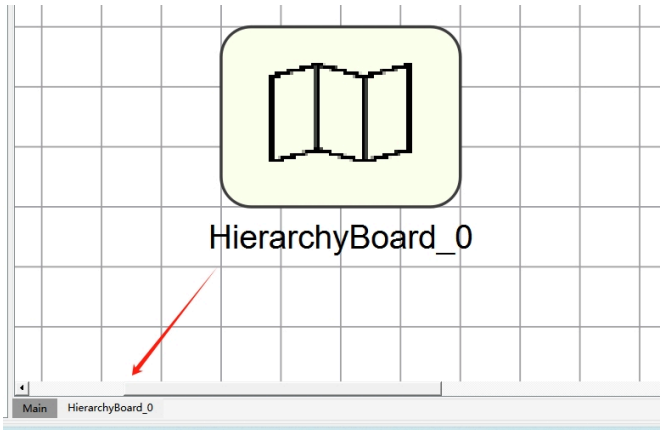


Figure Appendix 1.3-1 Hierarchy Board sub-diagram function

Appendix 1.4 Hierarchy Input

Function Description: Sub-diagram input, which corresponds to the input of the main diagram. When the number of input channels is increased, the input of the main diagram will also increase.

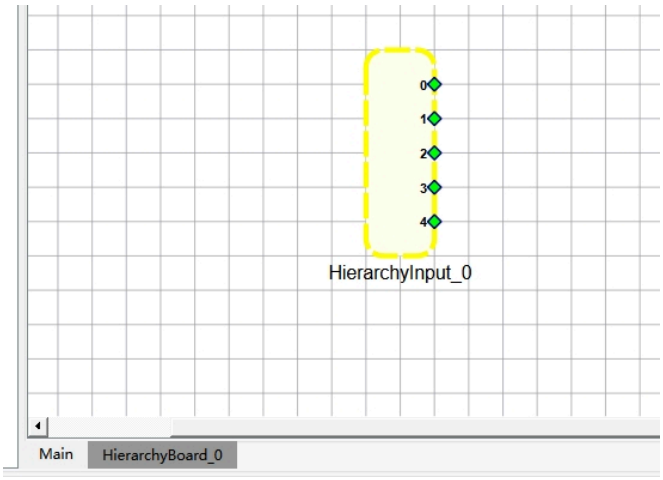


Figure Appendix 1.4-1 Hierarchy Input

Appendix 1.5 Hierarchy Output

Function Description: Sub-diagram output, which corresponds to the output of the main diagram. When the number of output channels is increased, the output of the main diagram will also increase.

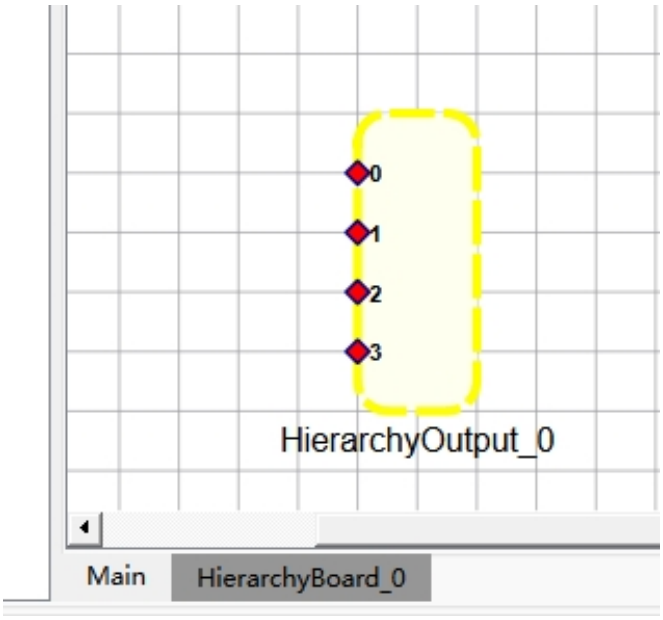


Figure Appendix 1.5-1 Hierarchy Output

Appendix 1.6 Alias

Function Description: Used for “fly-wire” connections. When routing connections, solid wires are hidden, making it easier to arrange plug-ins/modules and to view the signal connections. The connection mode is one-to-many: there is only one input, while the number of outputs can be freely increased via the right-click menu.

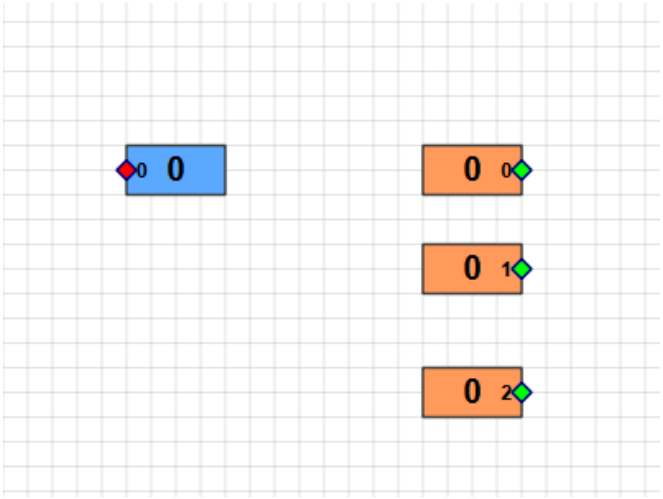


Figure Appendix 1.6-1 Alias

Appendix 2 Conversion Formulas

Appendix 2.1 Parameter Conversion Description

The parameters set in the module settings of the software (referred to as tool parameters), such as low-frequency/high-frequency cutoff frequencies and gain, are processed by the program and converted into corresponding parameters that are passed into the algorithms (referred to as Downlaod parameters). For specific parameter content, open the xxx_param.json file in the RK_Tool schematic project directory.

The following example uses the RK Algorithms -> Loudness -> Loudness(L&H)EXT module:

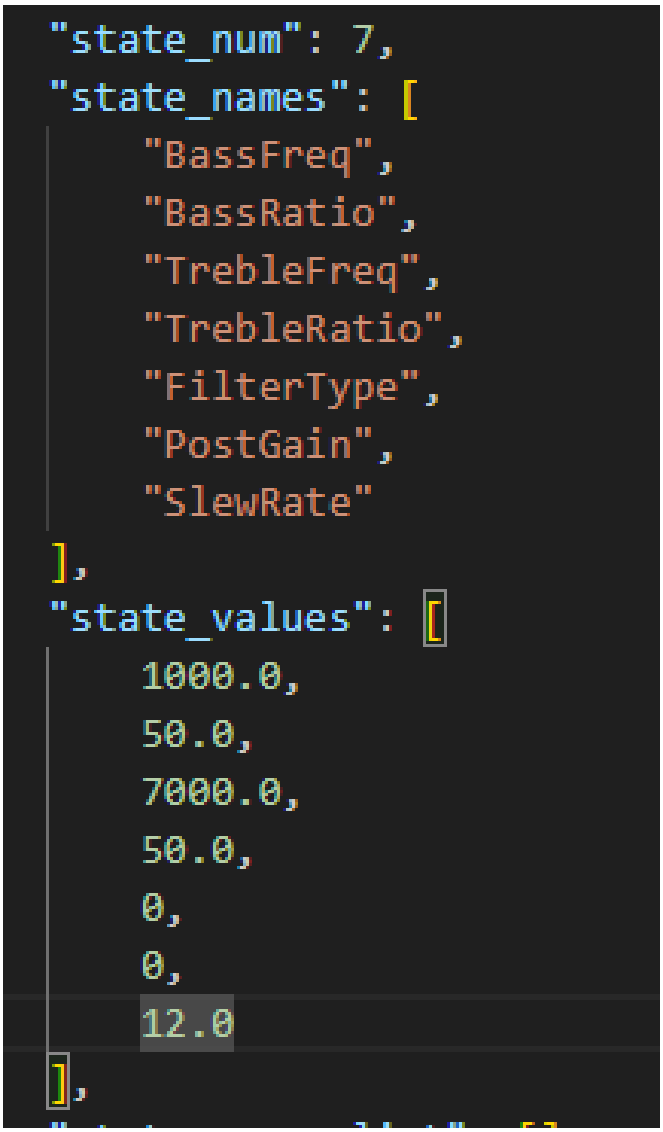


Figure Appendix 2.1-1 Tool Parameters, parameters set in the tool interface

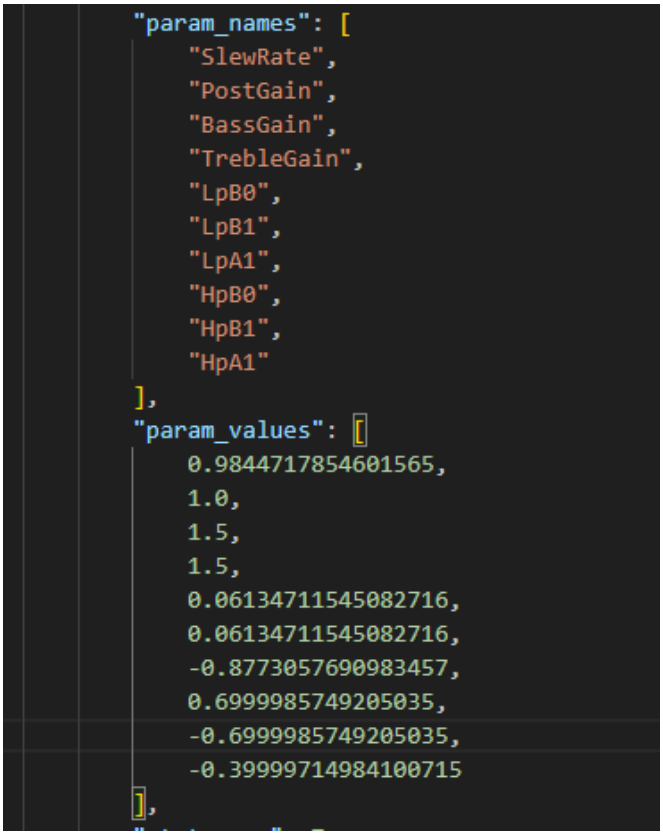


Figure Appendix 2.1-2 Download Parameters, parameters converted and passed into the algorithm

Appendix 2.2 IIR1st Calculation Formulas and Description

- **Conversion Logic**:
  - When Enable=0, b0=1, b1=0, a1=0.
  - When Polarity=1, b0 and b1 are negated based on the calculations below.

The following lists the calculation methods for the Download Parameters of five types of filters:

Input Description:

```
#      boost: The Boost (dB) value set in the tool for the filter gain
#      fc:    The cutoff frequency Fc (Hz) value set in the tool
#      gain:  The overall Gain (dB) value set in the tool
#      Fs:    Sampling frequency, assumed to be 48kHz here, used for calculating normalized frequency
```

```python
# LP1st     First-order Low-pass Filter
def calculate_lp1st_coeffs(fc: float, gain=0, Fs=48000):
    w0 = 2*np.pi*fc/Fs
    a1 = -pow(np.e, -w0)
    b0 = 0.5 * (1 + a1)
    b1 = b0

    gain_linear = 10**(gain/20)
    return [b0*gain_linear, b1*gain_linear], [1.0, a1]

# HP1st     First-order High-pass Filter
def calculate_hp1st_coeffs(fc: float, gain=0, Fs=48000):
    w0 = 2*np.pi*fc/Fs
    a1 = -pow(np.e, -w0)
    b0 = 0.5 * (1 - a1)
    b1 = -b0

    gain_linear = 10**(gain/20)
    return [b0*gain_linear, b1*gain_linear], [1.0, a1]

# AP1st     First-order All-pass Filter
def calculate_ap1st_coeffs(fc: float, gain=0, Fs=48000):
    w0 = 2*np.pi*fc/Fs
    a1 = -pow(np.e, -w0)
    b0 = a1
    b1 = 1

    gain_linear = 10**(gain/20)
    return [b0*gain_linear, b1*gain_linear], [1.0, a1]

# LS1st     First-order Low-shelf Filter
def calculate_ls1st_coeffs(fc: float, boost: float, gain=0, Fs=48000):
    w0 = 2*np.pi*fc/Fs
    A = pow(10.0, boost / 20.0)
    a0 = np.tan(w0/2.0) + np.sqrt(A)
    a1 = (np.tan(w0/2.0) - np.sqrt(A))
    b0 = (A*np.tan(w0/2.0) + np.sqrt(A))
    b1 = (A*np.tan(w0/2.0) - np.sqrt(A))

    a0_invert = 1 / a0
    gain_linear = 10**(gain/20)
    return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear], [1.0, a1*a0_invert]

# HS1st     First-order High-shelf Filter
def calculate_hs1st_coeffs(fc: float, boost: float, gain=0, Fs=48000):
    w0 = 2*np.pi*fc/Fs
    A = pow(10.0, boost / 20.0)
    a0 = np.sqrt(A)*np.tan(w0/2.0)+1
    a1 = (np.sqrt(A)*np.tan(w0/2.0)-1) / a0
```

```
    b0 = (np.sqrt(A)*np.tan(w0/2.0)+A) / a0
    b1 = (np.sqrt(A)*np.tan(w0/2.0)-A) / a0

    a0_invert = 1 / a0
    gain_linear = 10**(gain/20)
    return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear], [1.0, a1*a0_invert]
```

Appendix 2.3 IIR2nd Calculation Formulas and Description

- **Conversion Logic**:
  - When Enable=0, b0=1, b1=0, b2=0, a1=0, a2=0.
  - When Polarity=1, b0, b1, and b2 are negated based on the calculations below.

The following lists the calculation methods for the Download Parameters of 16 types of filters:

Input Description:

```
#      fc:    The cutoff frequency Fc (Hz) value set in the tool
#      boost: The Boost (dB) value set in the tool for the filter gain
#      Q:     The Q-factor value set in the tool for the filter
#      gain:  The overall Gain (dB) value set in the tool
#      Fs:    Sampling frequency, assumed to be 48kHz here, used for calculating normalized frequency
```

```python
# Parametric Parametric Filter
def calculate_param_coeffs(fc: float, boost: float, Q: float, gain=0, Fs=48000):
    w0 = 2 * np.pi * fc / Fs
    A = 10 ** (boost / 40.0)
    alpha = np.sin(w0) / (2.0 * Q)

    a0 = 1 + alpha / A
    a1 = -2 * np.cos(w0)
    a2 = 1 - alpha / A
    b0 = 1 + alpha * A
    b1 = -2 * np.cos(w0)
    b2 = 1 - alpha * A

    a0_invert = 1 / a0
    gain_linear = 10 ** (gain/20)

    return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_i

# LowShelf Low-shelf Filter
def calculate_lowshelf_coeffs(fc: float, boost: float, Q: float, gain=0, Fs=48000):
    w0 = 2 * np.pi * fc / Fs
    A = 10 ** (boost / 40.0)
    tmp = (A + 1/A)*(1/(Q + 1e-8) - 1) + 2
    tmp = max(tmp, 0)
    alpha = 0.5*np.sin(w0)*np.sqrt(tmp)

    a0 = (A + 1) + (A - 1)*np.cos(w0) + 2*np.sqrt(A)*alpha
    a1 = -2*((A - 1) + (A + 1)*np.cos(w0))
    a2 = (A + 1) + (A - 1)*np.cos(w0) - 2*np.sqrt(A)*alpha
    b0 = A*((A + 1) - (A - 1)*np.cos(w0) + 2*np.sqrt(A)*alpha)
    b1 = 2*A*((A - 1) - (A + 1)*np.cos(w0))
    b2 = A*((A + 1) - (A - 1)*np.cos(w0) - 2*np.sqrt(A)*alpha)

    a0_invert = 1 / a0
    gain_linear = 10 ** (gain/20)

    return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_i

# HighShelf High-shelf Filter
def calculate_highshelf_coeffs(fc: float, boost: float, Q: float, gain=0, Fs=48000):
    w0 = 2 * np.pi * fc / Fs
    A = 10 ** (boost / 40.0)
    tmp = (A + 1/A)*(1/(Q + 1e-8) - 1) + 2
    tmp = max(tmp, 0)
    alpha = 0.5*np.sin(w0)*np.sqrt(tmp)

    a0 = (A + 1) - (A - 1)*np.cos(w0) + 2*np.sqrt(A)*alpha
    a1 = 2*((A - 1) - (A + 1)*np.cos(w0))
    a2 = (A + 1) - (A - 1)*np.cos(w0) - 2*np.sqrt(A)*alpha
```

```
    b0 = A*((A + 1) + (A - 1)*np.cos(w0) + 2*np.sqrt(A)*alpha)
    b1 = -2*A*((A - 1) + (A + 1)*np.cos(w0))
    b2 = A*((A + 1) + (A - 1)*np.cos(w0) - 2*np.sqrt(A)*alpha)

    a0_invert = 1 / a0
    gain_linear = 10 ** (gain/20)

    return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_i

# GeneralLP Low-pass Filter
def calculate_genlp_coeffs(fc: float, Q: float, gain=0, Fs=48000):
    w0 = 2 * np.pi * fc / Fs
    alpha = np.sin(w0) / (2 * Q)

    a0 = 1 + alpha
    a1 = -2 * np.cos(w0)
    a2 = 1 - alpha
    b0 = 0.5 * (1 - np.cos(w0))
    b1 = 1 - np.cos(w0)
    b2 = 0.5 * (1 - np.cos(w0))

    a0_invert = 1 / a0
    gain_linear = 10 ** (gain/20)

    return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_i

# GeneralHP High-pass Filter
def calculate_genhp_coeffs(fc: float, Q: float, gain=0, Fs=48000):
    w0 = 2 * np.pi * fc / Fs
    alpha = np.sin(w0) / (2 * Q)

    a0 = 1 + alpha
    a1 = -2 * np.cos(w0)
    a2 = 1 - alpha
    b0 = 0.5 * (1 + np.cos(w0))
    b1 = -(1 + np.cos(w0))
    b2 = 0.5 * (1 + np.cos(w0))

    a0_invert = 1 / a0
    gain_linear = 10 ** (gain/20)

    return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_i

# GeneralBP Band-pass Filter
def calculate_genbp_coeffs(fc: float, Q: float, gain=0, Fs=48000):
    w0 = 2 * np.pi * fc / Fs
    BW = Q_to_BW(Q)
    alpha = np.sin(w0) * np.sinh(0.5*np.log(2)*BW*w0/np.sin(w0))
```

```
    a0 = 1 + alpha
    a1 = -2 * np.cos(w0)
    a2 = 1 - alpha
    b0 = alpha
    b1 = 0
    b2 = -alpha

    a0_invert = 1 / a0
    gain_linear = 10 ** (gain/20)

    return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_i

# GeneralBS Band-stop Filter
def calculate_genbs_coeffs(fc: float, Q: float, gain=0, Fs=48000):
    w0 = 2 * np.pi * fc / Fs
    BW = Q_to_BW(Q)
    alpha = np.sin(w0) * np.sinh(0.5*np.log(2)*BW*w0/(np.sin(w0)+1e-8))

    a0 = 1 + alpha
    a1 = -2 * np.cos(w0)
    a2 = 1 - alpha
    b0 = 1
    b1 = -2 * np.cos(w0)
    b2 = 1

    a0_invert = 1 / a0
    gain_linear = 10 ** (gain/20)

    return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_i

# AFP All-pass Filter
def calculate_apf_coeffs(fc: float, Q: float, gain=0, Fs=48000):
    w0 = 2 * np.pi * fc / Fs
    tmp = fc / Q
    K = np.pi*tmp/Fs
    K = K if K <= 0.95*np.pi/2 else 0.95*np.pi/2
    beta = 1 - 2/(np.tan(K) + 1)
    gamma = -np.cos(w0)

    a1 = gamma*(1 - beta)
    a2 = -beta
    b0 = -beta
    b1 = gamma*(1 - beta)
    b2 = 1

    gain_linear = 10 ** (gain/20)

    return [b0*gain_linear, b1*gain_linear, b2*gain_linear], [1.0, a1, a2]
```

```
# Peaking Peaking Filter
def calculate_peak_coeffs(fc: float, Q: float, gain=0, Fs=48000):
    w0 = 2 * np.pi * fc / Fs
    beta = 0.5 * (1 - np.tan(w0/(2*Q))) / (1 + np.tan(w0/(2*Q)))
    gamma = (0.5 + beta) * np.cos(w0)

    a1 = -2 * gamma
    a2 = 2 * beta
    b0 = 0.5 - beta
    b1 = 0
    b2 = -0.5 + beta

    gain_linear = 10 ** (gain/20)

    return [b0*gain_linear, b1*gain_linear, b2*gain_linear], [1.0, a1, a2]

# Notch Notch Filter
def calculate_notch_coeffs(fc: float, Q: float, gain=0, Fs=48000):
    w0 = 2 * np.pi * fc / Fs
    alpha = np.sin(w0) / (2 * Q)

    a0 = 1 + alpha
    a1 = -2 * np.cos(w0)
    a2 = 1 - alpha
    b0 = 1
    b1 = -2 * np.cos(w0)
    b2 = 1

    a0_invert = 1 / a0
    gain_linear = 10 ** (gain/20)

    return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_i

# ChebyshevLP Chebyshev Low-pass Filter
def calculate_cheby1lp_coeffs(fc: float, Q: float, gain=0, Fs=48000):
    gain_linear = 10 ** (gain / 20)
    b, a = cheby1(2, Q, fc, "lowpass", analog=False, fs=Fs)
    b = b * gain_linear
    b = b.tolist()
    a = a.tolist()
    return b, a

# ChebyshevHP Chebyshev High-pass Filter
def calculate_cheby1hp_coeffs(fc: float, Q: float, gain=0, Fs=48000):
    gain_linear = 10 ** (gain / 20)
    b, a = cheby1(2, Q, fc, "highpass", analog=False, fs=Fs)
    b = b * gain_linear
    b = b.tolist()
    a = a.tolist()
```

```
    return b, a

# BesselLP Bessel Low-pass Filter
def calculate_besselp_coeffs(fc: float, gain=0, Fs=48000):
    gain_linear = 10 ** (gain / 20)
    b, a = bessel(2, fc, "lowpass", analog=False, fs=Fs)
    b = b * gain_linear
    b = b.tolist()
    a = a.tolist()
    return b, a

# BesselHP Bessel High-pass Filter
def calculate_besselhp_coeffs(fc: float, gain=0, Fs=48000):
    gain_linear = 10 ** (gain / 20)
    b, a = bessel(2, fc, "highpass", analog=False, fs=Fs)
    b = b * gain_linear
    b = b.tolist()
    a = a.tolist()
    return b, a

# ButterLP Butterworth Low-pass Filter
def calculate_butterlp_coeffs(fc: float, gain=0, Fs=48000):
    gain_linear = 10 ** (gain / 20)
    b, a = butter(2, fc, "lowpass", analog=False, fs=Fs)
    b = b * gain_linear
    b = b.tolist()
    a = a.tolist()
    return b, a


# ButterHP Butterworth High-pass Filter
def calculate_butterhp_coeffs(fc: float, gain=0, Fs=48000):
    gain_linear = 10 ** (gain / 20)
    b, a = butter(2, fc, "highpass", analog=False, fs=Fs)
    b = b * gain_linear
    b = b.tolist()
    a = a.tolist()
    return b, a
```

Appendix 2.4 IIR_Nnd Calculation Formula and Explanation

- **Conversion Logic**:
  - When Enable=0, a0=1, b0=1, and all other parameters are 0.
  - When Polarity=1, the filter numerator parameters b are negated based on the following formula calculations.

    Input Description:

```
#   enable:       Corresponds to the tool parameter enable
#   polarity:     Corresponds to the tool parameter polarity
#   filter_type:  Corresponds to the tool parameter filter_type
#   order:        Corresponds to the tool parameter order
#   fc:           Corresponds to the tool parameter fc (cutoff frequency)
#   rpldb:        Corresponds to the tool parameter Rpl (dB) (maximum gain within the passband)
#   Fs:           Sampling frequency, default is 48k, used for calculating normalized frequency
```

```python
def calculate_filter_high_coeffs(enable: int,
                                 polarity: int,
                                 filter_type: int,
                                 order: int,
                                 fc: float,
                                 rpldb: float,
                                 Fs=48000):
    if not enable:
        num_sos = (order+1)//2
        return num_sos, [1, 0, 0]*num_sos, [1.0, 0, 0]*num_sos

    if filter_type == 0:    # "Butter LP (Butterworth Low-Pass Filter)"
        sos = butter(order, fc, btype='low', analog=False, output='sos', fs=Fs)
        b, a = sos_to_ba(sos)
    elif filter_type == 1:  # "Butter HP (Butterworth High-Pass Filter)"
        sos = butter(order, fc, btype='high', analog=False, output='sos', fs=Fs)
        b, a = sos_to_ba(sos)
    elif filter_type == 2:  # "Bessel LP (Bessel Low-Pass Filter)"
        sos = bessel(order, fc, btype='low', analog=False, output='sos', fs=Fs)
        b, a = sos_to_ba(sos)
    elif filter_type == 3:  # "Bessel HP (Bessel High-Pass Filter)"
        sos = bessel(order, fc, btype='high', analog=False, output='sos', fs=Fs)
        b, a = sos_to_ba(sos)
    elif filter_type == 4:  # "Cheby1 LP (Chebyshev Type I Low-Pass Filter)"
        sos = cheby1(order, rpldb, fc, btype='low', analog=False, output='sos', fs=Fs)
        b, a = sos_to_ba(sos)
    elif filter_type == 5:  # "Cheby1 HP (Chebyshev Type I High-Pass Filter)"
        sos = cheby1(order, rpldb, fc, btype='high', analog=False, output='sos', fs=Fs)
        b, a = sos_to_ba(sos)
    elif filter_type == 6:  # "Cheby2 LP (Chebyshev Type II Low-Pass Filter)"
        sos = cheby2(order, rpldb, fc, btype='low', analog=False, output='sos', fs=Fs)
        b, a = sos_to_ba(sos)
    elif filter_type == 7:  # "Cheby2 HP (Chebyshev Type II High-Pass Filter)"
        sos = cheby2(order, rpldb, fc, btype='high', analog=False, output='sos', fs=Fs)
        b, a = sos_to_ba(sos)
    else:
        raise ValueError("Unsupported filter type")

    if polarity == 1:
        b[0:3] = -b[0:3]
    b = b.tolist()
    a = a.tolist()
    return (order+1)//2, b, a

def sos_to_ba(sos):
    """
    将sos格式转换为ba格式
    :param sos:
    :return:
```

```
    """
    b_list = []
    a_list = []
    num_sos = len(sos)

    gain_all = sos[0][0]
    gain = gain_all**(1/num_sos)
    sos[0][:3] = sos[0][:3]/gain_all
    for section in sos:
        b_section = section[:3]*gain  # Extract the b coefficients of the current second-order section
        a_section = np.concatenate(([1], section[4:]))  # Extract the a coefficients of the current second-or

        # Add the b and a coefficients of the current section to the lists
        b_list.extend(b_section)
        a_list.extend(a_section)

    # Convert lists to numpy arrays
    b = np.array(b_list)
    a = np.array(a_list)

    return b, a
```

Appendix 2.5 Graphical EQ Filter Coefficient Calculation Formula and Explanation

Graphical EQ consists of multiple filters, each of which undergoes a transformation;

- **Conversion Logic**:
  - When bypass=1, b0=1, b1=0, a1=0
  - The calculation method for the download parameters is listed below:
    Input Description:

```
#      fc:       Cutoff frequency Fc (Hz) value set in the tool
#      Q:        Quality factor Q value set in the tool
#      boost:    Filter gain Boost (dB) value set in the tool
#      gain:     Overall gain Gain (dB) value set in the tool
#      filter_type: Numerical value of the filter type selected in the tool
#      bypass:   Bypass disable switch in the tool
#      Fs:       Sampling frequency, default is 48k, used for calculating normalized frequency
```

```python
def calculate_filter_coefficient(fc: float, Q: float, boost: float, gain: float = 0, filter_type: int = 0, by
    # Coefficient range check
    # fc
    fc = fc if fc > 20 else 20
    fc = fc if fc < Fs / 2 else Fs / 2
    fc = fc

    if bypass:
        return [1, 0, 0], [1, 0, 0]

    gain_linear = 10**(gain/20)

    if 0 == filter_type:  # Parametric
        b, a = calculate_param_coeffs(fc, boost, Q, gain, Fs)
    elif 1 == filter_type:  # LowShelf
        b, a = calculate_lowshelf_coeffs(fc, boost, Q, gain, Fs)
    elif 2 == filter_type:  # HighShelf
        b, a = calculate_highshelf_coeffs(fc, boost, Q, gain, Fs)
    elif 3 == filter_type:  # GeneralLP
        b, a = calculate_genlp_coeffs(fc, Q, gain, Fs)
    elif 4 == filter_type:  # GeneralHP
        b, a = calculate_genhp_coeffs(fc, Q, gain, Fs)
    elif 5 == filter_type:  # ButterLP
        b, a = butter(2, fc, btype='low', analog=False, output='ba', fs=Fs)
        b = b*gain_linear
        b = b.tolist()
        a = a.tolist()
    elif 6 == filter_type:  # ButterHP
        b, a = butter(2, fc, btype='high', analog=False, output='ba', fs=Fs)
        b = b*gain_linear
        b = b.tolist()
        a = a.tolist()
    elif 7 == filter_type:  # BesselLP
        b, a = bessel(2, fc, btype='low', analog=False, output='ba', fs=Fs)
        b = b*gain_linear
        b = b.tolist()
        a = a.tolist()
    elif 8 == filter_type:  # BesselHP
        b, a = bessel(2, fc, btype='high', analog=False, output='ba', fs=Fs)
        b = b*gain_linear
        b = b.tolist()
        a = a.tolist()
    elif 9 == filter_type:  # Allpass
        b, a = calculate_apf_coeffs(fc, Q, gain, Fs)
    return b, a
```

Appendix 2.6 Crossover Control Calculation Formulas and Explanations

- Crossover_2_Way separates low-frequency and high-frequency components; the control has two filters, one low-pass filter for the low-frequency path, and one high-pass filter for the high-frequency path.
- Crossover_3_Way separates low-frequency, mid-frequency, and high-frequency components; the control has four filters in total: one low-pass filter for the low-frequency path, one high-pass filter and one low-pass filter in series for the mid-frequency path, and one high-pass filter for the high-frequency path.

Low-pass and high-pass filters use different processing functions for conversion.

- **Conversion Logic**:
  - When bypass=1, b0=1, b1=0, a1=0
    The calculation method for the download parameters is listed below:
    Input Description:

```
#      fc:       The cutoff frequency Fc (Hz) value set in the tool
#      gain:     The overall gain Gain (dB) value set in the tool
#      filter_type: The numerical value of the filter type selected in the tool
#      Fs:       Sampling frequency, default is 48k, used for calculating normalized frequency
```

```
def calculate_crossover_lp_coefficient(fc: float, gain: float, filter_type: int, Fs=48000):
    # Coefficient range check
    # fc
    fc = fc if fc > 20 else 20
    fc = fc if fc < Fs / 2 else Fs / 2
    fc = fc
    # boost
    gain_linear = pow(10, gain / 20.0)

    a0 = 1
    a1 = 0
    a2 = 0
    b0 = 0
    b1 = 0
    b2 = 0

    omega_c = np.pi * fc
    theta_c = omega_c / Fs

    b = np.array([1.0, 0.0, 0.0])
    a = np.array([1.0, 0.0, 0.0])

    if CROSSOVER_FILTER_TYPE.Bypass == filter_type:
        b = None
        a = None
    elif CROSSOVER_FILTER_TYPE.LR12 == filter_type:
        k = omega_c / np.tan(theta_c)
        den = k * k + omega_c * omega_c + 2 * k * omega_c
        b1_Num = -2 * k * k + 2 * omega_c * omega_c
        b2_Num = -2 * k * omega_c + k * k + omega_c * omega_c

        b0 = omega_c * omega_c / den
        b1 = 2 * b0
        b2 = b0
        a1 = b1_Num / den
        a2 = b2_Num / den

        b = np.array([b0, b1, b2])
        a = np.array([1.0, a1, a2])
        b[:3] = b[:3] * gain_linear

    elif CROSSOVER_FILTER_TYPE.Butter12 == filter_type or CROSSOVER_FILTER_TYPE.LR24 == filter_type:
        k = 1.0 / np.tan(theta_c)

        b0 = 1.0 / (1.0 + RK_GLOBAL.SQRT_TWO * k + k * k)
        b1 = 2 * b0
        b2 = b0
        a1 = 2 * b0 * (1 - k * k)
        a2 = b0 * (1 - RK_GLOBAL.SQRT_TWO * k + k * k)
```

```
    b = np.array([b0, b1, b2])
    a = np.array([a0, a1, a2])
    if CROSSOVER_FILTER_TYPE.LR24 == filter_type:
        b = np.append(b, [b0, b1, b2])
        a = np.append(a, [a0, a1, a2])
    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Bessel12 == filter_type:
    k = 1.0 / np.tan(theta_c)

    b0 = 1.0 / (1.0 + RK_GLOBAL.SQRT_THREE * k + k * k)
    b1 = 2 * b0
    b2 = b0
    a1 = 2 * b0 * (1 - k * k)
    a2 = b0 * (1 - RK_GLOBAL.SQRT_THREE * k + k * k)

    b = np.array([b0, b1, b2])
    a = np.array([a0, a1, a2])
    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Butter18 == filter_type or CROSSOVER_FILTER_TYPE.LR36 == filter_type:
    sos = butter(3, fc, btype='low', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    if CROSSOVER_FILTER_TYPE.LR36 == filter_type:
        b = np.append(b, b)
        a = np.append(a, a)
    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Bessel18 == filter_type:
    sos = bessel(3, fc, btype='low', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Butter24 == filter_type or CROSSOVER_FILTER_TYPE.LR48 == filter_type:
    sos = butter(4, fc, btype='low', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    if CROSSOVER_FILTER_TYPE.LR48 == filter_type:
        b = np.append(b, b)
        a = np.append(a, a)
    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Bessel24 == filter_type:
```

```
        sos = bessel(4, fc, btype='low', analog=False, output='sos', fs=Fs)

        b, a = sos_to_ba(sos)

        b[:3] = b[:3] * gain_linear

    elif CROSSOVER_FILTER_TYPE.Butter36 == filter_type:
        sos = butter(6, fc, btype='low', analog=False, output='sos', fs=Fs)

        b, a = sos_to_ba(sos)

        b[:3] = b[:3] * gain_linear

    elif CROSSOVER_FILTER_TYPE.Bessel36 == filter_type:
        sos = bessel(6, fc, btype='low', analog=False, output='sos', fs=Fs)

        b, a = sos_to_ba(sos)

        b[:3] = b[:3] * gain_linear

    elif CROSSOVER_FILTER_TYPE.Butter48 == filter_type:
        sos = butter(8, fc, btype='low', analog=False, output='sos', fs=Fs)

        b, a = sos_to_ba(sos)

        b[:3] = b[:3] * gain_linear

    elif CROSSOVER_FILTER_TYPE.Bessel48 == filter_type:
        sos = bessel(8, fc, btype='low', analog=False, output='sos', fs=Fs)

        b, a = sos_to_ba(sos)

        b[:3] = b[:3] * gain_linear

    return b, a


def calculate_crossover_hp_coefficient(fc: float, gain: float, filter_type: int, Fs=48000):
    # Coefficient range check
    # fc
    fc = fc if fc > 20 else 20
    fc = fc if fc < Fs / 2 else Fs / 2
    fc = fc
    # boost
    gain_linear = pow(10, gain / 20.0)

    a0 = 1
    a1 = 0
    a2 = 0
```

```
b0 = 0
b1 = 0
b2 = 0

omega_c = np.pi * fc
theta_c = omega_c / Fs

b = np.array([1.0, 0.0, 0.0])
a = np.array([1.0, 0.0, 0.0])
if CROSSOVER_FILTER_TYPE.Bypass == filter_type:
    b = None
    a = None
elif CROSSOVER_FILTER_TYPE.LR12 == filter_type:
    k = omega_c / np.tan(theta_c)
    den = k * k + omega_c * omega_c + 2 * k * omega_c
    b1_Num = -2 * k * k + 2 * omega_c * omega_c
    b2_Num = -2 * k * omega_c + k * k + omega_c * omega_c

    b0 = k * k / den
    b1 = -2 * b0
    b2 = b0
    a1 = b1_Num / den
    a2 = b2_Num / den

    b = np.array([b0, b1, b2])
    a = np.array([a0, a1, a2])
    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Butter12 == filter_type or CROSSOVER_FILTER_TYPE.LR24 == filter_type:
    k = np.tan(theta_c)

    b0 = 1.0 / (1.0 + RK_GLOBAL.SQRT_TWO * k + k * k)
    b1 = -2 * b0
    b2 = b0
    a1 = 2 * b0 * (k * k - 1)
    a2 = b0 * (1 - RK_GLOBAL.SQRT_TWO * k + k * k)

    b = np.array([b0, b1, b2])
    a = np.array([a0, a1, a2])
    if CROSSOVER_FILTER_TYPE.LR24 == filter_type:
        b = np.append(b, [b0, b1, b2])
        a = np.append(a, [a0, a1, a2])
    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Bessel12 == filter_type:
    k = np.tan(theta_c)

    b0 = 1.0 / (1.0 + RK_GLOBAL.SQRT_THREE * k + k * k)
    b1 = -2 * b0
```

```
    b2 = b0
    a1 = 2 * b0 * (k * k - 1)
    a2 = b0 * (1 - RK_GLOBAL.SQRT_THREE * k + k * k)

    b = np.array([b0, b1, b2])
    a = np.array([a0, a1, a2])
    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Butter18 == filter_type or CROSSOVER_FILTER_TYPE.LR36 == filter_type:
    sos = butter(3, fc, btype='high', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    if CROSSOVER_FILTER_TYPE.LR36 == filter_type:
        b = np.append(b, b)
        a = np.append(a, a)
    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Bessel18 == filter_type:
    sos = bessel(3, fc, btype='high', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Butter24 == filter_type or CROSSOVER_FILTER_TYPE.LR48 == filter_type:
    sos = butter(4, fc, btype='high', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    if CROSSOVER_FILTER_TYPE.LR48 == filter_type:
        b = np.append(b, b)
        a = np.append(a, a)
    b[:3] = b[:3] * gain_linear

elif CROSSOVER_FILTER_TYPE.Bessel24 == filter_type:
    sos = bessel(4, fc, btype='high', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    b[:3] = b[:3] * gain_linear
elif CROSSOVER_FILTER_TYPE.Butter36 == filter_type:
    sos = butter(6, fc, btype='high', analog=False, output='sos', fs=Fs)

    b, a = sos_to_ba(sos)

    b[:3] = b[:3] * gain_linear
elif CROSSOVER_FILTER_TYPE.Bessel36 == filter_type:
    sos = bessel(6, fc, btype='high', analog=False, output='sos', fs=Fs)
```

```
        b, a = sos_to_ba(sos)

        b[:3] = b[:3] * gain_linear
    elif CROSSOVER_FILTER_TYPE.Butter48 == filter_type:
        sos = butter(8, fc, btype='high', analog=False, output='sos', fs=Fs)

        b, a = sos_to_ba(sos)

        b[:3] = b[:3] * gain_linear
    elif CROSSOVER_FILTER_TYPE.Bessel48 == filter_type:
        sos = bessel(8, fc, btype='high', analog=False, output='sos', fs=Fs)

        b, a = sos_to_ba(sos)

        b[:3] = b[:3] * gain_linear
    return b, a
```

Appendix 2.7 Using Python Library Functions to Calculate FIR Filter Coefficients

The `scipy.signal` library in Python provides two functions, `firwin()` and `firwin2()`, for calculating FIR filter coefficients.

- The `firwin()` function is relatively simple. It only requires basic parameters such as sampling frequency, cutoff frequency, filter type, and filter length to design the filter.
- `firwin2()` is more flexible than the former. It not only achieves the basic functionality of `firwin()` but also allows setting gains at specific frequency points.

Below, we use the `firwin()` function to design a 10th-order (corresponding to a filter length of 11) low-pass filter with a cutoff frequency of 10 kHz:

```
from scipy.signal import firwin

# Input parameter description:
# numtaps is the filter length, i.e., the number of filter coefficients
# fs is the sampling frequency, and cutoff is the cutoff frequency
# window specifies the corresponding window function; the default is the Hamming window
# pass_zero can specify the filter type
# Other parameters can be referenced from the source code of the corresponding Python library

coeffs = firwin(numtaps=11, fs=48000, cutoff=10000, window="hamming", pass_zero="lowpass")
print("filter coeffs:", coeffs)
```

The output result is:

```
filter coeff: [ 0.00131037 -0.01149941 -0.02967312  0.05396302  0.27879614  0.414206
0.27879614  0.05396302 -0.02967312 -0.01149941  0.00131037]
```

After obtaining the corresponding FIR filter coefficients, you can use the FIR control in Section 4.1.

Appendix 2.8 Ratio Gain Conversion to Linear Gain

The ratio gain set in the parameters is converted to linear gain. The conversion code is as follows:

```python
def ratio_to_linear(ratio):
    return 3 * ratio / 100
```

Appendix 2.9 SlewRate Parameter Conversion to Smoothing Coefficient

The SlewRate parameter will be converted into a smoothing coefficient, which depends on the sampling rate and block size. The conversion code is as follows:

```python
def slew_alpha_get(slew_rate, fs=48000, block_size=64):
    if slew_rate == 0:
        return 0
    tc = 0.0416 / block_size * math.pow(2, slew_rate - 1) / 1000
    alpha = math.exp(-1 / (tc * fs))
    return alpha
```

Appendix 2.10 Conversion for Delay- and Control-Time-Related Parameters

This converts the actual time (ms) into the corresponding number of samples. It is commonly used for parameters such as hold time, look-ahead time, and delay time.

```python
def ms_to_samples(ms, fs=48000):
    return float(round(ms * fs / 1000))
```

Appendix 2.11 Conversion for Smoothing-Time-Constant Parameters

This converts the smoothing time constant (ms) into a smoothing coefficient, which depends on the sampling rate and block size. It is commonly used for parameters such as rms_tc, attack_time, release_time, and decay_time.

```
def smooth_coeff_cal(smooth_time, fs=48000, block_size=64):
    if smooth_time == 0:
        return 1
    else:
        return 1 - math.exp(-1000 * block_size / (smooth_time * fs))
```

Appendix 2.12 Conversion for Gain-Change Rate Parameters

This converts the dB change rate (dbps, dB/sec) into a smoothing coefficient, which depends on the sampling rate and block size. It is commonly used for parameters such as RampUp, RampDown, ThldAttack, and ThldRelease.

```
def dbPs_to_linear(dbPs, fs=48000, block_size=64):
    return abs(1 - pow(10, dbPs * block_size / 10 / fs))
```

Appendix 2.13 Conversion for Gain-Related Parameters

This converts a dB value into a linear gain value, and is commonly used for gain-related parameter conversion.

You must distinguish whether the gain is based on RMS or peak:

In general, RMS-based dynamic plug-ins use RMS, such as compress rms and limit rms.

Other dynamics processors and gain applied directly to the output signal typically use peak, such as gain settings in mixer and gain modules.

```
def db_to_linear(db, en_rms=False):
    if db <= -200:
        return 0
    if en_rms:
        return 10**(db/10)
    else:
        return 10**(db/20)
```

Appendix 3 Glossary

Appendix 3.1 biquads:

In speech signal processing, biquads are generally second-order IIR filters. Higher-order IIR filters are prone to instability in engineering applications, while second-order IIR filters are more stable. Typically, higher-order filters are composed of cascaded second-order filters.

The parameters for a second-order IIR filter (biquad) are five (b0, b1, b2, a1, a2) (a0 is fixed at 1, so it is not listed in the parameter settings).

Appendix 3.2 attack_time:

Response time/attack time/rise time refers to the time from when a signal reaches the threshold to when it takes effect. For example, if we set a compressor with a threshold of -30 dB and a gain of -6 dB, and set the attack_time to 100 ms, then when a signal's amplitude slowly increases and reaches the -30 dB threshold, the compressor is triggered. However, the gain does not immediately change to -6 dB. Instead, it gradually changes from 0 dB to -6 dB over 100 ms. The time from triggering the threshold to reaching the operating point is called the attack_time, also known as response time or attack time.

Appendix 3.3 release_time:

Release time/decay time/fall time generally appears in pairs with attack_time. It refers to the time from the operating point to the release point. For example, if we set the release_time to 80 ms, returning to the Case set above with attack_time, after the signal reaches the operating point and then gradually decreases below the threshold (-30 dB), the gain does not immediately return from -6 dB to 0 dB. Instead, it gradually changes from -6 dB to 0 dB over 80 ms. The time from leaving the threshold to the release point is called the release_time, also known as release time or decay time.

Appendix 3.4 Quality Factor Q:

The quality factor Q of a digital filter describes the relationship between the bandwidth of the filter at the center frequency and the center frequency itself. Specifically, the higher the Q value, the narrower the filter bandwidth and the better the frequency selectivity; the lower the Q value, the wider the bandwidth and the poorer the frequency selectivity.

The following example uses a peaking filter:

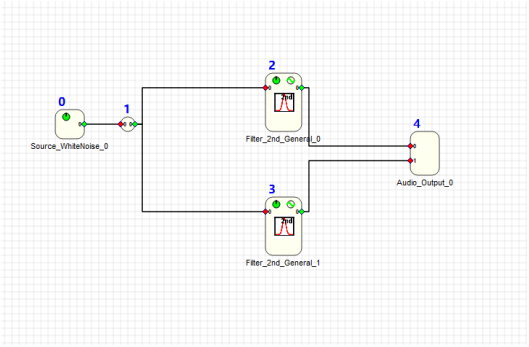


Figure Appendix 3.4-1 Control wiring diagram

The upper filter labeled 2 is set with a center frequency of 4000 Hz and a Q of 15 for a peaking filter; the lower filter labeled 3 is set with a center frequency of 4000 Hz and a Q of 1.41 for a peaking filter.

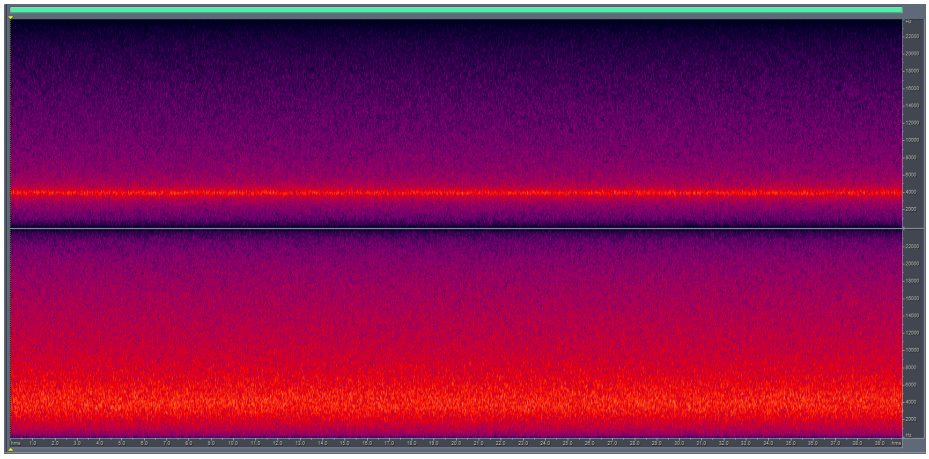


Figure Appendix 3.4-2 Output frequency domain diagram

The output result shows that the higher the Q value, the more concentrated the effect and the narrower the frequency band.

Appendix 3.5 Boost (Filter Gain) and Gain (Overall Output Gain):

First, let's explain the Gain parameter. Gain is the overall output gain, and all filters can set the Gain parameter, which is effective for the entire frequency band. The following example uses a Parametric filter, and the filter calculation function code is:

```
def calculate_param_coeffs(fc: float, boost: float, Q: float, gain=0, Fs=48000):
    w0 = 2 * np.pi * fc / Fs
    A = 10 ** (boost / 40.0)
    alpha = np.sin(w0) / (2.0 * Q)

    a0 = 1 + alpha / A
    a1 = -2 * np.cos(w0)
    a2 = 1 - alpha / A
    b0 = 1 + alpha * A
    b1 = -2 * np.cos(w0)
    b2 = 1 - alpha * A
    a0_invert = 1 / a0

# From the code below, we can see that the gain (dB) is converted to linear gain_linear,
# and then directly multiplied by the filter's numerator coefficients to get the new output coefficients.
    gain_linear = 10 ** (gain/20)
    return [b0*a0_invert*gain_linear, b1*a0_invert*gain_linear, b2*a0_invert*gain_linear], [1.0, a1*a0_invert
```

The following figure shows the amplitude-frequency response of a second-order Parametric filter to intuitively experience the effect of Gain. It shifts the entire curve up and down (the blue line corresponds to Gain=0 dB, and the yellow line corresponds to Gain=1 dB):

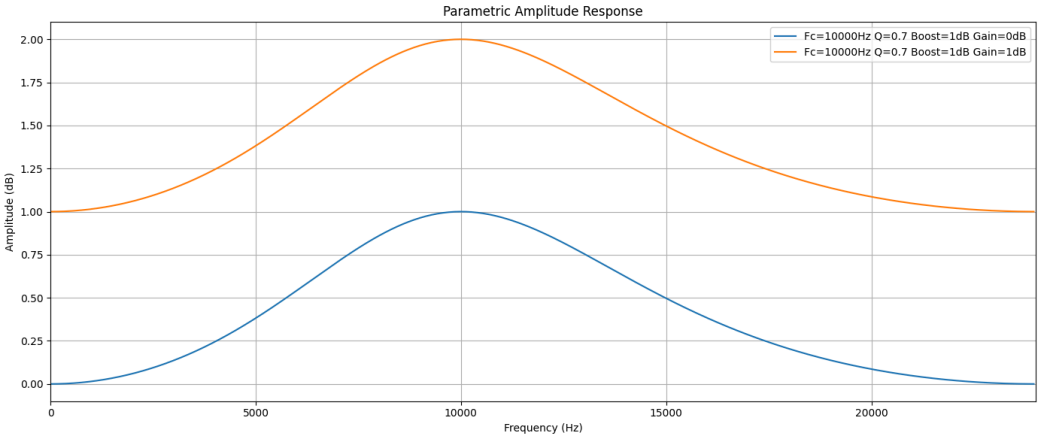


Figure Appendix 3.5-1 The Gain parameter shifts the filter's frequency response curve up and down.

Boost is the filter gain, and only the following filters have this parameter; other filters do not have the Boost parameter.

Type	Description
Parametric	Parametric filter
LowShelf	Low-shelf filter
HighShelf	High-shelf filter

For the Parametric filter, the Boost gain determines the gain at the center frequency Fc (as shown in the blue curve). If Fc is set to 10 kHz and Boost to 1 dB, the amplitude-frequency response curve will have a gain of 1 dB at 10 kHz.

For LowShelf (low-shelf filter) and HighShelf (high-shelf filter), the Boost is set to the maximum gain value, with a gain of 1/2 Boost (dB) at Fc.

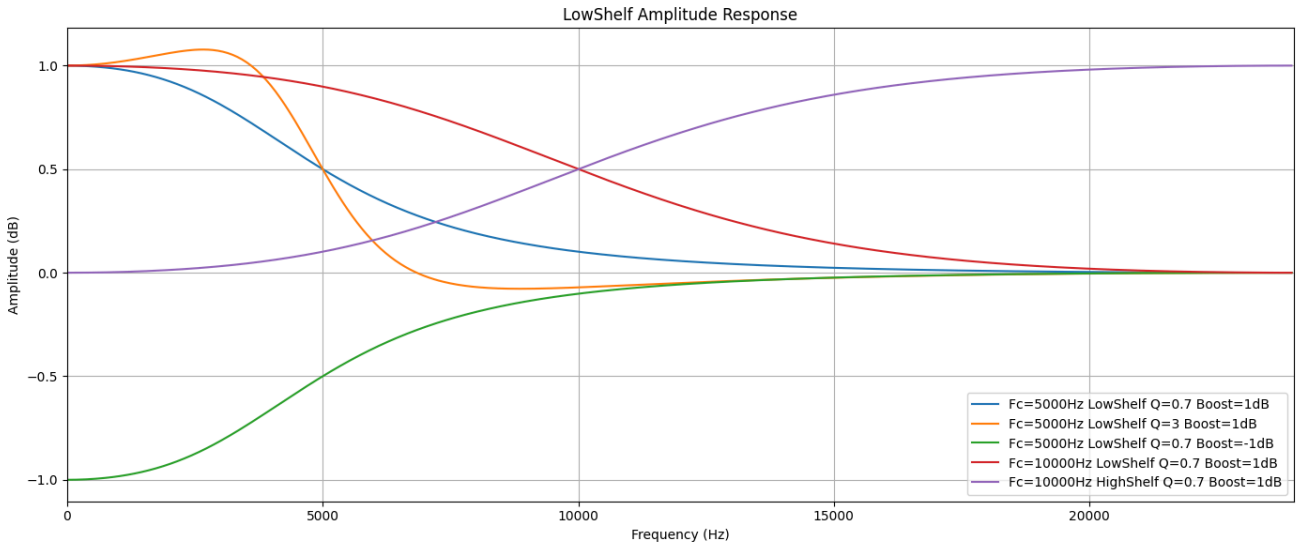


Figure Appendix 3.5-2 Theoretical frequency response curves of LowShelf and HighShelf filters

Appendix 3.6 Dry and Wet Signals:

In digital audio processing, the original, unprocessed audio signal is called dry, while the audio that has been processed by an audio processing system (such as reverb, delay, modulation, etc.) is called wet.

Appendix 3.7 Loudness and Sound Intensity:

Both loudness and sound intensity reflect the magnitude of sound, but they are not the same.

- Sound Intensity: An objective, physical measure of sound magnitude; it represents the sound energy passing through a unit area per unit time. It is typically measured in watts per square meter (W/m²).
- Loudness: A subjective perception of sound magnitude; it indicates how "loud" a sound is perceived. It is typically measured in phons or sones.

The human ear has different sensitivities to sounds of different frequencies. To perceive the same loudness, different frequencies require different sound intensities. As shown in Figure Appendix 3.7-1, the x-axis represents sound frequency, and the y-axis represents sound intensity. The red and blue curves show the required sound intensity at different frequencies to achieve a set loudness level.

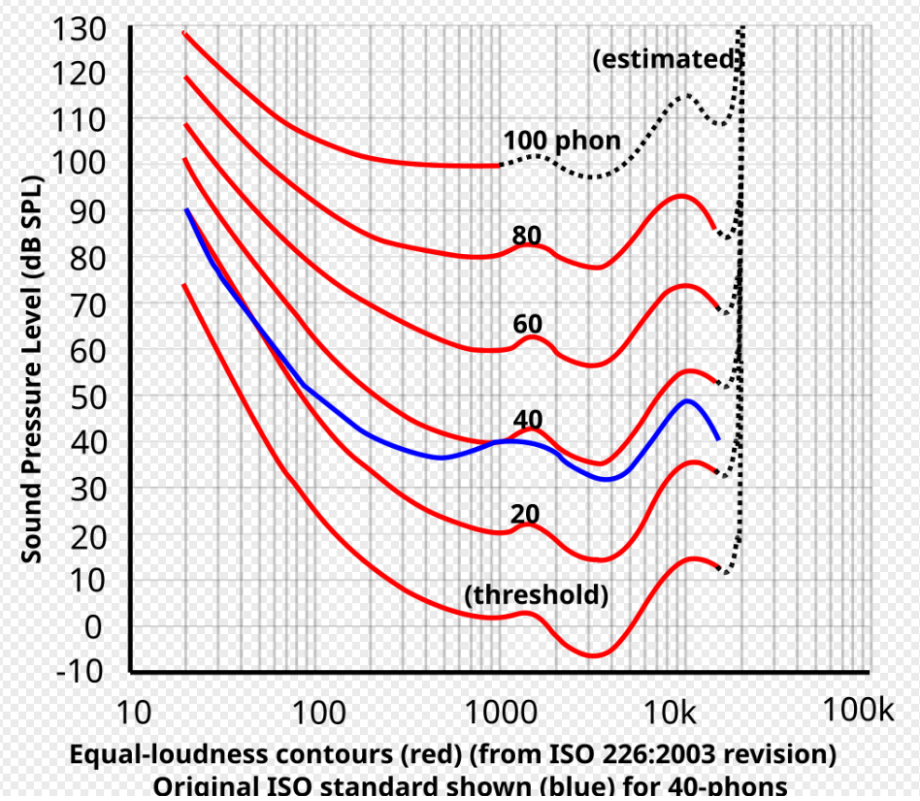


Figure Appendix 3.7-1 Correspondence curve between sound intensity and loudness at different frequencies

The figure shows that the human ear is most sensitive to signals around 3 kHz (the dip in the curve). Sounds at this frequency require lower intensity to achieve the set loudness effect, while other frequencies require higher intensity to achieve the same loudness effect.